Astra Trident 22.04 文档

Astra Trident

NetApp
August 12, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/trident-2204/index.html on August 12, 2022.
Always check docs.netapp.com for the latest.

目录

Astra Trident 22.04 文档 1
发行说明 2
22.04 中的新增功能 2
22.01.1 中的更改 3
22.01.0 中的更改 3
21.10.1 中的更改 3
21.10.0 中的更改 4
已知问题 5
了解更多信息 5
概念 6
Astra Trident 简介 6
ONTAP 驱动程序 7
配置 7
卷快照 8
虚拟存储池 8
卷访问组 9
入门 11
试用 11
要求 11
部署概述 15
使用 Trident 操作员进行部署 18
使用 tridentctl 进行部署 25
下一步是什么? 29
管理 Astra Trident 34
升级 Astra Trident 34
使用操作员升级 35
使用 tridentctl 进行升级 43
卸载 Astra Trident 46
降级 Astra Trident 48
使用 Astra Trident 52
配置后端 52
使用 kubectl 创建后端 122
使用 kubectl 执行后端管理 128
使用 tridentctl 执行后端管理 129
在后端管理选项之间移动 130
管理存储类 137
执行卷操作 139
准备工作节点 164
自动工作节点准备 167

监控 Astra Trident 167
适用于 Docker 的 Astra Trident 172
部署的前提条件 172
部署 Astra Trident 174
升级或卸载 Astra Trident 179
使用卷 181
收集日志 189
管理多个 Astra Trident 实例 190
存储配置选项 191
已知问题和限制 205
常见问题解答 207
一般问题 207
在 Kubernetes 集群上安装和使用 Astra Trident 207
故障排除和支持 208
升级 Astra Trident 210
管理后端和卷 210
支持 215
故障排除 216
常规故障排除 216
使用操作员对未成功的 Trident 部署进行故障排除 217
使用 tridentctl 对未成功的 Trident 部署进行故障排除 219
最佳实践和建议 220
部署 220
存储配置 220
集成 Astra Trident 226
数据保护 234
安全性 239
参考 241
Astra Trident 端口 241
Astra Trident REST API 241
命令行选项 242
与 Kubernetes 集成的 NetApp 产品 243
Kubernetes 和 Trident 对象 244
tridentctl 命令和选项 253
文档的早期版本 259
法律声明 260
版权 260
商标 260
专利 260
隐私政策 260
开放源代码 260

Astra Trident 22.04 文档

发行说明

发行说明提供了有关最新版本的 Astra Trident 中的新增功能，增强功能和错误修复的信息。

> 安装程序 zip 文件中提供的 `tridentctl` 二进制文件是经过测试且受支持的版本。请注意， zip 文件的 ` 或 Extras` 部分中提供的 `macos` 二进制文件未经过测试或不受支持。

22.04 中的新增功能

NetApp 不断改进和完善其产品和服务。以下是 Astra Trident 中的一些最新功能。对于先前版本，请参见 "文档的早期版本"。

> 如果要从先前的任何 Trident 版本升级并使用 Azure NetApp Files ，则 `location` config 参数现在是一个必填字段。

修复

- 改进了 iSCSI 启动程序名称的解析。（问题描述 [#681](https://github.com/NetApp/trident/issues/681))
- 修复了不允许使用 CSI 存储类参数的问题描述 。（问题描述 ##598](https://github.com/NetApp/trident/issues/598))
- 修复了 Trident CRD 中的重复密钥声明。（问题描述 [#671](https://github.com/NetApp/trident/issues/671))
- 修复了不准确的 CSI Snapshot 日志。（问题描述 [#629](https://github.com/NetApp/trident/issues/629))
- 修复了已删除节点上的卷已取消发布的问题描述 。（问题描述 [#691](https://github.com/NetApp/trident/issues/691))
- 增加了对块设备上文件系统不一致问题的处理。（问题描述 （ #656 ）(https://github.com/NetApp/trident/issues/656))
- 修复了在安装期间设置 `imageRegistry` 标志时问题描述 提取自动支持映像的问题。（问题描述 （ #715 ）(https://github.com/NetApp/trident/issues/715))
- 修复了问题描述 ，其中 ANF 驱动程序无法使用多个导出规则克隆卷。

增强功能

- 现在，与 Trident 安全端点的入站连接至少需要 TLS 1.3 。（问题描述 （第 698 号）(https://github.com/NetApp/trident/issues/698))
- 现在， Trident 会将 HSTS 标头添加到其安全端点的响应中。
- Trident 现在会尝试自动启用 Azure NetApp Files UNIX 权限功能。
- * Kubernetes *： Trident demonset 现在以 system-node-critical 优先级类运行。（问题描述 [#694/](https://github.com/NetApp/trident/issues/694))

删除

已删除 E 系列驱动程序（自 2007 年 20 月 20 日起禁用）。

22.01.1 中的更改

修复

- 修复了已删除节点上的卷已取消发布的问题描述 。（"GitHub 问题描述 第 691 号"）
- 修复了访问 ONTAP API 响应中聚合空间的 " 无 " 字段时的崩溃问题。

22.01.0 中的更改

修复

- * Kubernetes： * 增加大型集群的节点注册回退重试时间。
- 修复了问题描述 ，其中 azure-netapp-files 驱动程序可能会被同名的多个资源混淆。
- 如果使用括号指定 ONTAP SAN IPv6 数据 LIF ，则此 LIF 现在可以正常工作。
- 修复的问题描述 ，尝试导入已导入的卷时，返回的 EOF 将使 PVC 处于待定状态。（"GitHub 问题描述 489"）
- 修复了在 SolidFire 卷上创建超过 32 个快照时 Astra Trident 性能下降的问题描述 。
- 在创建 SSL 证书时将 SHA-1 替换为 SHA-256 。
- 固定的 ANF 驱动程序，允许重复的资源名称并将操作限制在一个位置。
- 固定的 ANF 驱动程序，允许重复的资源名称并将操作限制在一个位置。

增强功能

- Kubernetes 增强功能：
  - 增加了对 Kubernetes 1.23 的支持。
  - 通过 Trident 操作员或 Helm 安装 Trident Pod 时，为其添加计划选项。（"GitHub 问题描述 #651"）
- 在 GCP 驱动程序中允许跨区域卷。（"GitHub 问题描述 #633"）
- 为 ANF 卷增加了对 "unixPermissions" 选项的支持。（"GitHub 问题描述 #666"）

已弃用

Trident REST 接口只能在 127.0.0.1 或 [：：： 1 ）地址处侦听和提供服务

21.10.1 中的更改

> v21.10.0 版本具有一个问题描述，在删除节点并将其重新添加回 Kubernetes 集群时， Trident 控制器可以将其置于 CrashLoopBackOff 状态。此问题描述在 v21.10.1 中得到了修复（ GitHub 问题描述 669 ）。

修复

- 修复了在 GCP CVS 后端导入卷导致导入失败的潜在争用情况。
- 修复了一个问题描述，在删除节点并将其重新添加回 Kubernetes 集群时，可能会将 Trident 控制器置于 CrashLoopBackOff 状态（ GitHub 问题描述 669 ）。
- 修复了在未指定 SVM 名称的情况下不再发现 SVM 的问题描述（ GitHub 问题描述 612 ）。

21.10.0 中的更改

修复

- 修复了问题描述，其中无法将 XFS 卷的克隆挂载到与源卷相同的节点上（ GitHub 问题描述 514 ）。
- 修复了问题描述，其中 Astra Trident 在关闭时记录了致命错误（ GitHub 问题描述 597 ）。
- 与 Kubernetes 相关的修复程序：
  - 使用 `ontap-NAS` 和 `ontap-nas-flexgroup` 驱动程序创建快照时，将卷的已用空间返回为最小还原大小（ GitHub 问题描述 645 ）。
  - 修复了问题描述，其中 **无法扩展文件系统** 在调整卷大小后记录了错误（ GitHub 问题描述 560 ）。
  - 修复了 POD 可能停留在 **正在终止** 状态的问题描述（ GitHub 问题描述 572 ）。
  - 修复了 `ontap-san-economy-lun` FlexVol 可能已满快照 LUN 的情况（ GitHub 问题描述 533 ）。
  - 使用不同映像修复了自定义 YAML 安装程序问题描述（ GitHub 问题描述 613" ）。
  - 固定快照大小计算（ GitHub 问题描述 611 ）。
  - 修复了问题描述，其中所有 Astra Trident 安装程序都可以将纯 Kubernetes 标识为 OpenShift （ GitHub 问题描述 639 ）。
  - 修复了 Trident 操作员在无法访问 Kubernetes API 服务器时停止协调的问题（ GitHub 问题描述 599 ）。

增强功能

- 为 GCP-CVS 性能卷增加了对 `unixPermissions` 选项的支持。
- 增加了对 GCP 中 600 GiB 到 1 TiB 范围内的扩展优化 CVS 卷的支持。
- Kubernetes 相关增强功能：
  - 增加了对 Kubernetes 1.22 的支持。
  - 已启用 Trident 操作员和 Helm 图表以使用 Kubernetes 1.22 （ GitHub 问题描述 628 ）。
  - 已将操作员映像添加到 `tridentctl` images 命令（ GitHub 问题描述 570 ）中。

实验增强功能

- 在 `ontap-san` 驱动程序中增加了对卷复制的支持。
- 增加了对 `ontap-nas-flexgroup` ， `ontap-san` 和 `ontap-nas-economy` 驱动程序的 * 技术预览 * REST 支持。

已知问题

已知问题用于确定可能会阻止您成功使用本产品的问题。

- 对于在其 StorageClass 中未指定 `FSType` 的卷， Astra Trident 现在会强制使用空的 `FSType` （`FSTtype=""` ）。使用 Kubernetes 1.17 或更高版本时， Trident 支持为 NFS 卷提供一个空的 `FSType` 。对于 iSCSI 卷，在使用安全上下文强制执行 `fsGroup` 时，您需要在 StorageClass 上设置 `FSType` 。
- 在多个 Astra Trident 实例之间使用后端时，每个后端配置文件对于 ONTAP 后端应具有不同的 `storagePrefix` 值，或者对于 SolidFire 后端使用不同的 **租户名称** 。Astra Trident 无法检测其他 Astra Trident 实例创建的卷。尝试在 ONTAP 或 SolidFire 后端创建现有卷会成功，因为 Astra Trident 会将卷创建视为一项幂等操作。如果 `storagePrefix` 或 **租户名称** 不存在差异，则在同一后端创建的卷可能会发生名称冲突。
- 安装 Astra Trident （使用 `tridentctl` 或 Trident 操作员）并使用 `tridentctl` 管理 Astra Trident 时，应确保设置 `KUBECONFIG` 环境变量。这是 **需要对** tridentctl 进行处理的 Kubernetes 集群进行指示的。在使用多个 Kubernetes 环境时，您应确保准确获取 `KUBECONFIG` 文件。
- 要对 iSCSI PV 执行联机空间回收，工作节点上的底层操作系统可能需要将挂载选项传递到卷。这一点适用于需要 `discard` 的 RHEL/RedHat CoreOS 实例 "挂载选项"；确保中包含 Discard mountOption “s存储类” 支持联机块丢弃。
- 如果每个 Kubernetes 集群有多个 Astra Trident 实例，则 Astra Trident 将无法与其他实例通信，也无法发现它们创建的其他卷，如果集群中运行多个实例，则会导致意外的错误行为。每个 Kubernetes 集群只能有一个 Astra Trident 实例。
- 如果在 Astra Trident 脱机时从 Kubernetes 中删除了基于 Astra Trident 的 `StorageClass` 对象，则 Astra Trident 不会在恢复联机后从其数据库中删除相应的存储类。您应使用 `tridentctl` 或 REST API 删除这些存储类。
- 如果用户在删除相应的 PVC 之前删除了由 Astra Trident 配置的 PV ，则 Astra Trident 不会自动删除后备卷。您应通过 `tridentctl` 或 REST API 删除此卷。
- ONTAP 不能同时配置多个 FlexGroup ，除非聚合集对于每个配置请求是唯一的。
- 使用基于 IPv6 的 Astra Trident 时，应在后端定义中方括号内指定 `managementLIF` 和 `dataLIF` 。例如，`FD20 ： 8b1e ： b258 ： 2000 ： f816 ： 3effe ： feec ： 0` 。
- 如果将 `solidfire-san` 驱动程序与 OpenShift 4.5 结合使用，请确保底层工作节点使用 MD5 作为 CHAP 身份验证算法。

了解更多信息

- "Astra Trident GitHub"
- "Astra Trident 博客"

概念

Astra Trident 简介

Astra Trident 是一个完全受支持的开源项目，由 NetApp 在中维护 "Astra 产品系列"。它旨在帮助您使用容器存储接口（CSI）等行业标准接口满足容器化应用程序的持久性需求。

Astra Trident 作为 Pod 部署在 Kubernetes 集群中，并为您的 Kubernetes 工作负载提供动态存储编排服务。它可以让您的容器化应用程序快速轻松地使用 NetApp 广泛产品组合中的永久性存储，这些产品组合包括 ONTAP（AFF/FAS/Select/Cloud/Amazon FSX for NetApp ONTAP）， Element 软件（NetApp HCI/SolidFire），Astra 数据存储以及 Azure NetApp Files 服务以及 Google Cloud 上的 Cloud Volumes Service 。

Astra Trident 也是 NetApp Astra 的一项基础技术，利用 NetApp 行业领先的快照，备份，复制和克隆数据管理技术，可满足 Kubernetes 工作负载的数据保护，灾难恢复，可移动性和迁移用例。

支持的 Kubernetes 集群架构

以下 Kubernetes 架构支持 Astra Trident：

Kubernetes 集群架构	supported	默认安装
单个主节点，计算节点	是的。	是的。
多主机，计算	是的。	是的。
主， `etcd` ，计算	是的。	是的。
主机，基础架构，计算	是的。	是的。

什么是 Astra？

借助 Astra ，企业可以更轻松地在公有云内部和内部环境中管理，保护和移动 Kubernetes 上运行的数据丰富的容器化工作负载。Astra 使用 NetApp 在公有云和内部环境中成熟而广泛的存储产品组合中的 Astra Trident 配置和提供永久性容器存储。此外，它还提供了一组丰富的高级应用程序感知型数据管理功能，例如快照，备份和还原，活动日志和主动克隆，用于数据保护，灾难 / 数据恢复，数据审核以及 Kubernetes 工作负载的迁移用例。

您可以在 Astra 页面上注册免费试用版。

有关详细信息 ...

- "NetApp Astra 产品系列"
- "Astra Control Service 文档"
- "Astra 控制中心文档"
- "Astra Data Store 文档"
- "Astra API 文档"

ONTAP 驱动程序

Astra Trident 提供了五个唯一的 ONTAP 存储驱动程序，用于与 ONTAP 集群进行通信。详细了解每个驱动程序如何处理卷的创建，访问控制及其功能。

驱动程序	协议	卷模式	支持的访问模式	支持的文件系统
`ontap-NAS`	NFS	文件系统	rwo ， rwx ， rox	"" ， NFS
`ontap-nas-economy.`	NFS	文件系统	rwo ， rwx ， rox	"" ， NFS
`ontap-nas-flexgroup`	NFS	文件系统	rwo ， rwx ， rox	"" ， NFS
`ontap-san`	iSCSI	块	rwo ， rox ， rwx	无文件系统。原始块设备
`ontap-san`	iSCSI	文件系统	工单， ROX	`xfs` ， `ext3` ， `ext4`
`ontap-san-economy.`	iSCSI	块	rwo ， rox ， rwx	无文件系统。原始块设备
`ontap-san-economy.`	iSCSI	文件系统	工单， ROX	`xfs` ， `ext3` ， `ext4`

> 可以使用安全角色的登录凭据（用户名 / 密码）或使用 ONTAP 集群上安装的私钥和证书对 ONTAP 后端进行身份验证。您可以使用 `tridentctl update backend` 来更新现有后端，以便从一种身份验证模式移至另一种身份验证模式。

配置

在 Astra Trident 中配置有两个主要阶段。第一阶段会将存储类与一组合适的后端存储池相关联，并在配置之前进行必要的准备。第二阶段包括卷创建本身，需要从与待定卷的存储类关联的存储池中选择一个存储池。

将后端存储池与存储类关联取决于存储类请求的属性及其 `storagePools` ， `addtionalStoragePools` 和 `excludeStoragePools` 列表。创建存储类时， Trident 会将其每个后端提供的属性和池与存储类请求的属性和池进行比较。如果存储池的属性和名称与请求的所有属性和池名称匹配，则 Astra Trident 会将该存储池添加到该存储类的一组合适存储池中。此外， Astra Trident 会将 `addtionalStoragePools` 列表中列出的所有存储池添加到该集中，即使其属性不满足存储类请求的所有或任何属性也是如此。您应使用 `excludeStoragePools` 列表覆盖存储池并将其从存储类中删除。每次添加新后端时， Astra Trident 都会执行类似的过程，检查其存储池是否满足现有存储类的要求，并删除任何已标记为已排除的。

然后， Astra Trident 会使用存储类和存储池之间的关联来确定在何处配置卷。创建卷时， Astra Trident 会首先获取该卷的存储类的一组存储池， 此外，如果为卷指定协议，则 Astra Trident 会删除无法提供所请求协议的存储池（例如， NetApp HCI/SolidFire 后端无法提供基于文件的卷，而 ONTAP NAS 后端无法提供基于块的卷）。Astra Trident 会随机分配此结果集的顺序，以便均匀分布卷，然后迭代并依次尝试在每个存储池上配置卷。如果在一个上成功，则它会成功返回，并记录在此过程中遇到的任何故障。只有在 * 无法在 * 所有 * 可用于请求

的存储类和协议的存储池上配置时， Astra Trident 才会返回故障。

卷快照

详细了解 Astra Trident 如何为其驱动程序创建卷快照。

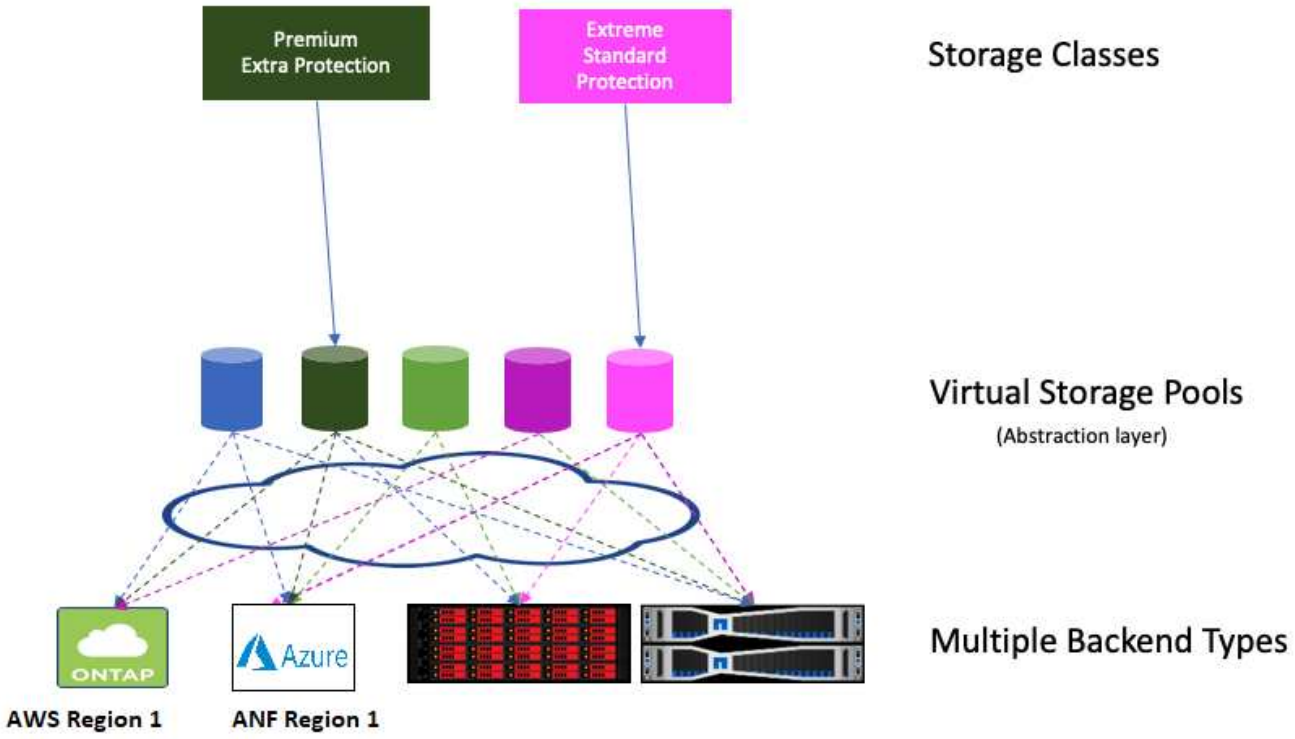
- 对于 `ontap-nas` ， `ontap-san` ， `GCP-CVS` 和 `azure-netapp-files` 驱动程序，每个永久性卷（ PV ）都会映射到一个 FlexVol 。因此，卷快照会创建为 NetApp 快照。与竞争对手的 Snapshot 技术相比， NetApp 的 Snapshot 技术可提供更高的稳定性，可扩展性，可恢复性和性能。无论是在创建 Snapshot 副本所需的时间还是在存储空间中，这些 Snapshot 副本都极为高效。
- 对于 `ontap-nas-flexgroup` 驱动程序，每个永久性卷（ PV ）都会映射到一个 FlexGroup 。因此，卷快照会创建为 NetApp FlexGroup 快照。与竞争对手的 Snapshot 技术相比， NetApp 的 Snapshot 技术可提供更高的稳定性，可扩展性，可恢复性和性能。无论是在创建 Snapshot 副本所需的时间还是在存储空间中，这些 Snapshot 副本都极为高效。
- 对于 `ontap-san-economy`-经济 驱动程序， PV 会映射到在共享 FlexVol 上创建的 LUN 。可以通过对关联 LUN 执行 FlexClones 来实现 PV 的卷快照。借助 ONTAP 的 FlexClone 技术，即使是最大的数据集，也可以几乎即时创建副本。副本与其父级共享数据块，除了元数据所需的存储之外，不会占用任何存储。
- 对于 `solidfire-san` 驱动程序，每个 PV 都会映射到在 NetApp Element 软件 /NetApp HCI 集群上创建的 LUN 。VolumeSnapshot 由底层 LUN 的 Element Snapshot 表示。这些快照是时间点副本，只占用少量系统资源和空间。
- 使用 `ontap-nas` 和 `ontap-san` 驱动程序时， ONTAP 快照是 FlexVol 的时间点副本，并占用 FlexVol 本身的空间。这样，在创建 / 计划快照时，卷中的可写空间量会随着时间的推移而减少。解决此问题的一个简单方法是，通过 Kubernetes 调整大小来增大卷的大小。另一个选项是删除不再需要的快照。删除通过 Kubernetes 创建的卷快照后， Astra Trident 将删除关联的 ONTAP 快照。也可以删除未通过 Kubernetes 创建的 ONTAP 快照。

借助 Astra Trident ，您可以使用 VolumeSnapshots 创建新的 PV 。通过对支持的 ONTAP 和 CVS 后端使用 FlexClone 技术，可以从这些快照创建 PV 。从快照创建 PV 时，后备卷是快照父卷的 FlexClone 。`solidfire-san` 驱动程序使用 Element 软件卷克隆从快照创建 PV 。此时，它将从 Element 快照创建一个克隆。

虚拟存储池

虚拟存储池在 Astra Trident 的存储后端与 Kubernetes 的 `StorageClasses` 之间提供了一个抽象层。通过这些功能，管理员可以在不受后端影响的情况下以一种通用方式为每个后端定义位置，性能和保护等方面，而无需使用 `StorageClass` 指定要用于满足所需条件的物理后端，后端池或后端类型。

存储管理员可以在 JSON 或 YAML 定义文件中的任一 Astra Trident 后端定义虚拟存储池。



在虚拟池列表之外指定的任何方面对于后端都是全局的，并将应用于所有虚拟池，而每个虚拟池可能会分别指定一个或多个方面（覆盖任何后端 - 全局方面）。

> 定义虚拟存储池时，请勿尝试在后端定义中重新排列现有虚拟池的顺序。此外，建议不要编辑 / 修改现有虚拟池的属性，而是定义新的虚拟池。

大多数方面都以后端特定术语来指定。重要的是，这些宽高值不会在后端驱动程序之外公开，也不能在 `StorageClasses` 中进行匹配。而是管理员为每个虚拟池定义一个或多个标签。每个标签都是一个键：值对，标签可能在唯一的后端通用。与其他方面一样，可以为每个池指定标签，也可以为后端指定全局标签。与具有预定义名称和值的方面不同，管理员可以根据需要全权定义标签键和值。

`StorageClass` 通过引用选择器参数中的标签来标识要使用的虚拟池。虚拟池选择器支持六个运算符：

运算符	示例	Description
`=`	性能 = 高级	池的标签值必须匹配
`!=`	性能！ = 至高	池的标签值不能匹配
在 中	位置（东部，西部）	池的标签值必须位于一组值中
注释	性能注释（银牌，铜牌）	池的标签值不能为 b

卷访问组

了解有关 Astra Trident 如何使用的更多信息 "卷访问组"。

> 如果使用的是 CHAP ，请忽略此部分，建议使用此部分来简化管理并避免下面所述的扩展限制。此外，如果您在 CSI 模式下使用 Astra Trident ，则可以忽略此部分。在作为增强型 CSI 配置程序安装时， Astra Trident 会使用 CHAP 。

Astra Trident 可以使用卷访问组来控制对其配置的卷的访问。如果禁用了 CHAP ，则它会查找名为 `trident` 的访问组，除非您在配置中指定了一个或多个访问组 ID 。

虽然 Astra Trident 会将新卷与已配置的访问组相关联，但它不会自行创建或管理访问组。在将存储后端添加到 Astra Trident 之前，访问组必须存在，并且这些访问组必须包含 Kubernetes 集群中每个节点的 iSCSI IQN ，这些节点可能会挂载该后端配置的卷。在大多数安装中，包括集群中的每个工作节点。

对于节点数超过 64 个的 Kubernetes 集群，您应使用多个访问组。每个访问组最多可以包含 64 个 IQN ，每个卷可以属于四个访问组。在最多配置四个访问组的情况下，集群中大小最多为 256 个节点的任何节点都可以访问任何卷。有关卷访问组的最新限制，请参见 "此处"。

如果要将配置从使用默认 `trident` 访问组的修改为也使用其他访问组的配置，请在列表中包含 `trident` 访问组的 ID 。

入门

试用

NetApp 提供了一个可随时使用的实验室映像，您可以通过该映像进行请求 "NetApp 试用"。此测试驱动器为您提供了一个沙盒环境，该环境附带安装和配置了三节点 Kubernetes 集群和 Astra Trident 。这是熟悉 Astra Trident 并了解其功能的好方法。

另一个选项是查看 "《 kubeadm 安装指南》" 由 Kubernetes 提供。

> 您不应在生产环境中使用使用这些说明构建的 Kubernetes 集群。使用您的分发版提供的生产部署指南创建可随时投入生产的集群。

如果这是您第一次使用 Kubernetes ，请熟悉相关概念和工具 "此处"。

要求

首先查看支持的前端，后端和主机配置。

> 要了解 Astra Trident 使用的端口，请参见 "此处"。

支持的前端（编排程序）

Astra Trident 支持多个容器引擎和流程编排程序，其中包括：

- Anthos on-Prem (VMware)和Anthos on bare metal 1.9、1.10、1.11
- Kubernetes 1.18 - 1.24
- Mirantis Kubernetes Engine 3.4
- OpenShift 4.7 ， 4.8 ， 4.9 ， 4.10

以下版本支持 Trident 操作符：

- Anthos on-Prem (VMware)和Anthos on bare metal 1.9、1.10、1.11
- Kubernetes 1.18 - 1.24
- OpenShift 4.7 ， 4.8 ， 4.9 ， 4.10

Astra Trident 还可与许多其他完全托管和自我管理的 Kubernetes 产品结合使用，包括 Google Kubernetes Engine （ GKEE ）， Amazon Elastic Kubernetes Services （ EKS ）， Azure Kubernetes Service （ AKS ）， Rancher 和 VMware Tanzu Portfolio 。

支持的后端（存储）

要使用 Astra Trident ，您需要以下一个或多个受支持的后端：

- 适用于 NetApp ONTAP 的 Amazon FSX

- Azure NetApp Files
- Astra 数据存储
- Cloud Volumes ONTAP
- 适用于 GCP 的 Cloud Volumes Service
- FAS/AFF/ 选择 9.3 或更高版本
- NetApp 全 SAN 阵列（ASA）
- NetApp HCI/ Element 软件 11 或更高版本

功能要求

下表总结了此版本的 Astra Trident 及其支持的 Kubernetes 版本提供的功能。

功能	**Kubernetes** 版本	是否需要功能安全门?
CSI Trident	1.18 - 1.24	否
卷快照	1.18 - 1.24	否
卷快照中的 PVC	1.18 - 1.24	否
iSCSI PV 调整大小	1.18 - 1.24	否
ONTAP 双向 CHAP	1.18 - 1.24	否
动态导出策略	1.18 - 1.24	否
Trident 运算符	1.18 - 1.24	否
自动工作节点准备（测试版）	1.18 - 1.24	否
CSI 拓扑	1.18 - 1.24	否

已测试主机操作系统

默认情况下， Astra Trident 在容器中运行，因此将在任何 Linux 工作程序上运行。但是，根据您使用的后端，这些员工需要能够使用标准 NFS 客户端或 iSCSI 启动程序挂载 Astra Trident 提供的卷。

虽然 Astra Trident 不会正式 " 支持 " 特定的操作系统，但已知以下 Linux 版本可以正常运行：

- OpenShift 容器平台支持的 RedHat CoreOS （RHCOS）版本
- RHEL 或 CentOS 7.4 或更高版本
- Ubuntu 18.04 或更高版本

`tridentctl` 实用程序也可在 Linux 的任何这些分发版上运行。

主机配置

根据所使用的后端，应在集群中的所有工作人员上安装 NFS 和 / 或 iSCSI 实用程序。请参见 "此处" 有关详细信息 ...

存储系统配置：

Astra Trident 可能需要先对存储系统进行一些更改，然后后端配置才能使用它。请参见 "此处" 了解详细信息。

容器映像以及相应的 **Kubernetes** 版本

对于带气的安装，下面列出了安装 Astra Trident 所需的容器映像。使用 `tridentctl images` 命令验证所需容器映像的列表。

Kubernetes 版本	容器映像
v1.18.0	• NetApp/trident：22.04.0 • netapp/trident autosupport： 22.04 • K8s.gcr.io/SIG-storage/Csl-provisioner： v2.2.2 • K8s.gcr.io/SIG-storage/Csl-attacher： v3.4.0 • K8s.gcr.io/SIG-storage/Csl-resizer： v1.4.0 • K8s.gcr.io/SIG-storage/Csl-snapshotter： v3.0.3 • K8s.gcr.io/SIG-storage/Csl-node-driver-registry:v2.5.0 • netapp/trident操作符：22.04.0 (可选)
v1.19.0	• NetApp/trident：22.04.0 • netapp/trident autosupport： 22.04 • K8s.gcr.io/SIG-storage/Csl-provisioner： v2.2.2 • K8s.gcr.io/SIG-storage/Csl-attacher： v3.4.0 • K8s.gcr.io/SIG-storage/Csl-resizer： v1.4.0 • K8s.gcr.io/SIG-storage/Csl-snapshotter： v3.0.3 • K8s.gcr.io/SIG-storage/Csl-node-driver-registry:v2.5.0 • netapp/trident操作符：22.04.0 (可选)

Kubernetes 版本	容器映像
v1.20.0	• NetApp/trident：22.04.0 • netapp/trident autosupport： 22.04 • K8s.gcr.io/SIG-storage/Csl-provisioner： v2.2.2 • K8s.gcr.io/SIG-storage/Csl-attacher： v3.4.0 • K8s.gcr.io/SIG-storage/Csl-resizer： v1.4.0 • K8s.gcr.io/SIG-storage/Csl-snapshotter： v3.0.3 • K8s.gcr.io/SIG-storage/Csl-node-driver-registry:v2.5.0 • netapp/trident操作符：22.04.0 (可选)
v1.21.0	• NetApp/trident：22.04.0 • netapp/trident autosupport： 22.04 • K8s.gcr.io/SIG-storage/Csl-provisioner： v2.2.2 • K8s.gcr.io/SIG-storage/Csl-attacher： v3.4.0 • K8s.gcr.io/SIG-storage/Csl-resizer： v1.4.0 • K8s.gcr.io/SIG-storage/Csl-snapshotter： v3.0.3 • K8s.gcr.io/SIG-storage/Csl-node-driver-registry:v2.5.0 • netapp/trident操作符：22.04.0 (可选)
v1.22.0	• NetApp/trident：22.04.0 • netapp/trident autosupport： 22.04 • K8s.gcr.io/SIG-storage/Csl-provisioner： v2.2.2 • K8s.gcr.io/SIG-storage/Csl-attacher： v3.4.0 • K8s.gcr.io/SIG-storage/Csl-resizer： v1.4.0 • K8s.gcr.io/SIG-storage/Csl-snapshotter： v3.0.3 • K8s.gcr.io/SIG-storage/Csl-node-driver-registry:v2.5.0 • netapp/trident操作符：22.04.0 (可选)

Kubernetes 版本	容器映像
v1.23.0	• NetApp/trident：22.04.0 • netapp/trident autosupport： 22.04 • K8s.gcr.io/SIG-storage/Csl-provisioner： v2.2.2 • K8s.gcr.io/SIG-storage/Csl-attacher： v3.4.0 • K8s.gcr.io/SIG-storage/Csl-resizer： v1.4.0 • K8s.gcr.io/SIG-storage/Csl-snapshotter： v3.0.3 • K8s.gcr.io/SIG-storage/Csl-node-driver-registry:v2.5.0 • netapp/trident操作符：22.04.0 (可选)

在Kubernetes 1.20及更高版本上、只有当`v1`版本提供了`volumesnapshots.snapshot.storage.k8s.io` CRD时、才使用经过验证的`K8s.gcr.io/SIG-storage/Csl-snapshotter：v5.x` image。如果 `v1bea1` 版本在使用 / 不使用 `v1` 版本的情况下为 CRD 提供服务，请使用经验证的 `K8s.gcr.io/SIG-storage/CsI-snapshotter ： v3.x` 映像。

部署概述

您可以使用 Trident 运算符或使用 `tridentctl` 来部署 Astra Trident 。

从 22.04 版开始，每次安装 Astra Trident 时， AES 密钥将不再重新生成。在此版本中， Astra Trident 将安装一个新的机密对象，该对象会在安装之间持续存在。这意味着， 22.04 中的 `tridentctl` 可以卸载先前版本的 Trident ，但早期版本无法卸载 22.04 安装。

选择部署方法

要确定要使用的部署方法，请考虑以下几点：

为什么要使用 **Trident** 运算符？

。 "Trident 运算符" 是动态管理 Astra Trident 资源并自动完成设置阶段的绝佳方式。必须满足某些前提条件。请参见 "要求"。

Trident 运算符提供了以下几项优势。

自我修复功能

您可以监控 Astra Trident 安装并主动采取措施来解决问题，例如删除部署或意外修改部署的时间。将此操作员设置为部署时，将创建 `trident` 操作符 `-<generated -id>` Pod 。此 POD 会将 `Trident Orchestrator` CR 与 Astra Trident 安装关联起来，并始终确保只有一个活动的 `TridentOrchestrator` 。换言之，操作员可确保集群中只有一个 Astra Trident 实例并控制其设置，从而确保安装有效。对安装进行更改（例如删除部署或节点取消设置）时，操作员会识别这些更改并逐个修复它们。

轻松更新现有安装

您可以使用操作员轻松更新现有部署。您只需编辑 `TridentOrchestrator` CR 即可更新安装。例如，请考虑需要启用 Astra Trident 以生成调试日志的情形。

为此，请修补您的 `TridentOrchestrator` 以将 `spec.debug` 设置为 `true`：

```
kubectl patch torc <trident-orchestrator-name> -n trident --type=merge -p
'{"spec":{"debug":true}}'
```

更新 `TridentOrchestrator` 后，操作员将处理更新并修补现有安装。这可能会触发创建新 Pod 以相应地修改安装。

自动处理 **Kubernetes** 升级

当集群的 Kubernetes 版本升级到受支持的版本时，操作员会自动更新现有的 Astra Trident 安装并进行更改，以确保其满足 Kubernetes 版本的要求。

> 如果集群升级到不受支持的版本，则操作员会阻止安装 Astra Trident 。如果已随操作员安装了 Astra Trident ，则会显示一条警告，指示 Astra Trident 安装在不受支持的 Kubernetes 版本上。

使用**Cloud Manager**管理**Kubernetes**集群

使用 "使用Cloud Manager的Astra Trident"、您可以升级到最新版本的Astra Trident、添加和管理存储类并将其连接到工作环境、以及使用Cloud Backup Service 备份永久性卷。Cloud Manager支持使用Trident操作员手动或使用Helm部署Astra Trident。

为什么要使用 **Helm**？

如果您还有其他要使用 Helm 管理的应用程序，从 Astra Trident 21.01 开始，您也可以使用 Helm 管理您的部署。

何时应使用 **tridenctl**？

如果您的现有部署必须升级到，或者您希望对部署进行高度自定义，则应查看使用 "Tridentctl"。这是部署 Astra Trident 的传统方法。

在部署方法之间移动时的注意事项

不难想象需要在部署方法之间移动的情形。在尝试从 `tridentctl` 部署迁移到基于操作员的部署之前，应考虑以下事项，反之亦然：

- 卸载 Astra Trident 时，请始终使用相同的方法。如果已使用 `tridentctl` 进行部署，则应使用适当版本的 `tridentctl` 二进制文件卸载 Astra Trident 。同样，如果要使用操作员进行部署，则应编辑 `TridentOrchestrator` CR 并设置 `sPec.uninstall=true` 以卸载 Astra Trident 。
- 如果您要删除基于操作员的部署并使用 `tridentctl` 来部署 Astra Trident ，则应先编辑 `TridentOrchestrator` 并设置 `sPec.uninstall=true` 以卸载 Astra Trident 。然后删除 `TridentOrchestrator` 和操作员部署。然后，您可以使用 `tridentctl` 进行安装。
- 如果您使用的是基于操作员的手动部署，并且要使用基于 Helm 的 Trident 操作员部署，则应先手动卸载此

操作员，然后再执行 Helm 安装。这样， Helm 就可以使用所需的标签和标注来部署 Trident 操作员。如果不执行此操作，则基于 Helm 的 Trident 操作员部署将失败，并显示标签验证错误和标注验证错误。如果您使用的是基于 `tridentctl` 的部署，则可以使用基于 Helm 的部署，而不会出现问题。

了解部署模式

部署 Astra Trident 的方法有三种。

标准部署

在 Kubernetes 集群上部署 Trident 会导致 Astra Trident 安装程序执行以下两项操作：

- 通过 Internet 提取容器映像
- 创建部署和 / 或节点取消设置，以便在 Kubernetes 集群中所有符合条件的节点上启动 Astra Trident Pod 。

此类标准部署可以通过两种不同的方式执行：

- 使用 `tridentctl install`
- 使用 Trident 运算符。您可以手动或使用 Helm 部署 Trident 操作员。

此安装模式是安装 Astra Trident 的最简单方法，适用于大多数不会实施网络限制的环境。

脱机部署

要执行空映射部署，您可以在调用 `tridentctl install` 时使用 ` -image-regRegistry` 标志来指向私有映像注册表。如果使用 Trident 操作符进行部署，则也可以在 `TridentOrchestrator` 中指定 `sPec.imageRegistry` 。此注册表应包含 "Trident 映像"， "Trident AutoSupport 映像"和 Kubernetes 版本所需的 CSI sidecar 映像。

要自定义部署，您可以使用 `tridentctl` 为 Trident 的资源生成清单。其中包括 Astra Trident 在安装过程中创建的部署，取消设置，服务帐户和集群角色。

有关自定义部署的详细信息，请参见以下链接：

- "自定义基于操作员的部署"
- "自定义基于 `tridentctl` 的部署"

> 如果您使用的是私有映像存储库、则应将`/SIG-storage`添加到专用注册表URL的末尾。在使用私有注册表进行 `tridentctl` 部署时，应将 ` -trident 映像` 和 ` -autosupport-image` 与 ` -image -regRegistry` 结合使用。如果要使用 Trident 操作符部署 Astra Trident ，请确保 Orchestrator CR 在安装参数中包含 `tridentImage` 和 `autosupportImage` 。

远程部署

下面简要概述了远程部署过程：

- 在要部署 Astra Trident 的远程计算机上部署适当版本的 `kubectl` 。
- 从 Kubernetes 集群复制配置文件，并在远程计算机上设置 `KUBECONFIG` 环境变量。

- 启动 `kubectl get nodes` 命令，验证您是否可以连接到所需的 Kubernetes 集群。
- 使用标准安装步骤从远程计算机完成部署。

其他已知配置选项

在 VMware Tanzu Portfolio 产品上安装 Astra Trident 时：

- 集群必须支持有权限的工作负载。
- ` -kubelet-dir` 标志应设置为 kubelet 目录的位置。默认情况下，此值为 ` /var/vcap/data/kubelet` 。

  已知使用 ` -kubelet-dir` 指定 kubelet 位置适用于 Trident Operator ， Helm 和 `tridentctl` 部署。

使用 Trident 操作员进行部署

您可以使用Trident操作员部署Astra Trident。您可以通过以下两种方式之一部署Trident操作员：

- 使用Trident "Helm图表"：Helm图表将部署Trident操作符并分步安装Trident。
- 手动：Trident提供 "bundle.yaml" 可用于安装操作员和创建关联对象的文件。

> 如果您尚未熟悉 "基本概念"，现在是一个实现这一目标的好时机。

要部署 Astra Trident ，应满足以下前提条件：

- 您对运行Kubernetes 1.18 - 1.24的受支持Kubernetes集群拥有完全权限。
- 您可以访问受支持的 NetApp 存储系统。
- 您可以从所有 Kubernetes 工作节点挂载卷。
- 您安装了一个安装了 `kubectl` （如果使用的是 OpenShift ，则为 `oc` ）的 Linux 主机，并将其配置为管理要使用的 Kubernetes 集群。
- 您已将 `KUBECONFIG` 环境变量设置为指向 Kubernetes 集群配置。
- 您已启用 "Astra Trident 所需的功能门"。
- 如果您将 Kubernetes 与 Docker Enterprise 结合使用， "按照其步骤启用 CLI 访问"。

明白了吗？太棒了！让我们开始吧。

部署Trident操作员并使用Helm安装Astra Trident

执行列出的步骤以使用 Helm 部署 Trident 操作员。

除了上述前提条件之外，要使用 Helm 部署 Trident 操作员，还需要满足以下条件：

- Kubernetes 1.18 - 1.24
- Helm 版本 3

步骤

1. 添加 Trident 的 Helm 存储库：

```
helm repo add netapp-trident https://netapp.github.io/trident-helm-chart
```

2. 使用 `helm install` 命令并为您的部署指定一个名称。请参见以下示例：

```
helm install <release-name> netapp-trident/trident-operator --version
22.4.0 --namespace <trident-namespace>
```

> 如果尚未为 Trident 创建命名空间，则可以将 ` -create-namespace` 参数添加到 `helm install` 命令中。然后， Helm 将自动为您创建命名空间。

在安装期间，可以通过两种方式传递配置数据：

- ` 值` （或 ` f` ）：指定包含覆盖的 YAML 文件。可以多次指定此值，最右侧的文件将优先。
- ` -set` ：在命令行上指定覆盖。

例如，要更改默认值 `debug` ，请运行以下 ` -set` 命令：

```
$ helm install <name> netapp-trident/trident-operator --version 22.4.0
--set tridentDebug=true
```

Helm 图表中的 `values.yaml` 文件提供了键及其默认值的列表。

`Helm list` 显示有关安装的详细信息，例如名称，命名空间，图表，状态， 应用程序版本，修订版号等。

手动部署 Trident 操作员

执行列出的步骤以手动部署 Trident 操作员。

第 1 步：确定 Kubernetes 集群的资格

首先，您需要登录到 Linux 主机并验证它是否正在管理 _b工作 _ ， "支持的 Kubernetes 集群" 您具有所需权限。

> 使用 OpenShift 时，在以下所有示例中使用 `oc` 而不是 `kubectl` ，并首先以 * 系统： admin* 的身份运行 `oc login -u system ： admin` 或 `oc login -u Kube-admin` 进行登录。

要验证Kubernetes版本、请运行以下命令：

```
kubectl version
```

要查看您是否具有 Kubernetes 集群管理员权限，请运行以下命令：

```
kubectl auth can-i '*' '*' --all-namespaces
```

要验证是否可以从 Docker Hub 启动使用映像的 POD 并通过 Pod 网络访问存储系统，请运行以下命令：

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

第 **2** 步：下载并设置操作员

> 从 21.01 开始， Trident 操作符的范围为集群范围。使用 Trident 操作符安装 Trident 需要创建 `TridentOrchestrator` Custom Resource Definition （ CRD ）并定义其他资源。在安装 Astra Trident 之前，您应执行以下步骤来设置操作员。

1. 下载最新版本的 "Trident 安装程序包" 从 _Downloads_部分 中提取该文件。

   ```
   wget
   https://github.com/NetApp/trident/releases/download/v22.04.0/trident-
   installer-22.04.0.tar.gz
   tar -xf trident-installer-22.04.0.tar.gz
   cd trident-installer
   ```

2. 使用适当的 CRD 清单创建 `TridentOrchestrator` CRD 。然后，您可以稍后创建一个 `TridentOrchestrator` Custom Resource ，以通过操作员实例化安装。

   运行以下命令：

   ```
   kubectl create -f
   deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
   ```

3. 创建 `TridentOrchestrator` CRD 后，创建操作员部署所需的以下资源：

   - 操作员的 ServiceAccount
   - 对 ServiceAccount 执行 ClusterRole 和 ClusterRoleBinding
   - 专用的 PodSecurityPolicy
   - 运算符本身

     Trident 安装程序包含用于定义这些资源的清单。默认情况下，操作符部署在 `trident` 命名空间中。如果 `trident` 命名空间不存在，请使用以下清单创建一个。

   ```
   $ kubectl apply -f deploy/namespace.yaml
   ```

4. 要在非默认 `trident` 命名空间中部署运算符，您应更新 `serviceaccount.yaml` ，`clusterrolebinding.yaml` 和 `operator.yaml` 清单并生成您的 `bundle.yaml` 。

   运行以下命令以更新 YAML 清单并使用 `kucstation.yaml` 生成您的 `bundle.yaml` ：

   ```
   kubectl kustomize deploy/ > deploy/bundle.yaml
   ```

   运行以下命令以创建资源并部署操作员：

   ```
   kubectl create -f deploy/bundle.yaml
   ```

5. 要在部署后验证操作员的状态，请执行以下操作：

   ```
   $ kubectl get deployment -n <operator-namespace>
   NAME                READY   UP-TO-DATE   AVAILABLE   AGE
   trident-operator    1/1     1            1           3m

   $ kubectl get pods -n <operator-namespace>
   NAME                                READY   STATUS             RESTARTS
   AGE
   trident-operator-54cb664d-lnjxh     1/1     Running            0
   3m
   ```

操作员部署成功创建了一个在集群中的一个工作节点上运行的 POD 。

> 在 Kubernetes 集群中只能有 * 一个操作符实例 * 。请勿创建 Trident 操作员的多个部署。

第 **3** 步：创建 **TridentOrchestrator** 并安装 **Trident**

现在，您可以使用操作员安装 Astra Trident 了！这需要创建 `TridentOrchestrator` 。Trident 安装程序附带了用于创建 `TridentOrchestrator` 的示例定义。这将在 `trident` 命名空间中启动安装。

```
$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created

$ kubectl describe torc trident
Name:        trident
Namespace:
Labels:      <none>
Annotations: <none>
API Version: trident.netapp.io/v1
Kind:        TridentOrchestrator
...
Spec:
  Debug:     true
  Namespace: trident
Status:
  Current Installation Params:
    IPv6:                      false
    Autosupport Hostname:
    Autosupport Image:         netapp/trident-autosupport:21.04
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:                     true
    Enable Node Prep:          false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:21.04.0
  Message:                  Trident installed  Namespace:
trident
  Status:                   Installed
  Version:                  v21.04.0
Events:
    Type Reason Age From Message ---- ------ ---- ---- -------Normal
    Installing 74s trident-operator.netapp.io Installing Trident Normal
    Installed 67s trident-operator.netapp.io Trident installed
```

使用 Trident 操作符，您可以使用 `TridentOrchestrator` 规范中的属性自定义 Astra Trident 的安装方式。请参见 "自定义 Trident 部署"。

状态 `TridentOrchestrator` 指示安装是否成功，并显示已安装的 Trident 版本。

Status	Description
安装	操作员正在使用此 `TridentOrchestrator` CR 安装 Astra Trident 。
已安装	Astra Trident 已成功安装。
正在卸载	操作符正在卸载 Astra Trident ，因为 `sPec.uninstall=true` 。
已卸载	Astra Trident 已卸载。
失败	操作员无法安装，修补，更新或卸载 Astra Trident ；操作员将自动尝试从此状态恢复。如果此状态仍然存在，则需要进行故障排除。
正在更新	操作员正在更新现有安装。
error	不使用 `TridentOrchestrator` 。另一个已存在。

在安装期间， `TridentOrchestrator` 的状态会从 `Installing` 更改为 `Installed` 。如果您观察到 `failed` 状态，并且操作员无法自行恢复，则应检查操作员的日志。请参见 "故障排除" 部分。

您可以通过查看已创建的 Pod 来确认 Astra Trident 安装是否已完成：

```
$ kubectl get pod -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-7d466bf5c7-v4cpw        5/5     Running   0           1m
trident-csi-mr6zc                   2/2     Running   0           1m
trident-csi-xrp7w                   2/2     Running   0           1m
trident-csi-zh2jt                   2/2     Running   0           1m
trident-operator-766f7b8658-ldzsv   1/1     Running   0           3m
```

您也可以使用 `tridentctl` 检查已安装的 Astra Trident 版本。

```
$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.04.0        | 21.04.0        |
+----------------+----------------+
```

现在，您可以继续创建后端。请参见 "部署后任务"。

有关在部署期间排除问题的信息，请参见 "故障排除" 部分。

自定义 Trident 操作员部署

使用 Trident 操作符，您可以使用 `TridentOrchestrator` 规范中的属性自定义 Astra Trident 的安装方式。

有关属性列表，请参见下表：

参数	Description	Default
命名空间	用于安装 Astra Trident 的命名空间	default
`debug`	为 Astra Trident 启用调试	false
`IPv6`	安装基于 IPv6 的 Astra Trident	false
`k8sTimeout`	Kubernetes 操作超时	30 秒
`sileAutoSupport`	不要自动向 NetApp 发送 AutoSupport 捆绑包	false
`enableNodePrep`	自动管理工作节点依赖关系（* 测试版 *）	false
`autosupport` 映像	AutoSupport 遥测的容器映像	"NetApp/trident autosupport：21.04.0"
`autosupport` 代理	用于发送 AutoSupport 遥测的代理的地址 / 端口	"<a href="http://proxy.example.com:8888"" class="bare">http://proxy.example.com:8888"</a>
卸载	用于卸载 Astra Trident 的标志	false
`logFormat`	要使用的 Astra Trident 日志记录格式 [text， json]	文本
`TridentImage`	要安装的 Astra Trident 映像	"NetApp/Trident： 21.04"
`imageRegistry`	内部注册表的路径，格式为 ` < 注册表 FQDN>[： 端口 ]/subpath]`	"K8s.gcr.io/SIG-storage (K8s 1.18+) 或quay.io/k8scsi "
`kubeletDir`	主机上的 kubelet 目录的路径	"/var/lib/kubelet"
`wipeout`	要删除以执行 Astra Trident 完全删除的资源列表	
`imagePullSecs`	从内部注册表中提取映像的机密信息	
控制器插件节点选择器	运行 Trident 控制器 CSI 插件的 Pod 的其他节点选择器。格式与 pod.spec.nodeSelector 相同。	无默认值；可选
控制器插件公差	覆盖运行 Trident 控制器 CSI 插件的 Pod 的容错。格式与 po.spec.Tolerations 相同。	无默认值；可选
`nodePluginNodeSelector`	运行 Trident Node CSI 插件的 Pod 的其他节点选择器。格式与 pod.spec.nodeSelector 相同。	无默认值；可选
`nodePluginTholeations`	覆盖运行 Trident Node CSI 插件的 Pod 的容错。格式与 po.spec.Tolerations 相同。	无默认值；可选

在 `TridentOrchestrator` 中指定 `spec.namespace` ，以表示安装了哪个命名空间 Astra Trident 。此参数 * 安装 Astra Trident 后无法更新 * 。尝试执行此操作会导致 `TridentOrchestrator` 的状态更改为 `Failed` 。Astra Trident 不适用于跨命名空间迁移。

自动员工节点准备是一项 * 测试版功能 * ，仅用于非生产环境。

有关格式化 POD 参数的详细信息，请参见 "将 Pod 分配给节点"。

在定义 `TridentOrchestrator` 时，您可以使用上述属性自定义安装。以下是一个示例：

```
$ cat deploy/crds/tridentorchestrator_cr_imagepullsecrets.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  imagePullSecrets:
  - thisisasecret
```

下面是另一个示例，说明如何使用节点选择器部署 Trident：

```
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  controllerPluginNodeSelector:
    nodetype: master
  nodePluginNodeSelector:
    storage: netapp
```

如果您希望自定义安装超出 `TridentOrchestrator` 参数的允许范围，则应考虑使用 `tridentctl` 生成自定义 YAML 清单，以便您可以根据需要进行修改。

使用 tridentctl 进行部署

您可以使用 `tridentctl` 来部署 Astra Trident 。

如果您尚未熟悉 "基本概念"，现在是一个实现这一目标的好时机。

要自定义部署，请参见 "此处"。

要部署 Astra Trident ，应满足以下前提条件：

- 您对受支持的 Kubernetes 集群具有完全权限。
- 您可以访问受支持的 NetApp 存储系统。
- 您可以从所有 Kubernetes 工作节点挂载卷。
- 您安装了一个安装了 `kubectl` （如果使用的是 OpenShift ，则为 `oc` ）的 Linux 主机，并将其配置为管理要使用的 Kubernetes 集群。
- 您已将 `KUBECONFIG` 环境变量设置为指向 Kubernetes 集群配置。
- 您已启用 "Astra Trident 所需的功能门"。
- 如果您将 Kubernetes 与 Docker Enterprise 结合使用， "按照其步骤启用 CLI 访问"。

明白了吗？太棒了！让我们开始吧。

有关自定义部署的信息，请参见 "此处"。

第 1 步：确定 Kubernetes 集群的资格

首先，您需要登录到 Linux 主机并验证它是否正在管理 _b工作 _ ， "支持的 Kubernetes 集群" 您具有所需权限。

使用 OpenShift 时，您可以在以下所有示例中使用 `oc` 而不是 `kubectl` ，并且应首先以 * 系统：admin* 的身份登录，方法是运行 `oc login -u system ： admin` 或 `oc login -u Kube-admin` 。

要检查 Kubernetes 版本，请运行以下命令：

```
kubectl version
```

要查看您是否具有 Kubernetes 集群管理员权限，请运行以下命令：

```
kubectl auth can-i '*' '*' --all-namespaces
```

要验证是否可以从 Docker Hub 启动使用映像的 POD 并通过 Pod 网络访问存储系统，请运行以下命令：

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

确定 Kubernetes 服务器版本。您将在安装 Astra Trident 时使用它。

第 2 步：下载并提取安装程序

> Trident 安装程序将创建 Trident Pod ，配置用于保持其状态的 CRD 对象，并初始化执行配置卷和将卷附加到集群主机等操作的 CSI sidecars 。

您可以下载最新版本的 "Trident 安装程序包" 从 _Downloads_部分 中提取该文件。

例如，如果最新版本为 21.07.1 ：

```
wget https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
tar -xf trident-installer-21.07.1.tar.gz
cd trident-installer
```

第 3 步：安装 Astra Trident

执行 `tridentctl install` 命令，在所需命名空间中安装 Astra Trident 。

```
$ ./tridentctl install -n trident
....
INFO Starting Trident installation.                namespace=trident
INFO Created service account.
INFO Created cluster role.
INFO Created cluster role binding.
INFO Added finalizers to custom resource definitions.
INFO Created Trident service.
INFO Created Trident secret.
INFO Created Trident deployment.
INFO Created Trident daemonset.
INFO Waiting for Trident pod to start.
INFO Trident pod started.                          namespace=trident
pod=trident-csi-679648bd45-cv2mx
INFO Waiting for Trident REST interface.
INFO Trident REST interface is up.                 version=21.07.1
INFO Trident installation succeeded.
....
```

安装程序完成后，其外观将会如此。根据 Kubernetes 集群中的节点数，您可能会发现更多 Pod ：

```
$ kubectl get pod -n trident
NAME                              READY   STATUS    RESTARTS   AGE
trident-csi-679648bd45-cv2mx      4/4     Running   0          5m29s
trident-csi-vgc8n                 2/2     Running   0          5m29s

$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.07.1        | 21.07.1        |
+----------------+----------------+
```

如果您看到与上述示例类似的输出，则表示您已完成此步骤，但尚未完全配置 Astra Trident 。请继续执行下一步。请参见 "部署后任务"。

但是，如果安装程序未成功完成或您未看到 * 正在运行 * `trident CSI -` ，则表示未安装此平台。

有关在部署期间排除问题的信息，请参见 "故障排除" 部分。

自定义 tridentctl 部署

使用 Trident 安装程序可以自定义属性。例如，如果已将 Trident 映像复制到专用存储库，则可以使用 ` -trident 映像` 指定映像名称。如果您已将 Trident 映像以及所需的 CSI sidecar 映像复制到专用存储库，则最好使用 ` -image-regRegistry` 开关指定该存储库的位置，该开关的格式为 ` < 注册表 FQDN>[： port]` 。

要让 Astra Trident 自动为您配置工作节点，请使用 ` -enable-node-prep` 。有关其工作原理的详细信息，请参见 "此处"。

自动员工节点准备是一项 * 测试版功能 * ，仅用于非生产环境。

如果您使用的是 Kubernetes 的分发版本，其中 `kubelet` 将其数据保存在通常的 ` /var/lib/kubelet` 以外的路径上，则可以使用 ` -kubelet-dir` 指定备用路径。

如果您需要自定义安装，使其超出安装程序参数的允许范围，则还可以自定义部署文件。使用 ` -generate -custom-yaml` 参数可在安装程序的 `setup` 目录中创建以下 YAML 文件：

- `trident — clusterrolebinding 。 yaml`
- `trident` 部署。 `yaml`
- `trident — crds.yaml`
- `trident` 集群角色。 `yaml`
- `trident — demonset.yaml`
- `trident service.yaml`
- `trident` 命名空间 `.yaml`
- `trident` 服务帐户。 `yaml`

生成这些文件后，您可以根据需要对其进行修改，然后使用 ` -use-custom-yaml` 安装自定义部署。

```
./tridentctl install -n trident --use-custom-yaml
```

下一步是什么?

部署 Astra Trident 后，您可以继续创建后端，创建存储类，配置卷以及将卷挂载到 Pod 中。

第 1 步：创建后端

现在，您可以继续创建一个后端，供 Astra Trident 配置卷使用。要执行此操作，请创建包含必要参数的 `backend.json` 文件。可以在 `sample-input` 目录中找到不同后端类型的示例配置文件。

请参见 "此处" 有关如何为后端类型配置文件的更多详细信息。

```
cp sample-input/<backend template>.json backend.json
vi backend.json
```

```
./tridentctl -n trident create backend -f backend.json
+-------------+----------------+--------------------------------------
+--------+---------+
|    NAME     | STORAGE DRIVER |                 UUID                 |
STATE   | VOLUMES |
+-------------+----------------+--------------------------------------
+--------+---------+
| nas-backend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       0 |
+-------------+----------------+--------------------------------------
+--------+---------+
```

如果创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
./tridentctl -n trident logs
```

解决问题后，只需返回到此步骤的开头并重试。有关更多故障排除提示，请参见 "故障排除" 部分。

第 2 步：创建存储类

Kubernetes 用户使用指定的永久性卷声明（ Persistent Volume Claim ， PVC ）配置卷 "存储类" 按名称。详细信息对用户隐藏，但存储类可标识用于该类的配置程序（在本例中为 Trident ）以及该类对配置程序的含义。

创建存储类 Kubernetes 用户将指定何时需要卷。该类的配置需要为上一步创建的后端建模，以便 Astra Trident 可以使用它来配置新卷。

首先要使用的最简单存储类是基于安装程序随附的 `sample-input/storage-class-csi 。 yaml.tempdl` 文件，将 `*backend_type _*` 替换为存储驱动程序名称。

```
./tridentctl -n trident get backend
+-------------+----------------+--------------------------------------
+--------+---------+
|    NAME     | STORAGE DRIVER |                 UUID                 |
STATE  | VOLUMES |
+-------------+----------------+--------------------------------------
+--------+---------+
| nas-backend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       0 |
+-------------+----------------+--------------------------------------
+--------+---------+

cp sample-input/storage-class-csi.yaml.templ sample-input/storage-class-
basic-csi.yaml

# Modify __BACKEND_TYPE__ with the storage driver field above (e.g.,
ontap-nas)
vi sample-input/storage-class-basic-csi.yaml
```

这是一个 Kubernetes 对象，因此您可以使用 `kubectl` 在 Kubernetes 中创建该对象。

```
kubectl create -f sample-input/storage-class-basic-csi.yaml
```

现在， Kubernetes 和 Astra Trident 都应显示 * 基本 -CSI * 存储类， Astra Trident 应已发现后端的池。

```
kubectl get sc basic-csi
NAME         PROVISIONER             AGE
basic-csi    csi.trident.netapp.io   15h

./tridentctl -n trident get storageclass basic-csi -o json
{
  "items": [
    {
      "Config": {
        "version": "1",
        "name": "basic-csi",
        "attributes": {
          "backendType": "ontap-nas"
        },
        "storagePools": null,
        "additionalStoragePools": null
      },
      "storage": {
        "ontapnas_10.0.0.1": [
          "aggr1",
          "aggr2",
          "aggr3",
          "aggr4"
        ]
      }
    }
  ]
}
```

第 3 步：配置第一个卷

现在，您已准备好动态配置第一个卷。可通过创建 Kubernetes 来完成此操作 "永久性卷声明" （ PVC ）对象。

为使用刚刚创建的存储类的卷创建 PVC 。

有关示例，请参见 `sample-input/vpva-basic CSI 。 yaml` 。确保存储类名称与您创建的名称匹配。

```
kubectl create -f sample-input/pvc-basic-csi.yaml

kubectl get pvc --watch
NAME      STATUS    VOLUME                                     CAPACITY
ACCESS MODES  STORAGECLASS   AGE
basic     Pending
basic          1s
basic     Pending   pvc-3acb0d1c-b1ae-11e9-8d9f-5254004dfdb7   0
basic          5s
basic     Bound     pvc-3acb0d1c-b1ae-11e9-8d9f-5254004dfdb7   1Gi
RWO           basic          7s
```

第 4 步：将卷挂载到 Pod 中

现在，让我们挂载卷。我们将启动一个 nginx pod ，将 PV 挂载到 ` /usr/share/nginx/html` 下。

```
cat << EOF > task-pv-pod.yaml
kind: Pod
apiVersion: v1
metadata:
  name: task-pv-pod
spec:
  volumes:
    - name: task-pv-storage
      persistentVolumeClaim:
       claimName: basic
  containers:
    - name: task-pv-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: task-pv-storage
EOF
kubectl create -f task-pv-pod.yaml
```

```
# Wait for the pod to start
kubectl get pod --watch

# Verify that the volume is mounted on /usr/share/nginx/html
kubectl exec -it task-pv-pod -- df -h /usr/share/nginx/html

# Delete the pod
kubectl delete pod task-pv-pod
```

此时， Pod （应用程序）不再存在，但卷仍在。如果需要，您可以从另一个 POD 使用它。

要删除卷，请删除声明：

```
kubectl delete pvc basic
```

现在，您可以执行其他任务，例如：

- "配置其他后端。"
- "创建其他存储类。"

管理 Astra Trident

升级 Astra Trident

Astra Trident 遵循季度发布节奏，每个日历年提供四个主要版本。每个新版本都是在先前版本的基础上构建的，可提供新功能和性能增强以及错误修复和改进功能。我们建议您每年至少升级一次，以利用 Astra Trident 中的新功能。

> 要升级到之前的五个版本，您需要执行多步升级。

确定要升级到的版本

- 您可以从 `YY-1.mm` 版本和任何介于两者之间的版本升级到 `YY.mm` 版本。例如，您可以从 19.07 及更高版本直接升级到 20.07 （包括 DOT 版本，例如 19.07.1 ）。
- 如果您使用的是早期版本，则应执行多步骤升级。这要求您首先升级到适合您的四个版本窗口的最新版本。例如，如果您运行的是 18.07 ，并希望升级到 20.07 版本，请按照以下所述执行多步骤升级过程：
  - 首次从 18.07 升级到 19.07 。有关升级的具体说明，请参见相应版本的文档。
  - 然后从 19.07 升级到 20.07 。

> 对于 19.04 及更早版本的所有升级，都需要将 Astra Trident 的元数据从其自身的 `etcd` 迁移到 CRD 对象。请务必查看此版本的文档，了解升级的工作原理。

> 升级时，请务必提供 `parameter.FSType` in `StorageClasses` used by Astra Trident 。您可以删除并重新创建 `StorageClasses` ，而不会中断已有卷。这是执行的一项 * 要求 * "安全上下文" SAN 卷。。 "输入示例：" 目录包含示例，例如 “storage-class-basic 。 yaml.temp” 和 “storage-class-brone-default.yaml”。有关详细信息，请参见 "已知问题"。

我应选择哪种升级路径?

您可以使用以下路径之一进行升级：

- 使用 Trident 运算符。
- 使用 `tridentctl` 。

> 从 Kubernetes 1.20 开始， CSI 卷快照现在是 GA 功能。在升级 Astra Trident 时，在执行升级之前，必须删除所有先前的 alpha snapshot CRS 和 CRD （卷快照类，卷快照和卷快照内容）。请参见 "本博客" 了解将 alpha 快照迁移到测试版 /GA 规范所涉及的步骤。

如果满足以下条件，则可以使用 Trident 操作符进行升级：

- 您正在运行 CSI Trident （ 19.07 及更高版本）。
- 您已安装基于 CRD 的 Trident 版本（ 19.07 及更高版本）。
- 您正在执行自定义安装（使用自定义 YAML ）。

> 如果您使用的是基于 `etcd` 的 Trident 版本（ 19.04 或更早版本），请勿使用运算符升级 Trident 。

如果您不想使用该运算符，或者您的自定义安装操作符不支持，则可以使用 `tridentctl` 进行升级。这是 Trident 19.04 及更早版本的首选升级方法。

对运算符进行了更改

Astra Trident 21.01 版为操作员引入了一些关键的架构变更，即：

- 操作符现在为 * 集群范围 * 。以前的 Trident 运算符实例（版本 20.04 到 20.10 ）为 * 命名空间范围 * 。集群范围内的运算符具有优势，原因如下：
  - 资源责任：操作员现在可以在集群级别管理与 Astra Trident 安装相关的资源。在安装 Astra Trident 过程中，操作员使用 `ownerReferences` 创建和维护多个资源。在集群范围的资源上维护 所有权参考 可能会在某些 Kubernetes 分销商（例如 OpenShift ）上引发错误。使用集群范围的运算符可缓解此问题。对于 Trident 资源的自动修复和修补，这是一项基本要求。
  - 卸载期间清理：要完全删除 Astra Trident ，需要删除所有关联的资源。命名空间范围的运算符可能会在删除集群范围的资源（例如 clusterRole ， ClusterRoleBinding-and PodSecurityPolicy ）时遇到问题，并导致清理不完整。集群范围的运算符可消除此问题描述。用户可以完全卸载 Astra Trident 并在需要时重新安装。
- `TridentProvider` 现已替换为 `TridentOrchestrator` 作为用于安装和管理 Astra Trident 的自定义资源。此外，还为 `TridentOrchestrator` 规范引入了一个新字段。用户可以使用 `spec.namespace` 字段指定必须安装 / 升级命名空间 Trident 。您可以查看一个示例 "此处"。

了解更多信息

- "使用 Trident 操作符进行升级"
- "使用 `tridentctl` 进行升级"

使用操作员升级

您可以使用操作员轻松升级现有的 Astra Trident 安装。

要使用运算符进行升级，应满足以下条件：

- 您应安装基于 CSI 的 Astra Trident 。要检查是否正在运行 CSI Trident ，请检查 Trident 命名空间中的 Pod 。如果它们遵循 `trident CSI - *` 命名模式，则表示您正在运行 CSI Trident 。
- 您应安装基于 CRD 的 Trident 。这表示 19.07 及更高版本的所有版本。如果您安装的是基于 CSI 的安装，则很可能是基于 CRD 的安装。
- 如果您已卸载 CSI Trident ，并且安装中的元数据仍然存在，则可以使用操作员进行升级。
- 在给定 Kubernetes 集群中的所有命名空间中，只应安装一个 Astra Trident 。
- 您应使用运行的 Kubernetes 集群 "版本1.18 - 1.24"。
- 如果存在字母快照 CRD ，则应使用 `tridentctl obliviate alpha-snapshot-crd` 将其删除。此操作将删除 alpha snapshot 规范的 CRD 。有关应删除 / 迁移的现有快照，请参见 "本博客"。

> 在 OpenShift 容器平台上使用操作符升级 Trident 时，应升级到 Trident 21.01.1 或更高版本。21.01.0 版发布的 Trident 运算符包含一个已知的问题描述，该 已在 21.01.1 中修复。有关详细信息，请参见 "GitHub 上的问题描述详细信息"。

升级集群范围的操作员安装

要从 * Trident 21.01 及更高版本 * 升级，请执行以下步骤。

步骤

1. 删除用于安装当前 Astra Trident 实例的 Trident 运算符。例如，如果要从 21.01 升级，请运行以下命令：

```
kubectl delete -f 21.01/trident-installer/deploy/bundle.yaml -n trident
```

2. （可选）如果要修改安装参数，请编辑在安装 Trident 时创建的 `TridentOrchestrator` 对象。这可能包括一些更改，例如修改自定义 Trident 映像，从其中提取容器映像的专用映像注册表，启用调试日志或指定映像提取密钥。
3. 使用为新版本设置 Trident 操作符的 `bundle.yaml` 文件安装 Astra Trident 。运行以下命令：

```
kubectl create -f 21.10.0/trident-installer/deploy/bundle.yaml -n
trident
```

在此步骤中， 21.10.0 Trident 操作员将确定现有的 Astra Trident 安装并将其升级到与操作员相同的版本。

升级命名空间范围的操作员安装

要从使用命名空间范围的运算符（版本 20.07 至 20.10 ）安装的 Astra Trident 实例进行升级，需要遵循以下步骤：

步骤

1. 验证现有 Trident 安装的状态。要执行此操作，请检查 `TridentProvider` 的 * 状态 * 。状态应为 `installed` 。

```
$ kubectl describe tprov trident -n trident | grep Message: -A 3
Message:  Trident installed
Status:   Installed
Version:  v20.10.1
```

> 如果状态显示 `Updating` ，请确保先解决此问题，然后再继续。有关可能的状态值列表，请参见 "此处"。

2. 使用 Trident 安装程序随附的清单创建 `TridentOrchestrator` CRD 。

```
# Download the release required [21.01]
$ mkdir 21.07.1
$ cd 21.07.1
$ wget
https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
$ tar -xf trident-installer-21.07.1.tar.gz
$ cd trident-installer
$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 使用其清单删除命名空间范围的运算符。要完成此步骤，您需要使用 `bundle.yaml` 文件来部署命名空间范围的运算符。您可以从获取 `bundle.yaml` "Trident 存储库"。请确保使用相应的分支。

> 在删除命名空间范围的运算符后以及安装集群范围的运算符之前，您应对 Trident 安装参数进行必要的更改（例如，更改 `tridentImage` ， `autosupportImage` ，私有映像存储库以及提供 `imagePullSecs` 的值）。有关可更新的参数的完整列表，请参见 "参数列表"。

```
#Ensure you are in the right directory
$ pwd
$ /root/20.10.1/trident-installer

#Delete the namespace-scoped operator
$ kubectl delete -f deploy/bundle.yaml
serviceaccount "trident-operator" deleted
clusterrole.rbac.authorization.k8s.io "trident-operator" deleted
clusterrolebinding.rbac.authorization.k8s.io "trident-operator" deleted
deployment.apps "trident-operator" deleted
podsecuritypolicy.policy "tridentoperatorpods" deleted

#Confirm the Trident operator was removed
$ kubectl get all -n trident
NAME                                READY   STATUS    RESTARTS   AGE
pod/trident-csi-68d979fb85-dsrmn    6/6     Running   12         99d
pod/trident-csi-8jfhf               2/2     Running   6          105d
pod/trident-csi-jtnjz               2/2     Running   6          105d
pod/trident-csi-lcxvh               2/2     Running   8          105d

NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
AGE
service/trident-csi   ClusterIP   10.108.174.125   <none>
34571/TCP,9220/TCP   105d

NAME                          DESIRED   CURRENT   READY   UP-TO-DATE
AVAILABLE   NODE SELECTOR                                       AGE
daemonset.apps/trident-csi   3         3         3       3            3
kubernetes.io/arch=amd64,kubernetes.io/os=linux   105d

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/trident-csi   1/1     1            1           105d

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/trident-csi-68d979fb85   1         1         1
105d
```

此时，将删除 `trident` 操作符 `-xxxxxx-xxxxx` Pod 。

4. （可选）如果需要修改安装参数，请更新 `TridentProvider` 规范。这些更改可能包括修改私有映像注册表以从中提取容器映像，启用调试日志或指定映像提取密钥等。

```
$  kubectl patch tprov <trident-provisioner-name> -n <trident-namespace>
--type=merge -p '{"spec":{"debug":true}}'
```

5. 安装集群范围的运算符。

> 安装集群范围的运算符会启动将 `TridentProvider` 对象迁移到 `TridentOrchestrator` 对象的操作，删除 `TridentProvider` 对象和 `tridentProvisioner` CRD ，并将 Astra Trident 升级到所使用的集群范围运算符版本。在以下示例中， Trident 会升级到 21.07.1 。

> 使用集群范围的运算符升级 Astra Trident 会将 `tridentProvider` 迁移到同名的 `tridentOrchestrator` 对象。此操作由操作员自动处理。在升级过程中， Astra Trident 也会安装在与之前相同的命名空间中。

```
#Ensure you are in the correct directory
$ pwd
$ /root/21.07.1/trident-installer

#Install the cluster-scoped operator in the **same namespace**
$ kubectl create -f deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created

#All tridentProvisioners will be removed, including the CRD itself
$ kubectl get tprov -n trident
Error from server (NotFound): Unable to list "trident.netapp.io/v1,
Resource=tridentprovisioners": the server could not find the requested
resource (get tridentprovisioners.trident.netapp.io)

#tridentProvisioners are replaced by tridentOrchestrator
$ kubectl get torc
NAME      AGE
trident   13s

#Examine Trident pods in the namespace
$ kubectl get pods -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          1m41s
trident-csi-xrst8                   2/2     Running   0          1m41s
trident-operator-5574dbbc68-nthjv   1/1     Running   0          1m52s

#Confirm Trident has been updated to the desired version
$ kubectl describe torc trident | grep Message -A 3
Message:                Trident installed
Namespace:              trident
Status:                 Installed
Version:                v21.07.1
```

升级基于 Helm 的操作员安装

要升级基于 Helm 的操作员安装，请执行以下步骤。

步骤

1. 下载最新的 Astra Trident 版本。
2. 使用 `helm upgrade` 命令。请参见以下示例：

```
$ helm upgrade <name> trident-operator-21.07.1.tgz
```

其中 `trident` 操作符 `-21.071.tgz` 反映了要升级到的版本。

3. 运行 `Helm list` 以验证图表和应用程序版本均已升级。

> 要在升级期间传递配置数据，请使用 `-set` 。

例如，要更改默认值 `tridentDebug` ，请运行以下命令：

```
$ helm upgrade <name> trident-operator-21.07.1-custom.tgz --set
tridentDebug=true
```

如果您运行的是 `$tridentctl logs` ，则可以看到调试消息。

> 如果在初始安装期间设置了任何非默认选项，请确保这些选项包含在 upgrade 命令中，否则，这些值将重置为其默认值。

从非操作员安装升级

如果您的 CSI Trident 实例满足上述前提条件，则可以升级到最新版本的 Trident 操作符。

步骤

1. 下载最新的 Astra Trident 版本。

```
# Download the release required [21.07.1]
$ mkdir 21.07.1
$ cd 21.07.1
$ wget
https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
$ tar -xf trident-installer-21.07.1.tar.gz
$ cd trident-installer
```

2. 从清单中创建 `tridentorchator` CRD 。

```
$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 部署操作员。

```
#Install the cluster-scoped operator in the **same namespace**
$ kubectl create -f deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created

#Examine the pods in the Trident namespace
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          150d
trident-csi-xrst8                   2/2     Running   0          150d
trident-operator-5574dbbc68-nthjv   1/1     Running   0          1m30s
```

4. 创建 `TridentOrchestrator` CR 以安装 Astra Trident 。

```
#Create a tridentOrchestrator to initate a Trident install
$ cat deploy/crds/tridentorchestrator_cr.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident

$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml

#Examine the pods in the Trident namespace
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          1m
trident-csi-xrst8                   2/2     Running   0          1m
trident-operator-5574dbbc68-nthjv   1/1     Running   0          5m41s

#Confirm Trident was upgraded to the desired version
$ kubectl describe torc trident | grep Message -A 3
Message:                Trident installed
Namespace:              trident
Status:                 Installed
Version:                v21.07.1
```

现有后端和 PVC 会自动可用。

使用 tridentctl 进行升级

您可以使用 `tridentctl` 轻松升级现有的 Astra Trident 安装。

升级到最新版本的 Astra Trident 时，请考虑以下事项：

- 从 Trident 20.01 开始，仅限测试版 "卷快照" 受支持。Kubernetes 管理员应注意安全地备份或将 alpha Snapshot 对象转换为测试版，以保留原有的 alpha Snapshot 。
- 测试版的卷快照引入了一组经过修改的 CRD 和一个快照控制器，这两个控制器都应在安装 Astra Trident 之前进行设置。

> "本博客" 讨论将 alpha 卷快照迁移到测试版格式所涉及的步骤。

卸载并重新安装 Astra Trident 可作为升级。卸载 Trident 时，不会删除 Astra Trident 部署所使用的永久性卷声明（ PVC ）和永久性卷（ PV ）。在 Astra Trident 脱机期间，已配置的 PV 仍可用，而 Astra Trident 将在恢复联机后为在此期间创建的任何 PVC 配置卷。

> 升级 Astra Trident 时，请勿中断升级过程。确保安装程序运行完毕。

升级后的后续步骤

要使用新 Trident 版本中提供的丰富功能（例如按需卷快照），您可以使用 `tridentctl upgrade` 命令升级卷。

如果存在旧卷，则应将其从 NFS/iSCSI 类型升级到 CSI 类型，以便能够使用 Astra Trident 中的一整套新功能。Trident 配置的原有 PV 支持传统功能集。

决定将卷升级到 CSI 类型时，请考虑以下事项：

- 您可能不需要升级所有卷。以前创建的卷将继续可访问并正常运行。
- 升级时， PV 可以作为部署 / 状态集的一部分挂载。不需要关闭部署 / 状态集。
- 升级时，您 * 无法 * 将 PV 连接到独立 POD 。在升级卷之前，您应关闭 POD 。
- 您只能升级绑定到 PVC 的卷。升级前，应删除和导入未绑定到 PVC 的卷。

卷升级示例

以下示例显示了如何执行卷升级。

1. 运行 `kubectl get pv` 以列出 PV 。

```
$ kubectl get pv
NAME                          CAPACITY     ACCESS MODES   RECLAIM POLICY
STATUS   CLAIM                   STORAGECLASS    REASON   AGE
default-pvc-1-a8475           1073741824   RWO            Delete
Bound    default/pvc-1           standard                 19h
default-pvc-2-a8486           1073741824   RWO            Delete
Bound    default/pvc-2           standard                 19h
default-pvc-3-a849e           1073741824   RWO            Delete
Bound    default/pvc-3           standard                 19h
default-pvc-4-a84de           1073741824   RWO            Delete
Bound    default/pvc-4           standard                 19h
trident                       2Gi          RWO            Retain
Bound    trident/trident                                   19h
```

目前， Trident 20.07 使用 `netapp.io/trident` 配置程序创建了四个 PV 。

2. 运行 `kubectl describe PV` 以获取 PV 的详细信息。

```
$ kubectl describe pv default-pvc-2-a8486

Name:            default-pvc-2-a8486
Labels:          <none>
Annotations:     pv.kubernetes.io/provisioned-by: netapp.io/trident
                 volume.beta.kubernetes.io/storage-class: standard
Finalizers:      [kubernetes.io/pv-protection]
StorageClass:    standard
Status:          Bound
Claim:           default/pvc-2
Reclaim Policy:  Delete
Access Modes:    RWO
VolumeMode:      Filesystem
Capacity:        1073741824
Node Affinity:   <none>
Message:
Source:
    Type:      NFS (an NFS mount that lasts the lifetime of a pod)
    Server:    10.xx.xx.xx
    Path:      /trid_1907_alpha_default_pvc_2_a8486
    ReadOnly:  false
```

PV 是使用 `netapp.io/trident` 配置程序创建的，其类型为 NFS 。要支持 Astra Trident 提供的所有新功能，应将此 PV 升级到 CSI 类型。

3. 运行 `tridenctl upgrade volume <name-o-trident -volume>` 命令将原有的 Astra Trident 卷升级到 CSI 规范。

```
$ ./tridentctl get volumes -n trident
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
|            NAME      |  SIZE   | STORAGE CLASS | PROTOCOL |
BACKEND UUID             | STATE  | MANAGED |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
| default-pvc-2-a8486 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-3-a849e | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-1-a8475 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-4-a84de | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+

$ ./tridentctl upgrade volume default-pvc-2-a8486 -n trident
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
|            NAME      |  SIZE   | STORAGE CLASS | PROTOCOL |
BACKEND UUID             | STATE  | MANAGED |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
| default-pvc-2-a8486 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
```

4. 运行 `kubectl describe PV` 以验证此卷是否为 CSI 卷。

```
$ kubectl describe pv default-pvc-2-a8486
Name:            default-pvc-2-a8486
Labels:          <none>
Annotations:     pv.kubernetes.io/provisioned-by: csi.trident.netapp.io
                 volume.beta.kubernetes.io/storage-class: standard
Finalizers:      [kubernetes.io/pv-protection]
StorageClass:    standard
Status:          Bound
Claim:           default/pvc-2
Reclaim Policy:  Delete
Access Modes:    RWO
VolumeMode:      Filesystem
Capacity:        1073741824
Node Affinity:   <none>
Message:
Source:
    Type:              CSI (a Container Storage Interface (CSI) volume
source)
    Driver:            csi.trident.netapp.io
    VolumeHandle:      default-pvc-2-a8486
    ReadOnly:          false
    VolumeAttributes:      backendUUID=c5a6f6a4-b052-423b-80d4-
8fb491a14a22

internalName=trid_1907_alpha_default_pvc_2_a8486
                           name=default-pvc-2-a8486
                           protocol=file
Events:                <none>
```

通过这种方式，您可以将由 Astra Trident 创建的 NFS/iSCSI 类型的卷逐个升级到 CSI 类型。

卸载 Astra Trident

根据 Astra Trident 的安装方式，有多种卸载方法。

使用 Helm 卸载

如果您使用 Helm 安装了 Astra Trident ，则可以使用 `helm uninstall` 将其卸载。

```
#List the Helm release corresponding to the Astra Trident install.
$ helm ls -n trident
NAME          NAMESPACE       REVISION        UPDATED
STATUS          CHART                           APP VERSION
trident       trident         1               2021-04-20
00:26:42.417764794 +0000 UTC deployed        trident-operator-21.07.1
21.07.1

#Uninstall Helm release to remove Trident
$ helm uninstall trident -n trident
release "trident" uninstalled
```

使用 Trident 操作符卸载

如果您使用操作符安装了 Astra Trident ，则可以通过执行以下操作之一卸载它：

- 编辑 TridentOrchestrator 设置卸载标志： * 您可以编辑 TridentOrchestrator s并设置 Pec.uninstall=true 。编辑 `TridentOrchestrator` CR 并设置 `uninstall` 标志，如下所示：

  ```
  $  kubectl patch torc <trident-orchestrator-name> --type=merge -p
  '{"spec":{"uninstall":true}}'
  ```

  当 `uninstall` 标志设置为 `true` 时， Trident 操作员将卸载 Trident ，但不会删除 Trident Orchestrator 本身。如果要重新安装 Trident ，应清理 Trident Orchestrator 并创建新的 Trident 。

- 删除 TridentOrchestrator： * 删除用于部署 `Astra Trident` 的 Trident Orchestrator CR 后，您可以指示操作员卸载 Trident 。操作员将处理 `TridentOrchestrator` 的删除，并继续删除 Astra Trident 部署和子集，同时删除在安装过程中创建的 Trident Pod 。要完全删除 Astra Trident （包括其创建的 CRD ）并有效地擦除板，您可以编辑 `TridentOrchestrator` 以传递 `wipeout` 选项。请参见以下示例：

```
$ kubectl patch torc <trident-orchestrator-name> --type=merge -p
'{"spec":{"wipeout":["crds"],"uninstall":true}}'
```

这将完全卸载 Astra Trident 并清除与后端及其管理的卷相关的所有元数据。后续安装将视为全新安装。

> 只有在执行完全卸载时，才应考虑擦除 CRD 。此操作无法撤消。* 除非您希望重新启动并创建全新的 Astra Trident 安装，否则请勿擦除 CRD 。

使用 tridentctl 卸载

在 `tridentctl` 中运行 `uninstall` 命令，如下所示删除除 CRD 和相关对象之外与 Astra Trident 关联的所有资源，从而可以轻松地重新运行安装程序以更新到最新版本。

```
./tridentctl uninstall -n <namespace>
```

要完全删除 Astra Trident ，您应删除由 Astra Trident 创建的 CRD 的最终结果并删除这些 CRD 。

降级 Astra Trident

了解降级到早期版本的 Astra Trident 所涉及的步骤。

您可能会考虑降级的各种原因，例如：

- 应急规划
- 立即修复因升级而发现的错误
- 依赖关系问题，升级失败和不完整

何时降级

迁移到使用 CRD 的 Astra Trident 版本时，应考虑降级。由于 Astra Trident 现在使用 CRD 来保持状态，因此创建的所有存储实体（后端，存储类， PV 和卷快照）都已关联 CRD 对象，而不是写入到 `trident` PV （由早期安装的 Astra Trident 版本使用）中的数据。新创建的 PV ，后端和存储类均作为 CRD 对象进行维护。如果需要降级，则只能对使用 CRD （ 19.07 及更高版本）运行的 Astra Trident 版本尝试执行此操作。这是为了确保在降级后，当前 Astra Trident 版本上执行的所有操作均可见。

何时不降级

您不应降级到使用 `etcd` 保持状态（ 19.04 及更早版本）的 Trident 版本。降级后，使用当前 Astra Trident 版本执行的所有操作都不会反映出来。在回滚到早期版本时，新创建的 PV 不可用。在迁移回早期版本时，对后端，PV ，存储类和卷快照（已创建 / 更新 / 删除）等对象所做的更改对 Astra Trident 不可见。恢复到早期版本不会中断对已使用早期版本创建的 PV 的访问，除非已对其进行升级。

使用操作员安装 Astra Trident 时的降级过程

对于使用 Trident 操作员完成的安装，降级过程有所不同，不需要使用 `tridentctl` 。

对于使用 Trident 操作符完成的安装， Astra Trident 可以降级为以下任一项：

- 使用命名空间范围的运算符（ 20.07 - 20.10 ）安装的版本。
- 使用集群范围运算符（ 21.01 及更高版本）安装的版本。

降级为集群范围的运算符

要将 Astra Trident 降级为使用集群范围运算符的版本，请执行以下步骤。

步骤

1. "卸载 Astra Trident"。* 除非要完全删除现有安装，否则请勿删除这些 CRD
2. 删除集群范围的运算符。为此，您需要使用清单来部署操作员。您可以从获取 "Trident GitHub 报告"。确保切换到所需的分支。

3. 通过安装所需版本的 Astra Trident 继续降级。按照所需版本的文档进行操作。

降级到命名空间范围的运算符

本节总结了降级到 20.07 到 20.10 范围内的 Astra Trident 版本所涉及的步骤，该版本将使用命名空间范围的运算符进行安装。

步骤

1. "卸载 Astra Trident"。除非要完全删除现有安装，否则请勿删除 `CRD` 。确保已删除 tridentOrchestrator 。

```
#Check to see if there are any tridentorchestrators present
$ kubectl get torc
NAME        AGE
trident     20h

#Looks like there is a tridentorchestrator that needs deleting
$ kubectl delete torc trident
tridentorchestrator.trident.netapp.io "trident" deleted
```

2. 删除集群范围的运算符。为此，您需要使用清单来部署操作员。您可以从此处获取 "Trident GitHub 报告"。确保切换到所需的分支。
3. 删除 `tridentorchator` CRD 。

```
#Check to see if ``tridentorchestrators.trident.netapp.io`` CRD is
present and delete it.
$ kubectl get crd tridentorchestrators.trident.netapp.io
NAME                                      CREATED AT
tridentorchestrators.trident.netapp.io   2021-01-21T21:11:37Z
$ kubectl delete crd tridentorchestrators.trident.netapp.io
customresourcedefinition.apiextensions.k8s.io
"tridentorchestrators.trident.netapp.io" deleted
```

   已卸载 Astra Trident 。

4. 通过安装所需版本继续降级。按照所需版本的文档进行操作。

使用 **Helm** 降级

要降级，请使用 `helm rollback` 命令。请参见以下示例：

```
$ helm rollback trident [revision #]
```

使用 tridentctl 安装 Astra Trident 时的降级过程

如果您使用 `tridentctl` 安装了 Astra Trident ，则降级过程涉及以下步骤。此顺序将指导您完成从 Astra Trident 21.07 迁移到 20.07 的降级过程。

> 在开始降级之前，您应创建 Kubernetes 集群的 `etcd` 的快照。这样，您就可以备份 Astra Trident 的 CRD 的当前状态了。

步骤

1. 确保使用 `tridentctl` 安装 Trident 。如果您不确定如何安装 Astra Trident ，请运行以下简单测试：
   a. 列出 Trident 命名空间中的 Pod 。
   b. 确定集群中运行的 Astra Trident 的版本。您可以使用 `tridentctl` 或查看 Trident Pod 中使用的图像。
   c. 如果您 * 未看到 * a `tridentOrchestrator` ，（或） a `tridentprovisioner` ，（或）名为 `trident — operator-xxxxxxxxxx-xxxxx` 的 Pod ，则已安装 Astra Trident * 和 `tridentctl` 。
2. 使用现有的 `tridentctl` 二进制文件卸载 Astra Trident 。在这种情况下，您将使用 21.07 二进制文件卸载。

```
$ tridentctl version -n trident
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.07.0        | 21.07.0        |
+----------------+----------------+

$ tridentctl uninstall -n trident
INFO Deleted Trident deployment.
INFO Deleted Trident daemonset.
INFO Deleted Trident service.
INFO Deleted Trident secret.
INFO Deleted cluster role binding.
INFO Deleted cluster role.
INFO Deleted service account.
INFO Deleted pod security policy.
podSecurityPolicy=tridentpods
INFO The uninstaller did not delete Trident's namespace in case it is
going to be reused.
INFO Trident uninstallation succeeded.
```

3. 完成此操作后，获取所需版本的 Trident 二进制文件（在此示例中为 20.07 ），并使用它安装 Astra Trident 。您可以为生成自定义 YAML "自定义安装" 如果需要，

```
$ cd 20.07/trident-installer/
$ ./tridentctl install -n trident-ns
INFO Created installer service account.
serviceaccount=trident-installer
INFO Created installer cluster role.               clusterrole=trident-
installer
INFO Created installer cluster role binding.
clusterrolebinding=trident-installer
INFO Created installer configmap.                  configmap=trident-
installer
...
...
INFO Deleted installer cluster role binding.
INFO Deleted installer cluster role.
INFO Deleted installer service account.
```

降级过程已完成。

使用 Astra Trident

配置后端

后端定义了 Astra Trident 与存储系统之间的关系。它告诉 Astra Trident 如何与该存储系统进行通信，以及 Astra Trident 如何从该存储系统配置卷。Astra Trident 将自动从后端提供符合存储类定义的要求的存储池。了解有关根据您拥有的存储系统类型配置后端的更多信息。

- "配置 Azure NetApp Files 后端"
- "配置适用于 Google 云平台的 Cloud Volumes Service 后端"
- "配置 NetApp HCI 或 SolidFire 后端"
- "使用 ONTAP 或 Cloud Volumes ONTAP NAS 驱动程序配置后端"
- "使用 ONTAP 或 Cloud Volumes ONTAP SAN 驱动程序配置后端"
- "将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSX 结合使用"

配置 Azure NetApp Files 后端

了解如何使用提供的示例配置将 Azure NetApp Files （ANF ）配置为 Astra Trident 安装的后端。

> Azure NetApp Files 服务不支持小于 100 GB 的卷。如果请求的卷较小，则 Astra Trident 会自动创建 100 GB 的卷。

配置和使用 "Azure NetApp Files" 后端，您需要满足以下要求：

- `ssubscriptionID` 来自启用了 Azure NetApp Files 的 Azure 订阅。
- 租户 `ID`， `clientID` 和 `clientSecret` 来自 "应用程序注册" 在 Azure Active Directory 中，具有足够的 Azure NetApp Files 服务权限。应用程序注册应使用` Azure 预定义的 `owner` 或 `Contributor… 角色。

  > 要了解有关 Azure 内置角色的详细信息，请参见 "Azure 文档"。

- 至少包含一个的 Azure 位置 "委派子网"。从 Trident 22.01 开始， `location` 参数是后端配置文件顶层的必填字段。在虚拟池中指定的位置值将被忽略。
- 如果您是首次使用 Azure NetApp Files 或在新位置使用，则需要进行一些初始配置。请参见 "《快速入门指南》"。

根据后端配置（子网，虚拟网络，服务级别和位置）， Trident 会在请求位置提供的容量池上创建 ANF 卷，并与请求的服务级别和子网匹配。

> 注意： Astra Trident 不支持手动 QoS 容量池。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	"Azure-netapp-files"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + 随机字符
ssubscriptionID 。	Azure 订阅中的订阅 ID	
租户 ID 。	应用程序注册中的租户 ID	
客户端 ID 。	应用程序注册中的客户端 ID	
clientSecret 。	应用程序注册中的客户端密钥	
s服务级别	S标准 ， 高级 或 超级 之一	"" （随机）
位置	要创建新卷的 Azure 位置的名称	
resourceGroups	用于筛选已发现资源的资源组列表	"[]" （无筛选器）
netappAccounts	用于筛选已发现资源的 NetApp 帐户列表	"[]" （无筛选器）
容量池	用于筛选已发现资源的容量池列表	"[]" （无筛选器，随机）
virtualNetwork	具有委派子网的虚拟网络的名称	""
ssubnet	委派给 Microsoft.Netapp/volumes 的子网的名称	""
nfsMountOptions	精细控制 NFS 挂载选项。	"nfsvers=3"
limitVolumeSize	如果请求的卷大小超过此值，则配置失败	"" （默认情况下不强制实施）
debugTraceFlags	故障排除时要使用的调试标志。示例，`\ {"api"： false ， "method"： true ， "discovery"： true} `。除非您正在进行故障排除并需要详细的日志转储，否则请勿使用此功能。	空

> 如果在尝试创建 PVC 时遇到 " 未找到容量池 " 错误，则您的应用程序注册可能没有关联的所需权限和资源（子网，虚拟网络，容量池）。启用调试后， Astra Trident 将记录在创建后端时发现的 Azure 资源。请务必检查是否正在使用适当的角色。

> 如果要使用 NFS 4.1 挂载卷，可以在逗号分隔的挂载选项列表中包含 nfsver=4 以选择 NFS v4.1 。存储类中设置的任何挂载选项都会覆盖后端配置文件中设置的挂载选项。

可以使用短名称或完全限定名称指定 resourceGroups ， netappAccounts ， capacityPools ， virtualNetwork 和 ssubnet 的值。短名称可以与多个同名资源匹配，因此在大多数情况下，建议使用完全限定名称。resourceGroups ， netappAccounts 和 capacityPools 值是一种筛选器，可将发现的一组资源限制为此存储后端提供的资源，并可通过任意组合方式指定。完全限定名称的格式如下：

Type	格式。
Resource group	< 资源组 >
NetApp 帐户	< 资源组 >/< NetApp 帐户 >
容量池	< 资源组 >/< NetApp 帐户 >/< 容量池 >
虚拟网络	< 资源组 >/< 虚拟网络 >
Subnet	< 资源组 >/< 虚拟网络 >/< 子网 >

您可以通过在配置文件的特殊部分中指定以下选项来控制默认配置每个卷的方式。请参见以下配置示例。

参数	Description	Default
`exportRule`	新卷的导出规则	"0.0.0.0/0
`snapshotDir`	控制 .snapshot 目录的可见性	false
`s`大小	新卷的默认大小	"100 克 "
`unixPermissions`	新卷的 UNIX 权限（ 4 个八进制数字）	"" （预览功能，需要在订阅中列入白名单）

`exportRule` 值必须是以 CIDR 表示法表示的 IPv4 地址或 IPv4 子网任意组合的逗号分隔列表。

> 对于在 ANF 后端创建的所有卷， Astra Trident 会在配置存储池时将存储池上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

示例 **1**： 最低配置

这是绝对的最低后端配置。使用此配置， Astra Trident 会发现在已配置位置委派给 ANF 的所有 NetApp 帐户，容量池和子网，并随机将新卷放置在其中一个池和子网上。

当您刚开始使用 ANF 并尝试执行相关操作时，此配置是理想的选择，但实际上，您希望为所配置的卷提供更多范围界定。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus"
}
```

示例 **2**： 使用容量池筛选器的特定服务级别配置

此后端配置会将卷放置在 Azure 的 `Easus` 位置的 **超高** 容量池中。Astra Trident 会自动发现该位置委派给 ANF

的所有子网，并随机在其中一个子网上放置一个新卷。

```
    {
        "version": 1,
        "storageDriverName": "azure-netapp-files",
        "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
        "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
        "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
        "clientSecret": "SECRET",
        "location": "eastus",
        "serviceLevel": "Ultra",
        "capacityPools": [
            "application-group-1/account-1/ultra-1",
            "application-group-1/account-1/ultra-2"
],
    }
```

示例 **3**： 高级配置

此后端配置进一步将卷放置范围缩小为一个子网，并修改了某些卷配置默认值。

```
    {
        "version": 1,
        "storageDriverName": "azure-netapp-files",
        "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
        "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
        "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
        "clientSecret": "SECRET",
        "location": "eastus",
        "serviceLevel": "Ultra",
        "capacityPools": [
            "application-group-1/account-1/ultra-1",
            "application-group-1/account-1/ultra-2"
],
        "virtualNetwork": "my-virtual-network",
        "subnet": "my-subnet",
        "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
        "limitVolumeSize": "500Gi",
        "defaults": {
            "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
            "snapshotDir": "true",
            "size": "200Gi",
            "unixPermissions": "0777"
=======
        }
    }
```

示例 **4**： 虚拟存储池配置

此后端配置可在一个文件中定义多个存储池。如果您有多个容量池支持不同的服务级别，并且您希望在 Kubernetes 中创建表示这些服务级别的存储类，则此功能非常有用。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus",
    "resourceGroups": ["application-group-1"],
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
    "labels": {
        "cloud": "azure"
    },
    "location": "eastus",

    "storage": [
        {
            "labels": {
                "performance": "gold"
            },
            "serviceLevel": "Ultra",
            "capacityPools": ["ultra-1", "ultra-2"]
        },
        {
            "labels": {
                "performance": "silver"
            },
            "serviceLevel": "Premium",
            "capacityPools": ["premium-1"]
        },
        {
            "labels": {
                "performance": "bronze"
            },
            "serviceLevel": "Standard",
            "capacityPools": ["standard-1", "standard-2"]
        }
    ]
}
```

以下 S存储类 定义是指上述存储池。通过使用 `parameters.selector` 字段，您可以为每个 `StorageClass` 指定用于托管卷的实际池。卷将在选定池中定义各个方面。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gold
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=gold"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: silver
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: bronze
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=bronze"
allowVolumeExpansion: true
```

下一步是什么?

创建后端配置文件后，运行以下命令：

```
tridentctl create backend -f <backend-file>
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
tridentctl logs
```

确定并更正配置文件中的问题后，您可以再次运行 create 命令。

配置 Astra 数据存储后端

了解如何使用提供的示例配置为 Astra Trident 安装配置 Astra 数据存储（ADS）后端。

以配置和使用 "Astra 数据存储" 后端，您需要满足以下要求：

- 支持的 ADS 存储系统。请参见 "Astra Data Store 预览文档" 了解详细信息。
- 托管 ADS 的 Kubernetes 集群的凭据。托管 ADS 的 Kubernetes 集群必须具有一个专用于此 Astra Trident 后端创建和管理的卷，快照和导出策略资源的命名空间。必须为托管支持以下功能的 ADS 的 Kubernetes 集群提供 `kubeconfig`：
  - 读取 `astrads-system` 命名空间中的所有对象
  - 读取 / 写入为该 Astra Trident 后端使用而创建的命名空间中的对象
  - 列出所有集群命名空间

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
`ve`版本		始终为 1
`storageDriverName`	存储驱动程序的名称	"astrads-NAS"
`backendName`	自定义名称或存储后端	ADS 集群名称
集群	AstraDSCluster 资源的名称	
命名空间	Astra Trident 将在其中创建所有 ADS 自定义资源的命名空间	
`kubeconfig`	ADS Kubernetes 集群的凭据（Base64 Compact JSON）	`nfsMountOptions`
精细控制 NFS 挂载选项	"vers=4.1"	`autosExportPolicy`
启用自动创建和更新导出策略 [布尔值]	false	`autosExportCIDR`
启用 `AutoExportPolicy` 时用于筛选 Kubernetes 节点 IP 的 CIDR 列表	["0.0.0.0/0， "：： /0"	`limitVolumeSize`
如果请求的卷大小超过此值，则配置失败	"" （默认情况下不强制实施）	`debugTraceFlags`
故障排除时要使用的调试标志。示例： { "api"： false， "method： true }	空	标签

请勿使用 `debugTraceFlags`，除非您正在进行故障排除并需要详细的日志转储。

您应先将 `kubeconfig` 值从 YAML 转换为紧凑型 JSON 格式，然后再转换为 Base64 格式，然后再将其包含在后端配置中。

每个后端都会在托管 ADS 的 Kubernetes 集群上的一个命名空间中配置卷。要在其他命名空间中创建卷，您可以定义其他后端。可以将 ADS 卷附加到托管集群中的任何命名空间，任何其他 Kubernetes 集群或可以挂载 NFS 共享的任何其他位置。

您可以在配置文件的特殊部分中使用这些选项控制每个卷的默认配置。

请参见以下配置示例。

参数	Description	Default
`exportPolicy`	要使用的导出策略	default
`unixPermissions`	新卷的模式必须为八进制，并且以 "0" 开头	"0777"
`sSnapshot` 预留	为快照预留的卷百分比	"5"
`snapshotDir`	控制 ` .snapshot` 目录的可见性	false
`qosPolicy`	要为创建的卷分配的 QoS 策略	""

> 对于在 ADS 后端创建的所有卷， Astra Trident 会在配置存储池时将存储池上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对每个存储池创建的所有卷进行分组。这样，您就可以根据后端配置文件中提供的一组可自定义标签来方便地区分卷了。

示例 **1**： 最小后端配置

这是绝对的最低后端配置。

示例 **1.** 示例


```
{
    "version": 1,
    "storageDriverName": "astrads-nas",
    "cluster": "astrads-sti-c6220-09-10-11-12",
    "namespace": "test",
    "kubeconfig":
"eyJjdXJyZW50LWNvbnRleHQiOiJmZWRlcmFsLWNvbnRleHQiLCJhcGlWZXJzaW9uIjoidj
EiLCJjbHVzdGVycyI6W3siY2x1c3RlciI6eyJhcGktdmVyc2lvbiI6InYxIiwic2VydmVyI
joiaHR0cDovL2Nvdy5vcmc6ODA4MCJ9LCJuYW1lIjoiY293LWNsdXN0ZXIifSx7ImNsdXN0
ZXIiOnsiY2VydGlmaWNhdGUtYXV0aG9yaXR5IjoicGF0aC90by9teS9jYWZpbGUiLCJzZXJ
2ZXIiOiJodHRwczovL2hvcnNlLm9yZzo0NDQzIn0sIm5hbWUiOiJob3JzZS1jbHVzdGVyIn
0seyJjbHVzdGVyIjp7Imluc2VjdXJlLXNraXAtdGxzLXZlcmlmeSI6dHJ1ZSwic2VydmVyI
joiaHR0cHM6Ly9waWcub3JnOjQ0MyJ9LCJuYW1lIjoicGlnLWNsdXN0ZXIifV0sImNvbnRl
eHRzIjpbeyJjb250ZXh0Ijp7ImNsdXN0ZXIiOiJob3JzZS1jbHVzdGVyIiwibmFtZXNwYWN
lIjoiY2hpc2VsLW5zIiwidXNlciI6ImdyZWVuLXVzZXIifSwibmFtZSI6ImZlZGVyYWwtY2
9udGV4dCJ9LHsiY29udGV4dCI6eyJjbHVzdGVyIjoicGlnLWNsdXN0ZXIiLCJuYW1lc3BhY
2UiOiJzYXctbnMiLCJ1c2VyIjoiYmxhY2stdXNlciJ9LCJuYW1lIjoicXVlZW4tYW5uZS1j
b250ZXh0In1dLCJraW5kIjoiQ29uZmlnIiwicHJlZmVyZW5jZXMiOnsiY29sb3JzIjp0cnV
lfSwidXNlcnMiOlt7Im5hbWUiOiJibHVlLXVzZXIiLCJ1c2VyIjp7InRva2VuIjoiYmx1ZS
10b2tlbiJ9fSx7Im5hbWUiOiJncmVlbi11c2VyIiwidXNlciI6eyJjbGllbnQtY2VydGlma
WNhdGUiOiJwYXRoL3RvL215L2NsaWVudC9jZXJ0IiwiY2xpZW50LWtleSI6InBhdGgvdG8v
bXkvY2xpZW50L2tleSJ9fV19"
}
```


示例 **2**： 单服务级别配置

此示例显示了一个后端文件，该文件对所有由 Astra Trident 创建的存储应用相同的方面。

示例 **2.** 示例


```
{
    "version": 1,
        "storageDriverName": "astrads-nas",
        "cluster": "astrads-sti-c6220-09-10-11-12",
        "namespace": "test",
        "kubeconfig":
"eyJjdXJyZW50LWNvbnRleHQiOiJmZWRlcmFsLWNvbnRleHQiLCJhcGlWZXJzaW9uIjoidj
EiLCJjbHVzdGVycyI6W3siY2x1c3RlciI6eyJhcGktdmVyc2lvbiI6InYxIiwic2VydmVyI
joiaHR0cDovL2Nvdy5vcmc6ODA4MCJ9LCJuYW1lIjoiY293LWNsdXN0ZXIifSx7ImNsdXN0
ZXIiOnsiY2VydGlmaWNhdGUtYXV0aG9yaXR5IjoicGF0aC90by9teS9jYWZpbGUiLCJzZXJ
2ZXIiOiJodHRwczovL2hvcnNlLm9yZzo0NDQzIn0sIm5hbWUiOiJob3JzZS1jbHVzdGVyIn
0seyJjbHVzdGVyIjp7Imluc2VjdXJlLXNraXAtdGxzLXZlcmlmeSI6dHJ1ZSwic2VydmVyI
joiaHR0cHM6Ly9waWcub3JnOjQ0MyJ9LCJuYW1lIjoicGlnLWNsdXN0ZXIifV0sImNvbnRl
eHRzIjpbeyJjb250ZXh0Ijp7ImNsdXN0ZXIiOiJob3JzZS1jbHVzdGVyIiwibmFtZXNwYWN
lIjoiY2hpc2VsLW5zIiwidXNlciI6ImdyZWVuLXVzZXIifSwibmFtZSI6ImZlZGVyYWwtY2
9udGV4dCJ9LHsiY29udGV4dCI6eyJjbHVzdGVyIjoicGlnLWNsdXN0ZXIiLCJuYW1lc3BhY
2UiOiJzYXctbnMiLCJ1c2VyIjoiYmxhY2stdXNlciJ9LCJuYW1lIjoicXVlZW4tYW5uZS1j
b250ZXh0In1dLCJraW5kIjoiQ29uZmlnIiwicHJlZmVyZW5jZXMiOnsiY29sb3JzIjp0cnV
lfSwidXNlcnMiOlt7Im5hbWUiOiJibHVlLXVzZXIiLCJ1c2VyIjp7InRva2VuIjoiYmx1ZS
10b2tlbiJ9fSx7Im5hbWUiOiJncmVlbi11c2VyIiwidXNlciI6eyJjbGllbnQtY2VydGlma
WNhdGUiOiJwYXRoL3RvL215L2NsaWVudC9jZXJ0IiwiY2xpZW50LWtleSI6InBhdGgvdG8v
bXkvY2xpZW50L2tleSJ9fV19",
        "defaults": {
            "exportPolicy": "myexportpolicy1",
            "qosPolicy": "bronze",
            "snapshotReserve": "10"
        },
        "labels": {"cloud": "on-prem", "creator": "ads-cluster-1",
"performance": "bronze"}
}
```


示例 **3**： 虚拟存储池配置

此示例显示了使用虚拟存储池配置的后端定义文件以及引用这些池的 StorageClasses 。

示例 **3.** 示例


```
{
    "version": 1,
    "storageDriverName": "astrads-nas",
    "cluster": "astrads-sti-c6220-09-10-11-12",
    "namespace": "test",
    "kubeconfig":
"eyJjdXJyZW50LWNvbnRleHQiOiJmZWRlcmFsLWNvbnRleHQiLCJhcGlWZXJzaW9uIjoidj
EiLCJjbHVzdGVycyI6W3siY2x1c3RlciI6eyJhcGktdmVyc2lvbiI6InYxIiwic2VydmVyI
joiaHR0cDovL2Nvdy5vcmc6ODA4MCJ9LCJuYW1lIjoiY293LWNsdXN0ZXIifSx7ImNsdXN0
ZXIiOnsiY2VydGlmaWNhdGUtYXV0aG9yaXR5IjoicGF0aC90by9teS9jYWZpbGUiLCJzZXJ
2ZXIiOiJodHRwczovL2hvcnNlLm9yZzo0NDQzIn0sIm5hbWUiOiJob3JzZS1jbHVzdGVyIn
0seyJjbHVzdGVyIjp7Imluc2VjdXJlLXNraXAtdGxzLXZlcmlmeSI6dHJ1ZSwic2VydmVyI
joiaHR0cHM6Ly9waWcub3JnOjQ0MyJ9LCJuYW1lIjoicGlnLWNsdXN0ZXIifV0sImNvbnRl
eHRzIjpbeyJjb250ZXh0Ijp7ImNsdXN0ZXIiOiJob3JzZS1jbHVzdGVyIiwibmFtZXNwYWN
lIjoiY2hpc2VsLW5zIiwidXNlciI6ImdyZWVuLXVzZXIifSwibmFtZSI6ImZlZGVyYWwtY2
9udGV4dCJ9LHsiY29udGV4dCI6eyJjbHVzdGVyIjoicGlnLWNsdXN0ZXIiLCJuYW1lc3BhY
2UiOiJzYXctbnMiLCJ1c2VyIjoiYmxhY2stdXNlciJ9LCJuYW1lIjoicXVlZW4tYW5uZS1j
b250ZXh0In1dLCJraW5kIjoiQ29uZmlnIiwicHJlZmVyZW5jZXMiOnsiY29sb3JzIjp0cnV
lfSwidXNlcnMiOlt7Im5hbWUiOiJibHVlLXVzZXIiLCJ1c2VyIjp7InRva2VuIjoiYmx1ZS
10b2tlbiJ9fSx7Im5hbWUiOiJncmVlbi11c2VyIiwidXNlciI6eyJjbGllbnQtY2VydGlma
WNhdGUiOiJwYXRoL3RvL215L2NsaWVudC9jZXJ0IiwiY2xpZW50LWtleSI6InBhdGgvdG8v
bXkvY2xpZW50L2tleSJ9fV19",

    "autoExportPolicy": true,
    "autoExportCIDRs": ["10.211.55.0/24"],

    "labels": {"cloud": "on-prem", "creator": "ads-cluster-1"},
    "defaults": {"snapshotReserve": "5"},

    "storage": [
        {
            "labels": {"performance": "gold", "cost": "3"},
            "defaults": {
                "qosPolicy": "gold",
                "snapshotReserve": "10"
            }
        },
        {
            "labels": {"performance": "silver", "cost": "2"},
            "defaults": {"qosPolicy": "silver"}
        },
        {
            "labels": {"performance": "bronze", "cost": "1"},
            "defaults": {"qosPolicy": "bronze"}
```

```
            }
        ]
}
```

以下 StorageClass 定义引用了上述存储池。通过使用 `parameters.selector` 字段，您可以为每个 StorageClass 指定用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

示例 **4.** 示例

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
allowVolumeExpansion: true

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads-gold
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
  selector: performance=gold
allowVolumeExpansion: true

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads-silver
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
  selector: performance=silver
allowVolumeExpansion: true

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ads-bronze
provisioner: csi.trident.netapp.io
parameters:
  backendType: astrads-nas
  selector: performance=bronze
allowVolumeExpansion: true
```

下一步是什么？

创建后端配置文件后，运行以下命令：

```
tridentctl create backend -f <backend-file>
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
tridentctl logs
```

确定并更正配置文件中的问题后，您可以再次运行 create 命令。

为 GCP 后端配置 CVS

了解如何使用提供的示例配置将适用于 Google 云平台（GCP）的 NetApp Cloud Volumes Service （CVS）配置为 Astra Trident 安装的后端。

> NetApp Cloud Volumes Service for Google Cloud 不支持大小小于 100 GiB 的 CVS-Performance 卷或小于 300 GiB 的 CVS 卷。如果请求的卷小于最小大小，则 Astra Trident 会自动创建最小大小的卷。

以配置和使用 "适用于 Google Cloud 的 Cloud Volumes Service" 后端，您需要满足以下要求：

- 配置了 NetApp CVS 的 Google Cloud 帐户
- Google Cloud 帐户的项目编号
- 具有 `netappcloudvolumes.admin` 角色的 Google Cloud 服务帐户
- CVS 服务帐户的 API 密钥文件

Astra Trident 现在支持使用默认值的较小卷 "GCP 上的 CVS 服务类型"。对于使用 `storageClass=software` 创建的后端，卷的最小配置大小将为 300 GiB 。CVS 目前在 " 受控可用性 " 下提供此功能，不提供技术支持。用户必须注册才能访问低于 1TiB 的卷 "此处"。NetApp 建议客户对非生产工作负载使用 1 TiB 以下的卷。

> 使用默认 CVS 服务类型（`storageClass=software`）部署后端时，用户必须获得 GCP 上有关项目编号和项目 ID 的 1 TiB 卷功能的访问权限。这对于 Astra Trident 配置低于 1TiB 的卷是必需的。否则，对于小于 600 GiB 的 PVC ，卷创建将失败。使用访问低于 1TiB 的卷 "此表单"。

由 Astra Trident 为默认 CVS 服务级别创建的卷将按以下方式进行配置：

- 小于 300 GiB 的 PVC 将导致 Astra Trident 创建 300 GiB CVS 卷。
- 如果 PVC 介于 300 GiB 到 600 GiB 之间，则 Astra Trident 将创建请求大小的 CVS 卷。
- 介于 600 GiB 和 1 TiB 之间的 PVC 将导致 Astra Trident 创建 1 TiB CVS 卷。
- 如果 PVC 大于 1 TiB ，则 Astra Trident 将创建请求大小的 CVS 卷。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	"GCP-CVS"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + API 密钥的一部分
s存储类	存储类型可选择 硬件 （性能优化）或 s软件 （ CVS 服务类型）	
projectNumber	Google Cloud 帐户项目编号。该值可在 Google Cloud 门户的主页页面上找到。	
区域	CVS 帐户区域。后端将在该区域配置卷。	
apiKey	具有 netappcloudvolumes.admin 角色的 Google Cloud 服务帐户的 API 密钥。它包括 Google Cloud 服务帐户专用密钥文件的 JSON 格式的内容（逐字复制到后端配置文件）。	
代理 URL	代理服务器需要连接到 CVS 帐户时的代理 URL 。代理服务器可以是 HTTP 代理，也可以是 HTTPS 代理。对于 HTTPS 代理，将跳过证书验证，以允许在代理服务器中使用自签名证书。不支持启用了身份验证的代理服务器。	
nfsMountOptions	精细控制 NFS 挂载选项。	"nfsvers=3"
limitVolumeSize	如果请求的卷大小超过此值，则配置失败	"" （默认情况下不强制实施）
s服务级别	新卷的 CVS 服务级别。这些值包括 "standard" ， "premer" 和 "Extreme " 。	标准
网络	用于CVS卷的GCP网络	default
debugTraceFlags	故障排除时要使用的调试标志。示例，`\ ｛"api"： false ， "method： true"｝`。除非您正在进行故障排除并需要详细的日志转储，否则请勿使用此功能。	空

如果使用共享 VPC 网络，则必须同时指定 projectNumber 和 hostProjectNumber 。在这种情况下，projectNumber 是服务项目，而 hostProjectNumber 是主机项目。

"apiRegion"` 表示 Astra Trident 在其中创建 CVS 卷的 GCP 区域。 在 `跨区域 Kubernetes 集群时，在

"apiRegion" 中创建的 CVS 卷可用于在多个 GCP 区域的节点上计划的工作负载。请注意，跨区域流量会产生额外成本。

- 要启用跨区域访问， allowedTopologies 的 StorageClass 定义必须包括所有地区。例如：

```
- key: topology.kubernetes.io/region
  values:
  - us-east1
  - europe-west1
```

- storageClass 是一个可选参数，您可以使用该参数选择所需的 "CVS 服务类型"。您可以从基本 CVS 服务类型（ storageClass=software ）或 CVS-Performance 服务类型（ storageClass=hardware ）中进行选择， Trident 默认使用此服务类型。请 指定一个 "apiRegion" ，用于在后端定义中提供相应的 CVS storageClass 。

Astra Trident 与 Google Cloud 上的基本 CVS 服务类型集成是一项 * 测试版功能 * ，不适用于生产工作负载。在 CVS-Performance 服务类型中， Trident 是 "* 完全支持 "* ，默认情况下会使用它。

每个后端都会在一个 Google Cloud 区域中配置卷。要在其他区域创建卷，您可以定义其他后端。

您可以通过在配置文件的特殊部分中指定以下选项来控制默认配置每个卷的方式。请参见以下配置示例。

参数	Description	Default
exportRule	新卷的导出规则	"0.0.0.0/0
snapshotDir	访问 `.snapshot` 目录	false
sSnapshot 预留	为快照预留的卷百分比	"" （接受 CVS 默认值为 0 ）
s大小	新卷的大小	"100Gi"

exportRule 值必须是以 CIDR 表示法表示的 IPv4 地址或 IPv4 子网任意组合的逗号分隔列表。

对于在 CVS Google Cloud 后端创建的所有卷， Trident 会在配置存储池时将其上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

示例 **1**： 最低配置

这是绝对的最低后端配置。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
```

```
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    }
}
```


示例 **2**： 基本 **CVS** 服务类型配置

此示例显示了使用基本 CVS 服务类型的后端定义，该服务类型适用于通用工作负载，可提供轻 / 中性能以及高区域可用性。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "storageClass": "software",
    "apiRegion": "us-east4",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
```

```
    }
}
```

示例 **3**： 单服务级别配置

此示例显示了一个后端文件，该文件对 Google Cloud us-west2 区域中由 Astra Trident 创建的所有存储应用相同的方面。此示例还显示了后端配置文件中使用的 `proxyURL` 。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
```

```
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "proxyURL": "http://proxy-server-hostname/",
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
    "limitVolumeSize": "10Ti",
    "serviceLevel": "premium",
    "defaults": {
        "snapshotDir": "true",
        "snapshotReserve": "5",
        "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
        "size": "5Ti"
    }
}
```

示例 **4**： 虚拟存储池配置

此示例显示了使用虚拟存储池配置的后端定义文件以及引用这些池的 `StorageClasses` 。

在下面显示的示例后端定义文件中，为所有存储池设置了特定的默认值，这些默认值会将 `snapshotReserve` 设置为 5% ，并将 `exportRule` 设置为 0.0.0.0/0 。虚拟存储池在 `s`存储 部分中进行定义。在此示例中，每个存储池都会设置自己的 `serviceLevel` ，而某些池会覆盖默认值。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
```

```
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",

    "defaults": {
        "snapshotReserve": "5",
        "exportRule": "0.0.0.0/0"
    },

    "labels": {
        "cloud": "gcp"
    },
    "region": "us-west2",

    "storage": [
        {
            "labels": {
                "performance": "extreme",
                "protection": "extra"
            },
            "serviceLevel": "extreme",
```

```
            "defaults": {
                "snapshotDir": "true",
                "snapshotReserve": "10",
                "exportRule": "10.0.0.0/24"
            }
        },
        {
            "labels": {
                "performance": "extreme",
                "protection": "standard"
            },
            "serviceLevel": "extreme"
        },
        {
            "labels": {
                "performance": "premium",
                "protection": "extra"
            },
            "serviceLevel": "premium",
            "defaults": {
                "snapshotDir": "true",
                "snapshotReserve": "10"
            }
        },

        {
            "labels": {
                "performance": "premium",
                "protection": "standard"
            },
            "serviceLevel": "premium"
        },

        {
            "labels": {
                "performance": "standard"
            },
            "serviceLevel": "standard"
        }
    ]
}
```

以下 StorageClass 定义引用了上述存储池。通过使用 `parameters.selector` 字段，您可以为每个 StorageClass 指定用于托管卷的虚拟池。卷将在选定池中定义各个方面。

第一个 StorageClass （`cvs-ext-protection` ）映射到第一个虚拟存储池。这是唯一一个可提供极高性能且

Snapshot 预留为 10% 的池。最后一个 StorageClass （`cvs-extra protection`）调用提供 10% 快照预留的任何存储池。Astra Trident 决定选择哪个虚拟存储池，并确保满足快照预留要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-extreme-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=extreme; protection=extra"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-extreme-standard-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=standard"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-premium-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=extra"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-premium
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=standard"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-standard
provisioner: netapp.io/trident
parameters:
  selector: "performance=standard"
```

```
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "protection=extra"
allowVolumeExpansion: true
```

下一步是什么？

创建后端配置文件后，运行以下命令：

```
tridentctl create backend -f <backend-file>
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
tridentctl logs
```

确定并更正配置文件中的问题后，您可以再次运行 create 命令。

配置 NetApp HCI 或 SolidFire 后端

了解如何在 Astra Trident 安装中创建和使用 Element 后端。

您需要什么？ **#8217**；将需要什么

- 运行 Element 软件的受支持存储系统。
- NetApp HCI/SolidFire 集群管理员或租户用户的凭据，可用于管理卷。
- 所有 Kubernetes 工作节点都应安装适当的 iSCSI 工具。请参见 "工作节点准备信息"。

`solidfire-san` 存储驱动程序支持两种卷模式： file 和 block 。对于 `filesystem` volumemode ， Astra Trident 将创建一个卷并创建一个文件系统。文件系统类型由 StorageClass 指定。

驱动程序	协议	卷模式	支持的访问模式	支持的文件系统
`solidfire-san`	iSCSI	块	rwo ， rox ， rwx	无文件系统。原始块设备。
`solidfire-san`	iSCSI	块	rwo ， rox ， rwx	无文件系统。原始块设备。

驱动程序	协议	卷模式	支持的访问模式	支持的文件系统
solidfire-san	iSCSI	文件系统	工单， ROX	xfs ， ext3 ， ext4
solidfire-san	iSCSI	文件系统	工单， ROX	xfs ， ext3 ， ext4

Astra Trident 在用作增强型 CSI 配置程序时使用 CHAP 。如果您使用的是 CHAP （这是 CSI 的默认设置），则无需进行进一步准备。建议明确设置 UseCHAP 选项，以便对非 CSI Trident 使用 CHAP 。否则，请参见 "此处"。

只有适用于 Astra Trident 的传统非 CSI 框架才支持卷访问组。如果配置为在 CSI 模式下运行，则 Astra Trident 将使用 CHAP 。

如果未设置 AccessGroups 或 UseCHAP ，则适用以下规则之一：

- 如果检测到默认的 trident 访问组，则会使用访问组。
- 如果未检测到访问组，并且 Kubernetes 版本为 1.7 或更高版本，则会使用 CHAP 。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	始终为 "solidfire-san"
backendName	自定义名称或存储后端	SolidFire + 存储（ iSCSI ） IP 地址
端点	使用租户凭据的 SolidFire 集群的 MVIP	
sVIP	存储（ iSCSI ） IP 地址和端口	
标签	要应用于卷的一组任意 JSON 格式的标签。	"
租户名称	要使用的租户名称（如果未找到，则创建）	
InitiatorIFace	将 iSCSI 流量限制为特定主机接口	default
UseCHAP	使用 CHAP 对 iSCSI 进行身份验证	true
访问组	要使用的访问组 ID 列表	查找名为 "trident " 的访问组的 ID
类型	QoS 规范	
limitVolumeSize	如果请求的卷大小超过此值，则配置失败	" （默认情况下不强制实施）

参数	Description	Default
debugTraceFlags	故障排除时要使用的调试标志。示例 { "api"： false ， "method "： true }	空

请勿使用 debugTraceFlags ，除非您正在进行故障排除并需要详细的日志转储。

对于创建的所有卷， Astra Trident 会在配置存储池时将存储池上的所有标签复制到备用存储 LUN 。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

示例 **1**： 具有三种卷类型的 **solidfire-san** 驱动程序的后端配置

此示例显示了一个后端文件，该文件使用 CHAP 身份验证并使用特定 QoS 保证对三种卷类型进行建模。然后，您很可能会使用 IOPS storage class 参数定义存储类以使用其中的每种类型。

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://<user>:<password>@<mvip>/json-rpc/8.0",
    "SVIP": "<svip>:3260",
    "TenantName": "<tenant>",
    "labels": {"k8scluster": "dev1", "backend": "dev1-element-cluster"},
    "UseCHAP": true,
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS": 2000,
"burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS": 6000,
"burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS": 8000,
"burstIOPS": 10000}}]
}
```

示例 **2**： 具有虚拟存储池的 **solidfire-san** 驱动程序的后端和存储类配置

此示例显示了使用虚拟存储池配置的后端定义文件以及引用这些池的 StorageClasses 。

在下面所示的示例后端定义文件中，为所有存储池设置了特定的默认值，即将 type 设置为 Silver 。虚拟存储池在 s存储 部分中进行定义。在此示例中，某些存储池设置了自己的类型，而某些池将覆盖上述默认值。

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://<user>:<password>@<mvip>/json-rpc/8.0",
    "SVIP": "<svip>:3260",
    "TenantName": "<tenant>",
    "UseCHAP": true,
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS": 2000,
"burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS": 6000,
"burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS": 8000,
"burstIOPS": 10000}}],

    "type": "Silver",
    "labels":{"store":"solidfire", "k8scluster": "dev-1-cluster"},
    "region": "us-east-1",

    "storage": [
        {
            "labels":{"performance":"gold", "cost":"4"},
            "zone":"us-east-1a",
            "type":"Gold"
        },
        {
            "labels":{"performance":"silver", "cost":"3"},
            "zone":"us-east-1b",
            "type":"Silver"
        },
        {
            "labels":{"performance":"bronze", "cost":"2"},
            "zone":"us-east-1c",
            "type":"Bronze"
        },
        {
            "labels":{"performance":"silver", "cost":"1"},
            "zone":"us-east-1d"
        }
    ]
}
```

以下 StorageClass 定义引用了上述虚拟存储池。使用 `parameters.selector` 字段，每个 StorageClass 都会调用可用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

第一个 StorageClass （ `solidfire-gold-四` ）将映射到第一个虚拟存储池。这是唯一一个性能卓越的池，

其 卷类型 `QoS` 值为金牌。最后一个 StorageClass （ `solidfire-silver` ）调用可提供银牌性能的任何存储池。Astra Trident 将决定选择哪个虚拟存储池，并确保满足存储要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-gold-four
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=gold; cost=4"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver-three
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver; cost=3"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-bronze-two
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=bronze; cost=2"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver-one
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver; cost=1"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver"
  fsType: "ext4"
```

了解更多信息

- "卷访问组"

使用 ONTAP 或 Cloud Volumes ONTAP SAN 驱动程序配置后端

了解如何使用 ONTAP 和 Cloud Volumes ONTAP SAN 驱动程序配置 ONTAP 后端。

- "准备"
- "配置和示例"

用户权限

Astra Trident 应以 ONTAP 或 SVM 管理员身份运行，通常使用 `admin` cluster 用户或 `vsadmin` SVM 用户，或者使用具有相同角色的其他名称的用户。对于适用于 NetApp ONTAP 的 Amazon FSX 部署， Astra Trident 应使用集群 `fsxadmin` user 或 `vsadmin` SVM 用户或具有相同角色的其他名称的用户作为 ONTAP 或 SVM 管理员运行。`fsxadmin` 用户是集群管理员用户的有限替代用户。

> 如果使用 `limitAggregateUsage` 参数，则需要集群管理员权限。将适用于 NetApp ONTAP 的 Amazon FSx 与 Astra Trident 结合使用时， `limitAggregateUsage` 参数不适用于 `vsadmin` 和 `fsxadmin` 用户帐户。如果指定此参数，配置操作将失败。

虽然可以在 ONTAP 中创建 Trident 驱动程序可以使用的限制性更强的角色，但我们不建议这样做。大多数新版本的 Trident 都会调用需要考虑的其他 API ，从而使升级变得困难且容易出错。

准备

了解如何准备使用 ONTAP SAN 驱动程序配置 ONTAP 后端。对于所有 ONTAP 后端， Astra Trident 需要至少为 SVM 分配一个聚合。

请记住，您还可以运行多个驱动程序，并创建指向其中一个驱动程序的存储类。例如，您可以配置使用 `ontap-san` 驱动程序的 `san-dev` 类和使用 `ontap-san-economy-one` 的 `san-default` 类。

所有 Kubernetes 工作节点都必须安装适当的 iSCSI 工具。请参见 "此处" 有关详细信息：

身份验证

Astra Trident 提供了两种对 ONTAP 后端进行身份验证的模式。

- Credential Based ：具有所需权限的 ONTAP 用户的用户名和密码。建议使用 `admin` 或 `vsadmin` 等预定义的安全登录角色，以确保与 ONTAP 版本的最大兼容性。
- 基于证书： Astra Trident 还可以使用后端安装的证书与 ONTAP 集群进行通信。此处，后端定义必须包含客户端证书，密钥和可信 CA 证书的 Base64 编码值（如果使用）（建议）。

用户还可以选择更新现有后端，选择从基于凭据迁移到基于证书，反之亦然。如果 * 同时提供了凭据和证书 * ，则 Astra Trident 将在发出警告以从后端定义中删除凭据时默认使用证书。

启用基于凭据的身份验证

Astra Trident 需要 SVM 范围 / 集群范围的管理员的凭据才能与 ONTAP 后端进行通信。建议使用标准的预定义

角色，例如 `admin` 或 `vsadmin` 。这样可以确保与未来的 ONTAP 版本向前兼容，这些版本可能会使功能 API 公开供未来的 Astra Trident 版本使用。可以创建自定义安全登录角色并将其用于 Astra Trident ，但不建议使用。

后端定义示例如下所示：

```
{
  "version": 1,
  "backendName": "ExampleBackend",
  "storageDriverName": "ontap-san",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "svm": "svm_nfs",
  "username": "vsadmin",
  "password": "secret",
}
```

请注意，后端定义是凭据以纯文本格式存储的唯一位置。创建后端后，用户名 / 密码将使用 Base64 进行编码并存储为 Kubernetes 密钥。创建 / 更新后端是唯一需要了解凭据的步骤。因此，这是一项仅由管理员执行的操作，由 Kubernetes 或存储管理员执行。

启用基于证书的身份验证

新的和现有的后端可以使用证书并与 ONTAP 后端进行通信。后端定义需要三个参数。

- clientCertificate ： 客户端证书的 Base64 编码值。
- clientPrivateKey ： 关联私钥的 Base64 编码值。
- trustedCACertifate ： 受信任 CA 证书的 Base64 编码值。如果使用可信 CA ，则必须提供此参数。如果不使用可信 CA ，则可以忽略此设置。

典型的工作流包括以下步骤。

步骤

1. 生成客户端证书和密钥。生成时，将公用名（ Common Name ， CN ）设置为要作为身份验证的 ONTAP 用户。

   ```
   openssl req -x509 -nodes -days 1095 -newkey rsa:2048 -keyout k8senv.key
   -out k8senv.pem -subj "/C=US/ST=NC/L=RTP/O=NetApp/CN=admin"
   ```

2. 将可信 CA 证书添加到 ONTAP 集群。此问题可能已由存储管理员处理。如果未使用可信 CA ，则忽略。

```
security certificate install -type server -cert-name <trusted-ca-cert-
name> -vserver <vserver-name>
ssl modify -vserver <vserver-name> -server-enabled true -client-enabled
true -common-name <common-name> -serial <SN-from-trusted-CA-cert> -ca
<cert-authority>
```

3. 在 ONTAP 集群上安装客户端证书和密钥（从步骤 1 开始）。

```
security certificate install -type client-ca -cert-name <certificate-
name> -vserver <vserver-name>
security ssl modify -vserver <vserver-name> -client-enabled true
```

4. 确认 ONTAP 安全登录角色支持 `cert` 身份验证方法。

```
security login create -user-or-group-name admin -application ontapi
-authentication-method cert
security login create -user-or-group-name admin -application http
-authentication-method cert
```

5. 使用生成的证书测试身份验证。将 <SVM 管理 LIF> 和 <SVM 名称 > 替换为管理 LIF IP 和 ONTAP 名称。

```
curl -X POST -Lk https://<ONTAP-Management-
LIF>/servlets/netapp.servlets.admin.XMLrequest_filer --key k8senv.key
--cert ~/k8senv.pem -d '<?xml version="1.0" encoding="UTF-8"?><netapp
xmlns="http://www.netapp.com/filer/admin" version="1.21"
vfiler="<vserver-name>"><vserver-get></vserver-get></netapp>'
```

6. 使用 Base64 对证书，密钥和可信 CA 证书进行编码。

```
base64 -w 0 k8senv.pem >> cert_base64
base64 -w 0 k8senv.key >> key_base64
base64 -w 0 trustedca.pem >> trustedca_base64
```

7. 使用从上一步获得的值创建后端。

```
$ cat cert-backend.json
{
"version": 1,
"storageDriverName": "ontap-san",
"backendName": "SanBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"clientCertificate": "Faaaakkkkeeee...Vaaallllluuuueeee",
"clientPrivateKey": "LS0tFaKE...0VaLuES0tLS0K",
"trustedCACertificate": "QNFinfO...SiqOyN",
"storagePrefix": "myPrefix_"
}

$ tridentctl create backend -f cert-backend.json -n trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                  |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| SanBackend | ontap-san      | 586b1cd5-8cf8-428d-a76c-2872713612c1 |
online |       0 |
+------------+----------------+--------------------------------------
+--------+---------+
```

更新身份验证方法或轮换凭据

您可以更新现有后端以使用其他身份验证方法或轮换其凭据。这两种方式都适用：使用用户名 / 密码的后端可以更新为使用证书；使用证书的后端可以更新为基于用户名 / 密码的后端。为此，请使用更新后的 `backend.json` 文件，该文件包含执行 `tridentctl` **后端更新** 所需的参数。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-san",
"backendName": "SanBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"username": "vsadmin",
"password": "secret",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend SanBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| SanBackend | ontap-san      | 586b1cd5-8cf8-428d-a76c-2872713612c1 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

> 轮换密码时，存储管理员必须先在 ONTAP 上更新用户的密码。然后进行后端更新。轮换证书时，可以向用户添加多个证书。之后，后端将更新以使用新证书，然后可以从 ONTAP 集群中删除旧证书。

更新后端不会中断对已创建卷的访问，也不会影响在之后建立的卷连接。成功的后端更新表明， Astra Trident 可以与 ONTAP 后端进行通信并处理未来的卷操作。

指定 igroup

Astra Trident 使用 igroup 来控制对其配置的卷（ LUN ）的访问。在为后端指定 igroup 时，管理员有两种选择：

- Astra Trident 可以自动为每个后端创建和管理 igroup 。如果后端定义中未包含 `igroupName` ，则 Astra Trident 会在 SVM 上创建一个名为 `trident -<backender-UUUUUID>` 的 igroup 。这将确保每个后端都有一个专用的 igroup ，并处理 Kubernetes 节点 IQN 的自动添加 / 删除。
- 或者，也可以在后端定义中提供预先创建的 igroup 。可以使用 `igroupName` config 参数来执行此操作。Astra Trident 会将 Kubernetes 节点 IQN 添加 / 删除到已有的 igroup 中。

对于已定义 `igroupName` 的后端，可以使用 `tridentctl` **后端更新** 删除 `igroupName` ，以使 Astra Trident 自动处理 igroup 。这样不会中断对已连接到工作负载的卷的访问。未来的连接将使用创建的 igroup Astra

Trident 进行处理。

> 为 Astra Trident 的每个唯一实例指定一个 igroup 是一个最佳实践，对 Kubernetes 管理员和存储管理员都很有用。CSI Trident 可自动向 igroup 添加和删除集群节点 IQN ，从而极大地简化了其管理。在 Kubernetes 环境（以及 Astra Trident 安装）中使用相同的 SVM 时，使用专用的 igroup 可确保对一个 Kubernetes 集群所做的更改不会影响与另一个 Kubernetes 集群关联的 igroup 。此外，还必须确保 Kubernetes 集群中的每个节点都具有唯一的 IQN 。如上所述，Astra Trident 会自动处理 IQN 的添加和删除。在多个主机之间重复使用 IQN 可能会导致出现主机相互错误并拒绝访问 LUN 的不希望出现的情况。

如果将 Astra Trident 配置为充当 CSI 配置程序，则 Kubernetes 节点 IQN 会自动添加到 igroup 中或从 igroup 中删除。将节点添加到 Kubernetes 集群后， `trident - CSI` DemonSet 会在新添加的节点上部署一个 Pod （`trident - CSI - xxxxx` ），并注册可将卷连接到的新节点。节点 IQN 也会添加到后端的 igroup 中。在对节点进行隔离，清空并从 Kubernetes 中删除时，可以执行一组类似的步骤来删除 IQN 。

如果 Astra Trident 未作为 CSI 配置程序运行，则必须手动更新 igroup ，以包含 Kubernetes 集群中每个工作节点的 iSCSI IQN 。需要将加入 Kubernetes 集群的节点的 IQN 添加到 igroup 中。同样，必须从 igroup 中删除从 Kubernetes 集群中删除的节点的 IQN 。

使用双向 **CHAP** 对连接进行身份验证

Astra Trident 可以使用双向 CHAP 对 `ontap-san` 和 `ontap-san-economy-sn` 驱动程序的 iSCSI 会话进行身份验证。这需要在后端定义中启用 `useCHAP` 选项。如果设置为 `true` ，则 Astra Trident 会将 SVM 的默认启动程序安全性配置为双向 CHAP ，并从后端文件设置用户名和密码。NetApp 建议使用双向 CHAP 对连接进行身份验证。请参见以下配置示例：

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "ontap_san_chap",
    "managementLIF": "192.168.0.135",
    "svm": "ontap_iscsi_svm",
    "useCHAP": true,
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "igroupName": "trident",
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
}
```

> `useCHAP` 参数是一个布尔选项，只能配置一次。默认情况下，此参数设置为 false 。将其设置为 true 后，无法将其设置为 false 。

除了 `useCHAP=true` 之外，后端定义还必须包括 `chapInitiatorSecret` ， `chapTargetInitiatorSecret` ， `chapTargetUsername` 和 `chapUsername` 字段。通过运行 `tridentctl update` 创建后端，可以更改这些密钥。

工作原理

通过将 `useCHAP` 设置为 true ，存储管理员指示 Astra Trident 在存储后端配置 CHAP 。其中包括：

- 在 SVM 上设置 CHAP ：
  - 如果 SVM 的默认启动程序安全类型为 none （默认设置） * 和 * 卷中没有已存在的 LUN ，则 Astra Trident 会将默认安全类型设置为 `CHAP` ，然后继续配置 CHAP 启动程序以及目标用户名和密码。
  - 如果 SVM 包含 LUN ，则 Astra Trident 不会在 SVM 上启用 CHAP 。这样可以确保对 SVM 上已存在的 LUN 的访问不受限制。
- 配置 CHAP 启动程序以及目标用户名和密码；必须在后端配置中指定这些选项（如上所示）。
- 管理向后端提供的 `igroupName` 添加启动程序的操作。如果未指定，则默认为 `trident` 。

创建后端后， Astra Trident 会创建相应的 `tridentbackend` CRD ，并将 CHAP 密码和用户名存储为 Kubernetes 密码。此后端由 Astra Trident 创建的所有 PV 都将通过 CHAP 进行挂载和连接。

轮换凭据并更新后端

您可以通过更新 `backend.json` 文件中的 CHAP 参数来更新 CHAP 凭据。这需要更新 CHAP 密码并使用 `tridentctl update` 命令反映这些更改。

> 更新后端的 CHAP 密码时，必须使用 `tridentctl` 来更新后端。请勿通过 CLI/ONTAP UI 更新存储集群上的凭据，因为 Astra Trident 将无法选取这些更改。

```
$ cat backend-san.json
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "ontap_san_chap",
    "managementLIF": "192.168.0.135",
    "svm": "ontap_iscsi_svm",
    "useCHAP": true,
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "igroupName": "trident",
    "chapInitiatorSecret": "cl9qxUpDaTeD",
    "chapTargetInitiatorSecret": "rqxigXgkeUpDaTeD",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
}

$ ./tridentctl update backend ontap_san_chap -f backend-san.json -n
trident
+----------------+----------------+--------------------------------------
+--------+---------+
|   NAME         | STORAGE DRIVER |                 UUID                 |
STATE   | VOLUMES |
+----------------+----------------+--------------------------------------
+--------+---------+
| ontap_san_chap | ontap-san      | aa458f3b-ad2d-4378-8a33-1a472ffbeb5c |
online |       7 |
+----------------+----------------+--------------------------------------
+--------+---------+
```

现有连接将不受影响；如果凭据由 SVM 上的 Astra Trident 更新，则这些连接将继续保持活动状态。新连接将使用更新后的凭据，现有连接将继续保持活动状态。断开并重新连接旧的 PV 将导致它们使用更新后的凭据。

配置选项和示例

了解如何在您的 Astra Trident 安装中创建和使用 ONTAP SAN 驱动程序。本节提供了后端配置示例以及有关如何将后端映射到 StorageClasses 的详细信息。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
`ve`版本		始终为 1

参数	Description	Default
storageDriverName	存储驱动程序的名称	"ontap-nas"， "ontap-nas-economy-"， "ontap-nas-flexgroup"， "ontap-san "， "ontap-san-economy-"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + dataLIF
m年 月 日	集群或 SVM 管理 LIF 的 IP 地址	"10.0.0.1 "， "2001： 1234： abcd：：： fefe] "
dataLIF	协议 LIF 的 IP 地址。对于 IPv6，请使用方括号。设置后无法更新	由 SVM 派生，除非另有说明
使用 CHAP	使用 CHAP 对 iSCSI 的 ONTAP SAN 驱动程序进行身份验证 [布尔值]	false
chapInitiatorSecret	CHAP 启动程序密钥。如果为 useCHAP=true，则为必需项	"
标签	要应用于卷的一组任意 JSON 格式的标签	"
chapTargetInitiatorSecret	CHAP 目标启动程序密钥。如果为 useCHAP=true，则为必需项	"
chapUsername	入站用户名。如果为 useCHAP=true，则为必需项	"
chapTargetUsername	目标用户名。如果为 useCHAP=true，则为必需项	"
客户端证书	客户端证书的 Base64 编码值。用于基于证书的身份验证	"
clientPrivateKey	客户端专用密钥的 Base64 编码值。用于基于证书的身份验证	"
trustedCACertifate	受信任 CA 证书的 Base64 编码值。可选。用于基于证书的身份验证	"
用户名	用于连接到集群 /SVM 的用户名。用于基于凭据的身份验证	"
密码	连接到集群 /SVM 的密码。用于基于凭据的身份验证	"
sVM	要使用的 Storage Virtual Machine	如果指定了 SVM managementLIF，则派生
igroupName	要使用的 SAN 卷的 igroup 的名称	"trident — < 后端 UUID >"
s存储前缀	在 SVM 中配置新卷时使用的前缀。设置后无法更新	Trident
limitAggregateUsage	如果使用量超过此百分比，则配置失败。* 不适用于适用于 ONTAP 的 Amazon FSx *	" （默认情况下不强制实施）

参数	Description	Default
limitVolumeSize	如果经济型驱动程序请求的卷大小超过此值，则配置失败。	" （默认情况下不强制实施）
lunsPerFlexvol	每个 FlexVol 的最大 LUN 数，必须在 50 ， 200 范围内	100
debugTraceFlags	故障排除时要使用的调试标志。示例 { "api" ： false ， "method " ： true }	空
useREST	用于使用 ONTAP REST API 的布尔参数。* 技术预览 *	false

> useREST 提供了一个 * 技术预览 * ，建议用于测试环境，而不是生产工作负载。如果设置为 true ，则 Astra Trident 将使用 ONTAP REST API 与后端进行通信。此功能需要使用 ONTAP 9.9 及更高版本。此外，使用的 ONTAP 登录角色必须能够访问 ONTAP 应用程序。这一点可通过预定义的 vsadmin 和 cluster-admin 角色来满足。

要与 ONTAP 集群通信，您应提供身份验证参数。这可以是安全登录的用户名 / 密码，也可以是已安装的证书。

> 如果您使用适用于 NetApp ONTAP 后端的 Amazon FSX ，请勿指定 limitAggregateUsage 参数。Amazon FSX for NetApp ONTAP 提供的 fsxadmin 和 vsadmin 角色不包含检索聚合使用情况并通过 Astra Trident 对其进行限制所需的访问权限。

> 请勿使用 debugTraceFlags ，除非您正在进行故障排除并需要详细的日志转储。

对于 ontap-san 驱动程序，默认使用 SVM 中的所有数据 LIF IP 并使用 iSCSI 多路径。为 ontap-san 驱动程序的 dataLIF 指定 IP 地址会强制其禁用多路径并仅使用指定的地址。

> 创建后端时，请记住，创建后无法修改 dataLIF 和 storagePrefix 。要更新这些参数，您需要创建一个新的后端。

可以将 igroupName 设置为已在 ONTAP 集群上创建的 igroup 。如果未指定，则 Astra Trident 会自动创建一个名为 trident -<backender-UUUUUID> 的 igroup 。如果要在环境之间共享 SVM ，则如果提供预定义的 igroupName ， NetApp 建议为每个 Kubernetes 集群使用一个 igroup 。这对于 Astra Trident 自动保持 IQN 添加 / 删除是必需的。

后端也可以在创建后更新 igroup ：

- 可以更新 igroupName 以指向在 Astra Trident 之外的 SVM 上创建和管理的新 igroup 。
- 可以省略 igroupName 。在这种情况下， Astra Trident 将自动创建和管理 trident -<backend-UUUUID> igroup 。

在这两种情况下，仍可访问卷附件。未来的卷附件将使用更新后的 igroup 。此更新不会中断对后端卷的访问。

可以为 managementLIF 选项指定完全限定域名 （ FQDN ）。

对于所有 ONTAP 驱动程序，也可以将 managementLIF 设置为 IPv6 地址。请务必使用 ` -use-ipv6` 标志安装 Trident 。必须注意在方括号内定义 managementLIF IPv6 地址。

> 使用 IPv6 地址时，请确保在方括号内定义 `managementLIF` 和 `dataLIF` （如果包含在后端定义中），例如 [28e8： d9fb： a825： b7bf： 69a8： d02f： 9e7b： 3555] 。如果未提供 `dataLIF` ，则 Astra Trident 将从 SVM 提取 IPv6 数据 LIF 。

要使 ontap-san 驱动程序能够使用 CHAP ，请在后端定义中将 `useCHAP` 参数设置为 `true` 。然后， Astra Trident 将配置双向 CHAP 并将其用作后端给定 SVM 的默认身份验证。请参见 "此处" 了解其工作原理。

对于 `ontap-san-economi` 驱动程序， `limitVolumeSize` 选项还会限制它为 qtree 和 LUN 管理的卷的最大大小。

> Astra Trident 会在使用 `ontap-san` 驱动程序创建的所有卷的 "Comments" 字段中设置配置标签。对于创建的每个卷， FlexVol 上的 "Comments" 字段将使用其所在存储池上的所有标签填充。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

用于配置卷的后端配置选项

您可以在配置的特殊部分中使用这些选项来控制默认配置每个卷的方式。有关示例，请参见以下配置示例。

参数	Description	Default
`spaceAllocation`	LUN 的空间分配	true
`s`页面预留	空间预留模式； " 无 " （精简）或 " 卷 " （厚）	无
`sSnapshot` 策略	要使用的 Snapshot 策略	无
`qosPolicy`	要为创建的卷分配的 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一	"
`adaptiveQosPolicy`	要为创建的卷分配的自适应 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一	"
`sSnapshot` 预留	为快照预留的卷百分比为 "0"	如果 `snapshotPolicy` 为 " 无 " ，则为 " 无 " ，否则为 " "
`splitOnClone`	创建克隆时，从其父级拆分该克隆	false
`splitOnClone`	创建克隆时，从其父级拆分该克隆	false
加密	启用 NetApp 卷加密	false
`securityStyle`	新卷的安全模式	"unix"
分层策略	使用 " 无 " 的分层策略	适用于 ONTAP 9.5 SVM-DR 之前的配置的 " 仅快照 "

> 在 Astra Trident 中使用 QoS 策略组需要 ONTAP 9.8 或更高版本。建议使用非共享 QoS 策略组，并确保策略组分别应用于每个成分卷。共享 QoS 策略组将对所有工作负载的总吞吐量实施上限。

下面是定义了默认值的示例：

```
{
 "version": 1,
 "storageDriverName": "ontap-san",
 "managementLIF": "10.0.0.1",
 "dataLIF": "10.0.0.2",
 "svm": "trident_svm",
 "username": "admin",
 "password": "password",
 "labels": {"k8scluster": "dev2", "backend": "dev2-sanbackend"},
 "storagePrefix": "alternate-trident",
 "igroupName": "custom",
 "debugTraceFlags": {"api":false, "method":true},
 "defaults": {
     "spaceReserve": "volume",
     "qosPolicy": "standard",
     "spaceAllocation": "false",
     "snapshotPolicy": "default",
     "snapshotReserve": "10"
 }
}
```

> 对于使用 `ontap-san` 驱动程序创建的所有卷， Astra Trident 会向 FlexVol 额外添加 10% 的容量，以容纳 LUN 元数据。LUN 将使用用户在 PVC 中请求的确切大小进行配置。Astra Trident 将 FlexVol 增加 10% （在 ONTAP 中显示为可用大小）。用户现在将获得所请求的可用容量。此更改还可防止 LUN 变为只读状态，除非已充分利用可用空间。这不适用于 ontap-san-economy.

对于定义 `snapshotReserve` 的后端， Astra Trident 将按如下方式计算卷的大小：

```
Total volume size = [(PVC requested size) / (1 - (snapshotReserve
percentage) / 100)] * 1.1
```

1.1 是 Astra Trident 向 FlexVol 额外添加 10% 以容纳 LUN 元数据。对于 `snapshotReserve` = 5% ， PVC 请求 = 5GiB ，卷总大小为 5.79GiB ，可用大小为 5.5GiB 。`volume show` 命令应显示与以下示例类似的结果：

```
Vserver   Volume       Aggregate    State      Type       Size  Available Used%
--------- ------------ ------------ ---------- ---- ---------- ---------- -----
          _pvc_89f1c156_3801_4de4_9f9d_034d54c395f4
                                  online     RW         10GB     5.00GB    0%
          _pvc_e42ec6fe_3baa_4af6_996d_134adbbb8e6d
                                  online     RW       5.79GB     5.50GB    0%
          _pvc_e8372153_9ad9_474a_951a_08ae15e1c0ba
                                  online     RW          1GB    511.8MB    0%
3 entries were displayed.
```

目前，调整大小是对现有卷使用新计算的唯一方法。

最低配置示例

以下示例显示了将大多数参数保留为默认值的基本配置。这是定义后端的最简单方法。

> 如果您正在将 NetApp ONTAP 上的 Amazon FSx 与 Astra Trident 结合使用，建议为 LIF 指定 DNS 名称，而不是 IP 地址。

具有基于证书的身份验证的 **ontap-san** 驱动程序

这是一个最低后端配置示例。`clientCertificate` ， `clientPrivateKey` 和 `trustedCACertifate` （如果使用可信 CA ，则可选）分别填充在 `backend.json` 中，并采用客户端证书，私钥和可信 CA 证书的 base64 编码值。

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "DefaultSANBackend",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
    "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
    "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz"
}
```

具有双向 **CHAP** 的 **ONTAP SAN** 驱动程序

这是一个最低后端配置示例。此基本配置将创建一个 `ontap-san` 后端，并将 `useCHAP` 设置为 `true` 。

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "labels": {"k8scluster": "test-cluster-1", "backend": "testcluster1-
sanbackend"},
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret"
}
```

ontap-san-economy-driver

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "svm": "svm_iscsi_eco",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret"
}
```

虚拟存储池后端示例

在下面显示的示例后端定义文件中，会为所有存储池设置特定的默认值，例如 `spaceReserve` at none ， `spaceAllocation` at false 和 `encryption` at false 。虚拟存储池在存储部分中进行定义。

在此示例中，某些存储池会设置自己的 `spaceReserve` ， `spaceAllocation` 和 `encryption` 值，而某些池会覆盖上述设置的默认值。

```
{
```

```
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceAllocation": "false",
          "encryption": "false",
          "qosPolicy": "standard"
    },
    "labels":{"store": "san_store", "kubernetes-cluster": "prod-cluster-
1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"protection":"gold", "creditpoints":"40000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "true",
                "adaptiveQosPolicy": "adaptive-extreme"
            }
        },
        {
            "labels":{"protection":"silver", "creditpoints":"20000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceAllocation": "false",
                "encryption": "true",
                "qosPolicy": "premium"
            }
        },
        {
            "labels":{"protection":"bronze", "creditpoints":"5000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceAllocation": "true",
```

```
                "encryption": "false"
            }
        }
    ]
}
```

以下是 `ontap-san-economy`-经济 驱动程序的 iSCSI 示例：

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "svm": "svm_iscsi_eco",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceAllocation": "false",
          "encryption": "false"
    },
    "labels":{"store":"san_economy_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"oracledb", "cost":"30"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "true"
            }
        },
        {
            "labels":{"app":"postgresdb", "cost":"20"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceAllocation": "false",
                "encryption": "true"
            }
        },
```

```
        {
            "labels":{"app":"mysqldb", "cost":"10"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "false"
            }
        }
    ]
}
```

将后端映射到 **StorageClasses**

以下 StorageClass 定义引用了上述虚拟存储池。使用 `parameters.selector` 字段，每个 StorageClass 都会调用可用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

- 第一个 StorageClass （`protection-gold`）将映射到 `ontap-nas-flexgroup` 后端的第一个，第二个虚拟存储池以及 `ontap-san` 后端的第一个虚拟存储池。这是唯一一个提供黄金级保护的池。
- 第二个 StorageClass （`protection-not-gold`）将映射到 `ontap-nas-flexgroup` 后端的第三个，第四个虚拟存储池以及 `ontap-san` 后端的第二个，第三个虚拟存储池。这些池是唯一提供黄金级以外保护级别的池。
- 第三个 StorageClass （`app-mysqldb`）将映射到 `ontap-NAS` 后端的第四个虚拟存储池和 `ontap-san-economy-backend` 的第三个虚拟存储池。这些池是唯一为 mysqldb 类型的应用程序提供存储池配置的池。
- 第四个存储类（`protection-silver-creditpoins-20k`）将映射到 `ontap-nas-flexgroup` 后端的第三个虚拟存储池和 `ontap-san` 后端的第二个虚拟存储池。这些池是唯一以 20000 个信用点提供黄金级保护的池。
- 第五个存储类（`credits-5k`）将映射到 `ontap-nas-economy-backend` 中的第二个虚拟存储池和 `ontap-san` 后端的第三个虚拟存储池。这些是唯一一款具有 5000 个信用点的池产品。

Astra Trident 将决定选择哪个虚拟存储池，并确保满足存储要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-not-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection!=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: app-mysqldb
provisioner: netapp.io/trident
parameters:
  selector: "app=mysqldb"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-silver-creditpoints-20k
provisioner: netapp.io/trident
parameters:
  selector: "protection=silver; creditpoints=20000"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: creditpoints-5k
provisioner: netapp.io/trident
parameters:
  selector: "creditpoints=5000"
  fsType: "ext4"
```

使用 ONTAP NAS 驱动程序配置后端

了解如何使用 ONTAP 和 Cloud Volumes ONTAP NAS 驱动程序配置 ONTAP 后端。

- "准备"
- "配置和示例"

用户权限

Astra Trident 应以 ONTAP 或 SVM 管理员身份运行，通常使用 `admin` cluster 用户或 `vsadmin` SVM 用户，或者使用具有相同角色的其他名称的用户。对于适用于 NetApp ONTAP 的 Amazon FSX 部署， Astra Trident 应使用集群 `fsxadmin` user 或 `vsadmin` SVM 用户或具有相同角色的其他名称的用户作为 ONTAP 或 SVM 管理员运行。`fsxadmin` 用户是集群管理员用户的有限替代用户。

> 如果使用 `limitAggregateUsage` 参数，则需要集群管理员权限。将适用于 NetApp ONTAP 的 Amazon FSx 与 Astra Trident 结合使用时， `limitAggregateUsage` 参数不适用于 `vsadmin` 和 `fsxadmin` 用户帐户。如果指定此参数，配置操作将失败。

虽然可以在 ONTAP 中创建 Trident 驱动程序可以使用的限制性更强的角色，但我们不建议这样做。大多数新版本的 Trident 都会调用需要考虑的其他 API ，从而使升级变得困难且容易出错。

准备

了解如何准备使用 ONTAP NAS 驱动程序配置 ONTAP 后端。对于所有 ONTAP 后端， Astra Trident 需要至少为 SVM 分配一个聚合。

对于所有 ONTAP 后端， Astra Trident 需要至少为 SVM 分配一个聚合。

请记住，您还可以运行多个驱动程序，并创建指向其中一个驱动程序的存储类。例如，您可以配置一个使用 `ontap-nas` 驱动程序` 的金牌类和一个使用 `ontap-nas-economy-one 的铜牌类。

所有 Kubernetes 工作节点都必须安装适当的 NFS 工具。请参见 "此处" 有关详细信息：

身份验证

Astra Trident 提供了两种对 ONTAP 后端进行身份验证的模式。

- Credential Based ：具有所需权限的 ONTAP 用户的用户名和密码。建议使用 `admin` 或 `vsadmin` 等预定义的安全登录角色，以确保与 ONTAP 版本的最大兼容性。
- 基于证书： Astra Trident 还可以使用后端安装的证书与 ONTAP 集群进行通信。此处，后端定义必须包含客户端证书，密钥和可信 CA 证书的 Base64 编码值（如果使用）（建议）。

用户还可以选择更新现有后端，选择从基于凭据迁移到基于证书，反之亦然。如果 * 同时提供了凭据和证书 * ，则 Astra Trident 将在发出警告以从后端定义中删除凭据时默认使用证书。

启用基于凭据的身份验证

Astra Trident 需要 SVM 范围 / 集群范围的管理员的凭据才能与 ONTAP 后端进行通信。建议使用标准的预定义角色，例如 `admin` 或 `vsadmin` 。这样可以确保与未来的 ONTAP 版本向前兼容，这些版本可能会使功能 API 公开供未来的 Astra Trident 版本使用。可以创建自定义安全登录角色并将其用于 Astra Trident ，但不建议使用。

后端定义示例如下所示：

```
{
  "version": 1,
  "backendName": "ExampleBackend",
  "storageDriverName": "ontap-nas",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "svm": "svm_nfs",
  "username": "vsadmin",
  "password": "secret"
}
```

请注意，后端定义是凭据以纯文本格式存储的唯一位置。创建后端后，用户名 / 密码将使用 Base64 进行编码并存储为 Kubernetes 密钥。创建 / 更新后端是唯一需要了解凭据的步骤。因此，这是一项仅由管理员执行的操作，由 Kubernetes 或存储管理员执行。

启用基于证书的身份验证

新的和现有的后端可以使用证书并与 ONTAP 后端进行通信。后端定义需要三个参数。

- clientCertificate ： 客户端证书的 Base64 编码值。
- clientPrivateKey ： 关联私钥的 Base64 编码值。
- trustedCACertifate ： 受信任 CA 证书的 Base64 编码值。如果使用可信 CA ，则必须提供此参数。如果不使用可信 CA ，则可以忽略此设置。

典型的工作流包括以下步骤。

步骤

1. 生成客户端证书和密钥。生成时，将公用名（ Common Name ， CN ）设置为要作为身份验证的 ONTAP 用户。

   ```
   openssl req -x509 -nodes -days 1095 -newkey rsa:2048 -keyout k8senv.key
   -out k8senv.pem -subj "/C=US/ST=NC/L=RTP/O=NetApp/CN=vsadmin"
   ```

2. 将可信 CA 证书添加到 ONTAP 集群。此问题可能已由存储管理员处理。如果未使用可信 CA ，则忽略。

   ```
   security certificate install -type server -cert-name <trusted-ca-cert-
   name> -vserver <vserver-name>
   ssl modify -vserver <vserver-name> -server-enabled true -client-enabled
   true -common-name <common-name> -serial <SN-from-trusted-CA-cert> -ca
   <cert-authority>
   ```

3. 在 ONTAP 集群上安装客户端证书和密钥（从步骤 1 开始）。

```
security certificate install -type client-ca -cert-name <certificate-
name> -vserver <vserver-name>
security ssl modify -vserver <vserver-name> -client-enabled true
```

4. 确认 ONTAP 安全登录角色支持 `cert` 身份验证方法。

```
security login create -user-or-group-name vsadmin -application ontapi
-authentication-method cert -vserver <vserver-name>
security login create -user-or-group-name vsadmin -application http
-authentication-method cert -vserver <vserver-name>
```

5. 使用生成的证书测试身份验证。将 <SVM 管理 LIF> 和 <SVM 名称 > 替换为管理 LIF IP 和 ONTAP 名称。您必须确保 LIF 的服务策略设置为 `default-data-management` 。

```
curl -X POST -Lk https://<ONTAP-Management-
LIF>/servlets/netapp.servlets.admin.XMLrequest_filer --key k8senv.key
--cert ~/k8senv.pem -d '<?xml version="1.0" encoding="UTF-8"?><netapp
xmlns="http://www.netapp.com/filer/admin" version="1.21"
vfiler="<vserver-name>"><vserver-get></vserver-get></netapp>'
```

6. 使用 Base64 对证书，密钥和可信 CA 证书进行编码。

```
base64 -w 0 k8senv.pem >> cert_base64
base64 -w 0 k8senv.key >> key_base64
base64 -w 0 trustedca.pem >> trustedca_base64
```

7. 使用从上一步获得的值创建后端。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "NasBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"clientCertificate": "Faaaakkkkeeee...Vaaallllluuuueeee",
"clientPrivateKey": "LS0tFaKE...0VaLuES0tLS0K",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend NasBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| NasBackend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

更新身份验证方法或轮换凭据

您可以更新现有后端以使用其他身份验证方法或轮换其凭据。这两种方式都适用：使用用户名 / 密码的后端可以更新为使用证书；使用证书的后端可以更新为基于用户名 / 密码的后端。为此，请使用更新后的 `backend.json` 文件，该文件包含执行 `tridentctl` 后端更新 所需的参数。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "NasBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"username": "vsadmin",
"password": "secret",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend NasBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| NasBackend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

> 轮换密码时，存储管理员必须先在 ONTAP 上更新用户的密码。然后进行后端更新。轮换证书时，可以向用户添加多个证书。之后，后端将更新以使用新证书，然后可以从 ONTAP 集群中删除旧证书。

更新后端不会中断对已创建卷的访问，也不会影响在之后建立的卷连接。成功的后端更新表明， Astra Trident 可以与 ONTAP 后端进行通信并处理未来的卷操作。

管理 NFS 导出策略

Astra Trident 使用 NFS 导出策略来控制对其配置的卷的访问。

使用导出策略时， Astra Trident 提供了两个选项：

- Astra Trident 可以动态管理导出策略本身；在此操作模式下，存储管理员会指定一个表示可接受 IP 地址的 CIDR 块列表。Astra Trident 会自动将属于这些范围的节点 IP 添加到导出策略中。或者，如果未指定任何 CIDR ，则在节点上找到的任何全局范围的单播 IP 都将添加到导出策略中。
- 存储管理员可以手动创建导出策略和添加规则。除非在配置中指定了不同的导出策略名称，否则 Astra Trident 将使用默认导出策略。

动态管理导出策略

CSI Trident 20.04 版可以动态管理 ONTAP 后端的导出策略。这样，存储管理员就可以为工作节点 IP 指定允许的地址空间，而不是手动定义显式规则。它大大简化了导出策略管理；修改导出策略不再需要手动干预存储集群。此外，这有助于将对存储集群的访问限制为仅允许 IP 位于指定范围内的工作节点访问，从而支持精细的自动化管理。

> 只有 CSI Trident 才支持动态管理导出策略。请务必确保工作节点未被 NAT 处理。

示例

必须使用两个配置选项。下面是一个后端定义示例：

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap_nas_auto_export,
    "managementLIF": "192.168.0.135",
    "svm": "svm1",
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "autoExportCIDRs": ["192.168.0.0/24"],
    "autoExportPolicy": true
}
```

> 使用此功能时，您必须确保 SVM 中的根接合具有预先创建的导出策略，并具有允许节点 CIDR 块的导出规则（例如默认导出策略）。请始终遵循 NetApp 建议的最佳实践，为 Astra Trident 专用 SVM 。

以下是使用上述示例对此功能的工作原理进行的说明：

- `autosExportPolicy` 设置为 `true` 。这表示 Astra Trident 将为 `svm1` SVM 创建导出策略，并使用 `autosExportCIDRS` 地址块处理规则的添加和删除。例如， UUID 为 403b5326-8482-40db-96d0-d83fb3f4daec 且 `autodExportPolicy` 设置为 `true` 的后端会在 SVM 上创建一个名为 `trident-403b5326-8482-40db-96d0-d83fb3f4daec` 的导出策略。
- `autosExportCIDR` 包含地址块列表。此字段为可选字段，默认为 "0.0.0.0/0 ， "：： /0" 。如果未定义，则 Astra Trident 会添加在工作节点上找到的所有全局范围的单播地址。

在此示例中，提供了 `192.168.0.0/24` 地址空间。这表示此地址范围内的 Kubernetes 节点 IP 将添加到 Astra Trident 创建的导出策略中。当 Astra Trident 注册其运行的节点时，它会检索该节点的 IP 地址，并根据 `autosExportCIDRS` 中提供的地址块对其进行检查。筛选 IP 后， Astra Trident 会为其发现的客户端 IP 创建导出策略规则，并为其标识的每个节点创建一个规则。

创建后，您可以为后端更新 `autosExportPolicy` 和 `autosExportCIDR` 。您可以为自动管理的后端附加新的 CIDR ，也可以删除现有的 CIDR 。删除 CIDR 时请务必小心，以确保现有连接不会断开。您也可以选择对后端禁用 `autosExportPolicy` ，并回退到手动创建的导出策略。这需要在后端配置中设置 `exportPolicy` 参数。

在 Astra Trident 创建或更新后端后，您可以使用 `tridentctl` 或相应的 `tridentbackend` CRD 检查后端：

```
$ ./tridentctl get backends ontap_nas_auto_export -n trident -o yaml
items:
- backendUUID: 403b5326-8482-40db-96d0-d83fb3f4daec
  config:
    aggregate: ""
    autoExportCIDRs:
    - 192.168.0.0/24
    autoExportPolicy: true
    backendName: ontap_nas_auto_export
    chapInitiatorSecret: ""
    chapTargetInitiatorSecret: ""
    chapTargetUsername: ""
    chapUsername: ""
    dataLIF: 192.168.0.135
    debug: false
    debugTraceFlags: null
    defaults:
      encryption: "false"
      exportPolicy: <automatic>
      fileSystemType: ext4
```

当节点添加到 Kubernetes 集群并向 Astra Trident 控制器注册后，现有后端的导出策略将会更新（前提是它们位于后端的 `autosExportCIDR` 中指定的地址范围内）。

删除节点后， Astra Trident 会检查所有联机后端，以删除该节点的访问规则。通过从受管后端的导出策略中删除此节点 IP ， Astra Trident 可防止恶意挂载，除非此 IP 可由集群中的新节点重复使用。

对于以前存在的后端，使用 `tridentctl update backend` 更新后端可确保 Astra Trident 自动管理导出策略。这将创建一个以后端 UUID 命名的新导出策略，后端上存在的卷将在重新挂载时使用新创建的导出策略。

> 删除具有自动管理导出策略的后端将删除动态创建的导出策略。如果重新创建后端，则会将其视为新的后端，并会创建新的导出策略。

如果更新了活动节点的 IP 地址，则必须在此节点上重新启动 Astra Trident Pod 。然后， Astra Trident 将更新其管理的后端的导出策略，以反映此 IP 更改。

配置选项和示例

了解如何在您的 Astra Trident 安装中创建和使用 ONTAP NAS 驱动程序。本节提供了后端配置示例以及有关如何将后端映射到 StorageClasses 的详细信息。

后端配置选项

有关后端配置选项，请参见下表：

参数	Description	Default
ve版本		始终为 1
storageDriverName	存储驱动程序的名称	"ontap-nas"， "ontap-nas-economy-"， "ontap-nas-flexgroup"， "ontap-san "， "ontap-san-economy-"
backendName	自定义名称或存储后端	驱动程序名称 + "_" + dataLIF
m年 月 日	集群或 SVM 管理 LIF 的 IP 地址	"10.0.0.1 "， "2001： 1234：abcd：：： fefe] "
dataLIF	协议 LIF 的 IP 地址。对于 IPv6，请使用方括号。设置后无法更新	由 SVM 派生，除非另有说明
autosExportPolicy	启用自动创建和更新导出策略 [布尔值]	false
autosExportCIDR	启用 AutoExportPolicy 时用于筛选 Kubernetes 节点 IP 的 CIDR 列表	["0.0.0.0/0 "， "：： /0 "]`
标签	要应用于卷的一组任意 JSON 格式的标签	"
客户端证书	客户端证书的 Base64 编码值。用于基于证书的身份验证	"
clientPrivateKey	客户端专用密钥的 Base64 编码值。用于基于证书的身份验证	"
trustedCACertifate	受信任 CA 证书的 Base64 编码值。可选。用于基于证书的身份验证	"
用户名	用于连接到集群 /SVM 的用户名。用于基于凭据的身份验证	
密码	连接到集群 /SVM 的密码。用于基于凭据的身份验证	
sVM	要使用的 Storage Virtual Machine	如果指定了 SVM managementLIF，则派生
igroupName	要使用的 SAN 卷的 igroup 的名称	"trident — < 后端 UUID >"
s存储前缀	在 SVM 中配置新卷时使用的前缀。设置后无法更新	Trident
limitAggregateUsage	如果使用量超过此百分比，则配置失败。* 不适用于适用于 ONTAP 的 Amazon FSx *	"（默认情况下不强制实施）
limitVolumeSize	如果经济型驱动程序请求的卷大小超过此值，则配置失败。	"（默认情况下不强制实施）
lunsPerFlexvol	每个 FlexVol 的最大 LUN 数，必须在 50， 200 范围内	100

参数	Description	Default
debugTraceFlags	故障排除时要使用的调试标志。示例 ｛"api"： false ， "method "： true｝	空
nfsMountOptions	NFS 挂载选项的逗号分隔列表	"
qtreesPerFlexvol	每个 FlexVol 的最大 qtree 数，必须在 50 ， 300 范围内	200
useREST	用于使用 ONTAP REST API 的布尔参数。* 技术预览 *	false

> useREST 提供了一个 * 技术预览 * ，建议用于测试环境，而不是生产工作负载。如果设置为 true ，则 Astra Trident 将使用 ONTAP REST API 与后端进行通信。此功能需要使用 ONTAP 9.9 及更高版本。此外，使用的 ONTAP 登录角色必须能够访问 ONTAP 应用程序。这一点可通过预定义的 vsadmin 和 cluster-admin 角色来满足。

要与 ONTAP 集群通信，您应提供身份验证参数。这可以是安全登录的用户名 / 密码，也可以是已安装的证书。

> 如果您使用适用于 NetApp ONTAP 后端的 Amazon FSX ，请勿指定 limitAggregateUsage 参数。Amazon FSX for NetApp ONTAP 提供的 fsxadmin 和 vsadmin 角色不包含检索聚合使用情况并通过 Astra Trident 对其进行限制所需的访问权限。

> 请勿使用 debugTraceFlags ，除非您正在进行故障排除并需要详细的日志转储。

> 创建后端时，请记住，创建后无法修改 dataLIF 和 storagePrefix 。要更新这些参数，您需要创建一个新的后端。

可以为 managementLIF 选项指定完全限定域名 （ FQDN ）。也可以为 dataLIF 选项指定 FQDN ，在这种情况下， FQDN 将用于 NFS 挂载操作。这样，您就可以创建循环 DNS ，以便在多个数据 LIF 之间实现负载平衡。

对于所有 ONTAP 驱动程序，也可以将 managementLIF 设置为 IPv6 地址。请务必使用 ` -use-ipv6` 标志安装 Astra Trident 。必须 m方括号内的 anagementLIF IPv6 地址。

> 使用 IPv6 地址时，请确保在方括号内定义 managementLIF 和 dataLIF （如果包含在后端定义中），例如 [28e8： d9fb： a825： b7bf： 69a8： d02f： 9e7b： 3555] 。如果未提供 dataLIF ，则 Astra Trident 将从 SVM 提取 IPv6 数据 LIF 。

使用 autosExportPolicy 和 autosExportCIDR 选项， CSI Trident 可以自动管理导出策略。所有 ontap-nas-* 驱动程序均支持此功能。

对于 ontap-nas-economy 驱动程序， limitVolumeSize 选项还会限制它为 qtree 和 FlexVol 管理的卷的最大大小，而 qtreesPerFlexvol 选项允许自定义每个 的最大 qtree 数。

可以使用 nfsMountOptions 参数指定挂载选项。Kubernetes 永久性卷的挂载选项通常在存储类中指定，但如果在存储类中未指定挂载选项，则 Astra Trident 将回退为使用存储后端配置文件中指定的挂载选项。如果在存储类或配置文件中未指定挂载选项，则 Astra Trident 不会在关联的永久性卷上设置任何挂载选项。

> Astra Trident 会在使用 `ontap-NAS` 和 `ontap-nas-flexgroup` 创建的所有卷的 "Comments" 字段中设置配置标签。根据所使用的驱动程序，注释将在 FlexVol （`ontap-NAS` ）或 FlexGroup （`ontap-nas-flexgroup` ）上进行设置。Astra Trident 会在配置存储池时将存储池上的所有标签复制到该存储卷。存储管理员可以为每个存储池定义标签，并对存储池中创建的所有卷进行分组。这样，您就可以根据后端配置中提供的一组可自定义标签来方便地区分卷了。

用于配置卷的后端配置选项

您可以在配置的特殊部分中使用这些选项来控制默认配置每个卷的方式。有关示例，请参见以下配置示例。

参数	**Description**	**Default**
`spaceAllocation`	LUN 的空间分配	true
`s`页面预留	空间预留模式； " 无 " （精简）或 " 卷 " （厚）	无
`sSnapshot` 策略	要使用的 Snapshot 策略	无
`qosPolicy`	要为创建的卷分配的 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一	"
`adaptiveQosPolicy`	要为创建的卷分配的自适应 QoS 策略组。选择每个存储池 / 后端的 qosPolicy 或 adaptiveQosPolicy 之一。不受 ontap-nas-economy.	"
`sSnapshot` 预留	为快照预留的卷百分比为 "0"	如果 `snapshotPolicy` 为 " 无 " ，则为 " 无 " ，否则为 " "
`splitOnClone`	创建克隆时，从其父级拆分该克隆	false
加密	启用 NetApp 卷加密	false
`securityStyle`	新卷的安全模式	"unix"
分层策略	使用 " 无 " 的分层策略	适用于 ONTAP 9.5 SVM-DR 之前的配置的 " 仅快照 "
unixPermissions	新卷的模式	777.
snapshotDir	控制 `.snapshot` 目录的可见性	false
导出策略	要使用的导出策略	default
securityStyle	新卷的安全模式	"unix"

> 在 Astra Trident 中使用 QoS 策略组需要 ONTAP 9.8 或更高版本。建议使用非共享 QoS 策略组，并确保策略组分别应用于每个成分卷。共享 QoS 策略组将对所有工作负载的总吞吐量实施上限。

下面是定义了默认值的示例：

```
{
  "version": 1,
  "storageDriverName": "ontap-nas",
  "backendName": "customBackendName",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "labels": {"k8scluster": "dev1", "backend": "dev1-nasbackend"},
  "svm": "trident_svm",
  "username": "cluster-admin",
  "password": "password",
  "limitAggregateUsage": "80%",
  "limitVolumeSize": "50Gi",
  "nfsMountOptions": "nfsvers=4",
  "debugTraceFlags": {"api":false, "method":true},
  "defaults": {
    "spaceReserve": "volume",
    "qosPolicy": "premium",
    "exportPolicy": "myk8scluster",
    "snapshotPolicy": "default",
    "snapshotReserve": "10"
  }
}
```

对于 `ontap-nas` 和 `ontap-nas-flexgroups` ， Astra Trident 现在使用新的计算方法来确保 FlexVol 的大小正确，并使用 snapshotReserve 百分比和 PVC 。当用户请求 PVC 时， Astra Trident 会使用新计算创建具有更多空间的原始 FlexVol 。此计算可确保用户在 PVC 中收到所请求的可写空间，而不是小于所请求的空间。在 v21.07 之前，如果用户请求 PVC （例如， 5GiB ），并且 snapshotReserve 为 50% ，则只会获得 2.5 GiB 的可写空间。这是因为用户请求的是整个卷，而 `snapshotReserve` 是其中的一个百分比。在 Trident 21.07 中，用户请求的是可写空间， Astra Trident 将 `snapshotReserve` number 定义为整个卷的百分比。这不适用于 `ontap-nas-economy"` 。请参见以下示例以了解其工作原理：

计算方法如下：

```
Total volume size = (PVC requested size) / (1 - (snapshotReserve
percentage) / 100)
```

对于 snapshotReserve = 50% ， PVC 请求 = 5GiB ，卷总大小为 2/.5 = 10GiB ，可用大小为 5GiB ，这是用户在 PVC 请求中请求的大小。`volume show` 命令应显示与以下示例类似的结果：

```
Vserver   Volume       Aggregate    State      Type       Size  Available Used%
--------- ------------ ------------ ---------- ---- ---------- ---------- -----
                 _pvc_89f1c156_3801_4de4_9f9d_034d54c395f4
                                      online     RW         10GB     5.00GB    0%
                 _pvc_e8372153_9ad9_474a_951a_08ae15e1c0ba
                                      online     RW          1GB    511.8MB    0%
2 entries were displayed.
```

在升级 Astra Trident 时，先前安装的现有后端将按照上述说明配置卷。对于在升级之前创建的卷，您应调整其卷的大小，以便观察到所做的更改。例如，一个 2 GiB PVC ，其 `snapshotReserve=50` earlier 会导致一个卷提供 1 GiB 的可写空间。例如，将卷大小调整为 3GiB 可为应用程序在一个 6 GiB 卷上提供 3GiB 的可写空间。

最低配置示例

以下示例显示了将大多数参数保留为默认值的基本配置。这是定义后端的最简单方法。

> 如果在采用 Trident 的 NetApp ONTAP 上使用 Amazon FSx ，建议为 LIF 指定 DNS 名称，而不是 IP 地址。

具有基于证书的身份验证的 **ONTAP NAS** 驱动程序

这是一个最低后端配置示例。`clientCertificate` ， `clientPrivateKey` 和 `trustedCACertifate` （如果使用可信 CA ，则可选）分别填充在 `backend.json` 中，并采用客户端证书，私钥和可信 CA 证书的 base64 编码值。

```
{
  "version": 1,
  "backendName": "DefaultNASBackend",
  "storageDriverName": "ontap-nas",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.15",
  "svm": "nfs_svm",
  "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
  "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
  "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz",
  "storagePrefix": "myPrefix_"
}
```

具有自动导出策略的 **ontap-NAS** 驱动程序

此示例显示了如何指示 Astra Trident 使用动态导出策略自动创建和管理导出策略。这对于 `ontap-nas-economy.` 和 `ontap-nas-flexgroup` 驱动程序也是如此。

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "labels": {"k8scluster": "test-cluster-east-1a", "backend": "test1-
nasbackend"},
    "autoExportPolicy": true,
    "autoExportCIDRs": ["10.0.0.0/24"],
    "username": "admin",
    "password": "secret",
    "nfsMountOptions": "nfsvers=4",
}
```

ontap-nas-flexgroup 驱动程序

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "labels": {"k8scluster": "test-cluster-east-1b", "backend": "test1-
ontap-cluster"},
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
}
```

使用 **IPv6** 的 **ONTAP NAS** 驱动程序

```
{
 "version": 1,
 "storageDriverName": "ontap-nas",
 "backendName": "nas_ipv6_backend",
 "managementLIF": "[5c5d:5edf:8f:7657:bef8:109b:1b41:d491]",
 "labels": {"k8scluster": "test-cluster-east-1a", "backend": "test1-ontap-
ipv6"},
 "svm": "nas_ipv6_svm",
 "username": "vsadmin",
 "password": "netapp123"
}
```

ontap-nas-economy-driver

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret"
}
```

虚拟存储池后端示例

在下面显示的示例后端定义文件中，会为所有存储池设置特定的默认值，例如 `spaceReserve` at none ， `spaceAllocation` at false 和 `encryption` at false 。虚拟存储池在存储部分中进行定义。

在此示例中，某些存储池会设置自己的 `spaceReserve` ， `spaceAllocation` 和 `encryption` 值，而某些池会覆盖上述设置的默认值。

ontap-NAS 驱动程序

```
{
    {
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "admin",
    "password": "secret",
    "nfsMountOptions": "nfsvers=4",

    "defaults": {
          "spaceReserve": "none",
          "encryption": "false",
          "qosPolicy": "standard"
    },
    "labels":{"store":"nas_store", "k8scluster": "prod-cluster-1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"msoffice", "cost":"100"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
```

```
                "encryption": "true",
                "unixPermissions": "0755",
                "adaptiveQosPolicy": "adaptive-premium"
            }
        },
        {
            "labels":{"app":"slack", "cost":"75"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"app":"wordpress", "cost":"50"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
            "labels":{"app":"mysqldb", "cost":"25"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

ontap-nas-flexgroup 驱动程序

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
```

```
    "defaults": {
          "spaceReserve": "none",
          "encryption": "false"
    },
    "labels":{"store":"flexgroup_store", "k8scluster": "prod-cluster-1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"protection":"gold", "creditpoints":"50000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"protection":"gold", "creditpoints":"30000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"protection":"silver", "creditpoints":"20000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
            "labels":{"protection":"bronze", "creditpoints":"10000"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

ontap-nas-economy-driver

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceReserve": "none",
          "encryption": "false"
    },
    "labels":{"store":"nas_economy_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"department":"finance", "creditpoints":"6000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"department":"legal", "creditpoints":"5000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"department":"engineering", "creditpoints":"3000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
```

```
            "labels":{"department":"humanresource",
"creditpoints":"2000"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

将后端映射到 **StorageClasses**

以下 StorageClass 定义引用了上述虚拟存储池。使用 `parameters.selector` 字段，每个 StorageClass 都会调用可用于托管卷的虚拟池。卷将在选定虚拟池中定义各个方面。

- 第一个 StorageClass （`protection-gold`）将映射到 `ontap-nas-flexgroup` 后端的第一个，第二个虚拟存储池以及 `ontap-san` 后端的第一个虚拟存储池。这是唯一一个提供黄金级保护的池。
- 第二个 StorageClass （`protection-not-gold`）将映射到 `ontap-nas-flexgroup` 后端的第三个，第四个虚拟存储池以及 `ontap-san` 后端的第二个，第三个虚拟存储池。这些池是唯一提供黄金级以外保护级别的池。
- 第三个 StorageClass （`app-mysqldb`）将映射到 `ontap-NAS` 后端的第四个虚拟存储池和 `ontap-san-economy-backend` 的第三个虚拟存储池。这些池是唯一为 mysqldb 类型的应用程序提供存储池配置的池。
- 第四个存储类（`protection-silver-creditpoins-20k`）将映射到 `ontap-nas-flexgroup` 后端的第三个虚拟存储池和 `ontap-san` 后端的第二个虚拟存储池。这些池是唯一以 20000 个信用点提供黄金级保护的池。
- 第五个存储类（`credits-5k`）将映射到 `ontap-nas-economy-backend` 中的第二个虚拟存储池和 `ontap-san` 后端的第三个虚拟存储池。这些是唯一一款具有 5000 个信用点的池产品。

Astra Trident 将决定选择哪个虚拟存储池，并确保满足存储要求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-not-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection!=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: app-mysqldb
provisioner: netapp.io/trident
parameters:
  selector: "app=mysqldb"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-silver-creditpoints-20k
provisioner: netapp.io/trident
parameters:
  selector: "protection=silver; creditpoints=20000"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: creditpoints-5k
provisioner: netapp.io/trident
parameters:
  selector: "creditpoints=5000"
  fsType: "ext4"
```

将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSX 结合使用

"适用于 NetApp ONTAP 的 Amazon FSX"是一种完全受管的 AWS 服务，支持客户启动和运行由 NetApp 的 ONTAP 存储操作系统提供支持的文件系统。Amazon FSX for NetApp ONTAP 支持您利用您熟悉的 NetApp 功能，性能和管理功能，同时利用在 AWS 上存储数据的简便性，灵活性，安全性和可扩展性。FSX 支持 ONTAP 的许多文件系统功能和管理 API 。

文件系统是 Amazon FSX 中的主要资源，类似于内部部署的 ONTAP 集群。在每个 SVM 中，您可以创建一个或多个卷，这些卷是将文件和文件夹存储在文件系统中的数据容器。借助适用于 NetApp ONTAP 的 Amazon FSX ， Data ONTAP 将作为云中的托管文件系统提供。新的文件系统类型称为 * NetApp ONTAP * 。

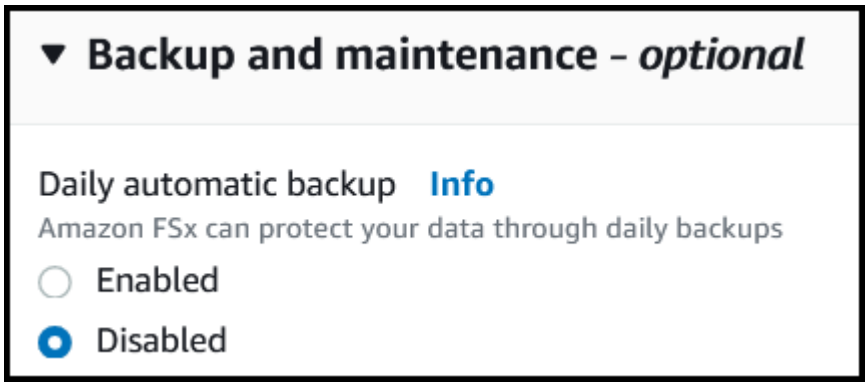
通过将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSx 结合使用，您可以确保在 Amazon Elastic Kubernetes Service （ EKS ）中运行的 Kubernetes 集群可以配置由 ONTAP 备份的块和文件永久性卷。

创建适用于 **ONTAP** 的 **Amazon FSX** 文件系统

Trident 无法删除在启用了自动备份的 Amazon FSx 文件系统上创建的卷。要删除 PVC ，您需要手动删除 PV 和 ONTAP 的 FSX 卷。

要防止此问题描述 ，请执行以下操作：

- 请勿使用 "* 快速创建 "* 来创建适用于 ONTAP 的 FSX 文件系统。快速创建工作流可启用自动备份，但不提供选择退出选项。
- 使用 "** 标准创建 " 时，禁用自动备份。禁用自动备份可以使 Trident 成功删除卷，而无需进一步手动干预。



了解 **Astra Trident**

如果您是 Astra Trident 的新用户，请使用下面提供的链接进行熟悉：

- "常见问题解答"
- "使用 Astra Trident 的要求"
- "部署 Astra Trident"
- "配置适用于 NetApp ONTAP 的 ONTAP， Cloud Volumes ONTAP 和 Amazon FSX 的最佳实践"
- "集成 Astra Trident"
- "ONTAP SAN 后端配置"
- "ONTAP NAS 后端配置"

详细了解驱动程序功能 "此处"。

适用于 NetApp ONTAP 的 Amazon FSX 使用 "FabricPool" 以管理存储层。通过它，您可以根据数据是否经常访问来将数据存储在层中。

Astra Trident应以`vsadmin` SVM用户或具有相同角色的其他名称的用户的身份运行。适用于NetApp ONTAP的Amazon FSX具有一个`fsxadmin`用户、该用户是ONTAP `admin` cluster用户的有限替代。建议不要在Trident中使用`fsxadmin`用户、因为`vsadmin` SVM用户可以访问更多Astra Trident功能。

驱动程序

您可以使用以下驱动程序将 Astra Trident 与适用于 NetApp ONTAP 的 Amazon FSx 集成：

- `ontap-san`：配置的每个 PV 都是其自己的 Amazon FSX for NetApp ONTAP 卷中的一个 LUN 。
- `ontap-san-economi`：为 NetApp ONTAP 卷配置的每个 PV 都是一个 LUN ，每个 Amazon FSX 具有可配置的 LUN 数量。
- `ONONTAP NAS`：配置的每个 PV 都是适用于 NetApp ONTAP 的完整 Amazon FSX 卷。
- `ontap-nas-economy.`：配置的每个 PV 都是一个 qtree ，对于 NetApp ONTAP 卷，每个 Amazon FSx 的 qtree 数量是可配置的。
- `ontap-nas-flexgroup`：配置的每个 PV 都是适用于 NetApp ONTAP FlexGroup 卷的完整 Amazon FSX 。

身份验证

Astra Trident 提供两种身份验证模式：

- 基于证书： Astra Trident 将使用 SVM 上安装的证书与 FSX 文件系统上的 SVM 进行通信。
- 基于凭据：您可以使用文件系统的 `fsxadmin` 用户或为 SVM 配置的 `vsadmin` 用户。

> 我们强烈建议使用`vsadmin` user而非`fsxadmin`来配置后端。Astra Trident 将使用此用户名和密码与 FSX 文件系统进行通信。

要了解有关身份验证的详细信息，请参见以下链接：

- "ONTAP NAS"
- "ONTAP SAN"

使用适用于 **NetApp ONTAP** 的 **Amazon FSX** 在 **EKS** 上部署和配置 **Astra Trident**

您需要什么？ **#8217**；将需要什么

- 已安装 `kubectl` 的现有 Amazon EKS 集群或自管理 Kubernetes 集群。
- 可从集群的工作节点访问的适用于 NetApp ONTAP 文件系统和 Storage Virtual Machine （ SVM ）的现有 Amazon FSX 。
- 为准备工作的工作节点 "NFS 和 / 或 iSCSI"。

> 确保按照 Amazon Linux 和 Ubuntu 所需的节点准备步骤进行操作 "Amazon Machine 映像"（AMIS），具体取决于您的 EKS AMI 类型。

有关其他 Astra Trident 要求，请参见 "此处"。

步骤

1. 使用 ./trident 部署 Astra Trident 。 Get-started/Kubernetes 部署 .html[部署方法^] 之一部署 Astra Trident 。
2. 按照以下步骤配置 Astra Trident：
   a. 收集 SVM 的管理 LIF DNS 名称。例如，通过使用 AWS 命令行界面，在运行以下命令后，在 `Endpoints` → `Management` 下找到 `DNSName` 条目：

   ```
   aws fsx describe-storage-virtual-machines --region <file system
   region>
   ```

3. 创建并安装用于身份验证的证书。如果您使用的是 `ontap-san` 后端，请参见 "此处"。如果您使用的是 `ontap-nas` 后端，请参见 "此处"。

   > 您可以从可以访问文件系统的任何位置使用 SSH 登录到文件系统（例如，安装证书）。使用 `fsxadmin` 用户，创建文件系统时配置的密码以及 `AWS FSx describe` **文件系统** 中的管理 DNS 名称。

4. 使用您的证书和管理 LIF 的 DNS 名称创建后端文件，如以下示例所示：

   ```
   {
     "version": 1,
     "storageDriverName": "ontap-san",
     "backendName": "customBackendName",
     "managementLIF": "svm-XXXXXXXXXXXXXXXXX.fs-XXXXXXXXXXXXXXXXX.fsx.us-
   east-2.aws.internal",
     "svm": "svm01",
     "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
     "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
     "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz",
    }
   ```

有关创建后端的信息，请参见以下链接：

- "使用 ONTAP NAS 驱动程序配置后端"
- "使用 ONTAP SAN 驱动程序配置后端"

> 请勿为 `ontap-san` 和 `ontap-san-economy-` 驱动程序指定 `dataLIF` ，以允许 Astra Trident 使用多路径。

> `limitAggregateUsage` 参数不适用于 `vsadmin` 和 `fsxadmin` 用户帐户。如果指定此参数，配置操作将失败。

部署完成后，执行以下步骤以创建 "存储类，配置卷以及将卷挂载到 Pod 中"。

了解更多信息

- "Amazon FSX for NetApp ONTAP 文档"
- "有关适用于 NetApp ONTAP 的 Amazon FSX 的博客文章"

使用 kubectl 创建后端

后端定义了 Astra Trident 与存储系统之间的关系。它告诉 Astra Trident 如何与该存储系统进行通信，以及 Astra Trident 如何从该存储系统配置卷。安装 Astra Trident 后，下一步是创建后端。使用 `TridentBackendConfig` Custom Resource Definition （ CRD ），您可以直接通过 Kubernetes 界面创建和管理 Trident 后端。为此，您可以对 Kubernetes 分发版使用 `kubectl` 或等效的命令行界面工具。

TridentBackendConfig

`TridentBackendConfig` （`tbc`， `tbconfig`， `tbackendconfig`）是一个前端命名的 CRD ，可用于使用 `kubectl` 管理 Astra Trident 后端。现在， Kubernetes 和存储管理员可以直接通过 Kubernetes 命令行界面创建和管理后端，而无需专用命令行实用程序（`tridentctl`）。

创建 `TridentBackendConfig` 对象时，将发生以下情况：

- Astra Trident 会根据您提供的配置自动创建后端。此值在内部表示为 `TridentBackend` （`tbe`， `tridentbackend`） CR 。
- `TridentBackendConfig` 唯一绑定到由 Astra Trident 创建的 `TridentBackend` 。

每个 `TridentBackendConfig` 都与 `TridentBackend` 保持一对一映射。前者是为用户提供的用于设计和配置后端的接口；后者是 Trident 表示实际后端对象的方式。

> `TridentBackend` CRS 由 Astra Trident 自动创建。您 * 不应 * 修改它们。如果要更新后端，请通过修改 `TridentBackendConfig` 对象来执行此操作。

有关 `TridentBackendConfig` CR 的格式，请参见以下示例：

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: backend-tbc-ontap-san
spec:
  version: 1
  backendName: ontap-san-backend
  storageDriverName: ontap-san
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  svm: trident_svm
  credentials:
    name: backend-tbc-ontap-san-secret
```

您还可以查看中的示例 "Trident 安装程序" 所需存储平台 / 服务的示例配置目录。

`sPec` 采用后端特定的配置参数。在此示例中，后端使用 `ontap-san` 存储驱动程序并使用此处所示的配置参数。有关所需存储驱动程序的配置选项列表，请参见 "存储驱动程序的后端配置信息"。

`sPec` 部分还包括 `credentials` 和 `deletionPolicy` 字段，这些字段在 `TridentBackendConfig` CR 中新增：

- `credentials` ： 此参数为必填字段，包含用于向存储系统 / 服务进行身份验证的凭据。此密码设置为用户创建的 Kubernetes Secret 。凭据不能以纯文本形式传递，因此会导致错误。
- `deeltionPolicy` ： 此字段定义删除 `TridentBackendConfig` 时应发生的情况。它可以采用以下两种可能值之一：
  - `delete` ： 这将同时删除 `TridentBackendConfig` CR 和关联后端。这是默认值。
  - `retain` ： 删除 `TrdentBackendConfig` CR 后，后端定义仍存在，可使用 `tridentctl` 进行管理。将删除策略设置为 `retain` 允许用户降级到早期版本（ 21.04 之前）并保留创建的后端。创建 `TridentBackendConfig` 后，可以更新此字段的值。

> 后端名称使用 `sPec.backendName` 设置。如果未指定，则后端的名称将设置为 `TridentBackendConfig` 对象的名称（ metadata.name ）。建议使用 `sPec.backendName` 显式设置后端名称。

> 使用 `tridentctl` 创建的后端没有关联的 `TridentBackendConfig` 对象。您可以通过创建 `TridentBackendConfig` CR ，选择使用 `kubectl` 来管理此类后端。必须注意指定相同的配置参数（例如 `sPec.backendName` ， `sPec.storagePrefix` ， `sPec.storageDriverName` 等）。Astra Trident 会自动将新创建的 `TridentBackendConfig` 与已有的后端绑定。

步骤概述

要使用 `kubectl` 创建新后端，应执行以下操作：

1. 创建 "Kubernetes 机密"。此密钥包含 Astra Trident 与存储集群 / 服务通信所需的凭据。

2. 创建 `TridentBackendConfig` 对象。其中包含有关存储集群 / 服务的详细信息，并引用了上一步中创建的密钥。

创建后端后，您可以使用 `kubectl get tbc <tbc-name> -n <trident` 命名空间 `>` 来观察其状态，并收集其他详细信息。

第 1 步：创建 Kubernetes 机密

创建一个机密，其中包含后端的访问凭据。这是每个存储服务 / 平台所特有的。以下是一个示例：

```
$ kubectl -n trident create -f backend-tbc-ontap-san-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: backend-tbc-ontap-san-secret
type: Opaque
stringData:
  username: cluster-admin
  password: t@Ax@7q(>
```

下表汇总了每个存储平台的机密中必须包含的字段：

存储平台机密字段问题描述	机密	字段问题描述
Azure NetApp Files	clientId	应用程序注册中的客户端 ID
适用于 GCP 的 Cloud Volumes Service	private_key_id	专用密钥的 ID 。具有 CVS 管理员角色的 GCP 服务帐户的 API 密钥的一部分
适用于 GCP 的 Cloud Volumes Service	private_key	专用密钥。具有 CVS 管理员角色的 GCP 服务帐户的 API 密钥的一部分
Element （ NetApp HCI/SolidFire ）	端点	使用租户凭据的 SolidFire 集群的 MVIP
ONTAP	username	用于连接到集群 /SVM 的用户名。用于基于凭据的身份验证
ONTAP	password	连接到集群 /SVM 的密码。用于基于凭据的身份验证
ONTAP	客户端权限密钥	客户端专用密钥的 Base64 编码值。用于基于证书的身份验证

存储平台机密字段问题描述	机密	字段问题描述
ONTAP	用户名	入站用户名。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.
ONTAP	chapInitiatorSecret	CHAP 启动程序密钥。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.
ONTAP	chapTargetUsername	目标用户名。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.
ONTAP	chapTargetInitiatorSecret	CHAP 目标启动程序密钥。如果 useCHAP=true ，则为必需项。适用于 `ontap-san` 和 `ontap-san-economy`.

在下一步中创建的 `TrdentBackendConfig` 对象的 `sPec.credentials` 字段将引用此步骤中创建的机密。

第 2 步：创建 TridentBackendConfig CR

现在，您可以创建 `TridentBackendConfig` CR 了。在此示例中，使用 `TriventBackendConfig` 对象创建使用` ontap-san `驱动程序的后端，如下所示：

```
$ kubectl -n trident create -f backend-tbc-ontap-san.yaml
```

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: backend-tbc-ontap-san
spec:
  version: 1
  backendName: ontap-san-backend
  storageDriverName: ontap-san
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  svm: trident_svm
  credentials:
    name: backend-tbc-ontap-san-secret
```

第 3 步：验证 TridentBackendConfig CR 的状态

现在，您已创建 `TridentBackendConfig` CR ，可以验证状态。请参见以下示例：

```
$ kubectl -n trident get tbc backend-tbc-ontap-san
NAME                      BACKEND NAME          BACKEND UUID
PHASE   STATUS
backend-tbc-ontap-san   ontap-san-backend     8d24fce7-6f60-4d4a-8ef6-
bab2699e6ab8   Bound   Success
```

已成功创建后端并将其绑定到 `TridentBackendConfig` CR 。

阶段可以采用以下值之一：

- `bound`： `TridentBackendConfig` CR 与后端关联，后端包含 `configRef` 设置为 `TridentBackendConfig` CR 的 UID 。
- `Unbound`：使用 `""` 表示。`TridentBackendConfig` 对象未绑定到后端。默认情况下，所有新创建的 `TridentBackendConfig` CRS 均处于此阶段。此阶段发生更改后，它将无法再次还原为 "Unbound （已取消绑定） " 。
- `deleting`： The `TridentBackendConfig` CR 's `deletionPolicy` was set to delete.删除 `TridentBackendConfig` CR 后，它将过渡到 Deleting 状态。
  - 如果后端不存在永久性卷请求（ PVC ），则删除 `TridentBackendConfig` 将导致 Astra Trident 删除后端以及 `TridentBackendConfig` CR 。
  - 如果后端存在一个或多个 PVC ，则会进入删除状态。`TridentBackendConfig` CR 随后也会进入删除阶段。只有在删除所有 PVC 后，才会删除后端和 `TridentBackendConfig` 。
- `Lost`：与 `TridentBackendConfig` CR 关联的后端被意外或故意删除， `TridentBackendConfig` CR 仍引用已删除的后端。无论 `detionPolicy` 值如何，仍可删除 `TridentBackendConfig` CR 。
- 未知： Astra Trident 无法确定与 `TridentBackendConfig` CR 关联的后端的状态或存在。例如，如果 API 服务器未响应或缺少 `tridentbackends.trident.netapp.io` CRD 。这可能需要用户干预。

在此阶段，已成功创建后端！此外，还可以处理多个操作，例如 "后端更新和后端删除"。

（可选）第 4 步：获取更多详细信息

您可以运行以下命令来获取有关后端的详细信息：

```
kubectl -n trident get tbc backend-tbc-ontap-san -o wide
```

```
NAME                      BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   8d24fce7-6f60-4d4a-8ef6-
bab2699e6ab8   Bound   Success   ontap-san        delete
```

此外，您还可以获取 YAML/JSON 转储 `TridentBackendConfig`。

```
$ kubectl -n trident get tbc backend-tbc-ontap-san -o yaml
```

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  creationTimestamp: "2021-04-21T20:45:11Z"
  finalizers:
  - trident.netapp.io
  generation: 1
  name: backend-tbc-ontap-san
  namespace: trident
  resourceVersion: "947143"
  uid: 35b9d777-109f-43d5-8077-c74a4559d09c
spec:
  backendName: ontap-san-backend
  credentials:
    name: backend-tbc-ontap-san-secret
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  storageDriverName: ontap-san
  svm: trident_svm
  version: 1
status:
  backendInfo:
    backendName: ontap-san-backend
    backendUUID: 8d24fce7-6f60-4d4a-8ef6-bab2699e6ab8
  deletionPolicy: delete
  lastOperationStatus: Success
  message: Backend 'ontap-san-backend' created
  phase: Bound
```

`backendInfo` 包含为响应 `TridentBackendConfig` CR 而创建的后端的 `backendName` 和 `backendUUID`。`lastOperationStatus` 字段表示 `TridentBackendConfig` CR 的上次操作状态，该操作可以是用户触发的（例如，用户在 `sPec` 中更改了某个内容），也可以是由 Astra Trident 触发的（例如，在 Astra Trident 重新启动期间）。可以是成功，也可以是失败。`phase` 表示 `TridentBackendConfig` CR 与后端之间关系的状态。在上面的示例中， `phase` 的值为 bound ，这意味着 `TridentBackendConfig` CR 与后端关联。

您可以运行 `kubectl -n trident describe tbc <tbc-cr-name>` 命令来获取事件日志的详细信息。

> 您不能使用 `tridentctl` 更新或删除包含关联的 `TridentBackendConfig` 对象的后端。要了解在 `tridentctl` 和 `TridentBackendConfig` 之间切换所涉及的步骤， "请参见此处"。

使用 kubectl 执行后端管理

了解如何使用 `kubectl` 执行后端管理操作。

删除后端

通过删除 `TridentBackendConfig` ，您可以指示 Astra Trident 删除 / 保留后端（基于 `detionPolicy` ）。要删除后端，请确保将 `deletionPolicy` 设置为 delete 。要仅删除 `TridentBackendConfig` ，请确保将 `deletionPolicy` 设置为 Retain 。这样可以确保后端仍然存在，并可使用 `tridentctl` 进行管理。

运行以下命令：

```
$ kubectl delete tbc <tbc-name> -n trident
```

Astra Trident 不会删除 `TridentBackendConfig` 使用的 Kubernetes 机密。Kubernetes 用户负责清理密钥。删除机密时必须小心。只有在后端未使用机密时，才应将其删除。

查看现有后端

运行以下命令：

```
$ kubectl get tbc -n trident
```

您也可以运行 `tridentctl get backend -n trident` 或 `tridentctl get backend -o YAML -n trident` 来获取存在的所有后端的列表。此列表还将包括使用 `tridentctl` 创建的后端。

更新后端

更新后端可能有多种原因：

- 存储系统的凭据已更改。要更新凭据，必须更新 `TridentBackendConfig` 对象中使用的 Kubernetes Secret 。Astra Trident 将使用提供的最新凭据自动更新后端。运行以下命令以更新 Kubernetes Secret：

```
$ kubectl apply -f <updated-secret-file.yaml> -n trident
```

- 需要更新参数（例如所使用的 ONTAP SVM 的名称）。在这种情况下，可以直接通过 Kubernetes 更新 `TridentBackendConfig` 对象。

```
$ kubectl apply -f <updated-backend-file.yaml>
```

或者，也可以运行以下命令来更改现有的 `TridentBackendConfig` CR：

```
$ kubectl edit tbc <tbc-name> -n trident
```

如果后端更新失败，则后端仍会保持在其上次已知配置中。您可以通过运行 `kubectl get tbc <tbc-name> -o yaml -n trident` 或 `kubectl describe tbc <tbc-name> -n trident` 来查看日志以确定发生原因。

确定并更正配置文件中的问题后，您可以重新运行 update 命令。

使用 tridentctl 执行后端管理

了解如何使用 `tridentctl` 执行后端管理操作。

创建后端

创建后 "后端配置文件"下，运行以下命令：

```
$ tridentctl create backend -f <backend-file> -n trident
```

如果后端创建失败，则后端配置出现问题。您可以运行以下命令来查看日志以确定发生原因：

```
$ tridentctl logs -n trident
```

确定并更正配置文件中的问题后，您只需再次运行 `create` 命令即可。

删除后端

要从 Astra Trident 中删除后端，请执行以下操作：

1. 检索后端名称：

   ```
   $ tridentctl get backend -n trident
   ```

2. 删除后端：

   ```
   $ tridentctl delete backend <backend-name> -n trident
   ```

> 如果 Astra Trident 从此后端配置了仍存在的卷和快照，则删除后端将阻止其配置新卷。后端将继续处于 " 删除 " 状态，而 Trident 将继续管理这些卷和快照，直到将其删除为止。

查看现有后端

要查看 Trident 了解的后端，请执行以下操作：

- 要获取摘要，请运行以下命令：

```
$ tridentctl get backend -n trident
```

- 要获取所有详细信息，请运行以下命令：

```
$ tridentctl get backend -o json -n trident
```

更新后端

创建新的后端配置文件后，运行以下命令：

```
$ tridentctl update backend <backend-name> -f <backend-file> -n trident
```

如果后端更新失败，则后端配置出现问题或您尝试的更新无效。您可以运行以下命令来查看日志以确定发生原因：

```
$ tridentctl logs -n trident
```

确定并更正配置文件中的问题后，您只需再次运行 `update` 命令即可。

确定使用后端的存储类

以下是您可以使用问题解答与 JSON 回答的问题的示例，这些问题会 `tridentctl` 后端对象的输出。此操作将使用 `JQ` 实用程序，您需要安装该实用程序。

```
$ tridentctl get backend -o json | jq '[.items[] | {backend: .name,
storageClasses: [.storage[].storageClasses]|unique}]'
```

这也适用于使用 `TridentBackendConfig` 创建的后端。

在后端管理选项之间移动

了解如何在 Astra Trident 中管理后端。随着 `TridentBackendConfig` 的推出，管理员现在可以通过两种独特的方式管理后端。这会提出以下问题：

- 使用 `tridentctl` 创建的后端是否可以使用 `TridentBackendConfig` 进行管理？

- 使用 `TridentBackendConfig` 创建的后端是否可以使用 `tridentctl` 进行管理?

使用 TridentBackendConfig 管理 tridentctl 后端

本节介绍通过创建 `TridentBackendConfig` 对象直接通过 Kubernetes 界面管理使用 `tridentctl` 创建的后端所需的步骤。

这适用于以下情形：

- 预先存在的后端，没有 `TridentBackendConfig` ，因为它们是使用 `tridentctl` 创建的。
- 使用 `tridentctl` 创建的新后端，而存在其他 `TridentBackendConfig` 对象。

在这两种情况下，后端仍会存在，其中 Astra Trident 会计划卷并对其进行操作。管理员可以选择以下两种方式之一：

- 继续使用 `tridentctl` 管理使用它创建的后端。
- 使用 `tridentctl` 创建的后端绑定到新的 `TridentBackendConfig` 对象。这样做意味着将使用 `kubectl` 而不是 `tridentctl` 来管理后端。

要使用 `kubectl` 管理已有后端，您需要创建一个绑定到现有后端的 `TridentBackendConfig` 。下面简要介绍了它的工作原理：

1. 创建 Kubernetes 机密。此密钥包含 Astra Trident 与存储集群 / 服务通信所需的凭据。
2. 创建 `TridentBackendConfig` 对象。其中包含有关存储集群 / 服务的详细信息，并引用了上一步中创建的密钥。必须注意指定相同的配置参数（例如 `sPec.backendName` ， `sPec.storagePrefix` ， `sPec.storageDriverName` 等）。`sPec.backendName` 必须设置为现有后端的名称。

第 **0** 步：确定后端

要创建绑定到现有后端的 `TridentBackendConfig` ，您需要获取后端的配置。在此示例中，假设已使用以下 JSON 定义创建了后端：

```
$ tridentctl get backend ontap-nas-backend -n trident
+---------------------+----------------
+--------------------------------------+--------+---------+
|          NAME       | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+---------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas-backend   | ontap-nas      | 52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7 | online |      25 |
+---------------------+----------------
+--------------------------------------+--------+---------+

$ cat ontap-nas-backend.json

{
```

```
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.10.10.1",
    "dataLIF": "10.10.10.2",
    "backendName": "ontap-nas-backend",
    "svm": "trident_svm",
    "username": "cluster-admin",
    "password": "admin-password",

    "defaults": {
        "spaceReserve": "none",
        "encryption": "false"
    },
    "labels":{"store":"nas_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"msoffice", "cost":"100"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"app":"mysqldb", "cost":"25"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

第 **1** 步：创建 **Kubernetes** 机密

创建一个包含后端凭据的机密，如以下示例所示：

```
$ cat tbc-ontap-nas-backend-secret.yaml

apiVersion: v1
kind: Secret
metadata:
  name: ontap-nas-backend-secret
type: Opaque
stringData:
  username: cluster-admin
  passWord: admin-password

$ kubectl create -f tbc-ontap-nas-backend-secret.yaml -n trident
secret/backend-tbc-ontap-san-secret created
```

第 **2** 步：创建 **TridentBackendConfig CR**

下一步是创建一个 `TridentBackendConfig` CR ，该 CR 将自动绑定到已有的 `ontap-nas-backend` （如本示例所示）。确保满足以下要求：

- 在 `sPec.backendName` 中定义了相同的后端名称。
- 配置参数与原始后端相同。
- 虚拟存储池（如果存在）必须与原始后端保持相同的顺序。
- 凭据通过 Kubernetes Secret 提供，而不是以纯文本形式提供。

在这种情况下， `TridentBackendConfig` 将如下所示：

```
$ cat backend-tbc-ontap-nas.yaml
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: tbc-ontap-nas-backend
spec:
  version: 1
  storageDriverName: ontap-nas
  managementLIF: 10.10.10.1
  dataLIF: 10.10.10.2
  backendName: ontap-nas-backend
  svm: trident_svm
  credentials:
    name: mysecret
  defaults:
    spaceReserve: none
    encryption: 'false'
  labels:
    store: nas_store
  region: us_east_1
  storage:
  - labels:
      app: msoffice
      cost: '100'
    zone: us_east_1a
    defaults:
      spaceReserve: volume
      encryption: 'true'
      unixPermissions: '0755'
  - labels:
      app: mysqldb
      cost: '25'
    zone: us_east_1d
    defaults:
      spaceReserve: volume
      encryption: 'false'
      unixPermissions: '0775'

$ kubectl create -f backend-tbc-ontap-nas.yaml -n trident
tridentbackendconfig.trident.netapp.io/tbc-ontap-nas-backend created
```

第 **3** 步：验证 **TridentBackendConfig CR** 的状态

创建 `TridentBackendConfig` 后，其阶段必须为 `bound` 。它还应反映与现有后端相同的后端名称和 UUID 。

```
$ kubectl -n trident get tbc tbc-ontap-nas-backend -n trident
NAME                    BACKEND NAME          BACKEND UUID
PHASE   STATUS
tbc-ontap-nas-backend  ontap-nas-backend     52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7   Bound   Success

#confirm that no new backends were created (i.e., TridentBackendConfig did
not end up creating a new backend)
$ tridentctl get backend -n trident
+----------------------+----------------
+--------------------------------------+--------+---------+
|          NAME        | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+----------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas-backend   | ontap-nas      | 52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7 | online |      25 |
+----------------------+----------------
+--------------------------------------+--------+---------+
```

现在，可以使用 `tbc-ontap-nas-backend` `TridentBackendConfig` 对象对后端进行全面管理。

使用 tridentctl 管理 TridentBackendConfig 后端

`tridentctl` 可用于列出使用 `TridentBackendConfig` 创建的后端。此外，管理员还可以选择通过 `tridentctl` 来完全管理此类后端，方法是删除 `TridentBackendConfig` 并确保将 `spec.deletionPolicy` 设置为 `retain` 。

第 0 步：确定后端

例如，假设使用 `TridentBackendConfig` 创建了以下后端：

```
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        delete

$ tridentctl get backend ontap-san-backend -n trident
+-------------------+----------------
+--------------------------------------+--------+---------+
|       NAME        | STORAGE DRIVER |                 UUID
| STATE  | VOLUMES |
+-------------------+----------------
+--------------------------------------+--------+---------+
| ontap-san-backend | ontap-san      | 81abcb27-ea63-49bb-b606-
0a5315ac5f82 | online |      33 |
+-------------------+----------------
+--------------------------------------+--------+---------+
```

从输出中可以看到 `TridentBackendConfig` 已成功创建并绑定到后端【观察后端的 UUUID] 。

第 1 步：确认 deletionPolicy 设置为 Retain

让我们来看看 `deletionPolicy` 的价值。需要将此值设置为 `retain` 。这样可以确保删除 `TridentBackendConfig` CR 时，后端定义仍存在，并可使用 `tridentctl` 进行管理。

```
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        delete

# Patch value of deletionPolicy to retain
$ kubectl patch tbc backend-tbc-ontap-san --type=merge -p
'{"spec":{"deletionPolicy":"retain"}}' -n trident
tridentbackendconfig.trident.netapp.io/backend-tbc-ontap-san patched

#Confirm the value of deletionPolicy
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        retain
```

请勿继续执行下一步，除非将 `deletionPolicy` 设置为 `retain` 。

第 2 步：删除 TridentBackendConfig CR

最后一步是删除 `TridentBackendConfig` CR 。确认 `deeltionPolicy` 设置为 `retain` 后，您可以继续执行删除：

```
$ kubectl delete tbc backend-tbc-ontap-san -n trident
tridentbackendconfig.trident.netapp.io "backend-tbc-ontap-san" deleted

$ tridentctl get backend ontap-san-backend -n trident
+--------------------+----------------
+--------------------------------------+--------+---------+
|       NAME         | STORAGE DRIVER |                  UUID
| STATE   | VOLUMES |
+--------------------+----------------
+--------------------------------------+--------+---------+
| ontap-san-backend | ontap-san      | 81abcb27-ea63-49bb-b606-
0a5315ac5f82 | online |      33 |
+--------------------+----------------
+--------------------------------------+--------+---------+
```

删除 `TridentBackendConfig` 对象后， Astra Trident 只需删除该对象，而无需实际删除后端本身。

管理存储类

查找有关创建存储类，删除存储类以及查看现有存储类的信息。

设计存储类

请参见 "此处" 有关什么是存储类以及如何配置这些类的详细信息，请参见。

创建存储类。

创建存储类文件后，运行以下命令：

```
kubectl create -f <storage-class-file>
```

` <storage-class-file>` 应替换为存储类文件名。

删除存储类

要从 Kubernetes 中删除存储类，请运行以下命令：

```
kubectl delete storageclass <storage-class>
```

`< 存储类 >` 应替换为您的存储类。

通过此存储类创建的任何永久性卷将保持不变， Astra Trident 将继续对其进行管理。

> Astra Trident 会为其创建的卷强制使用空的 `FSType` 。对于 iSCSI 后端，建议在 StorageClass 中强制实施 `parameters.FSType` 。您应删除现有 StorageClasses 并使用指定的 `parameters.FSType` 重新创建它们。

查看现有存储类

- 要查看现有 Kubernetes 存储类，请运行以下命令：

```
kubectl get storageclass
```

- 要查看 Kubernetes 存储类详细信息，请运行以下命令：

```
kubectl get storageclass <storage-class> -o json
```

- 要查看 Astra Trident 的同步存储类，请运行以下命令：

```
tridentctl get storageclass
```

- 要查看 Astra Trident 的同步存储类详细信息，请运行以下命令：

```
tridentctl get storageclass <storage-class> -o json
```

设置默认存储类

Kubernetes 1.6 增加了设置默认存储类的功能。如果用户未在永久性卷声明（ PVC ）中指定永久性卷，则此存储类将用于配置永久性卷。

- 通过在存储类定义中将标注 `storageclass.Kubernetes 。 io/is-default-class` 设置为 true 来定义默认存储类。根据规范，任何其他值或标注不存在均视为 false 。
- 您可以使用以下命令将现有存储类配置为默认存储类：

```
kubectl patch storageclass <storage-class-name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

- 同样，您也可以使用以下命令删除默认存储类标注：

```
kubectl patch storageclass <storage-class-name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
```

Trident 安装程序包中也有包含此标注的示例。

> 在任何给定时间，集群中只能有一个默认存储类。Kubernetes 在技术上不会阻止您拥有多个存储类，但其行为就像根本没有默认存储类一样。

确定存储类的后端

以下是您可以使用问题解答与 JSON 一起为 Astra Trident 后端对象输出的 `tridentctl` 类问题的示例。此操作将使用 `JQ` 实用程序，您可能需要先安装该实用程序。

```
tridentctl get storageclass -o json | jq  '[.items[] | {storageClass:
.Config.name, backends: [.storage]|unique}]'
```

执行卷操作

了解 Astra Trident 为管理卷提供的功能。

- "使用 CSI 拓扑"
- "使用快照"
- "展开卷"
- "导入卷"

使用 **CSI** 拓扑

Astra Trident 可以通过使用有选择地创建卷并将其附加到 Kubernetes 集群中的节点 "CSI 拓扑功能"。使用 CSI 拓扑功能，可以根据区域和可用性区域将对卷的访问限制为一小部分节点。如今，借助云提供商， Kubernetes 管理员可以生成基于分区的节点。节点可以位于一个区域内的不同可用性区域中，也可以位于不同区域之间。为了便于在多区域架构中为工作负载配置卷， Astra Trident 使用了 CSI 拓扑。

> 了解有关 CSI 拓扑功能的更多信息 "此处"。

Kubernetes 提供了两种唯一的卷绑定模式：

- 如果将 `VolumeBindingMode` 设置为 `immediate` ，则 Astra Trident 将创建卷，而不会感知任何拓扑。创建 PVC 时会处理卷绑定和动态配置。这是默认值 `VolumeBindingMode` ，适用于不强制实施拓扑约束的集群。创建永久性卷时，不会依赖于请求的 Pod 的计划要求。
- 如果将 `VolumeBindingMode` 设置为 `WaitForFirstConsuming` ，则为 PVC 创建和绑定永久性卷的操作将延迟，直到计划并创建使用 PVC 的 Pod 为止。这样，卷就会根据拓扑要求强制实施的计划限制来创建。

> `WaitForFirstConsumer"` 绑定模式不需要拓扑标签。此功能可独立于 CSI 拓扑功能使用。

要使用 CSI 拓扑，您需要满足以下条件：

- 运行 1.17 或更高版本的 Kubernetes 集群。

```
$ kubectl version
Client Version: version.Info{Major:"1", Minor:"19",
GitVersion:"v1.19.3",
GitCommit:"1e11e4a2108024935ecfcb2912226cedeafd99df",
GitTreeState:"clean", BuildDate:"2020-10-14T12:50:19Z",
GoVersion:"go1.15.2", Compiler:"gc", Platform:"linux/amd64"}
Server Version: version.Info{Major:"1", Minor:"19",
GitVersion:"v1.19.3",
GitCommit:"1e11e4a2108024935ecfcb2912226cedeafd99df",
GitTreeState:"clean", BuildDate:"2020-10-14T12:41:49Z",
GoVersion:"go1.15.2", Compiler:"gc", Platform:"linux/amd64"}
```

- 集群中的节点应具有可引入拓扑感知的标签（`topology.Kubernetes IO/` 区域 和 `topology.Kubernetes IO/` 区域 ）。在安装 Astra Trident 之前，集群中的节点上应存在这些标签 *，以使 Astra Trident 能够识别拓扑。

```
$ kubectl get nodes -o=jsonpath='{range .items[*]}[{.metadata.name},
{.metadata.labels}]{"\n"}{end}' | grep --color "topology.kubernetes.io"
[node1,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node1","kubernetes.io/
os":"linux","node-
role.kubernetes.io/master":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-a"}]
[node2,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node2","kubernetes.io/
os":"linux","node-
role.kubernetes.io/worker":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-b"}]
[node3,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node3","kubernetes.io/
os":"linux","node-
role.kubernetes.io/worker":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-c"}]
```

第 **1** 步：创建可感知拓扑的后端

可以设计 Astra Trident 存储后端，以便根据可用性区域有选择地配置卷。每个后端都可以包含一个可选的

`supportedTopatis` 块，该块表示必须支持的分区和区域的列表。对于使用此后端的 StorageClasses ，只有在受支持区域 / 区域中计划的应用程序请求时，才会创建卷。

下面是后端定义示例：

```
{
 "version": 1,
 "storageDriverName": "ontap-san",
 "backendName": "san-backend-us-east1",
 "managementLIF": "192.168.27.5",
 "svm": "iscsi_svm",
 "username": "admin",
 "password": "xxxxxxxxxxxx",
 "supportedTopologies": [
{"topology.kubernetes.io/region": "us-east1",
"topology.kubernetes.io/zone": "us-east1-a"},
{"topology.kubernetes.io/region": "us-east1",
"topology.kubernetes.io/zone": "us-east1-b"}
]
}
```

> `supportedTopologies` 用于提供每个后端的区域和分区列表。这些区域和分区表示可在 StorageClass 中提供的允许值列表。对于包含后端提供的部分区域和分区的 StorageClasses ，Astra Trident 将在后端创建卷。

您也可以为每个存储池定义 `supportedTopologies` 。请参见以下示例：

```
{"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "nas-backend-us-central1",
"managementLIF": "172.16.238.5",
"svm": "nfs_svm",
"username": "admin",
"password": "Netapp123",
"supportedTopologies": [
      {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-a"},
      {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-b"}
    ]
"storage": [
   {
       "labels": {"workload":"production"},
        "region": "Iowa-DC",
        "zone": "Iowa-DC-A",
        "supportedTopologies": [
            {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-a"}
        ]
    },
    {
        "labels": {"workload":"dev"},
         "region": "Iowa-DC",
         "zone": "Iowa-DC-B",
         "supportedTopologies": [
             {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-b"}
         ]
     }
]
}
```

在此示例中， `reGion` 和 `zone` 标签表示存储池的位置。`topology.Kubernees.io/zone` 和 `topology.Kubernees.io/zone` 指定存储池的使用位置。

第 **2** 步：定义可识别拓扑的 **StorageClasses**

根据为集群中的节点提供的拓扑标签，可以将 StorageClasses 定义为包含拓扑信息。这将确定用作 PVC 请求候选对象的存储池，以及可使用 Trident 配置的卷的节点子集。

请参见以下示例：

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: netapp-san-us-east1
provisioner: csi.trident.netapp.io
volumeBindingMode: WaitForFirstConsumer
allowedTopologies:
- matchLabelExpressions:
- key: topology.kubernetes.io/zone
  values:
  - us-east1-a
  - us-east1-b
- key: topology.kubernetes.io/region
  values:
  - us-east1
parameters:
  fsType: "ext4"
```

在上述 StorageClass 定义中， `volumeBindingMode` 设置为 `WaitForFirstConsumer"` 。在此存储类中请求的 PVC 在 Pod 中引用之前不会执行操作。此外， `allowedTopologies` 还提供了要使用的分区和区域。`NetApp-san-us-East1` StorageClass 将在上述 `san-backend-us-East1` 后端创建 PVC 。

第 **3** 步：创建和使用 **PVC**

创建 StorageClass 并将其映射到后端后，您现在可以创建 PVC 。

请参见以下示例 `sPec`：

```
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
name: pvc-san
spec:
accessModes:
  - ReadWriteOnce
resources:
  requests:
    storage: 300Mi
storageClassName: netapp-san-us-east1
```

使用此清单创建 PVC 将导致以下结果：

```
$ kubectl create -f pvc.yaml
persistentvolumeclaim/pvc-san created
$ kubectl get pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
AGE
pvc-san   Pending                                      netapp-san-us-east1
2s
$ kubectl describe pvc
Name:          pvc-san
Namespace:     default
StorageClass:  netapp-san-us-east1
Status:        Pending
Volume:
Labels:        <none>
Annotations:   <none>
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:
Access Modes:
VolumeMode:    Filesystem
Mounted By:    <none>
Events:
  Type    Reason                Age   From                         Message
  ----    ------                ----  ----                         -------
  Normal  WaitForFirstConsumer  6s    persistentvolume-controller  waiting
for first consumer to be created before binding
```

要使 Trident 创建卷并将其绑定到 PVC ，请在 Pod 中使用 PVC 。请参见以下示例：

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod-1
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: topology.kubernetes.io/region
            operator: In
            values:
            - us-east1
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: topology.kubernetes.io/zone
            operator: In
            values:
            - us-east1-a
            - us-east1-b
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
  volumes:
  - name: vol1
    persistentVolumeClaim:
      claimName: pvc-san
  containers:
  - name: sec-ctx-demo
    image: busybox
    command: [ "sh", "-c", "sleep 1h" ]
    volumeMounts:
    - name: vol1
      mountPath: /data/demo
    securityContext:
      allowPrivilegeEscalation: false
```

此 podSpec 指示 Kubernetes 在 `us-East1` 区域中的节点上计划 Pod ，并从 `us-East1-a` 或 `us-East1-b` 区域中的任何节点中进行选择。

请参见以下输出：

```
$ kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE   IP                NODE
NOMINATED NODE   READINESS GATES
app-pod-1   1/1     Running   0          19s   192.168.25.131   node2
<none>           <none>
$ kubectl get pvc -o wide
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS          AGE   VOLUMEMODE
pvc-san   Bound    pvc-ecb1e1a0-840c-463b-8b65-b3d033e2e62b   300Mi
RWO            netapp-san-us-east1   48s   Filesystem
```

更新后端以包括支持的拓扑

可以使用 `tridentctl backend update` 更新原有后端，以包含 `supportedTopologies` 列表。这不会影响已配置的卷，并且仅用于后续的 PVC 。

了解更多信息

- "管理容器的资源"
- "节点选择器"
- "关联性和反关联性"
- "损害和公差"

使用快照

从 Astra Trident 的 20.01 版开始，您可以在 Kubernetes 层创建 PV 快照。您可以使用这些快照维护由 Astra Trident 创建的卷的时间点副本，并计划创建其他卷（克隆）。卷快照受 `ontap-nas` ， `ontap-san` ， `ontap-san-economics` ， `solidfire-san` ， `GCP-CVS` 支持。 和 `azure-netapp-files` 驱动程序。

> 此功能可从 Kubernetes 1.17 （测试版）获得，从 1.20 开始正式上市。要了解从测试版迁移到 GA 所涉及的变化，请参见 "发布博客"。升级到 GA 后，将推出 `v1` API 版本，并向后兼容 `v1bea1` 快照。

您需要什么？ **#8217** ；将需要什么

- 创建卷快照需要创建外部快照控制器和自定义资源定义(Custom Resource Definitions、CRD)。这是所使用的Kubernetes流程编排程序(例如：Kubeadm、GKE、OpenShift)的责任。

如果您的Kubernetes分发版不包含快照控制器和CRD、则可以按如下所示进行部署。

1. 创建卷快照CRD。

   对于Kubernetes 1.20及更高版本、请使用v5.0或更高版本的Snapshot组件的v1 Snapshot CRD。对于Kubernetes 1.18和1.19版、请将v1bea1与v3.0.3 Snapshot组件结合使用。

5.0版组件

```
$ cat snapshot-setup.sh
#!/bin/bash
# Create volume snapshot CRDs
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotclasses.
yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotcontents
.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshots.yaml
```

v3.0.3组件

```
$ cat snapshot-setup.sh
#!/bin/bash
# Create volume snapshot CRDs
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshotclasses.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshotcontents.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshots.yaml
```

2. 在所需命名空间中创建Snapshot控制器。编辑以下 YAML 清单以修改命名空间。

   对于Kubernetes 1.20及更高版本、请使用5.0或更高版本。对于Kubernetes 1.18和1.19版、请使用v3.0.3

   > 如果在GKE-环境中设置按需卷快照、请勿创建快照控制器。GKE- 使用内置的隐藏快照控制器。

5.0版控制器

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/release-5.0/deploy/kubernetes/snapshot-controller/rbac-snapshot-controller.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/release-5.0/deploy/kubernetes/snapshot-controller/setup-snapshot-controller.yaml
```

v3.0.3控制器

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v3.0.3/deploy/kubernetes/snapshot-controller/rbac-snapshot-controller.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v3.0.3/deploy/kubernetes/snapshot-controller/setup-snapshot-controller.yaml
```

CSI Snapshotter 提供了 "正在验证 webhook" 帮助用户验证现有 v1bea1 快照并确认它们是有效的资源对象。正在验证的 webhook 会自动标记无效的快照对象，并防止将来创建无效对象。验证 webhook 由 Kubernetes 流程编排程序部署。请参见有关手动部署验证 webhook 的说明 "此处"。查找无效快照清单的示例 "此处"。

下面详细介绍的示例说明了使用快照所需的构造，并说明了如何创建和使用快照。

第 **1** 步：设置 **VolumeSnapshotClass**

在创建卷快照之前，请设置 “VolumeSnapshotClass”。

```
$ cat snap-sc.yaml
#Use apiVersion v1 for Kubernetes 1.20 and above. For Kubernetes 1.18 and 1.19, use apiVersion v1beta1.
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-snapclass
driver: csi.trident.netapp.io
deletionPolicy: Delete
```

d河 指向 Astra Trident 的 CSI 驱动程序。`deeltionPolicy` 可以是 `Delete` 或 `Retain` 。如果设置为 `Retain` ，则即使删除了 `VolumeSnapshot` 对象，存储集群上的底层物理快照也会保留。

第 **2** 步：创建现有 **PVC** 的快照

```
$ cat snap.yaml
#Use apiVersion v1 for Kubernetes 1.20 and above. For Kubernetes 1.18 and
1.19, use apiVersion v1beta1.
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: pvc1-snap
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: pvc1
```

正在为名为 `PVC1` 的 PVC 创建快照，并且快照名称设置为 `PVC1-Snap` 。

```
$ kubectl create -f snap.yaml
volumesnapshot.snapshot.storage.k8s.io/pvc1-snap created

$ kubectl get volumesnapshots
NAME                   AGE
pvc1-snap              50s
```

这样就创建了一个 `VolumeSnapshot` 对象。VolumeSnapshot 类似于 PVC ，并与代表实际快照的 `VolumeSnapshotContent` 对象关联。

可以通过描述来标识 `PVC1-Snap` VolumeSnapshot 的 `VolumeSnapshotContent` 对象。

```
$ kubectl describe volumesnapshots pvc1-snap
Name:         pvc1-snap
Namespace:    default
.
.
.
Spec:
  Snapshot Class Name:    pvc1-snap
  Snapshot Content Name:  snapcontent-e8d8a0ca-9826-11e9-9807-525400f3f660
  Source:
    API Group:
    Kind:       PersistentVolumeClaim
    Name:       pvc1
Status:
  Creation Time:  2019-06-26T15:27:29Z
  Ready To Use:   true
  Restore Size:   3Gi
.
.
```

`Snapshot Content Name` 用于标识提供此快照的 VolumeSnapshotContent 对象。`Ready to Use` 参数表示可使用 Snapshot 创建新的 PVC 。

第 3 步：从 VolumeSnapshots 创建 PVC

有关使用快照创建 PVC 的示例，请参见以下示例：

```
$ cat pvc-from-snap.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-from-snap
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: golden
  resources:
    requests:
      storage: 3Gi
  dataSource:
    name: pvc1-snap
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
```

`dataSource` 显示必须使用名为 `PVC1-Snap` 的 VolumeSnapshot 作为数据源来创建 PVC 。此操作将指示 Astra Trident 从快照创建 PVC 。创建 PVC 后，可以将其附加到 Pod 上，并像使用任何其他 PVC 一样使用。

> 删除具有关联快照的永久性卷时，相应的 Trident 卷将更新为 " 正在删除 " 状态。要删除 Astra Trident 卷，应删除该卷的快照。

了解更多信息

- "卷快照"
- “VolumeSnapshotClass”

展开卷

通过 Astra Trident ， Kubernetes 用户可以在创建卷后对其进行扩展。查找有关扩展 iSCSI 和 NFS 卷所需配置的信息。

展开 **iSCSI** 卷

您可以使用 CSI 配置程序扩展 iSCSI 永久性卷（ PV ）。

> iSCSI 卷扩展受 `ontap-san` ， `ontap-san-economy` ， `solidfire-san` 驱动程序支持，需要 Kubernetes 1.16 及更高版本。

扩展 iSCSI PV 包括以下步骤：

- 编辑 StorageClass 定义以将 `allowVolumeExpansion` 字段设置为 `true` 。
- 编辑 PVC 定义并更新 `sPec.resources.requests.storage` 以反映新需要的大小，该大小必须大于原始大小。
- 要调整 PV 大小，必须将 PV 连接到 Pod 。调整 iSCSI PV 大小时，有两种情况：
  - 如果 PV 连接到 Pod ，则 Astra Trident 会扩展存储后端的卷，重新扫描设备并调整文件系统大小。
  - 尝试调整未连接 PV 的大小时， Astra Trident 会扩展存储后端的卷。将 PVC 绑定到 Pod 后， Trident 会重新扫描设备并调整文件系统大小。然后， Kubernetes 会在扩展操作成功完成后更新 PVC 大小。

以下示例显示了扩展 iSCSI PV 的工作原理。

第 **1** 步：配置 **StorageClass** 以支持卷扩展

```
$ cat storageclass-ontapsan.yaml
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-san
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-san"
allowVolumeExpansion: True
```

对于已存在的 StorageClass ，请对其进行编辑，使其包含 `allowVolumeExpansion` 参数。

第 2 步：使用您创建的 StorageClass 创建 PVC

```
$ cat pvc-ontapsan.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: san-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: ontap-san
```

Astra Trident 会创建一个永久性卷（ PV ）并将其与此永久性卷声明（ PVC ）关联。

```
$ kubectl get pvc
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
san-pvc   Bound    pvc-8a814d62-bd58-4253-b0d1-82f2885db671   1Gi
RWO            ontap-san      8s

$ kubectl get pv
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM             STORAGECLASS   REASON   AGE
pvc-8a814d62-bd58-4253-b0d1-82f2885db671   1Gi        RWO
Delete           Bound    default/san-pvc   ontap-san               10s
```

第 3 步：定义连接 PVC 的 POD

在此示例中，创建了一个使用 `san-PVC` 的 Pod 。

```
$  kubectl get pod
NAME         READY   STATUS    RESTARTS   AGE
centos-pod   1/1     Running   0          65s

$  kubectl describe pvc san-pvc
Name:          san-pvc
Namespace:     default
StorageClass:  ontap-san
Status:        Bound
Volume:        pvc-8a814d62-bd58-4253-b0d1-82f2885db671
Labels:        <none>
Annotations:   pv.kubernetes.io/bind-completed: yes
               pv.kubernetes.io/bound-by-controller: yes
               volume.beta.kubernetes.io/storage-provisioner:
csi.trident.netapp.io
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:      1Gi
Access Modes:  RWO
VolumeMode:    Filesystem
Mounted By:    centos-pod
```

第 4 步：展开 PV

要将已创建的 PV 从 1Gi 调整为 2Gi ，请编辑 PVC 定义并将 `sPec.resources.requests.storage` 更新为 2Gi 。

```
$ kubectl edit pvc san-pvc
# Please edit the object below. Lines beginning with a '#' will be
ignored,
# and an empty file will abort the edit. If an error occurs while saving
this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  annotations:
    pv.kubernetes.io/bind-completed: "yes"
    pv.kubernetes.io/bound-by-controller: "yes"
    volume.beta.kubernetes.io/storage-provisioner: csi.trident.netapp.io
  creationTimestamp: "2019-10-10T17:32:29Z"
  finalizers:
  - kubernetes.io/pvc-protection
  name: san-pvc
  namespace: default
  resourceVersion: "16609"
  selfLink: /api/v1/namespaces/default/persistentvolumeclaims/san-pvc
  uid: 8a814d62-bd58-4253-b0d1-82f2885db671
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
 ...
```

第 **5** 步：验证扩展

您可以通过检查 PVC ， PV 和 Astra Trident 卷的大小来验证扩展是否正常运行：

```
$ kubectl get pvc san-pvc
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
san-pvc   Bound    pvc-8a814d62-bd58-4253-b0d1-82f2885db671   2Gi
RWO            ontap-san      11m
$ kubectl get pv
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM             STORAGECLASS   REASON   AGE
pvc-8a814d62-bd58-4253-b0d1-82f2885db671   2Gi        RWO
Delete           Bound    default/san-pvc   ontap-san               12m
$ tridentctl get volumes -n trident
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-8a814d62-bd58-4253-b0d1-82f2885db671 | 2.0 GiB | ontap-san     |
block    | a9b7bfff-0505-4e31-b6c5-59f492e02d33 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

展开 **NFS** 卷

Astra Trident 支持对在 `ontap-nas` ， `ontap-nas-economy` ， `ontap-nas-flexgroup` ， `GCP-CVS` 和 `azure-netapp-files` 后端配置的 NFS PV 进行卷扩展。

第 **1** 步：配置 **StorageClass** 以支持卷扩展

要调整 NFS PV 的大小，管理员首先需要将 `allowVolumeExpansion` 字段设置为 `true` 来配置存储类以允许卷扩展：

```
$ cat storageclass-ontapnas.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontapnas
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
allowVolumeExpansion: true
```

如果您已创建没有此选项的存储类，则只需使用 `kubectl edit storageclass` 编辑现有存储类即可进行卷扩展。

第 **2** 步：使用您创建的 **StorageClass** 创建 **PVC**

```
$ cat pvc-ontapnas.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: ontapnas20mb
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 20Mi
  storageClassName: ontapnas
```

Astra Trident 应为此 PVC 创建一个 20 MiB NFS PV：

```
$ kubectl get pvc
NAME           STATUS   VOLUME
CAPACITY     ACCESS MODES   STORAGECLASS    AGE
ontapnas20mb   Bound    pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   20Mi
RWO            ontapnas        9s

$ kubectl get pv pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM                  STORAGECLASS    REASON
AGE
pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   20Mi       RWO
Delete           Bound    default/ontapnas20mb   ontapnas
2m42s
```

第 **3** 步：展开 **PV**

要将新创建的 20MiB PV 调整为 1GiB ，请编辑 PVC 并将 `sPec.resources.requests.storage` 设置为 1GB：

```
$ kubectl edit pvc ontapnas20mb
# Please edit the object below. Lines beginning with a '#' will be
ignored,
# and an empty file will abort the edit. If an error occurs while saving
this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  annotations:
    pv.kubernetes.io/bind-completed: "yes"
    pv.kubernetes.io/bound-by-controller: "yes"
    volume.beta.kubernetes.io/storage-provisioner: csi.trident.netapp.io
  creationTimestamp: 2018-08-21T18:26:44Z
  finalizers:
  - kubernetes.io/pvc-protection
  name: ontapnas20mb
  namespace: default
  resourceVersion: "1958015"
  selfLink: /api/v1/namespaces/default/persistentvolumeclaims/ontapnas20mb
  uid: c1bd7fa5-a56f-11e8-b8d7-fa163e59eaab
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
...
```

第 **4** 步：验证扩展

您可以通过检查 PVC ， PV 和 Astra Trident 卷的大小来验证调整大小是否正常工作：

```
$ kubectl get pvc ontapnas20mb
NAME           STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS    AGE
ontapnas20mb   Bound    pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   1Gi
RWO            ontapnas        4m44s

$ kubectl get pv pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM                  STORAGECLASS    REASON
AGE
pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   1Gi        RWO
Delete           Bound    default/ontapnas20mb   ontapnas
5m35s

$ tridentctl get volume pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7 -n
trident
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7 | 1.0 GiB | ontapnas      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

导入卷

您可以使用 `tridentctl import` 将现有存储卷作为 Kubernetes PV 导入。

支持卷导入的驱动程序

下表介绍了支持导入卷的驱动程序及其引入的版本。

驱动程序	版本。
`ontap-NAS`	19.04
`ontap-nas-flexgroup`	19.04
`solidfire-san`	19.04
`azure-netapp-files`	19.04

驱动程序	版本。
GCP-CVS	19.04
ontap-san	19.04

为什么应导入卷?

将卷导入到 Trident 的使用情形有多种：

- 对应用程序进行容器化并重复使用其现有数据集
- 为临时应用程序使用数据集的克隆
- 重建发生故障的 Kubernetes 集群
- 在灾难恢复期间迁移应用程序数据

导入的工作原理是什么?

卷导入过程使用永久性卷声明（ PVC ）文件创建 PVC 。PVC 文件应至少包含 name ， namespace ， accessModes 和 storageClassName 字段，如以下示例所示。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: my_claim
  namespace: my_namespace
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: my_storage_class
```

使用 `tridentctl` 客户端导入现有存储卷。Trident 通过保留卷元数据并创建 PVC 和 PV 来导入卷。

```
$ tridentctl import volume <backendName> <volumeName> -f <path-to-pvc-
file>
```

要导入存储卷，请指定包含该卷的 Astra Trident 后端的名称以及用于唯一标识存储上的卷的名称（例如：ONTAP FlexVol ， Element Volume ， CVS 卷路径）。存储卷必须允许读 / 写访问，并可由指定的 Astra Trident 后端访问。需要 ` -f` 字符串参数，并指定 YAML 或 JSON PVC 文件的路径。

当 Astra Trident 收到导入卷请求时，系统会在 PVC 中确定并设置现有卷大小。存储驱动程序导入卷后，系统将创建 PV ，并为其创建一个 Claims Ref 。在 PV 中，回收策略最初设置为 `retain` 。Kubernetes 成功绑定 PVC 和 PV 后，将更新回收策略以匹配存储类的回收策略。如果存储类的回收策略为 `delete` ，则在删除 PV 时，存储卷将被删除。

使用 ` -no-manage` 参数导入卷时， Trident 不会在对象的生命周期内对 PVC 或 PV 执行任何其他操作。由于

Trident 会忽略 `-no-manage` 对象的 PV 和 PVC 事件，因此删除 PV 时不会删除存储卷。卷克隆和卷大小调整等其他操作也会被忽略。如果要对容器化工作负载使用 Kubernetes ，但希望在 Kubernetes 外部管理存储卷的生命周期，则此选项非常有用。

PVC 和 PV 中会添加一个标注，用于指示卷已导入以及 PVC 和 PV 是否已管理。不应修改或删除此标注。

Trident 19.07 及更高版本可处理 PV 的连接，并在导入卷时挂载该卷。对于使用早期版本的 Astra Trident 进行的导入，数据路径中不会执行任何操作，卷导入将不会验证是否可以挂载卷。如果卷导入出错（例如 StorageClass 不正确），您可以通过将 PV 上的回收策略更改为 `retain` ，删除 PVC 和 PV 并重试 volume import 命令来恢复。

ontap-nas 和 ontap-nas-flexgroup 导入

使用 `ontap-nas` 驱动程序创建的每个卷都是 ONTAP 集群上的一个 FlexVol 。使用 `ontap-NAS` 驱动程序导入 FlexVol 的工作原理相同。ONTAP 集群上已存在的 FlexVol 可以导入为 `ONONTAP -NAS` PVC 。同样，可以将 FlexGroup vols 导入为 `ontap-nas-flexgroup` PVC 。

> 要由 Trident 导入 ONTAP 卷，必须为 rw 类型。如果卷的类型为 DP ，则为 SnapMirror 目标卷；应先中断镜像关系，然后再将卷导入到 Trident 中。

> `ontap-NAS` 驱动程序无法导入和管理 qtree 。`ontap-nas` 和 `ontap-nas-flexgroup` 驱动程序不允许使用重复的卷名称。

例如，要在名为 `ontap_NAS` 的后端导入名为 `manage_volume` 的卷，请使用以下命令：

```
$ tridentctl import volume ontap_nas managed_volume -f <path-to-pvc-file>

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-bf5ad463-afbb-11e9-8d9f-5254004dfdb7 | 1.0 GiB | standard      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

要导入名为 `unmanageed_volume` 的卷（位于 `ontap_NAS` 后端 上），而 Trident 不会管理该卷，请使用以下命令：

```
$ tridentctl import volume nas_blog unmanaged_volume -f <path-to-pvc-file>
--no-manage

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-df07d542-afbc-11e9-8d9f-5254004dfdb7 | 1.0 GiB | standard      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | false   |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

使用 ` -no-manage` 参数时， Trident 不会重命名卷或验证卷是否已挂载。如果卷未手动挂载，则卷导入操作将失败。

> 先前存在的使用自定义 UnixPermissions 导入卷的错误已得到修复。您可以在 PVC 定义或后端配置中指定 unixPermissions ，并指示 Astra Trident 相应地导入卷。

ontap-san 导入

Astra Trident 还可以导入包含单个 LUN 的 ONTAP SAN FlexVol 。这与 `ontap-san` 驱动程序一致，该驱动程序会为 FlexVol 中的每个 PVC 和 LUN 创建一个 FlexVol 。您可以像在其他情况下一样使用 `tridentctl import` 命令：

- 请输入 `ontap-san` 后端的名称。
- 提供需要导入的 FlexVol 的名称。请记住，此 FlexVol 仅包含一个必须导入的 LUN 。
- 提供必须与 ` -f` 标志结合使用的 PVC 定义路径。
- 可以选择对 PVC 进行管理，也可以选择不对其进行管理。默认情况下， Trident 将管理 PVC 并重命名后端的 FlexVol 和 LUN 。要作为非受管卷导入，请传递 ` -no-manage` 标志。

> 导入非受管 `ontap-san` 卷时，应确保 FlexVol 中的 LUN 名为 `lun0` ，并已映射到具有所需启动程序的 igroup 。Astra Trident 会自动为受管导入处理此问题。

然后， Astra Trident 将导入 FlexVol 并将其与 PVC 定义关联。Astra Trident 还会将 FlexVol 重命名为 `vc-<uid>` 格式，并将 FlexVol 中的 LUN 重命名为 `lun0` 。

> 建议导入没有活动连接的卷。如果要导入当前使用的卷，请先克隆该卷，然后再执行导入。

示例

要导入 `ontap_san_default` 后端上存在的 `ontap-san-managed` FlexVol ，请运行 `tridentctl import` 命令：

```
$ tridentctl import volume ontapsan_san_default ontap-san-managed -f pvc-
basic-import.yaml -n trident -d

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-d6ee4f54-4e40-4454-92fd-d00fc228d74a | 20 MiB | basic         |
block    | cd394786-ddd5-4470-adc3-10c5ce4ca757 | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

ONTAP 卷的类型必须为 RW ，才能由 Astra Trident 导入。如果卷的类型为 DP ，则为 SnapMirror 目标卷；在将卷导入到 Astra Trident 之前，应中断镜像关系。

元素导入

您可以使用 Trident 将 NetApp Element 软件 /NetApp HCI 卷导入到 Kubernetes 集群中。您需要提供 Astra Trident 后端的名称以及卷和 PVC 文件的唯一名称作为 `tridentctl import` 命令的参数。

```
$ tridentctl import volume element_default element-managed -f pvc-basic-
import.yaml -n trident -d

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-970ce1ca-2096-4ecd-8545-ac7edc24a8fe | 10 GiB | basic-element |
block    | d3ba047a-ea0b-43f9-9c42-e38e58301c49 | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

Element 驱动程序支持重复的卷名称。如果卷名称重复，则 Trident 的卷导入过程将返回错误。作为临时解决策，克隆卷并提供唯一的卷名称。然后导入克隆的卷。

GCP-CVS 导入

要在 GCP 中导入由 NetApp Cloud Volumes Service 支持的卷，请按卷路径而非名称来标识该卷。

要在后端导入名为 gcpcvs_yEppr 的 GCP-CVS 卷，并将卷路径设置为 adrot-joly-swift ，请使用以下命令：

```
$ tridentctl import volume gcpcvs_YEppr adroit-jolly-swift -f <path-to-
pvc-file> -n trident

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-a46ccab7-44aa-4433-94b1-e47fc8c0fa55 | 93 GiB | gcp-storage   | file
| e1a6e65b-299e-4568-ad05-4f0a105c888f | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

> 卷路径是卷导出路径中： / 之后的部分。例如，如果导出路径为 10.0.0.1 ： /adrot-joly-swift ，则卷路径为 adrot-joly-swift 。

Azure-netapp-files 导入

要在后端导入 azure-netapp-files 卷，该卷名为 azurenetappfiles_40517 ，卷路径为 importvol1 ，请运行以下命令：

```
$ tridentctl import volume azurenetappfiles_40517 importvol1 -f <path-to-
pvc-file> -n trident

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-0ee95d60-fd5c-448d-b505-b72901b3a4ab | 100 GiB | anf-storage   |
file     | 1c01274f-d94b-44a3-98a3-04c953c9a51e | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

> ANF 卷的卷路径位于： / 之后的挂载路径中。例如，如果挂载路径为 10.0.0.2 ： /importvol1 ，则卷路径为 importvol1 。

准备工作节点

Kubernetes 集群中的所有工作节点都需要能够挂载为 Pod 配置的卷。如果您在其中一个后端使用 `ontap-nas`， `ontap-nas-economy` 或 `ontap-nas-flexgroup` 驱动程序，则您的工作节点需要 NFS 工具。否则，它们需要使用 iSCSI 工具。

默认情况下，最新版本的 RedHat CoreOS 同时安装了 NFS 和 iSCSI 。

安装 NFS 或 iSCSI 工具后，您应始终重新启动工作节点，否则将卷连接到容器可能会失败。

NFS volumes

协议	操作系统	命令
NFS	RHEL/CentOS	`sudo yum install -y nfs-utils`
NFS	Ubuntu 或 Debian	`sudo apt-get install -y nfs-common`

您应确保 NFS 服务在启动期间启动。

iSCSI 卷

使用 iSCSI 卷时，请考虑以下事项：

- Kubernetes 集群中的每个节点都必须具有唯一的 IQN 。* 这是必要的前提条件 * 。
- 如果将 RHCOS 4.5 或更高版本，或者将 RHEL 或 CentOS 8.2 或更高版本与 `solidfire-san` 驱动程序结合使用，请确保在 ` /etc/iscsi/iscsid.conf` 中将 CHAP 身份验证算法设置为 MD5 。

```
sudo sed -i 's/^\(node.session.auth.chap_algs\).*/\1 = MD5/'
/etc/iscsi/iscsid.conf
```

- 使用运行 RHEL/RedHat CoreOS 和 iSCSI PV 的工作节点时，请确保在 StorageClass 中指定 `discard` mountOption 以执行实时空间回收。请参见 "RedHat 的文档"。

协议	操作系统	命令

iSCSI	RHEL/CentOS	1. 安装以下系统软件包：
		
		`sudo yum install -y lsscsi iscsi-initiator-utils sg3_utils device-mapper-multipath`
		
		2. 检查 iscsi-initiator-utils 版本是否为 6.2.0.877-2.el7 或更高版本：
		
		`rpm -q iscsi-initiator-utils`
		
		3. 将扫描设置为手动：
		
		`sUdo sed -i 's/^ \ ( node.session.scan\).*/\1 = manual/' /etc/iscsi/iscsid.conf`
		
		4. 启用多路径：
		
		`sudo mpathconf -enable -for_multipathd y -find_multipaths n`
		
		ⓘ 确保 `detc/multipath.conf` contains `find_multipaths no` under` efaults `.
		
		5. 确保 `iscsid` 和 `multipathd` 正在运行：
		
		`sudo systemctl enable -now iscsid multipathd`
		
		6. 启用并启动 `iSCSI`：
		
		`sudo systemctl enable —` 现在为 `iSCSI`

协议	操作系统	命令

iSCSI	Ubuntu 或 Debian	1. 安装以下系统软件包：
		
		`sUdo apt-get install -y open-iscsi lsscsi sg3-utils multipath-tools scsitool`
		
		2. 检查 open-iscsi 版本是否为 2.0.877-5ubuntu2.10 或更高版本（对于双子系统）或 2.0.877-7.1ubuntu6.1 或更高版本（对于 Focal ）：
		
		`dpkg -l open-iscsi`
		
		3. 将扫描设置为手动：
		
		`sUdo sed -i 's/^ \ (node.session.scan\).*/\1 = manual/' /etc/iscsi/iscsid.conf`
		
		4. 启用多路径：
		
		`sUdo tee /etc/multipath.conf <←'EOF' defaults {user_friendly_names yes find_multipaths no } eOF sudo systemctl enable -now multipath-tools.service sudo service multipath-tools restart`
		
		ⓘ 确保 `detc/multipath.conf` contains `find_multipaths no` under` efaults `.
		
		5. 确保已启用并运行 `open-iscsi` 和 `multipath-tools` ：
		
		`sudo systemctl status multipath-tools sudo systemctl enable -now open-iscsi.service sudo systemctl status open-iscsi`

> 对于 Ubuntu 18.04 ，您必须先使用 `iscsiadm` 发现目标端口，然后再启动 `open-iscsi` ，iSCSI 守护进程才能启动。您也可以将 `iscsi` 服务修改为自动启动 `iscsid` 。

> 如果您希望了解有关自动员工节点准备的更多信息，这是一项测试功能，请参见 "此处"。

自动工作节点准备

Astra Trident 可以在 Kubernetes 集群中的节点上自动安装所需的 `NFS` 和 `iSCSI` 工具。这是一项 * 测试版功能 * ， * 不适用于 * 生产集群。目前，该功能可用于运行 * CentOS ， RHEL 和 Ubuntu * 的节点。

对于此功能， Astra Trident 提供了一个新的安装标志： ` -enable-node-prep` for installations deployed with `tridentctl` 。对于使用 Trident 运算符的部署，请使用布尔选项 `enableNodePrep` 。

> ` -enable-node-prep` 安装选项指示 Astra Trident 安装并确保在工作节点上挂载卷时 NFS 和 iSCSI 软件包和 / 或服务正在运行。这是一项 * 测试版功能 * ，用于 * 不符合生产条件 * 的开发 / 测试环境。

如果使用 `tridentctl` 部署的 Astra Trident 安装包含 ` -enable-node-prep` 标志，则会发生以下情况：

1. 在安装过程中， Astra Trident 会注册其运行所在的节点。
2. 发出永久性卷请求（ PVC ）请求后， Astra Trident 会从其管理的后端创建 PV 。
3. 在 Pod 中使用 PVC 需要使用 Astra Trident 在 Pod 运行的节点上挂载卷。Astra Trident 会尝试安装所需的 NFS/iSCSI 客户端实用程序，并确保所需服务处于活动状态。此操作会在挂载卷之前完成。

在首次尝试挂载卷时，只会对工作节点进行一次准备。只要 Astra Trident 之外的任何更改均未触及 `nfs` 和 `iscsi` 实用程序，所有后续卷挂载都应成功。

通过这种方式， Astra Trident 可以确保 Kubernetes 集群中的所有节点都具有挂载和连接卷所需的实用程序。对于 NFS 卷，导出策略还应允许挂载该卷。Trident 可以自动管理每个后端的导出策略；用户也可以管理带外导出策略。

监控 Astra Trident

Astra Trident 提供了一组 Prometheus 指标端点，您可以使用这些端点监控 Astra Trident 的性能。

通过 Astra Trident 提供的指标，您可以执行以下操作：

- 保留有关 Astra Trident 运行状况和配置的选项卡。您可以检查操作的成功程度以及它是否能够按预期与后端进行通信。
- 检查后端使用情况信息，并了解在后端配置的卷数量以及占用的空间量等。
- 维护可用后端配置的卷数量的映射关系。
- 跟踪性能。您可以了解 Astra Trident 与后端通信并执行操作所需的时间。

> 默认情况下， Trident 的指标会显示在 `/metrics` 端点的目标端口` 8001 ` 上。安装 Trident 时，这些指标默认为 * 已启用 * 。

您需要什么？ **#8217**；将需要什么

- 安装了 Astra Trident 的 Kubernetes 集群。
- 一个 Prometheus 实例。可以是 "容器化 Prometheus 部署" 或者，您也可以选择将 Prometheus 作为运行 "原生应用程序"。

第 1 步：定义 Prometheus 目标

您应定义一个 Prometheus 目标以收集指标并获取有关后端 Astra Trident 管理的信息，它创建的卷等。这 "博客" 介绍如何将 Prometheus 和 Grafana 与 Astra Trident 结合使用来检索指标。博客介绍了如何在 Kubernetes 集群中以操作员身份运行 Prometheus ，以及如何创建 ServiceMonitor 来获取 Astra Trident 的指标。

第 2 步：创建 Prometheus ServiceMonitor

要使用 Trident 指标，您应创建一个 Prometheus ServiceMonitor ，该监控器可监控 `trident CSI` 服务并侦听 `mtrigics` 端口。示例 ServiceMonitor 如下所示：

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: trident-sm
  namespace: monitoring
  labels:
      release: prom-operator
  spec:
    jobLabel: trident
    selector:
      matchLabels:
        app: controller.csi.trident.netapp.io
    namespaceSelector:
      matchNames:
      - trident
    endpoints:
    - port: metrics
      interval: 15s
```

此 ServiceMonitor 定义会检索 `trident CSI` 服务返回的指标，并专门查找服务的 `mtrics` 端点。因此，Prometheus 现在已配置为了解 Astra Trident 的指标。

除了直接从 Astra Trident 获得的指标之外， kubelet 还通过自己的指标端点公开了许多 `kubelet_volume_*` 指标。Kubelet 可以提供有关已连接的卷， Pod 及其处理的其他内部操作的信息。请参见 "此处"。

第 3 步：使用 PromQL 查询 Trident 指标

PromQL 非常适合创建返回时间序列或表格数据的表达式。

您可以使用以下 PromQL 查询：

获取 **Trident** 运行状况信息

- 来自 Astra Trident 的 HTTP 2XX 响应的百分比

```
(sum (trident_rest_ops_seconds_total_count{status_code=~"2.."} OR on()
vector(0)) / sum (trident_rest_ops_seconds_total_count)) * 100
```

- 通过状态代码来自 Astra Trident 的 REST 响应的百分比

```
(sum (trident_rest_ops_seconds_total_count) by (status_code)  / scalar
(sum (trident_rest_ops_seconds_total_count))) * 100
```

- * 由 Astra Trident 执行的操作的平均持续时间（毫秒） *

```
sum by (operation)
(trident_operation_duration_milliseconds_sum{success="true"}) / sum by
(operation)
(trident_operation_duration_milliseconds_count{success="true"})
```

获取 **Astra Trident** 使用信息

- 卷大小 * 平均值 *

```
trident_volume_allocated_bytes/trident_volume_count
```

- * 每个后端配置的卷总空间 *

```
sum (trident_volume_allocated_bytes) by (backend_uuid)
```

获取单个卷的使用情况

> 只有在同时收集 kubelet 指标时，才会启用此功能。

- * 每个卷的已用空间百分比 *

```
kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes *
100
```

了解有关 Astra Trident AutoSupport 遥测的信息

默认情况下， Astra Trident 会按每日节奏向 NetApp 发送 Prometheus 指标和基本后端信息。

- 要阻止 Astra Trident 向 NetApp 发送 Prometheus 指标和基本后端信息，请在 Astra Trident 安装期间传递 `-silning-autosupport` 标志。
- Astra Trident 还可以通过 `tridentctl send AutoSupport` 按需将容器日志发送到 NetApp 支持部门。您需要触发 Astra Trident 以上传其日志。在提交日志之前，您应接受 NetApp 的https://www.netapp.com/company/legal/privacy-policy/["隐私政策"^]。
- 除非另有说明，否则 Astra Trident 会从过去 24 小时提取日志。
- 您可以使用 `-since` 标志指定日志保留时间范围。例如： `tridentctl send AutoSupport -sase=1h` 。此信息通过与 Astra Trident 一起安装的 `trident — autosupport` 容器进行收集和发送。您可以从获取容器映像 "Trident AutoSupport"。
- Trident AutoSupport 不会收集或传输个人身份信息（ PiII ）或个人信息。它附带了 "EULA" 这不适用于 Trident 容器映像本身。您可以详细了解 NetApp 对数据安全和信任的承诺 "此处"。

Astra Trident 发送的有效负载示例如下：

```
{
  "items": [
    {
      "backendUUID": "ff3852e1-18a5-4df4-b2d3-f59f829627ed",
      "protocol": "file",
      "config": {
        "version": 1,
        "storageDriverName": "ontap-nas",
        "debug": false,
        "debugTraceFlags": null,
        "disableDelete": false,
        "serialNumbers": [
          "nwkvzfanek_SN"
        ],
        "limitVolumeSize": ""
      },
      "state": "online",
      "online": true
    }
  ]
}
```

- AutoSupport 消息将发送到 NetApp 的 AutoSupport 端点。如果使用私有注册表存储容器映像，则可以使用 `-image-regRegistry` 标志。
- 您也可以通过生成安装 YAML 文件来配置代理 URL 。为此，可以使用 `tridentctl install -generate-custom-yaml` 创建 YAML 文件，并在 `trident dedeployment.yaml` 中为 `trident autosupport` 容器添加 `-proxy-url` 参数。

禁用 **Astra Trident** 指标

要 ` 报告指标，您应生成自定义 YAML （使用` -generate-custom-yaml ` 标志）并对其进行编辑，以删除为 `trident` 主 容器调用的` -metrics 标志。

适用于 Docker 的 Astra Trident

部署的前提条件

在部署 Astra Trident 之前，您必须在主机上安装和配置必要的协议前提条件。

- 验证您的部署是否满足所有要求 "要求"。
- 验证您是否安装了受支持的 Docker 版本。如果您的 Docker 版本已过时， "安装或更新它"。

```
docker --version
```

- 验证是否已在主机上安装和配置协议前提条件：

协议	操作系统	命令
NFS	RHEL/CentOS	`sudo yum install -y nfs-utils`
NFS	Ubuntu 或 Debian	`sudo apt-get install -y nfs-common`

协议	操作系统	命令

iSCSI	RHEL/CentOS	1. 安装以下系统软件包：
		
		`sudo yum install -y lsscsi iscsi-initiator-utils sg3_utils device-mapper-multipath`
		
		2. 启动多路径守护进程：
		
		`sudo mpathconf -enable -with multipathd y`
		
		3. 确保已启用 `iscsid` 和 `multipathd` 并正在运行：
		
		`sudo systemctl enable iscsid multipathd` `sudo systemctl start iscsid multipathd`
		
		4. 发现 iSCSI 目标：
		
		`sudo iscsiadm -m discoverydb -t st -p <data_LIF_IP> -discover`
		
		5. 登录到发现的 iSCSI 目标：
		
		`sudo iscsiadm -m node -p <data_LIF_IP> -login`
		
		6. 启用并启动 `iSCSI`：
		
		`sudo systemctl enable iscsi` `sudo systemctl start iscsi`

协议	操作系统	命令

iSCSI	Ubuntu 或 Debian	1. 安装以下系统软件包：
		
		`sUdo apt-get install -y open-iscsi lsscsi sg3-utils multipath-tools scsitool`
		
		2. 启用多路径：
		
		`sUdo tee /etc/multipath.conf <←'EOF' defaults { user_friendly_names yes find_multipaths yes } eOF sudo service multipath-tools restart`
		
		3. 确保 `iscsid` 和 `multipathd` 正在运行：
		
		`sudo service open-iscsi start sudo service multipath-tools start`
		
		4. 发现 iSCSI 目标：
		
		`sudo iscsiadm -m discoverydb -t st -p <data_LIF_IP> -discover`
		
		5. 登录到发现的 iSCSI 目标：
		
		`sudo iscsiadm -m node -p <data_LIF_IP> -login`

部署 Astra Trident

适用于 Docker 的 Astra Trident 可直接与适用于 NetApp 存储平台的 Docker 生态系统集成。它支持从存储平台到 Docker 主机的存储资源配置和管理，并提供一个框架，用于在未来添加其他平台。

Astra Trident 的多个实例可以同时在同一主机上运行。这样可以同时连接到多个存储系统和存储类型，并能够自定义用于 Docker 卷的存储。

请参见 "部署的前提条件"。确保满足前提条件后，即可部署 Astra Trident 。

Docker 托管插件方法（1.13/17.03 及更高版本）

开始之前

如果您在传统守护进程方法中使用了 Astra Trident 之前的 Docker 1.13/ 17.03 ，请确保在使用受管插件方法之前停止 Astra Trident 进程并重新启动 Docker 守护进程。

1. 停止所有正在运行的实例：

```
pkill /usr/local/bin/netappdvp
pkill /usr/local/bin/trident
```

2. 重新启动 Docker 。

```
systemctl restart docker
```

3. 确保已安装 Docker 引擎 17.03 （新版本 1.13 ）或更高版本。

```
docker --version
```

如果您的版本已过期， "安装或更新安装"。

步骤

1. 创建配置文件并按如下所示指定选项：
   - `config` ：默认文件名为 `config.json` ，但您可以使用所选的任何名称，方法是使用文件名指定 `config` 选项。此配置文件必须位于主机系统上的 ` /etc/netappdvp` 目录中。
   - `log-level` ：指定日志记录级别（ `debug` ， `info` ， `warn` ， `error` ， `fal` ）。默认值为 `info` 。
   - `debug` ：指定是否启用调试日志记录。默认值为 false 。如果为 true ，则覆盖日志级别。
     i. 为配置文件创建一个位置：

     ```
     sudo mkdir -p /etc/netappdvp
     ```

     ii. 创建配置文件：

```
cat << EOF > /etc/netappdvp/config.json
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1"
}
EOF
```

2. 使用受管插件系统启动 Astra Trident 。

```
docker plugin install --grant-all-permissions --alias netapp
netapp/trident-plugin:21.07 config=myConfigFile.json
```

3. 开始使用 Astra Trident 消耗已配置系统中的存储。
   a. 创建名为 "firstVolume" 的卷：

   ```
   docker volume create -d netapp --name firstVolume
   ```

   b. 在容器启动时创建默认卷：

   ```
   docker run --rm -it --volume-driver netapp --volume
   secondVolume:/my_vol alpine ash
   ```

   c. 删除卷 "firstVolume"：

   ```
   docker volume rm firstVolume
   ```

传统方法（ 1.12 或更早版本）

开始之前

1. 确保您已安装 Docker 版本 1.10 或更高版本。

```
docker --version
```

如果您的版本已过期，请更新您的安装。

```
curl -fsSL https://get.docker.com/ | sh
```

或 "按照适用于您的分发版本的说明进行操作"。

2. 确保已为您的系统配置 NFS 和 / 或 iSCSI 。

步骤

1. 安装和配置 NetApp Docker 卷插件：
   a. 下载并解压缩应用程序：

   ```
   wget
   https://github.com/NetApp/trident/releases/download/v21.04.0/trident-
   installer-21.07.0.tar.gz
   tar zxf trident-installer-21.07.0.tar.gz
   ```

   b. 移动到托箱路径中的某个位置：

   ```
   sudo mv trident-installer/extras/bin/trident /usr/local/bin/
   sudo chown root:root /usr/local/bin/trident
   sudo chmod 755 /usr/local/bin/trident
   ```

   c. 为配置文件创建一个位置：

   ```
   sudo mkdir -p /etc/netappdvp
   ```

   d. 创建配置文件：

   ```
   cat << EOF > /etc/netappdvp/ontap-nas.json
   {
       "version": 1,
       "storageDriverName": "ontap-nas",
       "managementLIF": "10.0.0.1",
       "dataLIF": "10.0.0.2",
       "svm": "svm_nfs",
       "username": "vsadmin",
       "password": "secret",
       "aggregate": "aggr1"
   }
   EOF
   ```

2. 放置二进制文件并创建配置文件后，使用所需的配置文件启动 Trident 守护进程。

```
sudo trident --config=/etc/netappdvp/ontap-nas.json
```

> 除非指定，否则卷驱动程序的默认名称为 "netapp" 。

启动守护进程后，您可以使用 Docker 命令行界面创建和管理卷

3. 创建卷

```
docker volume create -d netapp --name trident_1
```

4. 启动容器时配置 Docker 卷：

```
docker run --rm -it --volume-driver netapp --volume trident_2:/my_vol
alpine ash
```

5. 删除 Docker 卷：

```
docker volume rm trident_1
docker volume rm trident_2
```

在系统启动时启动 Astra Trident

有关基于 systemd 的系统的示例单元文件，请参见 Git repo 中的 `contrib/trident.service.example` 。要将此文件与 CentOS/RHEL 结合使用，请执行以下操作：

1. 将文件复制到正确的位置。

   如果正在运行多个实例，则单元文件应使用唯一名称。

```
cp contrib/trident.service.example
/usr/lib/systemd/system/trident.service
```

2. 编辑文件，更改问题描述（第 2 行）以匹配驱动程序名称和配置文件路径（第 9 行）以反映您的环境。
3. 重新加载 systemd 以载入更改：

```
systemctl daemon-reload
```

4. 启用服务。

根据您在 ` /usr/lib/systemd/system` 目录中为文件命名的内容，此名称会有所不同。

```
systemctl enable trident
```

5. 启动服务。

```
systemctl start trident
```

6. 查看状态。

```
systemctl status trident
```

> 每当您修改单元文件时，请运行 `systemctl daemon-reload` 命令，使其了解所做的更改。

升级或卸载 Astra Trident

您可以安全地升级适用于 Docker 的 Astra Trident ，而不会对正在使用的卷产生任何影响。在升级过程中，指向插件的 `docker volume` 命令将在很短的时间内失败，并且应用程序将无法挂载卷，直到插件重新运行为止。在大多数情况下，这只需要几秒钟。

升级

执行以下步骤以升级适用于 Docker 的 Astra Trident 。

步骤

1. 列出现有卷：

```
docker volume ls
DRIVER              VOLUME NAME
netapp:latest       my_volume
```

2. 禁用插件：

```
docker plugin disable -f netapp:latest
docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
7067f39a5df5        netapp:latest       nDVP - NetApp Docker Volume
Plugin   false
```

3. 升级插件：

```
docker plugin upgrade --skip-remote-check --grant-all-permissions
netapp:latest netapp/trident-plugin:21.07
```

> Astra Trident 18.01 版取代了 nDVP 。您应直接从 `NetApp/ndvp-plugin` 映像升级到 `NetApp/trident` 插件 映像。

4. 启用插件：

```
docker plugin enable netapp:latest
```

5. 验证是否已启用此插件：

```
docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
7067f39a5df5        netapp:latest       Trident - NetApp Docker Volume
Plugin   true
```

6. 验证卷是否可见：

```
docker volume ls
DRIVER              VOLUME NAME
netapp:latest       my_volume
```

> 如果要从旧版本的 Astra Trident （ 20.10 之前的版本）升级到 Astra Trident 20.10 或更高版本，则可能会遇到错误。有关详细信息，请参见 "已知问题"。如果遇到此错误，应先禁用此插件，然后删除此插件，然后通过传递一个额外的配置参数来安装所需的 Astra Trident 版本 ： `docker plugin install netapp/trident plugin ： 20.10 —— alias netapp — -grant-all-permissions config.json`

卸载

执行以下步骤卸载适用于 Docker 的 Astra Trident 。

步骤

1. 删除插件创建的所有卷。
2. 禁用插件：

```
docker plugin disable netapp:latest
docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
7067f39a5df5        netapp:latest       nDVP - NetApp Docker Volume
Plugin   false
```

3. 删除插件：

```
docker plugin rm netapp:latest
```

使用卷

您可以根据需要使用标准的 `docker volume` 命令以及指定的 Astra Trident 驱动程序名称轻松创建，克隆和删除卷。

创建卷

- 使用默认名称创建包含驱动程序的卷：

```
docker volume create -d netapp --name firstVolume
```

- 使用特定的 Astra Trident 实例创建卷：

```
docker volume create -d ntap_bronze --name bronzeVolume
```

> 如果未指定任何 "选项"，将使用驱动程序的默认值。

- 覆盖默认卷大小。要使用驱动程序创建 20GiB 卷，请参见以下示例：

```
docker volume create -d netapp --name my_vol --opt size=20G
```

> 卷大小以字符串表示，该字符串包含一个包含可选单元的整数值（例如： 10 G ， 20 GB ， 3 TiB ）。如果未指定单位，则默认值为 G大小单位可以表示为 2 的幂（ B ， KiB ， MIB ， GiB ， TiB ）或 10 的幂（ B ， KB ， MB ， GB ， TB ）。速率单位使用 2 的电流（ G = GiB ， T = TiB ，…）。

删除卷

- 像删除任何其他 Docker 卷一样删除此卷：

```
docker volume rm firstVolume
```

> 使用 `solidfire-san` 驱动程序时，上述示例将删除并清除卷。

执行以下步骤以升级适用于 Docker 的 Astra Trident 。

克隆卷

使用 `ontap-nas` ， `ontap-san` ， `solidfire-san` 和 `GCP-CVS` **存储驱动程序** 时， Astra Trident 可以克隆卷。使用 `ontap-nas-flexgroup` 或 `ontap-nas-economy`-**经济** 驱动程序时，不支持克隆。从现有卷创建新卷将创建新快照。

- 检查卷以枚举快照：

```
docker volume inspect <volume_name>
```

- 从现有卷创建新卷。这将导致创建新快照：

```
docker volume create -d <driver_name> --name <new_name> -o
from=<source_docker_volume>
```

- 从卷上的现有快照创建新卷。此操作不会创建新快照：

```
docker volume create -d <driver_name> --name <new_name> -o
from=<source_docker_volume> -o fromSnapshot=<source_snap_name>
```

示例

```
[me@host ~]$ docker volume inspect firstVolume

[
    {
        "Driver": "ontap-nas",
        "Labels": null,
        "Mountpoint": "/var/lib/docker-volumes/ontap-
nas/netappdvp_firstVolume",
        "Name": "firstVolume",
        "Options": {},
        "Scope": "global",
        "Status": {
            "Snapshots": [
                {
                    "Created": "2017-02-10T19:05:00Z",
                    "Name": "hourly.2017-02-10_1505"
                }
            ]
        }
    }
]

[me@host ~]$ docker volume create -d ontap-nas --name clonedVolume -o
from=firstVolume
clonedVolume

[me@host ~]$ docker volume rm clonedVolume
[me@host ~]$ docker volume create -d ontap-nas --name volFromSnap -o
from=firstVolume -o fromSnapshot=hourly.2017-02-10_1505
volFromSnap

[me@host ~]$ docker volume rm volFromSnap
```

访问外部创建的卷

如果容器没有分区，并且 Astra Trident 支持其文件系统（例如： A `ext4` 格式的 ` /dev/sdc1` 将无法通过 Astra Trident 访问），则可以使用 Trident * only * 通过容器访问外部创建的块设备（或其克隆）。

驱动程序专用的卷选项

每个存储驱动程序都有一组不同的选项，您可以在创建卷时指定这些选项来自定义结果。有关适用于您配置的存储系统的选项，请参见以下内容。

在卷创建操作期间使用这些选项非常简单。在命令行界面操作期间，使用 ` -o` 运算符提供选项和值。这些参数将覆盖 JSON 配置文件中的任何等效值。

ONTAP 卷选项

NFS 和 iSCSI 的卷创建选项包括以下内容：

选项	Description
`s`大小	卷的大小默认为 1 GiB 。
`s`页面预留	精简或厚配置卷，默认为精简。有效值为 `none` （精简配置）和 `volume` （厚配置）。
`sSnapshot` 策略	此操作会将 Snapshot 策略设置为所需的值。默认值为 `none` ，表示不会自动为卷创建快照。除非存储管理员修改，否则所有 ONTAP 系统上都存在一个名为 "defaultion" 的策略，该策略会创建并保留六个每小时快照，两个每日快照和两个每周快照。通过浏览到卷中任意目录中的 ` .snapshot` 目录，可以恢复快照中保留的数据。
`sSnapshot` 预留	此操作会将快照预留设置为所需百分比。默认值为 no 值，这意味着如果您选择了 snapshotPolicy ， ONTAP 将选择 snapshotReserve （通常为 5% ）；如果 snapshotPolicy 为 none ，则选择 0% 。您可以在配置文件中为所有 ONTAP 后端设置默认 snapshotReserve 值，并可将其用作除 ontap-nas-economy. 以外的所有 ONTAP 后端的卷创建选项。
`splitOnClone`	克隆卷时，此操作将使发生原因 ONTAP 立即从其父卷拆分克隆。默认值为 `false` 。在克隆卷的某些使用情形中，最好在创建后立即将克隆从其父卷中拆分，因为不太可能有任何提高存储效率的机会。例如，克隆空数据库可以节省大量时间，但节省的存储很少，因此最好立即拆分克隆。
加密	这将在新卷上启用 NetApp 卷加密（ NVE ），默认为 `false` 。要使用此选项，必须在集群上获得 NVE 的许可并启用 NVE 。 Trident 目前不支持 NetApp 聚合加密（ NAE ）。
分层策略	设置要用于卷的分层策略。这将决定数据在变为非活动状态（冷）时是否移至云层。

以下附加选项适用于 NFS * 仅 *：

选项	Description
unixPermissions	此选项用于控制为卷本身设置的权限。默认情况下，权限将设置为 ` -rwrxr-xr-x ，或以数字表示法 0755 表示，并且 root 将为所有者。文本或数字格式均可使用。
snapshotDir	将此值设置为 true 将使访问卷的客户端能够看到 ` 。snapshot` 目录。默认值为 false ，这意味着默认情况下禁用 ` .snapshot` 目录的可见性。当 ` .snapshot` 目录可见时，某些映像（例如官方 MySQL 映像）无法按预期运行。
exportPolicy	设置要用于卷的导出策略。默认值为 default 。
securityStyle	设置用于访问卷的安全模式。默认值为 UNIX 。有效值为 UNIX 和 mi 已加传真 。

以下附加选项适用于 iSCSI * 仅 *：

选项	Description
fileSystemType	设置用于格式化 iSCSI 卷的文件系统。默认值为 ext4 。有效值为 ext3 ， ext4 和 xfs 。
spaceAllocation	如果将此值设置为 false ，则会关闭 LUN 的空间分配功能。默认值为 true ，表示当卷空间用尽且卷中的 LUN 无法接受写入时， ONTAP 会通知主机。此选项还允许 ONTAP 在主机删除数据时自动回收空间。

示例

请参见以下示例：

- 创建 10 GiB 卷：

```
docker volume create -d netapp --name demo -o size=10G -o
encryption=true
```

- 创建具有快照的 100GiB 卷：

```
docker volume create -d netapp --name demo -o size=100G -o
snapshotPolicy=default -o snapshotReserve=10
```

- 创建启用了 setuid 位的卷：

```
docker volume create -d netapp --name demo -o unixPermissions=4755
```

最小卷大小为 20MiB 。

如果未指定快照预留且快照策略为 `none` ，则 Trident 将使用 0% 的快照预留。

- 创建无快照策略且无快照预留的卷：

```
docker volume create -d netapp --name my_vol --opt snapshotPolicy=none
```

- 创建一个无快照策略且自定义快照预留为 10% 的卷：

```
docker volume create -d netapp --name my_vol --opt snapshotPolicy=none
--opt snapshotReserve=10
```

- 创建具有快照策略和 10% 自定义快照预留的卷：

```
docker volume create -d netapp --name my_vol --opt
snapshotPolicy=myPolicy --opt snapshotReserve=10
```

- 使用快照策略创建卷，并接受 ONTAP 的默认快照预留（通常为 5% ）：

```
docker volume create -d netapp --name my_vol --opt
snapshotPolicy=myPolicy
```

Element 软件卷选项

Element 软件选项会显示与卷关联的大小和服务质量（ QoS ）策略。创建卷时，将使用 ` -o type=service_level` 命名空间指定与其关联的 QoS 策略。

使用 Element 驱动程序定义 QoS 服务级别的第一步是至少创建一种类型，并指定与配置文件中的名称关联的最小，最大和突发 IOPS 。

其他 Element 软件卷创建选项包括：

选项	**Description**
`s`大小	卷的大小，默认为 1GiB 或配置条目 … " 默认值 " ：｛ "size" ： "5c" ｝ 。

选项	Description
块大小	使用 512 或 4096 ，默认为 512 或配置条目 DefaultBlockSize 。

示例

请参见以下包含 QoS 定义的示例配置文件：

```
{
    "...": "..."
    "Types": [
        {
            "Type": "Bronze",
            "Qos": {
                "minIOPS": 1000,
                "maxIOPS": 2000,
                "burstIOPS": 4000
            }
        },
        {
            "Type": "Silver",
            "Qos": {
                "minIOPS": 4000,
                "maxIOPS": 6000,
                "burstIOPS": 8000
            }
        },
        {
            "Type": "Gold",
            "Qos": {
                "minIOPS": 6000,
                "maxIOPS": 8000,
                "burstIOPS": 10000
            }
        }
    ]
}
```

在上述配置中，我们有三个策略定义：铜牌，银牌和金牌。这些名称是任意的。

- 创建 10 GiB 黄金卷：

```
docker volume create -d solidfire --name sfGold -o type=Gold -o size=10G
```

- 创建 100GiB 铜牌卷：

```
docker volume create -d solidfire --name sfBronze -o type=Bronze -o
size=100G
```

GCP 上的 CVS 卷选项

基于 GCP 的 CVS 驱动程序的卷创建选项包括以下内容：

选项	**Description**
`s`大小	卷的大小，默认情况下， CVS-Performance 卷为 100 GiB ， CVS 卷为 300 GiB 。
`s`服务级别	卷的 CVS 服务级别默认为标准。有效值包括标准，高级和极高。
`sSnapshot` 预留	此操作会将快照预留设置为所需百分比。默认值为 no 值，表示 CVS 将选择快照预留（通常为 0% ）。

示例

- 创建 2 TiB 卷：

```
docker volume create -d netapp --name demo -o size=2T
```

- 创建 5 TiB 高级卷：

```
docker volume create -d netapp --name demo -o size=5T -o
serviceLevel=premium
```

对于 CVS-Performance 卷，最小卷大小为 100 GiB ，对于 CVS 卷，最小卷大小为 300 GiB 。

Azure NetApp Files 卷选项

Azure NetApp Files 驱动程序的卷创建选项包括：

选项	**Description**
`s`大小	卷的大小默认为 100 GB 。

示例

- 创建 200 GiB 卷：

```
docker volume create -d netapp --name demo -o size=200G
```

最小卷大小为 100 GB 。

收集日志

您可以收集日志以帮助进行故障排除。收集日志的方法因运行 Docker 插件的方式而异。

收集日志以进行故障排除

步骤

1. 如果您使用建议的受管插件方法（例如，使用 `docker plugin` 命令）运行 Astra Trident ，请按如下所示查看它们：

```
# docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
4fb97d2b956b        netapp:latest       nDVP - NetApp Docker Volume
Plugin   false
# journalctl -u docker | grep 4fb97d2b956b
```

   标准日志记录级别应允许您诊断大多数问题。如果您发现这还不够，则可以启用调试日志记录。

2. 要启用调试日志记录，请安装启用了调试日志记录的插件：

```
docker plugin install netapp/trident-plugin:<version> --alias <alias>
debug=true
```

   或者，在已安装插件的情况下启用调试日志记录：

```
docker plugin disable <plugin>
docker plugin set <plugin> debug=true
docker plugin enable <plugin>
```

3. 如果在主机上运行二进制文件本身，则日志可从主机的 ` /var/log/netappdvp` 目录中获取。要启用调试日志记录，请在运行插件时指定 ` -debug` 。

一般故障排除提示

- 新用户遇到的最常见问题是配置不当，导致插件无法初始化。如果发生这种情况，在尝试安装或启用插件时，您可能会看到如下消息：

  守护进程的错误响应：拨打 `unix /run/docker/plugins/<id>/netapp.sock : connect : no such file or directory`

  这意味着插件无法启动。幸运的是，该插件已构建了全面的日志记录功能，可以帮助您诊断可能遇到的大多数问题。

- 如果在将 PV 挂载到容器时出现问题，请确保 `rpcbind` 已安装并正在运行。对主机操作系统使用所需的软件包管理器，并检查 `rpcbind` 是否正在运行。您可以通过运行 `systemctl status rpcbind` 或其等效项来检查 rpcbind 服务的状态。

管理多个 Astra Trident 实例

如果希望同时提供多个存储配置，则需要多个 Trident 实例。多个实例的关键在于，在主机上实例化 Trident 时，使用容器化插件的 ` -alias` 选项或 ` -volume-driver` 选项为其提供不同的名称。

Docker 托管插件（ 1.13/17.03 或更高版本）的步骤

1. 启动指定别名和配置文件的第一个实例。

```
docker plugin install --grant-all-permissions --alias silver
netapp/trident-plugin:21.07 config=silver.json
```

2. 启动第二个实例，指定其他别名和配置文件。

```
docker plugin install --grant-all-permissions --alias gold
netapp/trident-plugin:21.07 config=gold.json
```

3. 创建将别名指定为驱动程序名称的卷。

   例如，对于黄金卷：

```
docker volume create -d gold --name ntapGold
```

   例如，对于银牌卷：

```
docker volume create -d silver --name ntapSilver
```

传统（ 1.12 或更早版本）的步骤

1. 使用自定义驱动程序 ID 启动具有 NFS 配置的插件：

```
sudo trident --volume-driver=netapp-nas --config=/path/to/config
-nfs.json
```

2. 使用自定义驱动程序 ID 启动具有 iSCSI 配置的插件：

```
sudo trident --volume-driver=netapp-san --config=/path/to/config
-iscsi.json
```

3. 为每个驱动程序实例配置 Docker 卷：

   例如，对于 NFS ：

```
docker volume create -d netapp-nas --name my_nfs_vol
```

   例如，对于 iSCSI ：

```
docker volume create -d netapp-san --name my_iscsi_vol
```

存储配置选项

请参见适用于您的 Astra Trident 配置的配置选项。

全局配置选项

这些配置选项适用于所有 Astra Trident 配置，而不考虑所使用的存储平台。

选项	**Description**	示例
`ve`版本	配置文件版本号	1.
`storageDriverName`	存储驱动程序的名称	`ontap-nas` ， `ontap-san` ， `ontap-nas-economics` ， `ontap-nas-flexgroup` ， `solidfire-san` ， `azure-netapp-files` 或 `GCP-CVS`
`s`存储前缀	卷名称的可选前缀。默认值："netappdvp_ " 。	暂存 _

选项	Description	示例
limitVolumeSize	卷大小的可选限制。默认值：""（未强制实施）	10 G

> 请勿对元素后端使用 storagePrefix （包括默认值）。默认情况下， solidfire-san 驱动程序将忽略此设置，而不使用前缀。我们建议使用特定的租户 ID 进行 Docker 卷映射，或者在可能已使用任何名称的情况下使用 Docker 中填充的 Docker 版本，驱动程序信息和原始名称的属性数据。

您可以使用默认选项来避免在创建的每个卷上指定这些选项。s 选项适用于所有控制器类型。有关如何设置默认卷大小的示例，请参见 ONTAP 配置一节。

选项	Description	示例
s大小	新卷的可选默认大小。默认值："1G"	10 G

ONTAP 配置

除了上述全局配置值之外，在使用 ONTAP 时，还可以使用以下顶级选项。

选项	Description	示例
m年 月 日	ONTAP 管理 LIF 的 IP 地址。您可以指定完全限定域名（ FQDN ）。	10.0.0.1
dataLIF	协议 LIF 的 IP 地址；如果未指定，则派生此地址。对于 ontap-NAS 驱动程序 * 仅 *，您可以指定 FQDN，在这种情况下， FQDN 将用于 NFS 挂载操作。对于 ontap-san 驱动程序，默认使用 SVM 中的所有数据 LIF IP 并使用 iSCSI 多路径。为 ontap-san 驱动程序指定 dataLIF 的 IP 地址会强制驱动程序禁用多路径并仅使用指定的地址。	10.0.0.2
sVM	要使用的 Storage Virtual Machine（如果管理 LIF 为集群 LIF，则为必填项）	SVM_NFS
用户名	用于连接到存储设备的用户名	vsadmin
密码	用于连接到存储设备的密码	机密

选项	Description	示例
聚合	要配置的聚合（可选；如果设置了聚合，则必须将其分配给 SVM ）。对于 `ontap-nas-flexgroup` 驱动程序，此选项将被忽略。分配给 SVM 的所有聚合都用于配置 FlexGroup 卷。	aggr1.
`limitAggregateUsage`	可选，如果使用量超过此百分比，则配置失败	75%
`nfsMountOptions`	对 NFS 挂载选项进行精细控制；默认为 -o nfsver=3 。* 仅适用于 `ontap-nas` 和 `ontap-nas-` 经济驱动程序 * 。 "请参见此处的 NFS 主机配置信息"。	-o nfsver=4
`igroupName`	插件使用的 igroup ；默认为 "netappdvp " 。* 仅适用于 `ontap-san`driver* 。	myigroup
`limitVolumeSize`	可请求的最大卷大小和 qtree 父卷大小。* 对于 `ontap-nas-economy` 驱动程序，此选项还会限制其创建的 FlexVol 的大小 * 。	300 克
`qtreesPerFlexvol`	每个 FlexVol 的最大 qtree 数必须在 50 ， 300 范围内，默认值为 200 。* 对于 `ontap-nas-economy` 驱动程序，此选项允许自定义每个 qtree* 的最大 FlexVol 数。	300

您可以使用默认选项来避免在创建的每个卷上指定这些选项：

选项	Description	示例
s页面预留	空间预留模式； " 无 " （精简配置）或 " 卷 " （厚）	无
`sSnapshot` 策略	要使用的 Snapshot 策略，默认值为 " 无 "	无
`sSnapshot` 预留	Snapshot 预留百分比，默认值为 " " 以接受 ONTAP 的默认值	10
`splitOnClone`	创建克隆时将其从父级拆分，默认为 "false"	false

选项	Description	示例
加密	启用 NetApp 卷加密，默认为 "false"	true
unixPermissions	已配置 NFS 卷的 NAS 选项，默认为 "777"	777.
snapshotDir	用于访问 ` .snapshot` 目录的 NAS 选项，默认为 "false"	true
exportPolicy	要使用的 NFS 导出策略的 NAS 选项，默认为 " 默认 "	default
securityStyle	用于访问已配置 NFS 卷的 NAS 选项，默认为 "UNIX"	混合
fileSystemType	SAN 选项要选择文件系统类型，默认为 "ext4"	XFS
分层策略	要使用的分层策略，对于 ONTAP 9.5 SVM-DR 之前的配置，默认为 " 无 "； " 仅快照 "	无

扩展选项

ontap-nas 和 ontap-san 驱动程序会为每个 Docker 卷创建一个 ONTAP FlexVol 。对于每个集群节点，ONTAP 最多支持 1000 个 FlexVol ，而集群最多支持 12 ， 000 个 FlexVol 。如果您的 Docker 卷要求符合此限制，则由于 Flexvol 提供的其他功能（例如 Docker 卷粒度快照和克隆）， ontap-NAS 驱动程序是首选的 NAS 解决方案。

如果所需的 Docker 卷数超出 FlexVol 限制所能容纳的范围，请选择 ontap-nas-economy" 或 ontap-san-economy" 驱动程序。

ontap-nas-economy 驱动程序可在自动管理的 ONTAP 卷池中创建 Docker 卷作为 qtree 。qtree 的扩展能力远高于此，每个集群节点最多可扩展 100 ， 000 个，每个集群最多可扩展 2 ， 400 ， 000 个，但某些功能会受到影响。ontap-nas-economy 驱动程序不支持 Docker 卷粒度快照或克隆。

> Docker Swarm 目前不支持 ontap-nas-economy-经济 驱动程序，因为 Swarm 不会在多个节点之间协调卷创建。

ontap-san-economy-driver 可在一个由自动管理的 FlexVol 构成的共享池中将 Docker 卷创建为 ONTAP LUN 。这样，每个 FlexVol 就不会仅限于一个 LUN ，并且可以为 SAN 工作负载提供更好的可扩展性。根据存储阵列的不同， ONTAP 每个集群最多支持 16384 个 LUN 。由于卷是下面的 LUN ，因此此驱动程序支持 Docker 卷粒度快照和克隆。

选择 ontap-nas-flexgroup 驱动程序可将并行性提高到单个卷，该卷可以扩展到包含数十亿个文件的 PB 范围。FlexGroup 的一些理想用例包括 AI/ML/DL ，大数据和分析，软件构建，流式传输，文件存储库等。配置 FlexGroup 卷时， Trident 会使用分配给 SVM 的所有聚合。Trident 中的 FlexGroup 支持还需要注意以下事项：

- 需要 ONTAP 9.2 或更高版本。
- 截至本文撰写时， FlexGroup 仅支持 NFS v3 。
- 建议为 SVM 启用 64 位 NFSv3 标识符。
- 建议的最小 FlexGroup 大小为 100 GB 。
- FlexGroup 卷不支持克隆。

有关适用于 FlexGroup 的 FlexGroup 和工作负载的信息，请参见 "《 NetApp FlexGroup 卷最佳实践和实施指南》"。

要在同一环境中获得高级功能并实现大规模扩展，您可以运行多个 Docker 卷插件实例，其中一个使用 `ontap-NAS` ，另一` 个使用 `ontap-nas-economy.

ONTAP 配置文件示例

`ontap-nas` driver* 的 * NFS 示例

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1",
    "defaults": {
      "size": "10G",
      "spaceReserve": "none",
      "exportPolicy": "default"
    }
}
```

`ontap-nas-flexgroup` driver* 的 * NFS 示例

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "defaults": {
      "size": "100G",
      "spaceReserve": "none",
      "exportPolicy": "default"
    }
}
```

`ontap-nas-economy` driver* 的 * NFS 示例

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1"
}
```

`ontap-san` driver* 的 * iSCSI 示例

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1",
    "igroupName": "myigroup"
}
```

`ontap-san-economy-driver` 的 * NFS 示例

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi_eco",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1",
    "igroupName": "myigroup"
}
```

Element 软件配置

除了全局配置值之外，在使用 Element 软件（ NetApp HCI/SolidFire ）时，还可以使用这些选项。

选项	**Description**	示例
端点	https://<login>:<password>@<mvip>/json-rpc/<element-version>	https://admin:admin@192.168.160.3/json-rpc/8.0
`sVIP`	iSCSI IP 地址和端口	10.0.0.7： 3260
租户名称	要使用的 SolidFireF 租户（如果未找到，则创建）	Docker
`InitiatorIFace`	将 iSCSI 流量限制为非默认接口时，请指定接口	default
类型	QoS 规范	请参见以下示例
`LegendPrefix`	升级后的 Trident 安装的前缀。如果您使用的是 1.3.2 之前的 Trident 版本并对现有卷执行升级，则需要设置此值才能访问通过 volume-name 方法映射的旧卷。	"netappdvp- "

`solidfire-san` 驱动程序不支持 Docker Swarm 。

Element 软件配置文件示例

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://admin:admin@192.168.160.3/json-rpc/8.0",
    "SVIP": "10.0.0.7:3260",
    "TenantName": "docker",
    "InitiatorIFace": "default",
    "Types": [
        {
            "Type": "Bronze",
            "Qos": {
                "minIOPS": 1000,
                "maxIOPS": 2000,
                "burstIOPS": 4000
            }
        },
        {
            "Type": "Silver",
            "Qos": {
                "minIOPS": 4000,
                "maxIOPS": 6000,
                "burstIOPS": 8000
            }
        },
        {
            "Type": "Gold",
            "Qos": {
                "minIOPS": 6000,
                "maxIOPS": 8000,
                "burstIOPS": 10000
            }
        }
    ]
}
```

GCP 配置上的 Cloud Volumes Service （CVS）

Trident 现在支持使用默认 CVS 服务类型的较小卷 "GCP"。对于使用 `storageClass=software` 创建的后端，卷的最小配置大小将为 300 GiB 。* NetApp 建议客户将低于 1 TiB 的卷用于非生产工作负载 * 。CVS 目前在 " 受控可用性 " 下提供此功能，不提供技术支持。

> 注册以访问低于 1TiB 的卷 "此处"。

> 使用默认 CVS 服务类型 `storageClass=software` 部署后端时，您应获得 GCP 上有关项目编号和项目 ID 的子 1TiB 卷功能的访问权限。这对于 Trident 配置低于 1TiB 的卷是必需的。否则，对于小于 600 GiB 的 PVC ，卷创建 * 将失败 * 。使用访问低于 1TiB 的卷 "此表单"。

Trident 为默认 CVS 服务级别创建的卷将按以下方式进行配置：

- 小于 300 GiB 的 PVC 将导致 Trident 创建 300 GiB 的 CVS 卷。
- 如果 PVC 介于 300 GiB 到 600 GiB 之间，则 Trident 将创建一个请求大小的 CVS 卷。
- 介于 600 GiB 和 1 TiB 之间的 PVC 将导致 Trident 创建 1 TiB CVS 卷。
- 如果 PVC 大于 1 TiB ，则 Trident 将创建一个请求大小的 CVS 卷。

除了全局配置值之外，在 GCP 上使用 CVS 时，还可以使用这些选项。

选项	**Description**	示例
区域	CVS 帐户区域（必需）。是此后端将配置卷的 GCP 区域。	"us-west2"
`projectNumber`	GCP 项目编号（必需）。可以在 GCP Web 门户的主屏幕中找到。	"123456789012"
`hostProjectNumber`	GCP 共享 VPC 主机项目编号（如果使用共享 VPC ，则为必填项）	"098765432109"
`apiKey`	具有 CVS 管理员角色的 GCP 服务帐户的 API 密钥（必需）。是 GCP 服务帐户专用密钥文件的 JSON 格式内容（逐字复制到后端配置文件）。服务帐户必须具有 netappcloudvolumes.admin 角色。	（私钥文件的内容）
`secreKey`	CVS 帐户密钥（必需）。可以在 CVS Web 门户中的 "Account settings" >"API access" 中找到。	default
代理 `URL`	代理服务器需要连接到 CVS 帐户时的代理 URL 。代理服务器可以是 HTTP 代理，也可以是 HTTPS 代理。对于 HTTPS 代理，系统会跳过证书验证，以允许在代理服务器中使用自签名证书。* 不支持已启用身份验证的代理服务器 * 。	http://proxy-server-hostname/”
`nfsMountOptions`	NFS 挂载选项；默认为 -o nfsver=3	"nfsver=3 ， proto=tcp ， timeo=600"

选项	Description	示例
s服务级别	性能级别（标准，高级，极高），默认为 " 标准 "	高级版
网络	用于 CVS 卷的 GCP 网络，默认为 " 默认 "	default

> 如果使用共享 VPC 网络，则应同时指定 `projectNumber` 和 `hostProjectNumber` 。在这种情况下， `projectNumber` 是服务项目， `hostProjectNumber` 是主机项目。

> NetApp Cloud Volumes Service for GCP 不支持小于 100 GiB 的 CVS-Performance 卷或小于 300 GiB 的 CVS 卷。为了便于部署应用程序，如果请求的卷太小， Trident 会自动创建最小大小的卷。

在 GCP 上使用 CVS 时，可以使用这些默认卷选项设置。

选项	Description	示例
`exportRule`	NFS 访问列表（地址和 / 或 CIDR 子网），默认为 "0.0.0.0/0 "	"10.0.1.0/24 10.0.0.2.100"
`snapshotDir`	控制 ` .snapshot` 目录的可见性	false
`sSnapshot` 预留	Snapshot 预留百分比，默认值为 " " 以接受 CVS 默认值 0	10
s大小	卷大小，默认为 "100GiB"	"10T"

GCP 配置文件上的 CVS 示例

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
```

```
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "proxyURL": "http://proxy-server-hostname/"
}
```


Azure NetApp Files 配置

配置和使用 "Azure NetApp Files" 后端，您需要满足以下要求：

- `ssubscriptionID` 来自启用了 Azure NetApp Files 的 Azure 订阅
- 租户 `ID`， `clientID` 和 `clientSecret` 来自 "应用程序注册" 在 Azure Active Directory 中，具有足够的 Azure NetApp Files 服务权限
- 至少包含一个的 Azure 位置 "委派子网"

> 如果您是首次使用 Azure NetApp Files 或在新位置使用，则需要对进行一些初始配置 "《快速入门指南》" 将引导您完成操作。

Astra Trident 21.04.0 及更早版本不支持手动 QoS 容量池。

选项	Description	Default
ve版本	始终为 1	
storageDriverName	"azure-netapp-files"	
backendName	存储后端的自定义名称	驱动程序名称 + "_" + 随机字符
ssubscriptionID。	Azure 订阅中的订阅 ID	
租户 ID。	应用程序注册中的租户 ID	
客户端 ID。	应用程序注册中的客户端 ID	
clientSecret。	应用程序注册中的客户端密钥	
s服务级别	" 标准 " ， " 高级 " 或 " 超 " 之一	" （随机）
位置	将在中创建新卷的 Azure 位置名称	" （随机）
virtualNetwork	具有委派子网的虚拟网络的名称	" （随机）
ssubnet	委派给 Microsoft.Netapp/volumes 的子网的名称	" （随机）
nfsMountOptions	精细控制 NFS 挂载选项	-o nfsver=3
limitVolumeSize	如果请求的卷大小超过此值，则配置失败	" （默认情况下不强制实施）

Azure NetApp Files 服务不支持小于 100 GB 的卷。为了便于部署应用程序，如果请求的卷较小， Trident 会自动创建 100 GB 的卷。

您可以在配置的特殊部分中使用这些选项来控制默认配置每个卷的方式。

选项	Description	Default
exportRule	新卷的导出规则。必须是以 CIDR 表示法表示的任意 IPv4 地址或 IPv4 子网组合的逗号分隔列表。	"0.0.0.0/0 "

选项	Description	Default
snapshotDir	控制 ` .snapshot` 目录的可见性	false
s大小	新卷的默认大小	"100G"

Azure NetApp Files 配置示例

- 示例 1： azure-netapp-files* 的最小后端配置

这是绝对的最低后端配置。使用此配置， Trident 将发现全球每个位置委派给 ANF 的所有 NetApp 帐户，容量池和子网，并随机将新卷放置在其中一个上。

当您刚开始使用 ANF 并尝试执行以下操作时，此配置非常有用： 但实际上，您将需要为您配置的卷提供额外的范围界定，以确保这些卷具有所需的特征，并最终位于一个靠近使用该卷的计算的网络上。有关详细信息，请参见后续示例。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET"
}
```

- 示例 2： azure-netapp-files* 的单一位置和特定服务级别

此后端配置会将卷放置在 Azure 的 " 东向 " 位置的 " 高级 " 容量池中。Trident 会自动发现该位置委派给 ANF 的所有子网，并随机在其中一个子网上放置一个新卷。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus",
    "serviceLevel": "Premium"
}
```

- 示例 3： azure-netapp-files* 的高级配置

此后端配置进一步将卷放置范围缩小为一个子网，并修改了某些卷配置默认值。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus",
    "serviceLevel": "Premium",
    "virtualNetwork": "my-virtual-network",
    "subnet": "my-subnet",
    "nfsMountOptions": "nfsvers=3,proto=tcp,timeo=600",
    "limitVolumeSize": "500Gi",
    "defaults": {
        "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
        "size": "200Gi"
    }
}
```

- 示例 4： 使用 azure-netapp-files* 的虚拟存储池

此后端配置定义了多个 "存储池" 在单个文件中。如果您有多个容量池支持不同的服务级别，并且您希望在 Kubernetes 中创建表示这些服务级别的存储类，则此功能非常有用。

这只是擦除虚拟存储池及其标签的强大功能表面。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "nfsMountOptions": "nfsvers=3,proto=tcp,timeo=600",
    "labels": {
        "cloud": "azure"
    },
    "location": "eastus",

    "storage": [
        {
            "labels": {
                "performance": "gold"
            },
            "serviceLevel": "Ultra"
        },
        {
            "labels": {
                "performance": "silver"
            },
            "serviceLevel": "Premium"
        },
        {
            "labels": {
                "performance": "bronze"
            },
            "serviceLevel": "Standard",
        }
    ]
}
```

已知问题和限制

查找有关将 Astra Trident 与 Docker 结合使用时的已知问题和限制的信息。

将 **Trident Docker** 卷插件从旧版本升级到 **20.10** 及更高版本会导致升级失败，并且不会显示此类文件或目录错误。

临时解决策

1. 禁用插件。

```
docker plugin disable -f netapp:latest
```

2. 删除此插件。

```
docker plugin rm -f netapp:latest
```

3. 通过提供额外的 `config` 参数来重新安装插件。

```
docker plugin install netapp/trident-plugin:20.10 --alias netapp --grant
-all-permissions config=config.json
```

卷名称的长度必须至少为 2 个字符。

> 这是 Docker 客户端的限制。客户端会将单个字符名称解释为 Windows 路径。 "请参见错误 25773"。

Docker Swarm 的某些行为会阻止 Astra Trident 在每个存储和驱动程序组合中为其提供支持。

- Docker Swarm 目前使用卷名称而非卷 ID 作为其唯一卷标识符。
- 卷请求会同时发送到 Swarm 集群中的每个节点。
- 卷插件（包括 Astra Trident ）必须在 Swarm 集群中的每个节点上独立运行。由于 ONTAP 的工作方式以及 `ontap-NAS` 和 `ontap-san` 驱动程序的工作方式，它们恰好是唯一能够在这些限制范围内运行的驱动程序。

其余驱动程序可能会受到诸如争用情况等问题的影响，这些问题可能会导致为单个请求创建大量卷，而无需明确的 " 赢家 "；例如， Element 具有一项功能，允许卷具有相同的名称，但 ID 不同。

NetApp 已向 Docker 团队提供反馈，但没有任何迹象表明将来可以采用。

如果要配置 FlexGroup ，则在第二个 FlexGroup 具有一个或多个与要配置的 FlexGroup 相同的聚合时， ONTAP 不会配置第二个 FlexGroup 。

常见问题解答

查找有关 Astra Trident 的安装，配置，升级和故障排除的常见问题解答。

一般问题

Astra Trident 的发布频率如何？

Astra Trident 每三个月发布一次： 1 月， 4 月， 7 月和 10 月。这是 Kubernetes 发布后一个月。

Astra Trident 是否支持特定版本的 Kubernetes 中发布的所有功能？

Astra Trident 通常不支持 Kubernetes 中的 alpha 功能。在 Kubernetes 测试版之后的两个 Trident 版本中，Trident 可能支持测试版功能。

Astra Trident 的运行是否依赖于其他 NetApp 产品？

Astra Trident 与其他 NetApp 软件产品没有任何依赖关系，它可以作为独立应用程序运行。但是，您应具有 NetApp 后端存储设备。

如何获取完整的 Astra Trident 配置详细信息？

使用 `tridentctl get` 命令可获取有关 Astra Trident 配置的详细信息。

我能否获取有关 Astra Trident 如何配置存储的指标？

是的。Trident 20.01 引入了 Prometheus 端点，可用于收集有关 Astra Trident 操作的信息，例如托管的后端数量，配置的卷数量，占用的字节数等。您还可以使用 Cloud Insights 进行监控和分析。

使用 Astra Trident 作为 CSI 配置程序时，用户体验是否会发生变化？

否在用户体验和功能方面没有变化。使用的配置程序名称是 `csi.trident.netapp.io` 。如果要使用当前版本和未来版本提供的所有新功能，建议使用此方法安装 Astra Trident 。

在 Kubernetes 集群上安装和使用 Astra Trident

支持哪些版本的 etcd ？

Astra Trident 不再需要 `etcd` 。它使用 CRD 来保持状态。

Astra Trident 是否支持从专用注册表脱机安装？

可以， Astra Trident 可以脱机安装。请参见 "此处"。

是否可以远程安装 Astra Trident？

是的。Astra Trident 18.10 及更高版本支持从 `kubectl` 访问集群的任何计算机远程安装。验证 `kubectl` 访问后（例如，从远程计算机启动 `kubectl get nodes` 命令进行验证），请按照安装说明进行操作。

是否可以使用 Astra Trident 配置高可用性?

Astra Trident 以 Kubernetes 部署（ ReplicaSet ）的形式安装有一个实例，因此它具有内置的 HA 。您不应增加部署中的副本数量。如果安装了 Astra Trident 的节点丢失或 POD 无法访问， Kubernetes 会自动将 POD 重新部署到集群中运行正常的节点。Astra Trident 仅支持控制平面，因此，如果重新部署 Astra Trident ，当前安装的 Pod 不会受到影响。

Astra Trident 是否需要访问 Kube-system 命名空间?

Astra Trident 从 Kubernetes API 服务器读取数据，以确定应用程序何时请求新的 PVC ，因此需要访问 Kube-system 。

Astra Trident 使用哪些角色和特权?

Trident 安装程序会创建一个 Kubernetes ClusterRole ，该 ClusterRole 可对集群的 PersistentVolume ，PersistentVolumeClaim ， StorageClass 和 Kubernetes 集群的 Secret 资源进行特定访问。请参见 "此处"。

是否可以在本地生成 Astra Trident 用于安装的准确清单文件?

如果需要，您可以在本地生成和修改 Astra Trident 用于安装的确切清单文件。请参见 "此处"。

是否可以为两个单独的 Kubernetes 集群的两个单独的 Astra Trident 实例共享同一个 ONTAP 后端 SVM？

尽管不建议这样做，但您可以对两个 Astra Trident 实例使用同一个后端 SVM 。在安装期间为每个实例指定唯一的卷名称和 / 或在 `setup/backend.json` 文件中指定唯一的 `StoragePrefix` 参数。这是为了确保不会对这两个实例使用相同的 FlexVol 。

是否可以在 ContainerLinux （以前称为 CoreOS ）下安装 Astra Trident？

Astra Trident 只是 Kubernetes Pod ，可安装在 Kubernetes 运行的任何位置。

是否可以将 Astra Trident 与 NetApp Cloud Volumes ONTAP 结合使用?

是的， AWS ， Google Cloud 和 Azure 支持 Astra Trident 。

Astra Trident 是否支持 Cloud Volumes Services？

是的， Astra Trident 支持 Azure 中的 Azure NetApp Files 服务以及 GCP 中的 Cloud Volumes Service 。

故障排除和支持

NetApp 是否支持 Astra Trident？

虽然 Astra Trident 是开源且免费提供的，但只要您的 NetApp 后端受支持， NetApp 就会完全支持它。

如何提出支持案例?

要提交支持案例，请执行以下操作之一：

1. 请联系您的支持客户经理并获得帮助以提交服务单。
2. 联系以提出支持案例 "NetApp 支持"。

如何生成支持日志包?

您可以运行 `tridentctl logs -a` 来创建支持包。除了在捆绑包中捕获的日志之外，还可以捕获 kubelet 日志以诊断 Kubernetes 端的挂载问题。获取 kubelet 日志的说明因 Kubernetes 的安装方式而异。

如果需要提出新功能请求，我该怎么办?

在上创建问题描述 "Trident Github" 并在问题描述的主题和问题描述中提及 * RFE* 。

我应在何处提出缺陷?

在上创建问题描述 "Astra Trident Github"。请务必包含与问题描述相关的所有必要信息和日志。

如果我有有关 Astra Trident 的快速问题需要澄清，会发生什么情况？是否有社区或论坛？

如果您有任何问题，问题或请求，请通过我们的联系我们 "Slack" 团队或 GitHub 。

我的存储系统密码已更改， Astra Trident 不再工作，如何恢复?

使用 `tridentctl update backend myBackend -f </path/to_new_backend.json> -n trident` 更新后端的密码。将示例中的 `myBackend` 替换为后端名称，将 ` /path/to_new_backend.json` 替换为正确的 `backend.json` 文件的路径。

Astra Trident 找不到我的 Kubernetes 节点。如何修复此问题?

Astra Trident 无法找到 Kubernetes 节点的原因可能有两种。这可能是因为 Kubernetes 中的网络问题描述或 DNS 问题描述。在每个 Kubernetes 节点上运行的 Trident 节点取消设置必须能够与 Trident 控制器进行通信，以便向 Trident 注册该节点。如果在安装 Astra Trident 后发生网络更改，则只有在添加到集群中的新 Kubernetes 节点上才会遇到此问题。

如果 Trident POD 被销毁，是否会丢失数据?

如果 Trident POD 被销毁，数据不会丢失。Trident 的元数据存储在 CRD 对象中。已由 Trident 配置的所有 PV 都将正常运行。

升级 Astra Trident

是否可以直接从旧版本升级到新版本（跳过几个版本）？

NetApp 支持将 Astra Trident 从一个主要版本升级到下一个直接主要版本。您可以从 18.xx 升级到 19.xx ，从 19.xx 升级到 20.xx 等。在生产部署之前，您应在实验室中测试升级。

是否可以将 Trident 降级到先前版本?

如果要降级，需要评估许多因素。请参见 "有关降级的章节"。

管理后端和卷

是否需要在 ONTAP 后端定义文件中同时定义管理和数据 LIF ？

NetApp 建议在后端定义文件中同时包含这两者。但是，只有管理 LIF 是必需的。

Astra Trident 是否可以为 ONTAP 后端配置 CHAP ？

是的。从 20.04 开始， Astra Trident 支持 ONTAP 后端的双向 CHAP 。这需要在后端配置中设置 `useCHAP=true` 。

如何使用 Astra Trident 管理导出策略?

Astra Trident 可以从 20.04 版开始动态创建和管理导出策略。这样，存储管理员便可在其后端配置中提供一个或多个 CIDR 块，并使 Trident 将属于这些范围的节点 IP 添加到其创建的导出策略中。通过这种方式， Astra Trident 会自动管理为给定 CIDR 中具有 IP 的节点添加和删除规则的操作。此功能需要 CSI Trident 。

是否可以在 DataLIF 中指定端口?

Astra Trident 19.01 及更高版本支持在 DataLIF 中指定端口。在 `backend.json` 文件中将其配置为 `"managementLIF" ： <IP address> ： <port> "` 。例如，如果管理 LIF 的 IP 地址为 192.0.2.1 ，端口为 1000 ，请配置 `"managementLIF" ： "192.0.2.1 ： 1000"` 。

IPv6 地址是否可用于管理和数据 LIF ？

是的。Astra Trident 20.01 支持为管理 LIF 定义 IPv6 地址，并为 ONTAP 后端定义 dataLIF 参数。您应确保地址遵循 IPv6 语义，并且管理 LIF 在方括号内进行定义（例如， `[ec0d ： 6504 ： a9c1 ： ae67 ： 53d1 ： 4bdf ： ab32 ： e233]` ）。您还应确保使用 ` -use-ipv6` 标志安装 Astra Trident ，以使其能够在 IPv6 上正常运行。

是否可以在后端更新管理 LIF ？

可以，可以使用 `tridentctl update backend` 命令更新后端管理 LIF 。

是否可以更新后端的数据 LIF ？

不可以，无法更新后端的数据 LIF 。

是否可以在适用于 Kubernetes 的 Astra Trident 中创建多个后端?

Astra Trident 可以同时支持多个后端，可以使用相同的驱动程序，也可以使用不同的驱动程序。

Astra Trident 如何存储后端凭据?

Astra Trident 会将后端凭据存储为 Kubernetes Secretes 。

Astra Trident 如何选择特定后端?

如果无法使用后端属性自动为某个类选择合适的池，则会使用 `storagePools` 和 `addtionalStoragePools` 参数来选择一组特定的池。

如何确保 Astra Trident 不会从特定后端配置?

`excludeStoragePools` 参数用于筛选 Astra Trident 要用于配置的一组池，并将删除匹配的任何池。

如果存在多个相同类型的后端，则 Astra Trident 如何选择要使用的后端?

如果配置了多个相同类型的后端，则 Astra Trident 会根据 `StorageClass` 和 `PersistentVolumeClaim` 中的参数选择相应的后端。例如，如果有多个 ontap-NAS 驱动程序后端，则 Astra Trident 会尝试匹配 `StorageClass` 和 `PersistentVolumeClaim` 组合中的参数，并匹配后端，这样可以满足 `StorageClass` 和 `PersistentVolumeClaim` 中列出的要求。如果有多个后端与请求匹配，则 Astra Trident 会随机从其中一个后端中进行选择。

Astra Trident 是否支持 Element 或 SolidFire 的双向 CHAP ？

是的。

Astra Trident 如何在 ONTAP 卷上部署 qtree ？ 一个卷可以部署多少个 qtree ？

`ontap-nas-economy` 驱动程序可在同一个 FlexVol 中创建多达 200 个 qtree （可在 50 到 300 之间配置），每个集群节点创建 100 ， 000 个 qtree ，每个集群创建 2.4 M 个 qtree 。当您输入由经济型驱动程序提供服务的新的 `PersistentVolumeClaim` 时，驱动程序将查看是否已存在可为新的 qtree 提供服务的 FlexVol 。如果不存在可为 qtree 提供服务的 FlexVol ，则会创建一个新的 FlexVol 。

如何为在 ONTAP NAS 上配置的卷设置 Unix 权限?

您可以通过在后端定义文件中设置参数来对 Astra Trident 配置的卷设置 Unix 权限。

如何在配置卷时配置一组显式 ONTAP NFS 挂载选项?

默认情况下， Astra Trident 不会使用 Kubernetes 将挂载选项设置为任何值。要在 Kubernetes 存储类中指定挂载选项，请按照给定示例进行操作 "此处"。

如何将配置的卷设置为特定导出策略?

要允许相应的主机访问卷，请使用后端定义文件中配置的 `exportPolicy` 参数。

如何使用 ONTAP 通过 Astra Trident 设置卷加密?

您可以使用后端定义文件中的加密参数在 Trident 配置的卷上设置加密。

通过 Astra Trident 为 ONTAP 实施 QoS 的最佳方式是什么?

使用 `StorageClasses` 为 ONTAP 实施 QoS 。

如何通过 Astra Trident 指定精简配置或厚配置?

ONTAP 驱动程序支持精简或厚配置。ONTAP 驱动程序默认为精简配置。如果需要厚配置，则应配置后端定义文件或 `StorageClass` 。如果同时配置了这两者，则 `StorageClass` 优先。为 ONTAP 配置以下内容：

1. 在 `StorageClass` 上，将 `provisioningType` 属性设置为 thick 。
2. 在后端定义文件中，通过将 `backend spaceReserve` 参数 设置为 volume 来启用厚卷。

如何确保即使意外删除了 PVC 也不会删除所使用的卷?

从版本 1.10 开始， Kubernetes 会自动启用 PVC 保护。

是否可以扩展由 Astra Trident 创建的 NFS PVC ？

是的。您可以扩展由 Astra Trident 创建的 PVC 。请注意，卷自动增长是一项 ONTAP 功能，不适用于 Trident 。

如果我的卷是在 Astra Trident 外部创建的，是否可以将其导入到 Astra Trident ？

从 19.04 开始，您可以使用卷导入功能将卷引入 Kubernetes 。

是否可以在卷处于 SnapMirror 数据保护 （ DP ） 或脱机模式时导入它?

如果外部卷处于 DP 模式或脱机，则卷导入将失败。您会收到以下错误消息：

```
Error: could not import volume: volume import failed to get size of
volume: volume <name> was not found (400 Bad Request) command terminated
with exit code 1.
Make sure to remove the DP mode or put the volume online before importing
the volume.
```

是否可以扩展由 Astra Trident 创建的 iSCSI PVC ？

Trident 19.10 支持使用 CSI 配置程序扩展 iSCSI PV 。

如何将资源配额转换为 NetApp 集群？

只要 NetApp 存储具有容量， Kubernetes 存储资源配额就应起作用。当 NetApp 存储由于容量不足而无法支持 Kubernetes 配额设置时， Astra Trident 会尝试配置，但会出错。

是否可以使用 Astra Trident 创建卷快照？

是的。Astra Trident 支持从快照创建按需卷快照和永久性卷。要从快照创建 PV ，请确保已启用 `VolumeSnapshotDataSource` 功能门。

哪些驱动程序支持 Astra Trident 卷快照？

从目前开始，我们为 `ontap-nas` ， `ontap-nas-flexgroup` ， `ontap-san` ， `ontap-san-economics` ， `solidfire-san` ， `GCP-CVS` 和 `azure-netapp-files` 后端驱动程序。

如何为采用 ONTAP 的 Astra Trident 配置的卷创建快照备份？

此功能可从 `ontap-nas` ， `ontap-san` 和 `ontap-nas-flexgroup` 驱动程序获得。您也可以在 FlexVol 级别为 `ontap-san-economy` 驱动程序指定 `snapshotPolicy` 。

此功能也可在 `ontap-nas-economy` 驱动程序上使用，但可在 FlexVol 级别粒度上使用，而不能在 qtree 级别粒度上使用。要启用对由 Astra Trident 配置的卷创建快照的功能，请将后端参数选项 `snapshotPolicy` 设置为 ONTAP 后端定义的所需快照策略。Astra Trident 无法识别存储控制器创建的任何快照。

是否可以为通过 Astra Trident 配置的卷设置快照预留百分比？

可以，您可以通过在后端定义文件中设置 `snapshotReserve` 属性来预留特定百分比的磁盘空间，以便通过 Astra Trident 存储 Snapshot 副本。如果在后端定义文件中配置了 `snapshotPolicy` 和 `snapshotReserve` ，则会根据后端文件中提及的 `snapshotReserve` 百分比设置快照预留百分比。如果未提及 `snapshotReserve` 百分比数，则默认情况下， ONTAP 会将快照预留百分比设置为 5 。如果将 `snapshotPolicy` 选项设置为 none ，则快照预留百分比将设置为 0 。

是否可以直接访问卷快照目录和复制文件？

可以，您可以通过在后端定义文件中设置 `snapshotDir` 参数来访问 Trident 配置的卷上的 Snapshot 目录。

是否可以通过 Astra Trident 为卷设置 SnapMirror ？

目前，必须使用 ONTAP 命令行界面或 OnCommand 系统管理器在外部设置 SnapMirror 。

如何将永久性卷还原到特定 ONTAP 快照？

要将卷还原到 ONTAP 快照，请执行以下步骤：

1. 暂停正在使用永久性卷的应用程序 POD 。
2. 通过 ONTAP 命令行界面或 OnCommand 系统管理器还原到所需的快照。
3. 重新启动应用程序 POD 。

Trident是否可以在配置了负载共享镜像的SVM上配置卷？

可以为通过NFS提供数据的SVM的根卷创建负载共享镜像。ONTAP 会自动为Trident创建的卷更新负载共享镜像。这可能会导致卷挂载延迟。使用Trident创建多个卷时、配置卷取决于ONTAP 更新负载共享镜像。

如何区分每个客户 / 租户的存储类使用情况？

Kubernetes 不允许在命名空间中使用存储类。但是，您可以使用 Kubernetes 通过使用每个命名空间的存储资源配额来限制每个命名空间的特定存储类的使用。要拒绝特定命名空间对特定存储的访问，请将该存储类的资源配额设置为 0 。

支持

Astra Trident 是一个官方支持的 NetApp 项目。您可以使用任何标准机制联系 NetApp ，并获得所需的企业级支持。

此外，在 `containers` Channel on 上还有一个由容器用户（包括 Astra Trident 开发人员）组成的活跃公有社区 "NetApp 的工作时间很长"。这是一个很好的地方，可以提出有关项目的一般问题，并与志同道合的同行讨论相关主题。

故障排除

使用此处提供的指针排除安装和使用 Astra Trident 时可能遇到的问题。

> 要获得有关 Astra Trident 的帮助，请使用 `tridentctl logs -a -n trident` 创建支持包，并将其发送到 `NetApp` 支持 `<` 获取帮助 `>`。

> 有关故障排除文章的完整列表，请参见 "NetApp 知识库（需要登录）"。您还可以找到有关排除与 Astra 相关的问题的信息 "此处"。

常规故障排除

- 如果 Trident Pod 无法正常启动（例如，当 Trident Pod 停留在 `ContainerCreating` 阶段且已准备好的容器少于两个时），则运行 `kubectl -n trident describe deployment trident` and `kubectl -n trident describe pod trident -%%.%.%.* -*` 可以提供更多见解。获取 kubelet 日志（例如，通过 `jlogalctl -xeu kubelet`）也会很有帮助。
- 如果 Trident 日志中的信息不足，您可以根据安装选项将 ` -d` 标志传递到 install 参数，尝试为 Trident 启用调试模式。

  然后，使用 ` ./tridentctl logs -n trident` 并在日志中搜索 `level=debug msg` 来确认是否设置了调试。

  随操作员一起安装

  ```
  # kubectl patch torc trident -n <namespace> --type=merge -p
  '{"spec":{"debug":true}}'
  ```

  此操作将重新启动所有 Trident Pod，这可能需要几秒钟的时间。您可以通过观察 `kubectl get pod -n trident` 输出中的 "age" 列来检查此问题。

  对于 Astra Trident 20.07 和 20.10，请使用 `tprov` 代替 `Torc`。

  随 **Helm** 一起安装

  ```
  $ helm upgrade <name> trident-operator-21.07.1-custom.tgz --set
  tridentDebug=true`
  ```

  使用 **tridentctl** 安装

  ```
  ./tridentctl uninstall -n trident
  ./tridentctl install -d -n trident
  ```

- 您还可以通过在后端定义中包含 `debugTraceFlags` 来获取每个后端的调试日志。例如，包括 `debugTraceFlags : { "api" : true , "method " : true , }` 以获取 Trident 日志中的 API 调用和方法遍历。现有后端可以已配置 `debugTraceFlags` 并配置 `tridentctl` 后端更新。
- 使用 RedHat CoreOS 时，请确保在工作节点上启用了 `iscsid` 并默认启动。可以使用 OpenShift

MachineConfigs 或修改点燃模板来完成此操作。

- 使用 Trident 时可能会遇到的一个常见问题 "Azure NetApp Files" 租户和客户端密码来自权限不足的应用程序注册。有关 Trident 要求的完整列表，请参见 "Azure NetApp Files" Configuration
- 如果在将 PV 挂载到容器时出现问题，请确保 `rpcbind` 已安装并正在运行。对主机操作系统使用所需的软件包管理器，并检查 `rpcbind` 是否正在运行。您可以通过运行 `systemctl status rpcbind` 或其等效项来检查 `rpcbind` 服务的状态。
- 如果 Trident 后端报告其处于 `Failed` 状态，尽管之前已正常工作，但这可能是由于更改了与后端关联的 SVM/ 管理员凭据所致。使用 `tridentctl update backend` 更新后端信息或放弃 Trident Pod 将修复此问题描述。
- 如果要升级 Kubernetes 集群和 / 或 Trident 以使用测试版卷快照，请确保已完全删除所有现有的 alpha Snapshot CRS 。然后，您可以使用 `tridentctl obviate alpha-snapshot-crd` 命令删除 alpha snapshot CRD 。请参见 "本博客" 了解迁移 alpha 快照所涉及的步骤。
- 如果在容器运行时安装 Trident 时遇到权限问题，请使用 ` -in cluster=false` 标志尝试安装 Trident 。这不会使用安装程序 POD ，并可避免因 `trident` 安装程序 用户而出现权限问题。
- 使用 `uninstall` 参数 `<Uninstalling Trident >` 在运行失败后进行清理。默认情况下，该脚本不会删除 Trident 创建的 CRD ，因此即使在正在运行的部署中，也可以安全地卸载并重新安装。
- 如果要降级到早期版本的 Trident ，请先运行 `tridenctl uninstall` 命令以删除 Trident 。下载所需的 "Trident 版本" 并使用 `tridentctl install` 命令进行安装。只有在未创建新 PV 且未对现有 PV/ 后端 / 存储类进行更改的情况下，才考虑降级。由于 Trident 现在使用 CRD 来保持状态，因此创建的所有存储实体（后端，存储类， PV 和卷快照）都具有 关联的 `CRD` 对象 `<Kubernetes CustomResourceDefinition Objects>` ，而不是写入 PV 的数据，而 PV 中的数据是先前安装的 Trident 版本所使用的。* 新创建的 PV 在移回早期版本时不可用。 * 降级后， Trident 将无法看到对后端，PV ，存储类和卷快照（已创建 / 更新 / 删除）等对象所做的更改 * 。先前安装的 Trident 版本所使用的 PV 仍可供 Trident 查看。如果返回到早期版本，则不会中断对已使用旧版本创建的 PV 的访问，除非已对其进行升级。
- 要完全删除 Trident ，请运行 `tridentctl obviate crd` 命令。此操作将删除所有 CRD 对象并取消定义 CRD 。Trident 将不再管理其已配置的任何 PV 。

  > 之后，需要从头开始重新配置 Trident 。

- 成功安装后，如果 PVC 停留在 `Pending` 阶段，则运行 `kubectl describe PVC` 可以提供追加信息，说明 Trident 为何无法为此 PVC 配置 PV 。

使用操作员对未成功的 Trident 部署进行故障排除

如果使用操作员部署 Trident ，则 `TridentOrchestrator` 的状态将从 `Installing` 更改为 `Installed` 。如果您观察到 `failed` 状态，并且操作员无法自行恢复，则应运行以下命令来检查操作员的日志：

```
tridentctl logs -l trident-operator
```

跟踪 trident 操作器容器的日志可能会指向问题所在。例如，其中一个问题描述可能是无法从运行良好的环境中的上游注册表中提取所需的容器映像。

要了解 Trident 安装失败的原因，您应查看 `TridentOrchestrator` 的状态。

```
$ kubectl describe torc trident-2
Name:         trident-2
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  trident.netapp.io/v1
Kind:         TridentOrchestrator
...
Status:
  Current Installation Params:
    IPv6:
    Autosupport Hostname:
    Autosupport Image:
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:
    Enable Node Prep:
    Image Pull Secrets:         <nil>
    Image Registry:
    k8sTimeout:
    Kubelet Dir:
    Log Format:
    Silence Autosupport:
    Trident Image:
  Message:                      Trident is bound to another CR 'trident'
  Namespace:                    trident-2
  Status:                       Error
  Version:
Events:
  Type     Reason  Age                From                        Message
  ----     ------  ----               ----                        -------
  Warning  Error   16s (x2 over 16s)  trident-operator.netapp.io  Trident
is bound to another CR 'trident'
```

此错误表示已存在用于安装 Trident 的 `TridentOrchestrator` 。由于每个 Kubernetes 集群只能有一个 Trident 实例，因此操作员可确保在任意给定时间只存在一个可创建的活动 `TridentOrchestrator` 。

此外，观察 Trident Pod 的状态通常可以指示情况是否不正确。

```
$ kubectl get pods -n trident

NAME                                READY   STATUS             RESTARTS
AGE
trident-csi-4p5kq                   1/2     ImagePullBackOff   0
5m18s
trident-csi-6f45bfd8b6-vfrkw        4/5     ImagePullBackOff   0
5m19s
trident-csi-9q5xc                   1/2     ImagePullBackOff   0
5m18s
trident-csi-9v95z                   1/2     ImagePullBackOff   0
5m18s
trident-operator-766f7b8658-ldzsv   1/1     Running            0
8m17s
```

您可以清楚地看到，由于未提取一个或多个容器映像， Pod 无法完全初始化。

要解决此问题，您应编辑 `TridentOrchestrator` CR 。或者，您也可以删除 `TridentOrchestrator` ，并使用修改后的准确定义创建一个新的。

使用 tridentctl 对未成功的 Trident 部署进行故障排除

为了帮助您找出出现问题的原因，您可以使用 `-d` 参数再次运行安装程序，该参数将打开调试模式并帮助您了解问题所在：

```
./tridentctl install -n trident -d
```

解决此问题后，您可以按如下所示清理安装，然后再次运行 `tridentctl install` 命令：

```
./tridentctl uninstall -n trident
INFO Deleted Trident deployment.
INFO Deleted cluster role binding.
INFO Deleted cluster role.
INFO Deleted service account.
INFO Removed Trident user from security context constraint.
INFO Trident uninstallation succeeded.
```

最佳实践和建议

部署

在部署 Astra Trident 时，请使用此处列出的建议。

部署到专用命名空间

"命名空间" 在不同应用程序之间实现管理隔离，是资源共享的障碍。例如，一个命名空间中的 PVC 不能从另一个命名空间中使用。Astra Trident 为 Kubernetes 集群中的所有命名空间提供 PV 资源，从而利用具有提升权限的服务帐户。

此外，访问 Trident POD 可能会使用户能够访问存储系统凭据和其他敏感信息。请务必确保应用程序用户和管理应用程序无法访问 Trident 对象定义或 Pod 本身。

使用配额和范围限制来控制存储消耗

Kubernetes 具有两项功能，这些功能结合使用后，可提供一种功能强大的机制来限制应用程序的资源消耗。。 "存储配额机制" 使管理员能够在每个命名空间基础上实施全局容量和对象计数消耗限制以及特定于存储类的限制。此外，使用 "范围限制" 确保在将 PVC 请求转发给配置程序之前，该请求同时处于最小值和最大值范围内。

这些值是按命名空间定义的，这意味着每个命名空间都应定义符合其资源要求的值。有关信息，请参见此处 "如何利用配额"。

存储配置

NetApp 产品组合中的每个存储平台都具有独特的功能，无论应用程序是容器化还是非容器化，都能从中受益。Trident 可与 ONTAP 和 Element 结合使用。没有一个平台比另一个平台更适合所有应用程序和场景，但是，在选择平台时，应考虑应用程序和设备管理团队的需求。

您应遵循使用所使用协议的主机操作系统的基线最佳实践。或者，您也可以考虑将应用程序最佳实践（如果有）与后端，存储类和 PVC 设置结合使用，以便为特定应用程序优化存储。

ONTAP 和 Cloud Volumes ONTAP 最佳实践

了解为 Trident 配置 ONTAP 和 Cloud Volumes ONTAP 的最佳实践。

以下建议是为容器化工作负载配置 ONTAP 的准则，容器化工作负载会占用 Trident 动态配置的卷。应考虑并评估每个问题在您的环境中的适用性。

使用专用于 **Trident** 的 **SVM**

Storage Virtual Machine （ SVM ）可在 ONTAP 系统上的租户之间实现隔离和管理隔离。通过将 SVM 专用于应用程序，可以委派特权并应用最佳实践来限制资源消耗。

可通过多种方法管理 SVM：

- 在后端配置中提供集群管理接口以及相应的凭据，并指定 SVM 名称。
- 使用 ONTAP 系统管理器或命令行界面为 SVM 创建专用管理接口。

- 与 NFS 数据接口共享管理角色。

在每种情况下，接口都应位于 DNS 中，配置 Trident 时应使用 DNS 名称。这有助于在不使用网络身份保留的情况下实施某些灾难恢复方案，例如 SVM-DR 。

在为 SVM 配置专用管理 LIF 或共享管理 LIF 之间没有任何偏好，但是，您应确保网络安全策略与您选择的方法一致。无论如何，管理 LIF 应可通过 DNS 访问，以实现最大的灵活性 "SVM-DR" 与 Trident 结合使用。

限制最大卷数

ONTAP 存储系统具有最大卷数，具体取决于软件版本和硬件平台。请参见 "NetApp Hardware Universe" 以 ONTAP 确定确切限制。当卷计数用尽时，配置操作不仅会对 Trident 失败，而且会对所有存储请求失败。

Trident 的 `ontap-nas` 和 `ontap-san` 驱动程序为创建的每个 Kubernetes 永久性卷（ PV ）配置一个 FlexVolume 。`ontap-nas-economy` 驱动程序会为每 200 个 PV 创建大约一个 FlexVolume （可在 50 到 300 之间配置）。`ontap-san-economy`-经济 驱动程序会为每 100 个 PV 创建大约一个 FlexVolume （可配置为 50 到 200 ）。要防止 Trident 占用存储系统上的所有可用卷，您应对 SVM 设置限制。您可以从命令行执行此操作：

```
vserver modify -vserver <svm_name> -max-volumes <num_of_volumes>
```

`max-volumes` 的值因环境的特定条件而异：

- ONTAP 集群中现有卷的数量
- 希望在 Trident 之外为其他应用程序配置的卷数
- Kubernetes 应用程序预期占用的永久性卷数

`max-volumes` 值是在 ONTAP 集群中的所有节点上配置的总卷数，而不是在单个 ONTAP 节点上配置的总卷数。因此，在某些情况下， ONTAP 集群节点所配置的 Trident 卷可能远远多于或少于其他节点。

例如，一个双节点 ONTAP 集群最多可以托管 2000 个 FlexVolume 。将最大卷数设置为 1250 似乎非常合理。但是，如果只是 "聚合" 从一个节点分配给 SVM ，或者从一个节点分配的聚合无法配置（例如，由于容量），则另一个节点将成为所有 Trident 配置卷的目标。这意味着，在达到 `max-volumes` 值之前，可能会达到该节点的卷限制，从而影响使用该节点的 Trident 和其他卷操作。* 您可以通过确保将集群中每个节点的聚合分配给 Trident 使用的 SVM 来避免这种情况。 *

限制 **Trident** 创建的卷的最大大小

要为 Trident 可以创建的卷配置最大大小，请在 `backend.json` 定义中使用 `limitVolumeSize` 参数。

除了控制存储阵列上的卷大小之外，您还应利用 Kubernetes 功能。

配置 **Trident** 以使用双向 **CHAP**

您可以在后端定义中指定 CHAP 启动程序以及目标用户名和密码，并在 SVM 上启用 Trident CHAP 。在后端配置中使用 `useCHAP` 参数， Trident 使用 CHAP 对 ONTAP 后端的 iSCSI 连接进行身份验证。Trident 20.04 及更高版本支持双向 CHAP 。

创建并使用 **SVM QoS** 策略

利用应用于 SVM 的 ONTAP QoS 策略，限制 Trident 配置的卷可使用的 IOPS 数量。这有助于实现 "防止抢占资源" 或控制不足的容器，以使其不会影响 Trident SVM 以外的工作负载。

您可以通过几个步骤为 SVM 创建 QoS 策略。有关最准确的信息，请参见适用于您的 ONTAP 版本的文档。以下示例将创建一个 QoS 策略，将 SVM 可用的总 IOPS 限制为 5000 。

```
# create the policy group for the SVM
qos policy-group create -policy-group <policy_name> -vserver <svm_name>
-max-throughput 5000iops

# assign the policy group to the SVM, note this will not work
# if volumes or files in the SVM have existing QoS policies
vserver modify -vserver <svm_name> -qos-policy-group <policy_name>
```

此外，如果您的 ONTAP 版本支持此功能，则可以考虑使用最低 QoS 来保证容器化工作负载的吞吐量。自适应 QoS 与 SVM 级别策略不兼容。

专用于容器化工作负载的 IOPS 数量取决于许多方面。其中包括：

- 使用存储阵列的其他工作负载。如果存在与 Kubernetes 部署无关的其他工作负载，则应注意利用存储资源，以确保这些工作负载不会意外受到不利影响。
- 容器中运行的预期工作负载。如果 IOPS 要求较高的工作负载将在容器中运行，则 QoS 策略较低会导致出现不良体验。

请务必记住，在 SVM 级别分配的 QoS 策略会导致配置到 SVM 的所有卷共享同一个 IOPS 池。如果一个或少量容器化应用程序的 IOPS 要求较高，则可能会成为其他容器化工作负载的抢占资源的应用程序。如果是这种情况，您可能需要考虑使用外部自动化来分配每个卷的 QoS 策略。

> 如果 ONTAP 版本早于 9.8 ，则应将此 QoS 策略组分配给 SVM * 仅 * 。

为 **Trident** 创建 **QoS** 策略组

服务质量（ QoS ）可确保关键工作负载的性能不会因争用工作负载而降级。ONTAP QoS 策略组为卷提供 QoS 选项，并使用户能够为一个或多个工作负载定义吞吐量上限。有关 QoS 的详细信息，请参见 "通过 QoS 保证吞吐量"。您可以在后端或存储池中指定 QoS 策略组，这些策略组将应用于该池或后端创建的每个卷。

ONTAP 有两种类型的 QoS 策略组：传统和自适应。传统策略组以 IOPS 为单位提供固定的最大（或最小）吞吐量。自适应 QoS 会根据工作负载大小自动扩展吞吐量，并在工作负载大小发生变化时保持 IOPS 与 TBSGB 的比率。如果您要在大型部署中管理数百或数千个工作负载，则这将带来显著优势。

创建 QoS 策略组时，请考虑以下事项：

- 您应在后端配置的 `defaults` 块中设置 `qosPolicy` 密钥。请参见以下后端配置示例：

```
{
  "version": 1,
  "storageDriverName": "ontap-nas",
  "managementLIF": "0.0.0.0",
  "dataLIF": "0.0.0.0",
  "svm": "svm0",
  "username": "user",
  "password": "pass",
  "defaults": {
    "qosPolicy": "standard-pg"
  },
  "storage": [
    {
      "labels": {"performance": "extreme"},
      "defaults": {
        "adaptiveQosPolicy": "extremely-adaptive-pg"
      }
    },
    {
      "labels": {"performance": "premium"},
      "defaults": {
        "qosPolicy": "premium-pg"
      }
    }
  ]
}
```

- 您应该对每个卷应用策略组，以便每个卷都获得策略组指定的整个吞吐量。不支持共享策略组。

有关 QoS 策略组的详细信息，请参见 "ONTAP 9.8 QoS 命令"。

将存储资源访问限制为 **Kubernetes** 集群成员

限制对 Trident 创建的 NFS 卷和 iSCSI LUN 的访问是 Kubernetes 部署安全状况的重要组成部分。这样可以防止不属于 Kubernetes 集群的主机访问卷并可能意外修改数据。

请务必了解命名空间是 Kubernetes 中资源的逻辑边界。假设同一命名空间中的资源可以共享，但重要的是，没有跨命名空间功能。这意味着，即使 PV 是全局对象，但在绑定到 PVC 时，它们只能由同一命名空间中的 Pod 访问。* 请务必确保使用命名空间在适当时提供分隔。 *

大多数组织在 Kubernetes 环境中的数据安全方面的主要顾虑是，容器中的进程可以访问挂载到主机但不适用于容器的存储。 "命名空间" 旨在防止这种类型的损害。但是，存在一个例外：特权容器。

有权限的容器是指运行时拥有比正常情况更多主机级别权限的容器。默认情况下，这些选项不会被拒绝，因此请确保使用禁用此功能 "POD 安全策略"。

对于需要从 Kubernetes 和外部主机访问的卷，应采用传统方式管理存储，并由管理员引入 PV ，而不是由

Trident 管理。这样可以确保只有在 Kubernetes 和外部主机断开连接且不再使用此卷时，才会销毁此存储卷。此外，还可以应用自定义导出策略，以便从 Kubernetes 集群节点和 Kubernetes 集群以外的目标服务器进行访问。

对于具有专用基础架构节点（例如 OpenShift ）或其他不能为用户应用程序计划的节点的部署，应使用单独的导出策略进一步限制对存储资源的访问。其中包括为部署到这些基础架构节点的服务（例如 OpenShift 指标和日志记录服务）以及部署到非基础架构节点的标准应用程序创建导出策略。

使用专用导出策略

您应确保每个后端都有一个导出策略，该策略仅允许访问 Kubernetes 集群中的节点。从 20.04 版开始， Trident 可以自动创建和管理导出策略。通过这种方式， Trident 会限制对其配置给 Kubernetes 集群中节点的卷的访问，并简化节点的添加 / 删除。

或者，您也可以手动创建导出策略，并使用一个或多个导出规则来填充此策略，这些导出规则用于处理每个节点访问请求：

- 使用 `vserver export-policy create` ONTAP 命令行界面命令创建导出策略。
- 使用 `vserver export-policy rule create` ONTAP 命令行界面命令向导出策略添加规则。

通过运行这些命令，您可以限制哪些 Kubernetes 节点可以访问数据。

为应用程序 **SVM** 禁用 **showmount**

通过 `showmount` 功能， NFS 客户端可以向 SVM 查询可用 NFS 导出列表。部署到 Kubernetes 集群的 Pod 可以对数据 LIF 执行问题描述 `showmount -e` 命令，并接收可用挂载列表，包括其无权访问的挂载。虽然这本身并不会影响安全，但它确实会提供不必要的信息，可能有助于未经授权的用户连接到 NFS 导出。

您应使用 SVM 级别的 ONTAP 命令行界面命令禁用 `showmount`：

```
vserver nfs modify -vserver <svm_name> -showmount disabled
```

SolidFire 最佳实践

了解为 Trident 配置 SolidFire 存储的最佳实践。

创建 **SolidFire** 帐户

每个 SolidFire 帐户都代表一个唯一的卷所有者，并接收自己的一组质询握手身份验证协议（ Challenge-Handshake Authentication Protocol ， CHAP ）凭据。您可以使用帐户名称和相对 CHAP 凭据或通过卷访问组访问分配给帐户的卷。一个帐户最多可以分配 2 ， 000 个卷，但一个卷只能属于一个帐户。

创建 **QoS** 策略

如果要创建并保存可应用于多个卷的标准化服务质量设置，请使用 SolidFire 服务质量（ QoS ）策略。

您可以按卷设置 QoS 参数。通过设置三个可配置的参数来定义 QoS ，可以确保每个卷的性能：最小 IOPS ，最大 IOPS 和突发 IOPS 。

以下是 4 KB 块大小的可能最小，最大和突发 IOPS 值。

IOPS 参数	定义	最小value	默认值	最大值（4 KB）
最小 IOPS	卷的性能保障级别。	50	50	15000
最大 IOPS	性能不会超过此限制。	50	15000	200,000
突发 IOPS	在短时突发情形下允许的最大 IOPS 。	50	15000	200,000

> 虽然最大 IOPS 和突发 IOPS 可设置为高达 200， 000，但卷的实际最大性能受集群使用情况和每节点性能的限制。

块大小和带宽会直接影响 IOPS 数量。随着块大小的增加，系统会将带宽增加到处理较大块大小所需的级别。随着带宽的增加，系统能够达到的 IOPS 数量也会减少。请参见 "SolidFire 服务质量" 有关 QoS 和性能的详细信息。

SolidFire 身份验证

Element 支持两种身份验证方法： CHAP 和卷访问组（VAG）。CHAP 使用 CHAP 协议向后端对主机进行身份验证。卷访问组控制对其配置的卷的访问。NetApp 建议使用 CHAP 进行身份验证，因为它更简单，并且没有扩展限制。

> 具有增强型 CSI 配置程序的 Trident 支持使用 CHAP 身份验证。VAG 只能在传统的非 CSI 操作模式下使用。

只有基于帐户的访问控制才支持 CHAP 身份验证（验证启动程序是否为目标卷用户）。如果使用 CHAP 进行身份验证，则可以使用两个选项：单向 CHAP 和双向 CHAP 。单向 CHAP 使用 SolidFire 帐户名称和启动程序密钥对卷访问进行身份验证。双向 CHAP 选项可提供最安全的卷身份验证方法，因为卷会通过帐户名称和启动程序密钥对主机进行身份验证，然后主机通过帐户名称和目标密钥对卷进行身份验证。

但是，如果无法启用 CHAP 且需要使用 VAG ，请创建访问组并将主机启动程序和卷添加到此访问组。添加到访问组的每个 IQN 都可以使用或不使用 CHAP 身份验证访问组中的每个卷。如果将 iSCSI 启动程序配置为使用 CHAP 身份验证，则会使用基于帐户的访问控制。如果 iSCSI 启动程序未配置为使用 CHAP 身份验证，则会使用卷访问组访问控制。

如何查找更多信息

下面列出了一些最佳实践文档。搜索 "NetApp 库" 对于最新版本。

- ONTAP *
- "NFS 最佳实践和实施指南"
- "《 SAN 管理指南》" （对于 iSCSI ）
- "适用于 RHEL 的 iSCSI 快速配置"
- Element 软件 *
- "配置适用于 Linux 的 SolidFire"
- NetApp HCI *

- "NetApp HCI 部署前提条件"
- "访问 NetApp 部署引擎"
- 应用程序最佳实践信息 *
- "基于 ONTAP 的 MySQL 最佳实践"
- "基于 SolidFire 的 MySQL 最佳实践"
- "NetApp SolidFire 和 Cassandra"
- "SolidFire 上的 Oracle 最佳实践"
- "SolidFire 上的 PostgreSQL 最佳实践"

并非所有应用程序都有特定的准则，与您的 NetApp 团队合作并使用非常重要 "NetApp 库" 以查找最新文档。

集成 Astra Trident

要集成 Astra Trident ，需要集成以下设计和架构要素：驱动程序选择和部署，存储类设计，虚拟存储池设计，永久性卷声明（ PVC ）对存储配置的影响，卷操作以及使用 Astra Trident 部署 OpenShift 服务。

驱动程序选择和部署

为 **ONTAP** 选择后端驱动程序

ONTAP 系统可使用四种不同的后端驱动程序。这些驱动程序根据所使用的协议以及在存储系统上配置卷的方式进行了区分。因此，请仔细考虑要部署的驱动程序。

更高级别的是，如果您的应用程序中的组件需要共享存储（多个 Pod 访问同一个 PVC ），则基于 NAS 的驱动程序将成为默认选项，而基于块的 iSCSI 驱动程序则可满足非共享存储的需求。根据应用程序要求以及存储和基础架构团队的舒适程度选择协议。一般来说，对于大多数应用程序来说，它们之间没有什么区别，因此通常是根据是否需要共享存储（多个 POD 需要同时访问）来决定的。

下面列出了 ONTAP 后端的五个驱动程序：

- `ontap-NAS`： 配置的每个 PV 都是一个完整的 ONTAP FlexVolume 。
- `ontap-nas-economy.`： 配置的每个 PV 都是一个 qtree ，每个 FlexVolume 的 qtree 数量是可配置的（默认值为 200 ）。
- `ontap-nas-flexgroup`： 使用配置为完整 ONTAP FlexGroup 的每个 PV ，以及分配给 SVM 的所有聚合。
- `ontap-san`： 配置的每个 PV 都是其自身 FlexVolume 中的一个 LUN 。
- `ontap-san-economi`： 配置的每个 PV 都是一个 LUN ，每个 FlexVolume 具有可配置的 LUN 数量（默认值为 100 ）。

在三个 NAS 驱动程序之间进行选择会对应用程序可用的功能产生一些影响。

请注意，在下表中，并非所有功能都通过 Astra Trident 公开。如果需要某些功能，存储管理员必须在配置后应用这些功能。上标脚注区分了每个功能和驱动程序的功能。

ONTAP NAS 驱动程序	快照	克隆	动态导出策略	多连接	QoS	调整大小	Replication
ontap-NAS	是的。	是的。	是脚注：5[]	是的。	是脚注：1[]	是的。	是脚注：1[]
ontap-nas-economy.	是脚注：3[]	是脚注：3[]	是脚注：5[]	是的。	是脚注：3[]	是的。	是脚注：3[]
ontap-nas-flexgroup	是脚注：1[]	否	是脚注：5[]	是的。	是脚注：1[]	是的。	是脚注：1[]

Astra Trident 为 ONTAP 提供了 2 个 SAN 驱动程序，其功能如下所示。

ONTAP SAN 驱动程序	快照	克隆	多连接	双向 CHAP	QoS	调整大小	Replication
ontap-san	是的。	是的。	是脚注：4[]	是的。	是脚注：1[]	是的。	是脚注：1[]
ontap-san-economy.	是的。	是的。	是脚注：4[]	是的。	是脚注：3[]	是脚注：1[]	是脚注：3[]

> 上述表的脚注：是脚注： 1[]： 不受 Astra Trident 管理是脚注： 2[]： 由 Astra Trident 管理，但不是 PV 粒度 Yesfootnote： 3[]： 不受 Astra Trident 管理，也不是 PV 粒度 Yesnote： 4[]： 支持原始块卷是脚注： 5[]： CSI Trident 支持

非 PV 粒度功能将应用于整个 FlexVolume ，而所有 PV （即共享 FlexVol 中的 qtree 或 LUN ）将共享一个通用计划。

如上表所示， ontap-nas 与 ontap-nas-economy 之间的大部分功能是相同的。但是，由于 ontap-nas-economy 驱动程序限制了按 PV 粒度控制计划的能力，因此这可能会特别影响灾难恢复和备份规划。对于希望在 ONTAP 存储上利用 PVC 克隆功能的开发团队，只有在使用 ontap-NAS ， ontap-san 或 ontap-san-economy-经济 驱动程序时，才可能实现这一点。

> solidfire-san 驱动程序还能够克隆 PVC 。

为 **Cloud Volumes ONTAP** 选择后端驱动程序

Cloud Volumes ONTAP 可为各种使用情形提供数据控制以及企业级存储功能，包括文件共享和为 NAS 和 SAN 协议（ NFS ， SMB/CIFS 和 iSCSI ）提供服务的块级存储。Cloud Volume ONTAP 的兼容驱动程序包括 ontap-nas ， ontap-nas-economy. ontap-san 和 `ontap-san-economy.`它们适用于适用于 Azure 的 Cloud Volume ONTAP ，适用于 GCP 的 Cloud Volume ONTAP 。

为适用于 **ONTAP** 的 **Amazon FSX** 选择后端驱动程序

借助适用于 ONTAP 的 Amazon FSX ，客户可以利用他们熟悉的 NetApp 功能，性能和管理功能，同时利用在 AWS 上存储数据的简便性，灵活性，安全性和可扩展性。FSX for ONTAP 支持 ONTAP 的许多文件系统功能和管理 API 。Cloud Volume ONTAP 的兼容驱动程序包括 ontap-nas ， ontap-nas-economy. ontap-nas-flexgroup ， ontap-san 和 ontap-san-economy.

为 **NetApp HCI/SolidFire** 选择后端驱动程序

与 NetApp HCI/SolidFire 平台结合使用的 `solidfire-san` 驱动程序可帮助管理员根据 QoS 限制为 Trident 配置 Element 后端。如果您希望设计后端，以便为 Trident 配置的卷设置特定的 QoS 限制，请在后端文件中使用 `type` 参数。管理员还可以使用 `limitVolumeSize` 参数限制可在存储上创建的卷大小。目前， `solidfire-san` 驱动程序不支持卷大小调整和卷复制等 Element 存储功能。这些操作应通过 Element Software Web UI 手动完成。

SolidFire 驱动程序	快照	克隆	多连接	**CHAP**	**QoS**	调整大小	**Replication**
`solidfire-san`	是的。	是的。	是脚注：2[]	是的。	是的。	是的。	是脚注：1[]

> 脚注：是脚注： 1[]： 不由 Astra Trident 管理是脚注： 2[]： 支持原始块卷

为 **Azure NetApp Files** 选择后端驱动程序

Astra Trident 使用 `azure-netapp-files` 驱动程序来管理 "Azure NetApp Files" 服务

有关此驱动程序及其配置方法的详细信息，请参见 "适用于 Azure NetApp Files 的 Astra Trident 后端配置"。

Azure NetApp Files 驱动程序	快照	克隆	多连接	**QoS**	展开	**Replication**
`azure-netapp-files`	是的。	是的。	是的。	是的。	是的。	是脚注： 1[]

> 脚注：是脚注： 1[]： 不由 Astra Trident 管理

为采用 **GCP** 的 **Cloud Volumes Service** 选择后端驱动程序

Astra Trident 使用 `GCP-CVS` 驱动程序与 GCP 后端的 Cloud Volumes Service 链接。要在 Trident 上配置 GCP 后端，您需要在后端文件中指定 `projectNumber` ， `apiRegion` 和 `apiKey` 。项目编号可在 GCP Web 门户中找到，而 API 密钥必须从您在 GCP 上为 Cloud Volumes 设置 API 访问时创建的服务帐户专用密钥文件中获取。Astra Trident 可以在两个卷中创建 CVS 卷之一 "服务类型"：

1. * CVS*： 基本 CVS 服务类型，可提供较高的区域可用性，但性能级别有限 / 中等。
2. * CVS-Performance*： 经过性能优化的服务类型最适合重视性能的生产工作负载。从三个独特的服务级别中进行选择 `standard` ， `Premium` 和 `Extreme` 。目前， 100 GiB 是要配置的最小 CVS-Performance 卷大小，而 CVS 卷必须至少为 300 GiB 。未来版本的 CVS 可能会删除此限制。

> 使用默认 CVS 服务类型 `storageClass=software` 部署后端时，用户 * 必须获得 * 对 GCP 上相关项目编号和项目 ID 的子 1TiB 卷功能的访问权限。这对于 Trident 配置低于 1TiB 的卷是必需的。否则，对于小于 600 GiB 的 PVC ，卷创建 * 将失败 * 。使用 … "此表单" 以访问低于 1 TiB 的卷。

适用于 **GCP** 的 **CVS** 驱动程序	快照	克隆	多连接	**QoS**	展开	**Replication**
`GCP-CVS`	是的。	是的。	是的。	是的。	是的。	是脚注： 1[]

> 脚注：是脚注： 1[]：不由 Astra Trident 管理

`GCP-CVS` 驱动程序使用虚拟存储池。虚拟存储池会对后端进行抽象化，从而使 Astra Trident 决定卷的放置。管理员在 backend.json 文件中定义虚拟存储池。存储类使用标签标识虚拟存储池。

存储类设计

要创建 Kubernetes 存储类对象，需要配置并应用各个存储类。本节讨论如何为您的应用程序设计存储类。

针对特定后端利用率的存储类设计

可以在特定存储类对象中使用筛选功能来确定要将哪个存储池或一组池与该特定存储类结合使用。可以在存储类中设置三组筛选器： `storagePools` ， `addtionalStoragePools` 和 / 或 `excludeStoragePools` 。

`storagePools` 参数有助于将存储限制为与任何指定属性匹配的一组池。`addtionalStoragePools` 参数用于扩展 Astra Trident 用于配置的池集以及由属性和 `storagePools` 参数选择的池集。您可以单独使用参数，也可以同时使用这两个参数，以确保选择适当的存储池集。

`excludeStoragePools` 参数用于明确排除列出的一组与属性匹配的池。

用于模拟 **QoS** 策略的存储类设计

如果要设计存储类以模拟服务质量策略，请创建一个存储类，其中 `mmedia` attribute 为 `HDD` 或 `sSD` 。根据存储类中提及的 `mmedia` 属性， Trident 将选择提供 `HDD` 或 `sSD` 聚合的相应后端，以便与介质属性匹配，然后将卷配置定向到特定聚合。因此，我们可以创建一个存储类高级版，该高级版会将 `mmedia` attribute 设置为 `sSD` ，该高级版可归类为高级 QoS 策略。我们可以创建另一个存储类标准，该标准会将介质属性设置为 `HDD' ，并可归类为标准 QoS 策略。我们还可以使用存储类中的 ``IOPS' 属性将配置重定向到可定义为 QoS 策略的 Element 设备。

存储类设计，可根据特定功能利用后端

存储类可设计为在启用了精简和厚配置，快照，克隆和加密等功能的特定后端直接配置卷。要指定要使用的存储，请创建存储类，以指定启用了所需功能的相应后端。

虚拟存储池的存储类设计

所有 Astra Trident 后端均可使用虚拟存储池。您可以使用 Astra Trident 提供的任何驱动程序为任何后端定义虚拟存储池。

通过虚拟存储池，管理员可以在后端创建一个抽象级别，并可通过存储类进行引用，从而提高卷在后端的灵活性和效率。可以使用相同的服务类定义不同的后端。此外，可以在同一后端创建多个存储池，但其特征不同。如果为存储类配置了具有特定标签的选择器，则 Astra Trident 会选择与所有选择器标签匹配的后端来放置卷。如果存储类选择器标签与多个存储池匹配，则 Astra Trident 将选择其中一个存储池来配置卷。

虚拟存储池设计

创建后端时，通常可以指定一组参数。管理员无法使用相同的存储凭据和一组不同的参数创建另一个后端。随着虚拟存储池的推出，此问题描述得以缓解。虚拟存储池是在后端和 Kubernetes 存储类之间引入的级别抽象，因此管理员可以定义参数以及标签，这些参数和标签可以通过 Kubernetes 存储类作为选择器进行引用，并且与后端无关。可以使用 Astra Trident 为所有受支持的 NetApp 后端定义虚拟存储池。该列表包括 SolidFire/NetApp HCI ， ONTAP ， GCP 上的 Cloud Volumes Service 以及 Azure NetApp Files 。

> 定义虚拟存储池时，建议不要尝试在后端定义中重新排列现有虚拟池的顺序。此外，建议不要编辑 / 修改现有虚拟池的属性，而是定义新的虚拟池。

设计虚拟存储池以模拟不同的服务级别 **/QoS**

可以设计虚拟存储池来模拟服务类。使用适用于 Azure NetApp Files 的云卷服务的虚拟池实施，让我们来了解一下如何设置不同的服务类。为 ANF 后端配置多个标签，以表示不同的性能级别。将 `servicelevel` Aspect 设置为适当的性能级别，并在每个标签下添加其他所需的内容。现在，创建可映射到不同虚拟存储池的不同 Kubernetes 存储类。使用 `parameters.selector` 字段，每个 StorageClass 都会调用可用于托管卷的虚拟池。

设计用于分配特定方面的虚拟池

可以从一个存储后端设计具有一组特定方面的多个虚拟存储池。为此，请为后端配置多个标签，并在每个标签下设置所需的方面。现在，使用 `parameters.selector` 字段创建不同的 Kubernetes 存储类，该字段将映射到不同的虚拟存储池。在后端配置的卷将在选定的虚拟存储池中定义相关方面。

影响存储配置的 **PVC** 特征

创建 PVC 时，请求的存储类以外的某些参数可能会影响 Astra Trident 的配置决策过程。

访问模式

通过 PVC 请求存储时，访问模式为必填字段之一。所需的模式可能会影响所选的托管存储请求的后端。

Astra Trident 将尝试与根据下表指定的访问方法所使用的存储协议匹配。这独立于底层存储平台。

	ReadWriteOnce	**ReadOnlyMany**	读取写入任何
iSCSI	是的。	是的。	是（原始块）
NFS	是的。	是的。	是的。

如果在未配置 NFS 后端的情况下向 Trident 部署提交了 ReadWriteMany PVC 请求，则不会配置任何卷。因此，请求者应使用适合其应用程序的访问模式。

卷操作

修改永久性卷

除了两个例外，永久性卷是 Kubernetes 中不可变的对象。创建后，可以修改回收策略和大小。但是，这并不会阻止在 Kubernetes 外部修改卷的某些方面。为了针对特定应用程序自定义卷，确保容量不会意外占用，或者出于任何原因将卷移动到其他存储控制器，这一点可能是理想的。

> 目前， Kubernetes 树中配置程序不支持对 NFS 或 iSCSI PV 执行卷大小调整操作。Astra Trident 支持扩展 NFS 和 iSCSI 卷。

创建 PV 后，无法修改其连接详细信息。

创建按需卷快照

Astra Trident 支持按需创建卷快照，并使用 CSI 框架从快照创建 PVC 。快照提供了一种维护数据时间点副本的便捷方法，并且生命周期独立于 Kubernetes 中的源 PV 。这些快照可用于克隆 PVC 。

从快照创建卷

Astra Trident 还支持从卷快照创建 PersistentVolumes 。为此，只需创建 PersistentVolumeClaim 并注明 `datasource` 作为创建卷所需的快照即可。Astra Trident 将通过创建包含快照上的数据的卷来处理此 PVC 。通过此功能，可以跨区域复制数据，创建测试环境，整体更换损坏或损坏的生产卷，或者检索特定文件和目录并将其传输到另一个连接的卷。

移动集群中的卷

存储管理员可以在 ONTAP 集群中的聚合和控制器之间无中断地将卷移动到存储使用者。此操作不会影响 Astra Trident 或 Kubernetes 集群，只要目标聚合是 Astra Trident 所使用的 SVM 有权访问的聚合即可。重要的是，如果已将聚合新添加到 SVM ，则需要通过将后端重新添加到 Astra Trident 来刷新后端。这将触发 Astra Trident 对 SVM 重新进行清单配置，以便识别新聚合。

但是， Astra Trident 不支持在后端之间自动移动卷。这包括在同一集群中的 SVM 之间，集群之间或不同存储平台上（即使该存储系统是连接到 Astra Trident 的存储系统也是如此）。

如果将卷复制到其他位置，则可以使用卷导入功能将当前卷导入到 Astra Trident 中。

展开卷

Astra Trident 支持调整 NFS 和 iSCSI PV 的大小。这样，用户就可以直接通过 Kubernetes 层调整其卷的大小。所有主要 NetApp 存储平台均可进行卷扩展，包括 ONTAP ， SolidFire/NetApp HCI 和 Cloud Volumes Service 后端。要稍后允许扩展，请在与卷关联的 StorageClass 中将 `allowVolumeExpansion` 设置为 `true` 。每当需要调整永久性卷的大小时，请在永久性卷声明中编辑 `sPec.resources.requests.storage` 标注以指定所需的卷大小。Trident 会自动调整存储集群上卷的大小。

将现有卷导入到 **Kubernetes** 中

通过卷导入，可以将现有存储卷导入到 Kubernetes 环境中。目前， `ontap-nas` ， `ontap-nas-flexgroup` ， `solidfire-san` ， `azure-netapp-files` 和 `GCP-CVS` 驱动程序均支持此功能。在将现有应用程序移植到 Kubernetes 或在灾难恢复场景中，此功能非常有用。

使用 ONTAP 和 `solidfire-san` 驱动程序时，请使用命令 `tridentctl import volume <backend-name> <volume-name> -f /path/PVC.YAML` 将现有卷导入到要由 Astra Trident 管理的 Kubernetes 中。导入卷命令中使用的 PVC YAML 或 JSON 文件指向将 Astra Trident 标识为配置程序的存储类。使用 NetApp HCI/SolidFire 后端时，请确保卷名称是唯一的。如果卷名称重复，请将卷克隆为唯一名称，以便卷导入功能可以区分它们。

如果使用了 `azure-netapp-files` 或 `gcp-cvs` 驱动程序，请使用命令 `tridentctl import volume <backend-name> <volume path> -f /path/vc.yaml` 将卷导入到 Kubernetes 中，以供 Astra Trident 管理。这样可以确保卷引用是唯一的。

执行上述命令后， Astra Trident 将在后端找到卷并读取其大小。它将自动添加（并在必要时覆盖）已配置的 PVC 卷大小。然后， Astra Trident 会创建新的 PV ， Kubernetes 会将 PVC 绑定到 PV 。

如果部署的容器需要特定的导入 PVC ，则容器将保持待定状态，直到通过卷导入过程绑定 PVC/PV 对为止。在绑定 PVC/PV 对后，如果没有其他问题，应启动容器。

部署 OpenShift 服务

OpenShift 增值集群服务为集群管理员和要托管的应用程序提供了重要功能。这些服务使用的存储可以使用节点本地资源进行配置，但这通常会限制服务的容量，性能，可恢复性和可持续性。利用企业级存储阵列为这些服务提供容量可以显著改善服务，但是，与所有应用程序一样， OpenShift 和存储管理员应密切合作，为每个服务确定最佳选项。应大量利用 Red Hat 文档来确定要求并确保满足规模估算和性能需求。

注册表服务

有关为注册表部署和管理存储的文档，请参见 "netapp.io" 在中 "博客"。

日志记录服务

与其他 OpenShift 服务一样，日志记录服务也是使用清单文件（也称为）提供的配置参数 Ansible 部署的主机，提供给攻略手册。其中包括两种安装方法：在初始 OpenShift 安装期间部署日志记录以及在安装 OpenShift 之后部署日志记录。

> 自 Red Hat OpenShift 3.9 版开始，官方文档出于对数据损坏的担忧，建议不要对日志记录服务使用 NFS 。这是基于 Red Hat 对其产品的测试得出的。ONTAP 的 NFS 服务器不存在这些问题，可以轻松地备份日志记录部署。最终，您可以选择日志记录服务的协议，只需了解这两种协议在使用 NetApp 平台时都能很好地发挥作用，如果您愿意，也没有理由避免使用 NFS 。

如果您选择将 NFS 与日志记录服务结合使用，则需要将 Ansible 变量 `OpenShift_enable_unsupported_configurations` 设置为 `true` ，以防止安装程序失败。

入门

可以选择为这两个应用程序以及 OpenShift 集群本身的核心操作部署日志记录服务。如果选择部署操作日志记录，则通过将变量 `OpenShift_logging_use_ops` 指定为 `true` ，将创建两个服务实例。控制操作日志记录实例的变量包含 "ops" ，而应用程序实例则不包含 "ops" 。

要确保底层服务使用正确的存储，必须根据部署方法配置 Ansible 变量。让我们来了解一下每种部署方法的选项。

> 下表仅包含与存储配置相关的变量，因为这些变量与日志记录服务相关。您可以在中找到其他选项 "RedHat OpenShift 日志记录文档" 应根据您的部署情况查看，配置和使用。

下表中的变量将导致 Ansible 攻略手册使用提供的详细信息为日志记录服务创建 PV 和 PVC 。与在 OpenShift 安装后使用组件安装攻略手册相比，此方法的灵活性明显降低，但是，如果您有可用的现有卷，则可以选择此方法。

变量	详细信息
`OpenShift_logging_storage_kind`	设置为 `NFS` ，以使安装程序为日志记录服务创建 NFS PV 。
`OpenShift_logging_storage_host`	NFS 主机的主机名或 IP 地址。此值应设置为虚拟机的数据 LIF 。
`OpenShift_logging_storage_nfs_directory`	NFS 导出的挂载路径。例如，如果卷接合为 `/OpenShift_logging` ，则应使用该路径作为此变量。
`OpenShift_logging_storage_volume_name`	要创建的 PV 的名称，例如 `PV_ose_logs` 。

变量	详细信息
`OpenShift_logging_storage_volume_size`	NFS 导出的大小，例如 `100Gi` 。

如果 OpenShift 集群已在运行，因此已部署和配置 Trident ，则安装程序可以使用动态配置来创建卷。需要配置以下变量。

变量	详细信息
`OpenShift_logging_es_vc_dynamic`	设置为 true 可使用动态配置的卷。
`OpenShift_logging_es_vc_storage_class_name`	要在 PVC 中使用的存储类的名称。
`OpenShift_logging_es_vc_size`	在 PVC 中请求的卷大小。
`OpenShift_logging_es_vc_prefix`	日志记录服务使用的 PVC 的前缀。
`OpenShift_logging_es_ops_vc_dynamic`	设置为 `true` 可对操作日志记录实例使用动态配置的卷。
`OpenShift_logging_es_ops_vc_storage_class_name`	操作日志记录实例的存储类的名称。
`OpenShift_logging_es_ops_vc_size`	操作实例的卷请求大小。
`OpenShift_logging_es_ops_vc_prefix`	操作实例 PVC 的前缀。

部署日志记录堆栈

如果要在初始 OpenShift 安装过程中部署日志记录，则只需遵循标准部署过程即可。Ansible 将配置和部署所需的服务和 OpenShift 对象，以便在 Ansible 完成后立即提供此服务。

但是，如果在初始安装后进行部署，则 Ansible 需要使用组件攻略手册。此过程可能会因 OpenShift 的不同版本而略有变化，因此请务必阅读并遵循 "RedHat OpenShift Container Platform 3.11 文档" 适用于您的版本。

指标服务

指标服务可为管理员提供有关 OpenShift 集群的状态，资源利用率和可用性的宝贵信息。此外， POD 自动调整功能也需要使用此功能，许多组织会将来自指标服务的数据用于其成本分摊和 / 或成本分摊应用程序。

与日志记录服务和 OpenShift 作为一个整体一样， Ansible 用于部署指标服务。此外，与日志记录服务一样，可以在集群初始设置期间或使用组件安装方法运行之后部署指标服务。下表包含在为指标服务配置永久性存储时非常重要的变量。

> 下表仅包含与存储配置相关的变量，因为这些变量与指标服务相关。文档中还有许多其他选项，应根据您的部署情况进行查看，配置和使用。

变量	详细信息
`OpenShift_metrics_storage_kind`	设置为 `NFS` ，以使安装程序为日志记录服务创建 NFS PV 。
`OpenShift_metrics_storage_host`	NFS 主机的主机名或 IP 地址。此值应设置为 SVM 的数据 LIF 。

变量	详细信息
`OpenShift_metrics_storage_nfs_directory`	NFS 导出的挂载路径。例如，如果卷接合为 `/OpenShift_metrics`，则您将使用该路径作为此变量。
`OpenShift_metrics_storage_volume_name`	要创建的 PV 的名称，例如 `PV_ose_metrics` 。
`OpenShift_metrics_storage_volume_size`	NFS 导出的大小，例如 `100Gi` 。

如果 OpenShift 集群已在运行，因此已部署和配置 Trident ，则安装程序可以使用动态配置来创建卷。需要配置以下变量。

变量	详细信息
`OpenShift_metrics_Cassandra_vc_prefix`	用于衡量指标 PVC 的前缀。
`OpenShift_metrics_Cassandra_vc_size`	要请求的卷的大小。
`OpenShift_metrics_Cassandra_storage_type`	要用于度量指标的存储类型，必须将此类型设置为动态， Ansible 才能创建具有相应存储类的 PVC 。
`OpenShift_metrics_cassanda_vc_storage_class_name`	要使用的存储类的名称。

部署指标服务

使用在主机 / 清单文件中定义的适当 Ansible 变量，使用 Ansible 部署服务。如果您在 OpenShift 安装时进行部署，则系统将自动创建和使用 PV 。如果您使用组件攻略手册进行部署，则在 OpenShift 安装之后， Ansible 将创建所需的任何 PVC ，并在 Astra Trident 为其配置存储后部署该服务。

上述变量以及部署过程可能会随 OpenShift 的每个版本而发生变化。确保您查看并遵循 "RedHat 的 OpenShift 部署指南" 为您的版本配置，以便为您的环境进行配置。

数据保护

了解 NetApp 存储平台提供的数据保护和可恢复性选项。Astra Trident 可以配置可利用其中某些功能的卷。对于具有持久性要求的每个应用程序，您都应制定一个数据保护和恢复策略。

备份 etcd 集群数据

Astra Trident 会将其元数据存储在 Kubernetes 集群的 `etcd` 数据库中。定期备份 `etcd` 集群数据对于在发生灾难时恢复 Kubernetes 集群非常重要。

步骤

1. 使用 `etcdctl snapshot save` 命令可以为 `etcd` 集群创建时间点快照：

```
sudo docker run --rm -v /backup:/backup \
  --network host \
  -v /etc/kubernetes/pki/etcd:/etc/kubernetes/pki/etcd \
  --env ETCDCTL_API=3 \
  k8s.gcr.io/etcd-amd64:3.2.18 \
  etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
  --key=/etc/kubernetes/pki/etcd/healthcheck-client.key \
  snapshot save /backup/etcd-snapshot.db
```

此命令通过旋转 etcd 容器来创建 etcd 快照，并将其保存在 ` /backup` 目录中。

2. 发生灾难时，您可以使用 etcd 快照启动 Kubernetes 集群。使用 `etcdctl snapshot restore` 命令还原为 ` /var/lib/etcd` 文件夹创建的特定快照。还原后，确认 ` /var/lib/etcd` 文件夹是否已填充 `mMember` 文件夹。以下是 `etcdctl snapshot restore` 命令的示例：

```
# etcdctl snapshot restore '/backup/etcd-snapshot-latest.db' ; mv
/default.etcd/member/ /var/lib/etcd/
```

3. 在初始化 Kubernetes 集群之前，请复制所有必要的证书。
4. 使用 `-ignore-prefly-errors=DirAvailable-var-lib-etcd` 标志创建集群。
5. 集群启动后，请确保 Kube-system Pod 已启动。
6. 使用 `kubectl get crd` 命令验证 Trident 创建的自定义资源是否存在，并检索 Trident 对象以确保所有数据均可用。

使用 ONTAP 快照恢复日期

快照通过为应用程序数据提供时间点恢复选项发挥着重要作用。但是，快照本身并不是备份，它们无法防止存储系统故障或其他灾难。但是，在大多数情况下，它们是一种方便，快速和轻松的数据恢复方式。了解如何使用 ONTAP 快照技术为卷创建备份以及如何还原这些备份。

- 如果未在后端定义 Snapshot 策略，则默认使用 `none` 策略。这会导致 ONTAP 不会自动创建快照。但是，存储管理员可以通过 ONTAP 管理界面手动创建快照或更改快照策略。这不会影响 Trident 操作。
- 默认情况下， Snapshot 目录处于隐藏状态。这有助于最大程度地兼容使用 `ontap-NAS` 和 `ontap-nas-经济` 驱动程序配置的卷。使用 `ontap-nas` 和 `ontap-nas-economy` 驱动程序时，请启用 ` .snapshot` 目录，以使应用程序能够直接从快照恢复数据。
- 使用 `volume snapshot restore` ONTAP 命令行界面命令将卷还原到先前快照中记录的状态。还原快照副本时，还原操作会覆盖现有卷配置。创建 Snapshot 副本后对卷中数据所做的任何更改都将丢失。

```
cluster1::*> volume snapshot restore -vserver vs0 -volume vol3 -snapshot
vol3_snap_archive
```

使用 ONTAP 复制数据

复制数据在防止存储阵列故障导致的数据丢失方面发挥着重要作用。

> 要了解有关 ONTAP 复制技术的详细信息，请参见 "ONTAP 文档"。

SnapMirror Storage Virtual Machine （ SVM ）复制

您可以使用 "SnapMirror" 复制完整的 SVM ，其中包括其配置设置及其卷。发生灾难时，您可以激活 SnapMirror 目标 SVM 以开始提供数据。系统还原后，您可以切换回主系统。

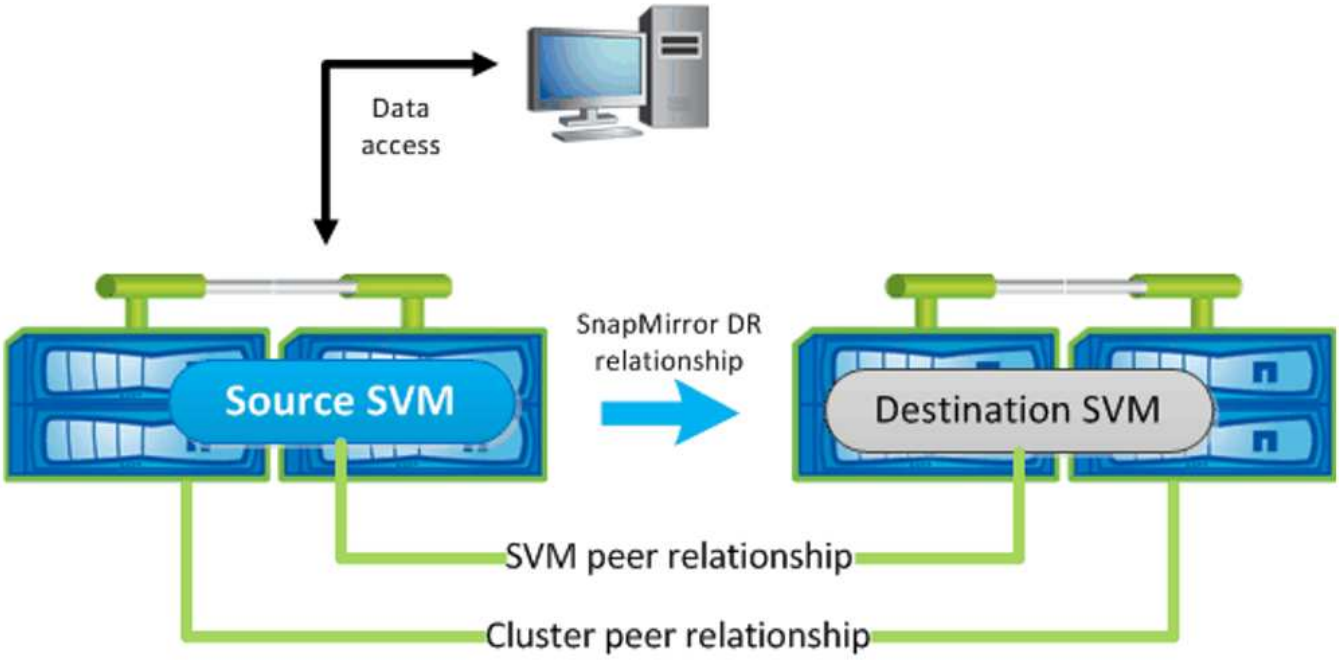
Astra Trident 无法自行配置复制关系，因此存储管理员可以使用 ONTAP 的 SnapMirror SVM 复制功能将卷自动复制到灾难恢复（ Disaster Recovery ， DR ）目标。

如果您计划使用 SnapMirror SVM 复制功能或当前正在使用此功能，请考虑以下事项：

- 您应为每个 SVM 创建一个单独的后端，此后端已启用 SVM-DR 。
- 您应配置存储类，以便在需要时不选择复制的后端。这一点对于避免将不需要复制关系保护的卷配置到支持 SVM-DR 的后端非常重要。
- 应用程序管理员应了解与复制数据相关的额外成本和复杂性，在利用数据复制之前，应确定恢复计划。
- 在激活 SnapMirror 目标 SVM 之前，请停止所有计划的 SnapMirror 传输，中止所有正在进行的 SnapMirror 传输，中断复制关系，停止源 SVM ，然后启动 SnapMirror 目标 SVM 。
- Astra Trident 不会自动检测 SVM 故障。因此，发生故障时，管理员应运行 `tridentctl backend update` 命令以触发 Trident 向新后端的故障转移。

下面简要介绍了 SVM 设置步骤：

- 在源和目标集群以及 SVM 之间设置对等关系。
- 使用 ` 子类型 dp-destination` 选项创建目标 SVM 。
- 创建复制作业计划以确保按所需时间间隔进行复制。
- 使用 ` -identity-preserve true` 选项创建从目标 SVM 到源 SVM 的 SnapMirror 复制，以确保将源 SVM 配置和源 SVM 接口复制到目标。从目标 SVM 初始化 SnapMirror SVM 复制关系。



Trident 的灾难恢复工作流

Astra Trident 19.07 及更高版本使用 Kubernetes CRD 存储和管理其自身状态。它使用 Kubernetes 集群的 `etcd` 存储其元数据。此处我们假定 Kubernetes `etcd` 数据文件和证书存储在 NetApp FlexVolume 上。此 FlexVolume 驻留在 SVM 中， SVM 与二级站点的目标 SVM 具有 SnapMirror SVM-DR 关系。

以下步骤介绍如何在发生灾难时使用 Astra Trident 恢复单个主 Kubernetes 集群：

1. 如果源 SVM 发生故障，请激活 SnapMirror 目标 SVM 。为此，您应停止计划的 SnapMirror 传输，中止正在进行的 SnapMirror 传输，中断复制关系，停止源 SVM 并启动目标 SVM 。
2. 从目标 SVM 中，将包含 Kubernetes `etcd` 数据文件和证书的卷挂载到要设置为主节点的主机上。
3. 将与 Kubernetes 集群相关的所有必需证书复制到 ` /etc/Kubernetes /PKI` 下，并将 etcd `mMember` files 复制到 ` /var/lib/etcd` 下。
4. 使用 `kubeadm init` 命令和 ` -ignore-prefly-errors=DirAvailable-var-lib/etcd` 标志创建 Kubernetes 集群。Kubernetes 节点使用的主机名应与源 Kubernetes 集群相同。
5. 运行 `kubectl get crd` 命令验证所有 Trident 自定义资源是否均已启动并检索 Trident 对象以验证所有数据是否可用。
6. 运行 ` ./tridentctl update backend <backend-name> -f <backend-json-file> -n <namespace>` 命令，更新所有必需的后端以反映新的目标 SVM 名称。

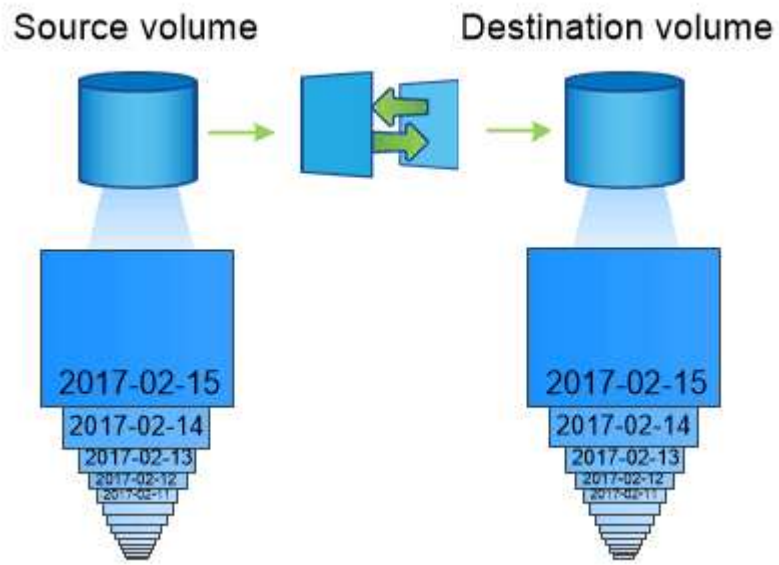
> 对于应用程序永久性卷，激活目标 SVM 后，由 Trident 配置的所有卷将开始提供数据。按照上述步骤在目标端设置 Kubernetes 集群后，所有部署和 Pod 均会启动，容器化应用程序应正常运行。

SnapMirror 卷复制

ONTAP SnapMirror 卷复制是一项灾难恢复功能，可用于在卷级别从主存储故障转移到目标存储。SnapMirror 通过同步快照在二级存储上创建主存储的卷副本或镜像。

下面简要介绍了 ONTAP SnapMirror 卷复制设置步骤：

- 在卷所在的集群与从卷提供数据的 SVM 之间设置对等关系。
- 创建一个 SnapMirror 策略，用于控制关系的行为并指定该关系的配置属性。
- 使用在目标卷和源卷之间创建 SnapMirror 关系 "`snapmirror create` 命令" 并分配相应的 SnapMirror 策略。
- 创建 SnapMirror 关系后，初始化此关系，以便完成从源卷到目标卷的基线传输。



Trident 的 SnapMirror 卷灾难恢复工作流

以下步骤介绍如何使用 Astra Trident 恢复单个主 Kubernetes 集群。

1. 发生灾难时，停止所有计划的 SnapMirror 传输并中止所有正在进行的 SnapMirror 传输。中断目标卷和源卷之间的复制关系，使目标卷变为读 / 写卷。
2. 从目标 SVM 中，将包含 Kubernetes `etcd` 数据文件和证书的卷挂载到将设置为主节点的主机上。
3. 将与 Kubernetes 集群相关的所有必需证书复制到 `/etc/Kubernetes /PKI` 下，并将 etcd `mMember` files 复制到 `/var/lib/etcd` 下。
4. 通过运行 `kubeadm init` 命令和 `-ignore-prefly-errors=DirAvailable-var-lib/etcd` 标志来创建 Kubernetes 集群。主机名应与源 Kubernetes 集群相同。
5. 运行 `kubectl get crd` 命令以验证所有 Trident 自定义资源是否均已启动并检索 Trident 对象，以确保所有数据均可用。
6. 清理先前的后端并在 Trident 上创建新的后端。指定目标 SVM 的新管理和数据 LIF ，新 SVM 名称和密码。

应用程序永久性卷的灾难恢复工作流

以下步骤介绍了在发生灾难时如何为容器化工作负载提供 SnapMirror 目标卷：

1. 停止所有计划的 SnapMirror 传输并中止所有正在进行的 SnapMirror 传输。中断目标卷和源卷之间的复制关系，使目标卷变为读 / 写卷。清理使用与源 SVM 上的卷绑定的 PVC 的部署。
2. 按照上述步骤在目标端设置 Kubernetes 集群后，请从 Kubernetes 集群中清理部署， PVC 和 PV 。

3. 通过指定目标 SVM 的新管理和数据 LIF ，新 SVM 名称和密码，在 Trident 上创建新的后端。
4. 使用 Trident 导入功能将所需卷作为 PV 导入，并绑定到新 PVC 。
5. 使用新创建的 PVC 重新部署应用程序部署。

使用 Element 快照恢复数据

通过为 Element 卷设置快照计划并确保按所需间隔创建快照来备份此卷上的数据。您应使用 Element UI 或 API 设置快照计划。目前，无法通过 `solidfire-san` 驱动程序为卷设置快照计划。

如果发生数据损坏，您可以使用 Element UI 或 API 选择特定快照并手动将卷回滚到快照。此操作将还原自创建快照以来对卷所做的任何更改。

安全性

使用此处列出的建议确保您的 Astra Trident 安装安全。

在自己的命名空间中运行 Astra Trident

请务必防止应用程序，应用程序管理员，用户和管理应用程序访问 Astra Trident 对象定义或 Pod ，以确保存储可靠并阻止潜在的恶意活动。

要将其他应用程序和用户与 Astra Trident 分开，请始终在自己的 Kubernetes 命名空间（`trident` ）中安装 Astra Trident 。将 Astra Trident 置于自己的命名空间中可确保只有 Kubernetes 管理人员才能访问 Astra Trident Pod 以及存储在命名空间 CRD 对象中的项目（如适用，还包括后端和 CHAP 密码）。您应确保仅允许管理员访问 Astra Trident 命名空间，从而访问 `tridentctl` 应用程序。

对 ONTAP SAN 后端使用 CHAP 身份验证

Astra Trident 支持对 ONTAP SAN 工作负载进行基于 CHAP 的身份验证（使用 `ontap-san` 和 `ontap-san-economy`. 驱动程序）。NetApp 建议将双向 CHAP 与 Astra Trident 结合使用，以便在主机和存储后端之间进行身份验证。

对于使用 SAN 存储驱动程序的 ONTAP 后端， Astra Trident 可以通过 `tridentctl` 设置双向 CHAP 并管理 CHAP 用户名和密码。请参见 "此处" 了解 Astra Trident 如何在 ONTAP 后端配置 CHAP 。

> Trident 20.04 及更高版本支持 ONTAP 后端的 CHAP 。

对 NetApp HCI 和 SolidFire 后端使用 CHAP 身份验证

NetApp 建议部署双向 CHAP ，以确保主机与 NetApp HCI 和 SolidFire 后端之间的身份验证。Astra Trident 使用一个机密对象，每个租户包含两个 CHAP 密码。当 Trident 作为 CSI 配置程序安装时，它会管理 CHAP 密码并将其存储在相应 PV 的 `tridentvolume` CR 对象中。创建 PV 时， CSI Astra Trident 会使用 CHAP 密码启动 iSCSI 会话并通过 CHAP 与 NetApp HCI 和 SolidFire 系统进行通信。

> CSI Trident 创建的卷不与任何卷访问组关联。

在非 CSI 前端中，作为辅助节点上的设备连接卷的操作由 Kubernetes 处理。创建卷后，如果该租户的密钥尚不存在，则 Astra Trident 会对 NetApp HCI/SolidFire 系统进行 API 调用，以检索这些密钥。然后， Astra Trident

将这些机密传递给 Kubernetes 。位于每个节点上的 kubelet 通过 Kubernetes API 访问这些机密，并使用它们在访问卷的每个节点与卷所在的 NetApp HCI/SolidFire 系统之间运行 / 启用 CHAP 。

参考

Astra Trident 端口

详细了解 Astra Trident 所通过的端口。

Astra Trident 通过以下端口进行通信：

Port	目的
8443	后通道 HTTPS
8001	Prometheus 指标端点
8000	Trident REST 服务器
17546	Trident demonset Pod 使用的活动性 / 就绪性探测端口

> 可以在安装期间使用 ` -probe-port` 标志更改活跃度 / 就绪性探测端口。请务必确保此端口未被工作节点上的其他进程使用。

Astra Trident REST API

同时 "tridentctl 命令和选项" 是与 Astra Trident 的 REST API 进行交互的最简单方式，您可以根据需要直接使用 REST 端点。

这对于在非 Kubernetes 部署中使用 Astra Trident 作为独立二进制文件的高级安装非常有用。

为了提高安全性，在 Pod 内部运行时， Astra Trident 的 `reST API` 默认限制为 localhost 。要更改此行为，您需要在其 Pod 配置中设置 Astra Trident 的 ` 地址` 参数。

API 的工作原理如下：

获取

- `get <trident` 地址 `>/trident /v1/<` 对象类型 `>`：列出此类型的所有对象。
- `get <trident` 地址 `>/trident /v1/<` 对象类型 `>/<` 对象名称 `>`：获取命名对象的详细信息。

发布

`post <trident` 地址 `>/trident /v1/<` 对象类型 `>`：创建指定类型的对象。

- 需要为要创建的对象配置 JSON 。有关每种对象类型的规范，请参见 "`tridentctl` 命令和选项"。
- 如果对象已存在，则行为会有所不同：后端更新现有对象，而所有其他对象类型将使操作失败。

删除

`delete <trident` 地址 `>/trident /v1/<` 对象类型 `>/<` 对象名称 `>`：删除指定的资源。

> 与后端或存储类关联的卷将继续存在；必须单独删除这些卷。有关详细信息，请参见 "`tridentctl` 命令和选项"。

有关如何调用这些 API 的示例，请传递 debug （` -d` ）标志。有关详细信息，请参见 "`tridentctl` 命令和选项 "。

命令行选项

Astra Trident 为 Trident 流程编排程序提供了多个命令行选项。您可以使用这些选项修改部署。

日志记录

- ` -debug` ： 启用调试输出。
- ` -loglevel <level>` ： 设置日志记录级别（调试，信息，警告，错误，致命）。默认为 INFO 。

Kubernetes

- ` -K8s_pod` ： 使用此选项或 ` -K8s_api_server` 启用 Kubernetes 支持。如果设置此值，则 Trident 将使用其所属 POD 的 Kubernetes 服务帐户凭据来联系 API 服务器。只有当 Trident 在启用了服务帐户的 Kubernetes 集群中作为 POD 运行时，此功能才有效。
- ` -K8s_api_server < 不安全地址：不安全端口 >` ： 使用此选项或 ` -K8s_pod` 启用 Kubernetes 支持。指定后， Trident 将使用提供的不安全地址和端口连接到 Kubernetes API 服务器。这样可以在 Pod 之外部署 Trident ；但是，它仅支持与 API 服务器的不安全连接。要安全连接，请使用 ` -K8s_pod` 选项将 Trident 部署在 Pod 中。
- ` -K8s_config_path <file>` ： 必需；必须指定 KubeConfig 文件的路径。

Docker

- ` -volume_driver < 名称 >` ： 注册 Docker 插件时使用的驱动程序名称。默认为 `NetApp` 。
- ` -driver_port <port-number>` ： 侦听此端口，而不是 UNIX 域套接字。
- ` 配置 < 文件 >` ： 必需；您必须指定后端配置文件的路径。

REST

- ` 地址 <ip-or-host>` ： 指定 Trident 的 REST 服务器应侦听的地址。默认为 localhost 。在本地主机上侦听并在 Kubernetes Pod 中运行时，无法从 Pod 外部直接访问 REST 接口。使用 ` 地址 ""` 可从 Pod IP 地址访问 REST 接口。

> 可以将 Trident REST 接口配置为仅以 127.0.0.1 （对于 IPv4 ）或（：： 1 ）（对于 IPv6 ）侦听和提供服务。

- ` -port <port-number>` ： 指定 Trident 的 REST 服务器应侦听的端口。默认为 8000 。
- ` -REST` ： 启用 REST 接口。默认为 true 。

与 Kubernetes 集成的 NetApp 产品

NetApp 存储产品组合可与 Kubernetes 集群的许多不同方面相集成，从而提供高级数据管理功能，从而增强 Kubernetes 部署的功能，功能，性能和可用性。

Astra

"Astra" 使企业能够更轻松地管理，保护和移动在公有云内部和内部环境中以及之间的 Kubernetes 上运行的数据丰富的容器化工作负载。Astra 使用 NetApp 在公有云和内部环境中成熟而广泛的存储产品组合中的 Trident 配置和提供永久性容器存储。此外，它还提供了一组丰富的高级应用程序感知型数据管理功能，例如快照，备份和还原，活动日志和主动克隆，用于数据保护，灾难 / 数据恢复，数据审核以及 Kubernetes 工作负载的迁移用例。

ONTAP

ONTAP 是 NetApp 的多协议统一存储操作系统，可为任何应用程序提供高级数据管理功能。ONTAP 系统采用全闪存，混合或全 HDD 配置，并提供多种不同的部署模式，包括专门设计的硬件（ FAS 和 AFF ），白盒（ ONTAP Select ）和纯云（ Cloud Volumes ONTAP ）。

> Trident 支持上述所有 ONTAP 部署模式。

Cloud Volumes ONTAP

"Cloud Volumes ONTAP" 是一款纯软件存储设备，可在云中运行 ONTAP 数据管理软件。您可以将 Cloud Volumes ONTAP 用于生产工作负载、灾难恢复、 DevOps 、文件共享和数据库管理。它通过提供存储效率，高可用性，数据复制，数据分层和应用程序一致性，将企业级存储扩展到云。

适用于 NetApp ONTAP 的 Amazon FSX

"适用于 NetApp ONTAP 的 Amazon FSX" 是一种完全托管的 AWS 服务，可使客户启动和运行由 NetApp ONTAP 存储操作系统提供支持的文件系统。借助适用于 ONTAP 的 FSX ，客户可以利用他们熟悉的 NetApp 功能，性能和管理功能，同时利用在 AWS 上存储数据的简便性，灵活性，安全性和可扩展性。FSX for ONTAP 支持 ONTAP 的许多文件系统功能和管理 API 。

Element 软件

"Element" 通过保证性能并简化存储占用空间，使存储管理员能够整合工作负载。Element 与 API 相结合，可实现存储管理各个方面的自动化，可帮助存储管理员事半功倍。

NetApp HCI

"NetApp HCI" 通过自动化执行日常任务并使基础架构管理员能够专注于更重要的功能，简化数据中心的管理和扩展。

Trident 完全支持 NetApp HCI 。Trident 可以直接在底层 NetApp HCI 存储平台上为容器化应用程序配置和管理存储设备。

Azure NetApp Files

"Azure NetApp Files" 是一种企业级 Azure 文件共享服务，由 NetApp 提供支持。您可以在 Azure 中以本机方式

运行要求最苛刻的基于文件的工作负载，同时享受 NetApp 应有的性能和丰富的数据管理功能。

适用于 Google Cloud 的 Cloud Volumes Service

"适用于 Google Cloud 的 NetApp Cloud Volumes Service" 是一种云原生文件服务，可通过 NFS 和 SMB 提供具有全闪存性能的 NAS 卷。此服务支持在 GCP 云中运行任何工作负载，包括原有应用程序。它提供了一种完全托管的服务，可提供稳定一致的高性能，即时克隆，数据保护以及对 Google Compute Engine （ GCE ）实例的安全访问。

Kubernetes 和 Trident 对象

您可以通过读取和写入资源对象来使用 REST API 与 Kubernetes 和 Trident 进行交互。Kubernetes 与 Trident ， Trident 与存储以及 Kubernetes 与存储之间的关系由多个资源对象决定。其中一些对象通过 Kubernetes 进行管理，而另一些对象则通过 Trident 进行管理。

对象如何相互交互?

了解对象，对象的用途以及对象交互方式的最简单方法可能是，遵循 Kubernetes 用户的单个存储请求：

1. 用户创建一个 `PersistentVolumeClaim` ，请求管理员先前配置的 Kubernetes `StorageClass` 中具有特定大小的新 `PersistentVolume` 。
2. Kubernetes `StorageClass` 将 Trident 标识为其配置程序，并包含一些参数，用于指示 Trident 如何为请求的类配置卷。
3. Trident 会查看自己的 `StorageClass` 并使用相同的名称来标识匹配的 `Backend` 和 `StoragePools` ，它可以使用这些卷为该类配置卷。
4. Trident 会在匹配的后端配置存储并创建两个对象： Kubernetes 中的 A `PersistentVolume` 用于告知 Kubernetes 如何查找，挂载和处理卷； Trident 中的一个卷用于保留 `PersistentVolume` 与实际存储之间的关系。
5. Kubernetes 会将 `PersistentVolumeClaim` 绑定到新的 `PersistentVolume` 。包含 `PersistentVolumeClaim` 的 Pod 会将此 PersistentVolume 挂载到其运行所在的任何主机上。
6. 用户使用指向 Trident 的 `VolumeSnapshotClass` 创建现有 PVC 的 `VolumeSnapshot` 。
7. Trident 标识与 PVC 关联的卷，并在其后端创建卷的快照。此外，它还会创建一个 `VolumeSnapshotContent` ，指示 Kubernetes 如何识别快照。
8. 用户可以使用 `VolumeSnapshot` 作为源创建 `PersistentVolumeClaim` 。
9. Trident 可识别所需的快照，并执行与创建 `PersistentVolume` 和 `Volume` 相同的一组步骤。

要进一步了解 Kubernetes 对象，强烈建议您阅读 "永久性卷" Kubernetes 文档的一节。

Kubernetes PersistentVolumeClaim 对象

Kubernetes `PersistentVolumeClaim` 对象是 Kubernetes 集群用户发出的存储请求。

除了标准规范之外，如果用户要覆盖在后端配置中设置的默认值， Trident 还允许用户指定以下特定于卷的标注：

标注	卷选项	支持的驱动程序
trident.netapp.io/fileSystem	文件系统	ontap-san ， solidfire-san ， E 系列 -iscsi ， ontap-san-economy.
trident.netapp.io/cloneFromPVC	cloneSourceVolume	ontap-nas ， ontap-san ， solidfire-san ， azure-netapp-files ， gcp-cvs ， ontap-san-economy.
trident.netapp.io/splitOnClone	splitOnClone	ontap-NAS ， ontap-san
trident.netapp.io/protocol	协议	任意
trident.netapp.io/exportPolicy	导出策略	ontap-nas ， ontap-nas-economy- 、 ontap-nas-flexgroup
trident.netapp.io/snapshotPolicy	snapshotPolicy	ontap-nas ， ontap-nas-economy. ontap-nas-flexgroup ， ontap-san
trident.netapp.io/snapshotReserve	SnapshotReserve	ontap-nas ， ontap-nas-flexgroup ， ontap-san ， GCP-CVS
trident.netapp.io/snapshotDirectory	snapshotDirectory	ontap-nas ， ontap-nas-economy- 、 ontap-nas-flexgroup
trident.netapp.io/unixPermissions	unixPermissions	ontap-nas ， ontap-nas-economy- 、 ontap-nas-flexgroup
trident.netapp.io/blockSize	块大小	solidfire-san

如果创建的 PV 具有 `Delete` reclaime 策略，则在释放 PV 时（即用户删除 PVC 时）， Trident 会同时删除 PV 和后备卷。如果删除操作失败， Trident 会将 PV 标记为相应的 PV ，并定期重试此操作，直到操作成功或 PV 手动删除为止。如果 PV 使用 `retain` 策略， Trident 会忽略该策略，并假定管理员将从 Kubernetes 和后端清理该策略，以便在删除卷之前对其进行备份或检查。请注意，删除 PV 不会通过发生原因 Trident 删除后备卷。您应使用 REST API （`tridentctl` ）将其删除。

Trident 支持使用 CSI 规范创建卷快照：您可以创建卷快照并将其用作数据源来克隆现有 PVC 。这样， PV 的时间点副本就可以以快照的形式公开给 Kubernetes 。然后，可以使用快照创建新的 PV 。请查看 `+ 按需卷快照 +` 了解其工作原理。

Trident 还提供了用于创建克隆的 `cloneFromPVC` 和 `splitOnClone` 标注。您可以使用这些标注克隆 PVC ，而无需使用 CSI 实施（在 Kubernetes 1.13 及更早版本上），或者您的 Kubernetes 版本不支持测试版卷快照（ Kubernetes 1.16 及更早版本）。请注意， Trident 19.10 支持从 PVC 克隆的 CSI 工作流。

> 您可以将 `cloneFromPVC` 和 `splitOnClone` 标注与 CSI Trident 以及传统的非 CSI 前端结合使用。

例如：如果用户已经有一个名为 `mysql` 的 PVC ，则用户可以使用标注创建一个名为 `mysqlclone` 的新 PVC ，例如 `trident.netapp.io/cloneFromPVC: mysql` 。设置了此标注后， Trident 将克隆与 mysql PVC 对应的卷，而不是从头开始配置卷。

请考虑以下几点：

- 建议克隆空闲卷。
- 一个 PVC 及其克隆应位于同一个 Kubernetes 命名空间中，并具有相同的存储类。
- 使用 `ontap-nas` 和 `ontap-san` 驱动程序时，可能需要将 PVC 标注

trident.netapp.io/splitOnClone 与 trident.netapp.io/cloneFromPVC 结合使用。当 trident.netapp.io/splitOnClone 设置为 true 时， Trident 会将克隆的卷与父卷拆分，从而将克隆的卷与其父卷的生命周期完全分离，从而降低存储效率。如果不设置 trident.netapp.io/splitOnClone 或将其设置为 false ，则会减少后端的空间占用，而会在父卷和克隆卷之间创建依赖关系，因此除非先删除克隆，否则无法删除父卷。拆分克隆是有意义的一种情形，即克隆空数据库卷时，该卷及其克隆会发生很大的差异，无法从 ONTAP 提供的存储效率中受益。

sample-input 目录包含用于 Trident 的 PVC 定义示例。有关与 Trident 卷关联的参数和设置的完整问题描述，请参见 Trident 卷对象。

Kubernetes PersistentVolume 对象

Kubernetes PersistentVolume 对象表示可供 Kubernetes 集群使用的一段存储。它的生命周期与使用它的 POD 无关。

> Trident 会创建 PersistentVolume 对象，并根据其配置的卷自动将其注册到 Kubernetes 集群中。您不应自行管理它们。

创建引用基于 Trident 的 S存储类 的 PVC 时， Trident 会使用相应的存储类配置新卷并为该卷注册新的 PV 。在配置已配置的卷和相应的 PV 时， Trident 会遵循以下规则：

- Trident 会为 Kubernetes 生成 PV 名称及其用于配置存储的内部名称。在这两种情况下，它都可以确保名称在其范围内是唯一的。
- 卷的大小与 PVC 中请求的大小尽可能匹配，但可能会根据平台将其取整为最接近的可分配数量。

Kubernetes StorageClass 对象

Kubernetes StorageClass 对象在 PersistentVolumeClass 中按名称指定，用于使用一组属性配置存储。存储类本身可标识要使用的配置程序，并按配置程序所了解的术语定义该属性集。

它是需要由管理员创建和管理的两个基本对象之一。另一个是 Trident 后端对象。

使用 Trident 的 Kubernetes StorageClass 对象如下所示：

```
apiVersion: storage.k8s.io/v1beta1
kind: StorageClass
metadata:
  name: <Name>
provisioner: csi.trident.netapp.io
mountOptions: <Mount Options>
parameters:
  <Trident Parameters>
allowVolumeExpansion: true
volumeBindingMode: Immediate
```

这些参数是 Trident 专用的，可告诉 Trident 如何为类配置卷。

存储类参数包括：

属性	Type	Required	Description
属性	map[string]string	否	请参见下面的属性部分
存储池	map[string]StringList	否	后端名称映射到中的存储池列表
附加 StoragePools	map[string]StringList	否	后端名称映射到中的存储池列表
排除 StoragePools	map[string]StringList	否	后端名称映射到中的存储池列表

存储属性及其可能值可以分类为存储池选择属性和 Kubernetes 属性。

存储池选择属性

这些参数决定了应使用哪些 Trident 管理的存储池来配置给定类型的卷。

属性	Type	值	优惠	请求	支持
介质[1]	string	HDD ，混合，SSD	Pool 包含此类型的介质；混合表示两者	指定的介质类型	ontap-nas ，ontap-nas-economy. ontap-nas-flexgroup ，ontap-san ，solidfire-san
配置类型	string	精简，厚	Pool 支持此配置方法	指定的配置方法	厚：全 ONTAP 和 E 系列 iSCSI ；精简：全 ONTAP 和 solidfire-san
后端类型	string	ontap-nas ，ontap-nas-economy. ontap-nas-flexgroup ，ontap-san ，solidfire-san ， E 系列 iSCSI ，GCP-CVS ，azure-netapp-files ， ontap-san-economy.	池属于此类型的后端	指定后端	所有驱动程序
snapshots	池	true false	Pool 支持具有快照的卷	启用了快照的卷	ontap-nas ，ontap-san ，solidfire-san ，gcp-cvs
克隆	池	true false	Pool 支持克隆卷	启用了克隆的卷	ontap-nas ，ontap-san ，solidfire-san ，gcp-cvs

属性	**Type**	值	优惠	请求	支持
加密	池	true false	池支持加密卷	已启用加密的卷	ontap-nas ，ontap-nas-economy-、ontap-nas-flexgroups ，ontap-san
IOPS	内部	正整数	Pool 能够保证此范围内的 IOPS	卷保证这些 IOPS	solidfire-san

[1]： ONTAP Select 系统不支持

在大多数情况下，请求的值直接影响配置；例如，请求厚配置会导致卷配置较厚。但是， Element 存储池会使用其提供的 IOPS 最小值和最大值来设置 QoS 值，而不是请求的值。在这种情况下，请求的值仅用于选择存储池。

理想情况下，您可以单独使用 `attributes` 来为满足特定类需求所需的存储质量建模。Trident 会自动发现并选择与您指定的 **属性** 的 *all* 匹配的存储池。

如果您发现自己无法使用 `attributes` 自动为某个类选择合适的池，则可以使用 `storagePools` 和 `addtionalStoragePools` 参数进一步细化池，甚至可以选择一组特定的池。

您可以使用 `storagePools` 参数进一步限制与任何指定的 `attributes` 匹配的池集。换言之， Trident 会使用 `attributes` 和 `storagePools` 参数标识的池的交叉点进行配置。您可以单独使用参数，也可以同时使用这两者。

您可以使用 `addtionalStoragePools` 参数扩展 Trident 用于配置的池集，而不管 `attributes` 和 `storagePools` 参数选择的任何池如何。

您可以使用 `excludeStoragePools` 参数筛选 Trident 用于配置的池集。使用此参数将删除任何匹配的池。

在 `storagePools` 和 `addtionalStoragePools` 参数中，每个条目的格式为 `<backend>：<storagePoolList>` ，其中 `<storagePoolList>` 是指定后端的存储池列表，以逗号分隔。例如， `addtionalStoragePools` 的值可能类似于 `ontapnas_192.168.1.100 ： aggr1 ， aggr2 ； solidfire_192.168.1.101 ： bronze` 。这些列表接受后端值和列表值的正则表达式值。您可以使用 `tridentctl get backend` 来获取后端及其池的列表。

Kubernetes 属性

这些属性不会影响 Trident 在动态配置期间选择的存储池 / 后端。相反，这些属性仅提供 Kubernetes 永久性卷支持的参数。工作节点负责文件系统创建操作，并且可能需要文件系统实用程序，例如 xfsprogs 。

属性	Type	值	Description	相关驱动程序	Kubernetes 版本
FSType	string	ext4 ， ext3 ， xfs 等	块卷的文件系统类型	solidfire-san ， ontap-nas ， ontap-nas-economy. ontap-nas-flexgroup ， ontap-san ， ontap-san-economy-、 E 系列 -iscsi	全部
允许卷扩展	boolean	true false	启用或禁用对增加 PVC 大小的支持	ontap-nas ， ontap-nas-economy. ontap-nas-flexgroup ， ontap-san ， ontap-san-economy. solidfire-san ， gcp-cvs ， azure-netapp-files	1.11 及更高版本
卷绑定模式	string	即时，WaitForFirstConsumer"	选择何时进行卷绑定和动态配置	全部	1.18 - 1.24

- `FSType` 参数用于控制 SAN LUN 所需的文件系统类型。此外， Kubernetes 还会使用存储类中存在的 `FSType` 来指示文件系统已存在。只有在设置了 `FSType` 的情况下，才能使用 Pod 的 `fsGroup` 安全上下文来控制卷所有权。请参见 "Kubernetes ： 为 Pod 或容器配置安全上下文" 有关使用 `fsGroup` 上下文设置卷所有权的概述。只有在以下情况下， Kubernetes 才会应用 `fsGroup` 值：
  - 在存储类中设置 `FSType` 。
  - PVC 访问模式为 RW 。

  对于 NFS 存储驱动程序， NFS 导出中已存在文件系统。要使用 `fsGroup` ，存储类仍需要指定 `FSType` 。您可以将其设置为 `NFS` 或任何非空值。
- 请参见 "展开卷" 有关卷扩展的更多详细信息。
- Trident 安装程序包提供了几个示例存储类定义，可用于 `sample-input/storage-class-*` 。 `yaml` 中的 Trident 。删除 Kubernetes 存储类也会删除相应的 Trident 存储类。

Kubernetes VolumeSnapshotClass 对象

Kubernetes `VolumeSnapshotClass` 对象类似于 `StorageClasses` 。它们有助于定义多个存储类，并由卷快照引用以将快照与所需的快照类关联。每个卷快照都与一个卷快照类相关联。

要创建快照，管理员应定义 `VolumeSnapshotClass` 。此时将使用以下定义创建卷快照类：

```yaml
apiVersion: snapshot.storage.k8s.io/v1beta1
kind: VolumeSnapshotClass
metadata:
  name: csi-snapclass
driver: csi.trident.netapp.io
deletionPolicy: Delete
```

`driver` 指定给 Kubernetes ，由 Trident 处理对 `csI-snapclass` 类的卷快照请求。`deeltionPolicy` 指定必须删除快照时要执行的操作。如果将 `deletionPolicy` 设置为 `Delete` ，则在删除快照时，卷快照对象以及存储集群上的底层快照将被删除。或者，如果将其设置为 `Retain` ，则表示保留 `VolumeSnapshotContent` 和物理快照。

Kubernetes VolumeSnapshot 对象

Kubernetes `VolumeSnapshot` 对象是创建卷快照的请求。就像 PVC 代表用户对卷发出的请求一样，卷快照也是用户为现有 PVC 创建快照的请求。

收到卷快照请求后， Trident 会自动管理在后端为卷创建快照的操作，并通过创建唯一的 `VolumeSnapshotContent` 对象公开快照。您可以从现有 PVC 创建快照，并在创建新 PVC 时将这些快照用作 DataSource 。

> VolumeSnapshot 的生命周期与源 PVC 无关：即使删除了源 PVC ，快照也会持续存在。删除具有关联快照的 PVC 时， Trident 会将此 PVC 的后备卷标记为 " 正在删除 " 状态，但不会将其完全删除。删除所有关联快照后，卷将被删除。

Kubernetes VolumeSnapshotContent 对象

Kubernetes `VolumeSnapshotContent` 对象表示从已配置的卷创建的快照。它类似于 `PersistentVolume` ，表示存储集群上配置的快照。与 `PersistentVolumeClaim` 和 `PersistentVolume` 对象类似，创建快照时， `VolumeSnapshotContent` 对象会与请求创建快照的 `VolumeSnapshot` 对象保持一对一映射。

> Trident 会创建 `VolumeSnapshotContent` 对象，并根据其配置的卷自动将其注册到 Kubernetes 集群中。您不应自行管理它们。

`VolumeSnapshotContent` 对象包含用于唯一标识快照的详细信息，例如 `snapshotHandle` 。此 `snapshotHandle` 是 PV 名称和 `VolumeSnapshotContent` 对象名称的唯一组合。

收到快照请求后， Trident 会在后端创建快照。创建快照后， Trident 会配置一个 `VolumeSnapshotContent` 对象，从而将快照公开到 Kubernetes API 。

Kubernetes CustomResourceDefinition 对象

Kubernetes 自定义资源是 Kubernetes API 中的端点，由管理员定义并用于对类似对象进行分组。Kubernetes 支持创建自定义资源以存储对象集合。您可以通过运行 `kubectl get crds` 来获取这些资源定义。

自定义资源定义（ CRD ）及其关联的对象元数据由 Kubernetes 存储在其元数据存储中。这样就无需为 Trident 创建单独的存储。

从 19.07 版开始， Trident 会使用许多 `CustomResourceDefinition` 对象来保留 Trident 对象的身份，例如 Trident 后端， Trident 存储类和 Trident 卷。这些对象由 Trident 管理。此外， CSI 卷快照框架还引入了一些定义卷快照所需的 CRD 。

CRD 是一种 Kubernetes 构造。上述资源的对象由 Trident 创建。例如，使用 `tridentctl` 创建后端时，会创建一个对应的 `tridentbackend` CRD 对象，供 Kubernetes 使用。

有关 Trident 的 CRD ，请注意以下几点：

- 安装 Trident 时，系统会创建一组 CRD ，并可像使用任何其他资源类型一样使用。
- 从先前版本的 Trident （使用 `etcd` 保持状态的版本）升级时， Trident 安装程序会从 `etcd` key-value 数据存储库迁移数据并创建相应的 CRD 对象。
- 使用 `tridentctl uninstall` 命令卸载 Trident 时， Trident Pod 会被删除，但不会清理创建的 CRD 。请参见 "卸载 Trident" 了解如何从头开始完全删除和重新配置 Trident 。

Trident StorageClass 对象

Trident 会为 Kubernetes `StorageClass` 在其配置程序字段中指定 `csi.trident.netapp.io` /`netapp.io/trident` 的对象创建匹配的存储类。存储类名称与 Kubernetes `StorageClass` 其所代表的对象的名称匹配。

> 使用 Kubernetes 时，如果注册了使用 Trident 作为配置程序的 Kubernetes `StorageClass` ，则会自动创建这些对象。

存储类包含一组卷要求。Trident 会将这些要求与每个存储池中的属性进行匹配；如果匹配，则该存储池是使用该存储类配置卷的有效目标。

您可以使用 REST API 创建存储类配置以直接定义存储类。但是，对于 Kubernetes 部署，我们希望在注册新的 Kubernetes `StorageClass` 对象时创建这些部署。

Trident 后端对象

后端表示存储提供程序，其中 Trident 配置卷；单个 Trident 实例可以管理任意数量的后端。

> 这是您自己创建和管理的两种对象类型之一。另一个是 Kubernetes `StorageClass` 对象。

有关如何构建这些对象的详细信息，请参见后端配置。

Trident StoragePool 对象

存储池表示可在每个后端配置的不同位置。对于 ONTAP ，这些聚合对应于 SVM 中的聚合。对于 NetApp HCI/SolidFire ，这些 QoS 分段对应于管理员指定的 QoS 分段。对于 Cloud Volumes Service ，这些区域对应于云提供商区域。每个存储池都有一组不同的存储属性，用于定义其性能特征和数据保护特征。

与此处的其他对象不同，存储池候选对象始终会自动发现和管理。

Trident Volume 对象

卷是基本配置单元，由后端端点组成，例如 NFS 共享和 iSCSI LUN 。在 Kubernetes 中，这些卷直接对应于

`PersistentVolumes` 。创建卷时，请确保其具有存储类，此类可确定可配置该卷的位置以及大小。

> 在 Kubernetes 中，这些对象会自动进行管理。您可以查看它们以查看 Trident 配置的内容。

> 删除具有关联快照的 PV 时，相应的 Trident 卷将更新为 * 正在删除 * 状态。要删除 Trident 卷，您应删除该卷的快照。

卷配置定义了配置的卷应具有的属性。

属性	Type	Required	Description
version	string	否	Trident API 版本（ "1" ）
name	string	是的。	要创建的卷的名称
存储类	string	是的。	配置卷时要使用的存储类
size	string	是的。	要配置的卷大小（以字节为单位）
协议	string	否	要使用的协议类型； "file" 或 "block"
内部名称	string	否	存储系统上的对象名称；由 Trident 生成
cloneSourceVolume	string	否	ONTAP （ NAS ， SAN ）和 SolidFire — *： 要从中克隆的卷的名称
splitOnClone	string	否	ONTAP （ NAS ， SAN ）： 将克隆从其父级拆分
snapshotPolicy	string	否	Snapshot-*： 要使用的 ONTAP 策略
SnapshotReserve	string	否	Snapshot-*： 为快照预留的卷百分比 ONTAP
导出策略	string	否	ontap-nas*： 要使用的导出策略
snapshotDirectory	池	否	ontap-nas*： 是否显示快照目录
unixPermissions	string	否	ontap-nas*： 初始 UNIX 权限
块大小	string	否	SolidFire — *： 块 / 扇区大小
文件系统	string	否	文件系统类型

创建卷时， Trident 会生成 `internalName` 。这包括两个步骤。首先，它会将存储前缀（默认值 `trident` 或后端配置中的前缀）预先添加到卷名称中，从而使名称格式为 `<prefix>-<volume-name>` 。然后，它将继续清理名称，替换后端不允许使用的字符。对于 ONTAP 后端，它会将连字符替换为下划线（因此，内部名称将变为 `<prefix>_<volume-name>` ）。对于 Element 后端，它会将下划线替换为连字符。

您可以使用卷配置使用 REST API 直接配置卷，但在 Kubernetes 部署中，我们希望大多数用户使用标准的

Kubernetes `PersistentVolumeClaim` 方法。Trident 会在配置过程中自动创建此卷对象。

Trident Snapshot 对象

快照是卷的时间点副本，可用于配置新卷或还原状态。在 Kubernetes 中，这些对象直接对应于 `VolumeSnapshotContent` 对象。每个快照都与一个卷相关联，该卷是快照的数据源。

每个 `Snapshot` 对象包括以下属性：

属性	Type	Required	Description
version	string	是的。	Trident API 版本（"1"）
name	string	是的。	Trident Snapshot 对象的名称
内部名称	string	是的。	存储系统上 Trident Snapshot 对象的名称
volumeName	string	是的。	为其创建快照的永久性卷的名称
volumeInternalName	string	是的。	存储系统上关联的 Trident 卷对象的名称

在 Kubernetes 中，这些对象会自动进行管理。您可以查看它们以查看 Trident 配置的内容。

创建 Kubernetes `VolumeSnapshot` 对象请求时， Trident 会在备用存储系统上创建 Snapshot 对象。此快照对象的 `internalName` 是通过将前缀 `snapshot-` 与 `VolumeSnapshot` 对象的 `UID` （例如， `snapshot-e8d8a0ca-9826-11e9-9807-525400f3f660` ）组合而生成的。`VolumeName` 和 `volumeInternalName` 可通过获取后备卷的详细信息来填充。

tridentctl 命令和选项

。 "Trident 安装程序包" 包括一个命令行实用程序 `tridentctl` ，可用于轻松访问 Astra Trident 。具有足够权限的 Kubernetes 用户可以使用它来安装 Astra Trident ，并直接与其交互以管理包含 Astra Trident Pod 的命名空间。

有关使用情况信息，请运行 `tridentctl -help` 。

可用的命令和全局选项包括：

```
Usage:
  tridentctl [command]
```

可用命令：

- `create` ： 将资源添加到 Astra Trident 。
- `d`删除： 从 Astra Trident 中删除一个或多个资源。

- **获取**： 从 Astra Trident 获取一个或多个资源。
- **帮助**： 有关任何命令的帮助。
- `images`： 打印一个包含 Astra Trident 所需容器映像的表。
- `import`： 将现有资源导入到 Astra Trident 。
- **安装**： 安装 Astra Trident 。
- `logs`： 从 Astra Trident 打印日志。
- `sEnd`： 从 Astra Trident 发送资源。
- **卸载**： 卸载 Astra Trident 。
- `update`： 修改 Astra Trident 中的资源。
- **升级**： 升级 Astra Trident 中的资源。
- `version`： 打印 Astra Trident 的版本。

flags

- ` `-d` ， `-debug`： 调试输出。
- ` `-h` ， `-help`： 帮助 `tridentctl` 。
- ` `-n` ， `-namespace string`： Astra Trident 部署的命名空间。
- ` `-o` ， `-output string`： 输出格式。json_yaml_name_wide|ps 之一（默认）。
- ` `-s` ， `-server string`： Astra Trident REST 接口的地址 / 端口。

> 可以将 Trident REST 接口配置为仅以 127.0.0.1 （对于 IPv4 ）或（：： 1 ）（对于 IPv6 ）侦听和提供服务。

> 可以将 Trident REST 接口配置为仅以 127.0.0.1 （对于 IPv4 ）或（：： 1 ）（对于 IPv6 ）侦听和提供服务。

创建

您可以使用 run the `create` 命令向 Astra Trident 添加资源。

```
Usage:
  tridentctl create [option]
```

可用选项： `backend`： 将后端添加到 Astra Trident 。

删除

您可以运行 `delete` 命令从 Astra Trident 中删除一个或多个资源。

```
Usage:
  tridentctl delete [option]
```

可用选项：

- 后端： 从 Astra Trident 中删除一个或多个存储后端。
- `node`： 从 Astra Trident 中删除一个或多个 CSI 节点。
- `snapshot`： 从 Astra Trident 中删除一个或多个卷快照。
- `storageclass`： 从 Astra Trident 中删除一个或多个存储类。
- `volume`： 从 Astra Trident 中删除一个或多个存储卷。

获取

您可以运行 `get` 命令从 Astra Trident 获取一个或多个资源。

```
Usage:
  tridentctl get [option]
```

可用选项：

- 后端： 从 Astra Trident 获取一个或多个存储后端。
- `snapshot`： 从 Astra Trident 获取一个或多个快照。
- `storageclass`： 从 Astra Trident 获取一个或多个存储类。
- `volume`： 从 Astra Trident 获取一个或多个卷。

映像

您可以运行 `images` 标志来打印包含 Astra Trident 所需容器映像的表。

```
Usage:
  tridentctl images [flags]
```

标志： `* -h ， -help` ` ： 图像帮助。* -v ， -k8s-version 字符串` ： Kubernetes 集群的语义版本。

导入卷

您可以运行 `import volume` 命令将现有卷导入到 Astra Trident 。

```
Usage:
  tridentctl import volume <backendName> <volumeName> [flags]
```

别名： `volume` ， `v`

flags

- ` -f ， -filename string`： YAML 或 JSON PVC 文件的路径。
- ` -h ， -help`： 卷帮助。
- ` -no-manage`： 仅创建 PV/PVC 。不要假定卷生命周期管理。

安装

您可以运行 `install` 标志来安装 Astra Trident 。

```
Usage:
  tridentctl install [flags]
```

flags

- ` -autosupport-image string`： AutoSupport 遥测的容器映像（默认为 "netapp/trident autosupport ： 20.07.0" ）。
- ` -autosupport-proxy string`： 用于发送 AutoSupport 遥测的代理的地址 / 端口。
- ` -CSI`： 安装 CSI Trident （仅对 Kubernetes 1.13 进行覆盖，需要功能安全门）。
- ` -enable-node-prep`： 尝试在节点上安装所需的软件包。
- ` -generate-custom-yaml`： 在不安装任何内容的情况下生成 YAML 文件。
- ` -h ， -help`： 安装帮助。
- ` -http-request-timeout`： 覆盖 Trident 控制器的 REST API 的 HTTP 请求超时（默认值为 1m30s ）。
- ` -image-regRegistry string`： 内部映像注册表的地址 / 端口。
- ` -K8s-timeout duration`： 所有 Kubernetes 操作的超时（默认值为 3 毫秒）。
- ` -kubelet-dir string`： kubelet 内部状态的主机位置（默认为 "/var/lib/kubelet" ）。
- ` -log-format string`： Astra Trident 日志记录格式（文本， json ）（默认为 "text" ）。
- ` -pv` **字符串**： Astra Trident 使用的原有 PV 名称，用于确保此名称不存在（默认为 "trident " ）。
- ` -PVC` **字符串**： Astra Trident 使用的原有 PVC 的名称，用于确保此名称不存在（默认为 "trident " ）。
- ` -silning-autosupport`： 不要自动向 NetApp 发送 AutoSupport 捆绑包（默认为 true ）。
- ` –静默`： 在安装期间禁用大多数输出。
- ` -trident` **映像字符串**： 要安装的 Astra Trident 映像。

- ` -use-custom-yaml`： 使用设置目录中现有的任何 YAML 文件。
- ` -use-ipv6`： 使用 IPv6 进行 Astra Trident 的通信。

日志

您可以运行 `logs` 标志从 Astra Trident 打印日志。

```
Usage:
  tridentctl logs [flags]
```

flags

- ` A ， -archive`： 使用所有日志创建支持归档，除非另有说明。
- ` -h ， -help`： 日志帮助。
- ` -l ， -log` 字符串： 要显示的 Astra Trident 日志。Trident 中的一个 "auto"|trident 操作符 "All" （默认为 "auto" ）。
- ` -node string`： 用于收集节点 Pod 日志的 Kubernetes 节点名称。
- ` -p ， -previous`： 获取先前容器实例（如果存在）的日志。
- ` -sidecar`： 获取 sidecar 容器的日志。

发送

您可以运行 `send` 命令从 Astra Trident 发送资源。

```
Usage:
  tridentctl send [option]
```

可用选项： `AutoSupport`： 将 AutoSupport 归档发送到 NetApp 。

卸载

您可以运行 `uninstall` 标志来卸载 Astra Trident 。

```
Usage:
  tridentctl uninstall [flags]
```

标志： *`-h ， -help`： 卸载帮助。*`—静默`： 在卸载期间禁用大多数输出。

更新

您可以运行 `update` 命令来修改 Astra Trident 中的资源。

```
Usage:
  tridentctl update [option]
```

可用选项： `backend` ： 在 Astra Trident 中更新后端。

升级

您可以运行 `upgrade` 命令来升级 Astra Trident 中的资源。

```
Usage:
tridentctl upgrade [option]
```

可用选项： `volume` ： 将一个或多个永久性卷从 NFS/iSCSI 升级到 CSI 。

version

您可以运行 `version` 标志来打印 `tridentctl` 的版本以及正在运行的 Trident 服务。

```
Usage:
  tridentctl version [flags]
```

标志： * ` -client` ： 仅限客户端版本（不需要服务器）。* ` -h ， -help` ： 版本帮助。

文档的早期版本

如果您运行的不是 Astra Trident 22.04 ，则可以参考先前版本的文档。

> 对于 21.07 之前的版本，您将重定向到旧版文档站点。

- "Astra Trident 22.01"
- "Astra Trident 21.10"
- "Astra Trident 21.07"
- "Astra Trident 21.04"
- "Astra Trident 21.01"
- "Astra Trident 20.10"
- "Astra Trident 20.07"
- "Astra Trident 20.04"
- "Astra Trident 20.01"
- "Astra Trident 19.10"
- "Astra Trident 19.07"
- "Astra Trident 19.04"
- "Astra Trident 19.01"

法律声明

法律声明提供对版权声明、商标、专利等的访问。

版权

http://www.netapp.com/us/legal/copyright.aspx

商标

NetApp 、 NetApp 徽标和 NetApp 商标页面上列出的标记是 NetApp 、 Inc. 的商标。其他公司和产品名称可能是其各自所有者的商标。

http://www.netapp.com/us/legal/netapptmlist.aspx

专利

有关 NetApp 拥有的专利的最新列表，请访问：

https://www.netapp.com/us/media/patents-page.pdf

隐私政策

https://www.netapp.com/us/legal/privacypolicy/index.aspx

开放源代码

您可以在每个版本的通知文件中查看用于 Astra Trident 的 NetApp 软件中使用的第三方版权和许可证 https://github.com/NetApp/trident/。

版权信息

版权所有©2022 NetApp、Inc.。保留所有权利。Printed in the U.S.版权所涵盖的本文档的任何部分不得以任何形式或任何手段复制、包括影印、录制、 磁带或存储在电子检索系统中—未经版权所有者事先书面许可。

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

本软件由NetApp按"原样"提供、不含任何明示或默示担保、包括但不限于适销性和特定用途适用性的默示担保、特此声明不承担任何责任。IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice.NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp.The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S.patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

商标信息

NetApp、NetApp标识和中列出的标记 http://www.netapp.com/TM 是NetApp、Inc.的商标。其他公司和产品名称可能是其各自所有者的商标。